Native Inference

From grounded judgments to composable proof, computation and meaning

Mettapedia Working Notes

20 October 2026

Abstract

What must a reasoner supply for another reasoner to check an inference? We begin with observations, explicitly admitted assumptions and licensed rule instances. The formal foundation is an effective finitary presentation of judgments, with contexts and binding kept explicit. The open derivations over such a presentation form the free cartesian multicategory on its rule instances; certificate trees are its closed morphisms and shared derivation graphs unfold into them. Translations between kernels are multifunctors, determined by the images of rule instances. Logical work is additive under composition; contraction in a rule image separates tree from graph representations, while extension by definitions changes the multicategory itself. Atomic multi-pattern steps without consumption are graph replay and are independent of the order of independent steps; a consuming split/join process exceeds tree-shaped single-wire-cut composition. A finite evidence checker connects circular strong Kleene specifications to the same proof interface while distinguishing ungroundedness from unfinished checking. Buddhist epistemology and the physical, qualitative, totality and indexical distinctions of scrutability theory motivate explicit evidence boundaries; these philosophical interpretations are distinguished from formal claims. The ledger in Section 14 records names, axioms and status, and Section 15 lists what is open.

Contents

1 What makes a step of reasoning? 2

1.1 Observations, assumptions and checked consequences . . . 3
1.2 A dialogue with Buddhist epistemology . . . 3
1.3 Phenomenal and indexical grounding . . . 4
1.4 From a local step to a proof category . . . 4

2 Scope and relation to the abstract-machine notes 5

3 Circular specifications and finite evidence 5

3.1 Several meanings of an absent answer . . . 6
3.2 Reflection, observation and open-endedness . . . 6

4 The checking ladder 7

4.1 Weighted answers, effects and paid work . . . 8
4.2 Three faces across the ladder . . . 8

5 Rule signatures and the free cartesian multicategory 9

6 Kernels as free multicategories 10

7 Translations as multifunctors **11**

8 Costs: logical work, sharing and extension **12**

9 Atomic multi-pattern steps and schedules **13**

10 Admission and revalidation of retained evidence **14**

11 Module coverage and the LRAT action adapter **14**

12 The costed HOL Light to MM0 translation **15**

13 Dependent judgments, theorem authority and set models **16**

13.1 Source queries, retained execution and checked publication 16

13.2 An exact reason to require dependency 19

13.3 Finite dependent evidence through the common interfaces 26

13.4 The comparison to sets 26

13.5 Operational logical form and relative checking bases 28

13.6 A computational STT carve-out, with a proved boundary 29

13.7 Conversion policies, proof elaboration and operational fibres 31

13.8 Selected iota roots and checked dependent instantiation 33

13.9 Generated rules, bidirectional evidence and dependent append 35

13.10A Lean-family branch: exact checking, reconstruction and model soundness 52

13.11Four inductive petals and agent-facing data 54

14 Status ledger **83**

15 Open problems **155**

1 What makes a step of reasoning?

A person sees smoke above a hill and concludes that there is fire. A program matches a stored equation against a subterm and replaces it. A proof assistant applies a lemma to two earlier results. These activities differ in their objects, their sources of authority and their treatment of resources, but they share a question: *what licenses this conclusion from these particular inputs, here?*

Consider the small mathematical inference

$$\frac{e_A : A \qquad e_{A\Rightarrow B} : A \Rightarrow B}{\mathsf{apply}(e_{A\Rightarrow B}, e_A) : B}.$$

The labels e_A and $e_{A\Rightarrow B}$ record evidence, rather than an unexplained assertion that A and $A \Rightarrow B$ are true. Either input may be an assumption, in which case the conclusion remains conditional on it. Alternatively it may have a checked derivation. Recording the dependencies lets another reader distinguish these cases. Modus ponens is one possible licensed rule; it is not built into every rewrite language or every model of reasoning considered here.

The corresponding rewrite example is

$$(\mathsf{add}\ \mathsf{Z}\ n) \longrightarrow n, \qquad \mathsf{S}(\mathsf{add}\ \mathsf{Z}\ \mathsf{Z}) \longrightarrow \mathsf{S}(\mathsf{Z}).$$

A fully specified recipe cites the admitted rule, the inner site, the substitution $n := \mathsf{Z}$ and any required side evidence. The checker replays those choices. The rule does not automatically apply under every constructor: the selected language must license the surrounding context. An equality judgment, a directed execution step and a claim about an evaluator's complete result multiset are different conclusions.

This is a foundation for checking licensed inferences, not a claim that all reasoning is deduction. Induction, abduction, analogy and proof search suggest claims or revise assumptions. Their own presentations may license probabilistic updates or defeasible conclusions rather than truth-preserving deduction. A checked Bayesian update, for example, certifies the stated calculation under its model and observations; a high resulting probability is not a deductive proof of the event. The common floor asks for the contract of each step.

1.1 Observations, assumptions and checked consequences

An observation enters a formal account through an interface: a sensor report, a human assertion, a database snapshot, or a chosen mathematical model. Checking a consequence of that report does not check that the sensor was accurate. A certificate can establish

$$\Gamma, \mathsf{Report}(i, o) \vdash C$$

while leaving the reliability and interpretation of the report as explicit premises. This is useful even when a premise is defeasible: changing it identifies which retained conclusions need revalidation.

We therefore separate three obligations. *Admission* states which assumptions, observations and declarations are available. *Inference checking* validates a finite composition under those admissions. *Semantic soundness* relates the admitted rules and their consequences to a stated interpretation. None of these obligations is discharged by merely printing a theorem-shaped atom. A sealed theorem value can enforce the construction discipline inside an implementation; an exported certificate permits independent replay. Their logical comparison is separate from the runtime's unforgeability and lifetime guarantees.

1.2 A dialogue with Buddhist epistemology

The school of Dignaga and Dharmakirti distinguishes perception from inference, and inference for oneself from the presentation of an inference to another. Dharmakirti's *Nyayabindu* develops inferential knowledge through a reason whose relevant characteristics must be established[1]. The familiar smoke/fire example makes the distinction practical: recognizing smoke at this hill and licensing the connection to fire are different tasks. A formal derivation records the connection and the cited instance; it cannot manufacture reliable perception from the appearance of a valid derivation.

This motivates a division between finding a reason and presenting a checkable reason. The connection to certificate checking is architectural, not an identity between Buddhist epistemology and a modern proof calculus. In particular, inference as valid cognition includes epistemic demands that the replay of a formal certificate does not settle. The companion *Formal Buddhist Logic* studies precise readings of valid reasons, non-perception, exclusion and four-corner evidence separately.

One consequence matters immediately: failure to observe is not, by itself, evidence of absence. A negative conclusion requires a premise such as adequate coverage, observability or an exhaustive finite check. Similarly, failure to find a proof is not a proof of negation. Four-corner evidence, where positive and negative support can both occur or both be absent, is useful for some knowledge stores. It is not automatically the logic of every native kernel, and neither four-corner evidence nor strong Kleene partial truth is simply identified with the Buddhist fourfold analysis.

1.3 Phenomenal and indexical grounding

Chalmers's scrutability programme distinguishes physical information P, phenomenal information Q, totality information T and indexical information I[2, 3]. Indexical information supplies a location within an account; the totality component addresses what the account excludes. These distinctions help ask what a formal reasoning system has been given, but the scrutability thesis does not provide an executable kernel.

Our use is deliberately open-ended. Let $i : I$ specify an agent, stage or location; let $p : P(i)$ specify publicly represented conditions; and let $Q(i, p)$ record further qualitative distinctions. The indexed total space

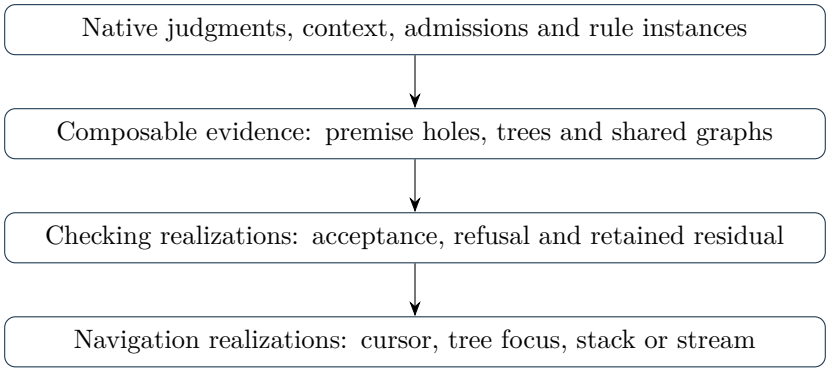
$$\mathcal{E} = \sum_{i:I} \sum_{p:P(i)} Q(i, p)$$

keeps the coordinates distinct. It does not assert that these families exhaust possible experience or that qualitative information is recovered from public data. Replacing the physical vocabulary by abstract laws A gives an $A/Q/I$ variant of this descriptive scheme; it is not a renamed assertion of Chalmers's full $P/Q/T/I$ thesis. A local coverage certificate may authorize a particular negative conclusion without committing to a global "that's all" claim about the world.

The immediate grounding for a human reasoner may include phenomenal experience. A machine account instead requires a specified observation interface; this paper does not establish that the interface is conscious or that recording an experience report captures the experience itself. The operational, intensional and extensional faces of inference are another distinction: an execution, its retained proof evidence and its interpretation. Their comparison maps are mathematical obligations. The three faces are not identified term for term with P, Q and I; in particular a proof's meaning in a Henkin model is not thereby a phenomenal state.

1.4 From a local step to a proof category

A rule has typed input slots and a typed output. Filling its slots with derivations constructs a larger derivation. Filling slots in stages or all at once agrees by associativity; a premise hole is an identity. Multiple input slots lead to a multicategory. A unary rewrite sequence gives the path-category case. This is the elementary categorical content of proof composition, rather than a demand that every language be encoded in one privileged logic.



These are dependency levels, not successive authorities. A cursor's current focus can be made observable, like a reader's position in a proof, but a source calculus need not contain that position. Conversely a language that makes scheduling or resource ownership observable needs a realization theorem preserving precisely those observations. The model should follow those semantics rather than the convenience of a particular storage backend.

2 Scope and relation to the abstract-machine notes

The companion notes *Abstract Machines* (Section 13, "Proof machines: scoped, licensed composition") describe how source judgments, composable evidence, checking realizations and navigation adapters are layered, and which checking machines realize which kernels. This note does not repeat that material. It supplies the mathematical structure of the evidence layer: what composition of derivations is, how translations act on it, what costs are attached to it, and how atomic multi-premise execution relates to it.

Section map.

Section	Content
3	Circular specifications, finite evidence and semantic partiality
4	The checking ladder and its distinct comparison axes
5	Rule signatures, open derivations and freeness
6	Kernels as free multicategories: trees, graphs and authored calculi
7	Translations as multifunctors
8	Costs: logical work, sharing (SHARE) and extension (EXTEND)
9	Atomic multi-pattern steps and schedules
10	Admission and revalidation of retained evidence
11	Module coverage and the LRAT action adapter
12	The costed HOL Light to MM0 translation
13	Dependent judgments, theorem authority and set models
14	Status ledger
15	Open problems

3 Circular specifications and finite evidence

Circular syntax and circular justification are different. A finite collection of named propositions can contain a reference cycle, while a particular justification for one of its conclusions is still a finite, well-founded tree. For example, if a grounded fact G is true, then $L \vee G$ is true in strong Kleene semantics even when L refers negatively to itself. The evidence uses G; it does not need a proof of L.

Let each judgment $j : J$ have a specification $\mathsf{spec}(j)$ built from base facts, truth references, negation, conjunction and disjunction. A valuation is a pair (H, F) of positive and negative extensions, ordered by inclusion in both coordinates. Strong Kleene evaluation induces a monotone jump $\Phi(H, F)$. Starting with $(\varnothing, \varnothing)$ selects its least fixed point (H_*, F_*)[4]. With consistent base facts the two extensions are disjoint. They need not cover J.

The liar $L = \neg\mathsf{truth}(L)$ receives neither polarity. The truth-teller $U = \mathsf{truth}(U)$ also receives neither in the least fixed point, although other fixed points can make it true or false. Merely checking that an offered valuation is a fixed point therefore does not establish groundedness.

Theorem 3.1 (Grounded evidence as a native proof profile)**.** *For the existing strong Kleene specification system with decidable base admission and decidable equality of judgment and base labels, ten local rule codes give an exact rule-witness interface. A judgment j has a positive or negative grounded verdict exactly when there exists an accepted finite certificate for its specification at that polarity. Equivalently, that signed specification has a closed proof in the free multicategory of the interface. These statements do not require J finite.*

Lean. `KripkeTruth.Checking.interface`, `Checking.check_sound`, `Checking.coverage`, `Checking.judgment_coverage`, `Checking.closed_proof_iff_grounded`

The local rules are the source evidence constructors: admitted positive and negative base leaves; explicit unfolding at a truth reference; polarity reversal at negation; two positive premises for

conjunction and either negative premise to refute it; either positive premise for disjunction and two negative premises to refute it. There is no rule that establishes a judgment merely by citing its own label. Generic tree and graph checking apply without a new account of proof composition. This is one particular partial-truth logic hosted by the framework, not a change of every kernel to a three-valued object logic.

Theorem 3.2 (Finite decision agrees with the certificate authority)**.** *Under these assumptions, if J is finite, iteration from the empty valuation reaches (H_*, F_*) after at most $2|J|$ jumps. The implemented Boolean-pair iteration returns a polarity exactly when some finite certificate for that polarity is accepted.*

Lean. `KripkeTruth.System.stage_eq_least`, `System.groundedDecide_spec`, `Checking.finite_decision_iff_certificate`

The bound counts semantic jumps. It neither makes a jump constant time nor proves that the current recursive evaluator avoids repeated work. It depends on a finite judgment domain and decidable base admission. It is not a bound for an unrestricted self-referential truth language with quantifiers.

Example 3.3 (Completion is not bivalence)**.** The finite example computes the twice-ascribed fact as positive, a conjunction with a grounded false component as negative, and the liar and truth-teller as undetermined. The completed liar computation is not unfinished work. Conversely the initially unknown fact becomes true at a later stage, so an intermediate unknown value does not establish ungroundedness. The fact certificate with a forged negative claim is refused, and a self-reference with no supplied premise is refused. The liar has no finite certificate for either polarity.

Lean. `KripkeTruth.Example.decided_verdicts`, `Checking.Controls.fact_accepts`, `Checking.Controls.forged_false_refuses`, `Checking.Controls.absent_premise_refuses`, `Checking.Controls.false_base_leaf_refuses`, `Checking.Controls.cyclic_graph_refuses`, `Checking.Controls.liar_has_no_certificate`

3.1 Several meanings of an absent answer

Outcome	What has been established
Accepted certificate	This evidence checks for this claim under these admissions.
Refused certificate	This candidate fails checking; another candidate may prove the claim.
Negative evidence	A stated negative judgment, or an obstruction under the selected authority.
Grounded undetermined	Neither polarity belongs to the selected least fixed point.
Unfinished check	A retained residual remains; no final verdict follows.
Outside an effective profile	The selected checking method has no authority for this request.
Conflicting reports	A knowledge store has both supports; the chosen logic must specify their consequence.

The shared authority algebra already distinguishes evidence, obstruction, fragment boundaries and restartable resource frontiers. The partial-truth profile gives one mathematically stable meaning to a boundary. It must not turn a timeout, a missing proof or an untrusted report into a falsehood. Nor should the three-valued semantic result replace the richer execution status. Both coordinates can be retained.

Lean. `KripkeTruth.Checking.finite_undetermined_iff_no_certificates`

3.2 Reflection, observation and open-endedness

Self-reference imposes a precise local obstruction: when the liar equation is required, no total Boolean fixed-point valuation satisfies the system. This is not a theorem that every language

containing a truth symbol is inconsistent. Gödel incompleteness is a different obstruction: an appropriate consistent, effective arithmetical theory cannot capture every truth of its intended standard model. Reflection levels record which theory a claim concerns; they do not grant an unqualified rule asserting the current theory's own soundness. The existing consistency-tower results are parameterized by their arithmetical hypotheses, which must be instantiated before a concrete base theory is claimed.

Model observations have their own limitations. Indistinguishability by the selected formulas is weaker than isomorphism; downward Lowenheim–Skolem requires the relevant language and parameter cardinal bounds. First-order theories can be categorical at a specified infinite cardinal, so the finite-model observation cannot be generalized to prohibit such categoricity. These model-theoretic qualifications do not alter the local rule checker.

Neither hypersets nor a terminal coalgebra are required for the evidence profile above. A final coalgebra is final for a specified functor in a specified category, not an automatic universe containing every possible logic. Likewise, observer-relative indistinguishability does not justify substitution in arbitrary contexts: a replacement theorem must establish the congruence or naturality law for the contexts actually permitted.

4 The checking ladder

Tier zero names a checking discipline: supply finite evidence for licensed rule applications, with explicit dependencies and enough local choices for a terminating checker. It does not name a weakest axiom system, a unique primitive basis, a universal notion of truth, or a promise of polynomial checking for every possible encoding. A powerful set theory and a modest rewrite calculus can both have this discipline.

Presentation	Supplied evidence	Principal checking obligation
Explicit rule replay	Rule instances, premises, substitutions and side witnesses	Replay each licensed composition.
Retained sharing	A dependency graph and selected root	Validate references and local instances without unfolding shared premises.
Compressed computation	A strategy, fuel and optional checkpoints	Reproduce the specified computation and its endpoint.
Theory extension	Declarations, definitions and admission evidence	State conservativity or new assumptions and revalidate reuse.
Reconstruction	Untrusted tactic or prover output	Produce a certificate that the destination independently checks.

This is a family of comparisons rather than one total ordering. Logical strength, source expressiveness, certificate size, checking time, search power and trusted implementation are separate axes. A proof-size translation does not by itself prove a runtime bound. A faithful map on existing proofs need not reflect the existence of proofs in its target. The intuitionistic-to-classical inclusion below is a concrete example.

Fuel illustrates both the benefit and the price of compression. If a specified machine accepts after n reduction steps, supplying n makes bounded replay decidable and reproduces that successful run. A binary count needs only logarithmically many bits, but replay can still take n steps. Checkpoints need justification between them; an explicit trace writes more information. A trace's linear number of local checks is not automatically linear bit-time if those local checks manipulate large substitutions or expressions.

The library already formalizes fuelled conversion evidence, explicit equality recipes and reproducible strategy hints. It also keeps the Lean4Lean algorithmic checker separate from ideal typing and

conversion. More fuel recovers a successful finite run; it cannot make an incomplete algorithm complete for a larger ideal relation. The present executable Lean-related instance is a specified head-conversion profile, not a proof that the full Lean kernel has been reconstructed. The Lean4Less boundary records this distinction rather than treating implementation heartbeats as portable reduction counts.

The comparison target matters more than the implementation language. McTT proves soundness and completeness of an extracted checker for its specified Martin-Lof type theory[9]. The observational theory TT_{obs} has a normalizing reduction and decidable conversion and type checking for its own rules[12]. These are useful reference profiles; neither result is a completeness theorem for Lean's different judgment, nor does implementing a checker in Rocq identify its object theory with Rocq's full calculus. Runtime efficiency requires a separate measurement.

4.1 Weighted answers, effects and paid work

A useful companion model of an effectful computation is

$$\mathsf{Spec}(K, C, W, X) = W \longrightarrow \mathsf{Ans}_K(X \times W \times C), \qquad \mathsf{Ans}_K(Y) = Y \to_{\mathrm{fin}} K.$$

Here K is a commutative semiring of answer annotations, C a commutative monoid of branch costs, and W a world or space. Sequencing combines weights, threads each branch's state and accumulates its cost. This supplies one lawful model, not a complete semantics of every language that returns several answers. Reads and writes need not commute. A weight is not automatically a probability, a degree of truth or an unresolved scheduler choice. In particular the existing no-distributive-law result concerns probability distributions over finite sets with specified naturality and unit laws; it is not a prohibition on every combination of probabilistic and nondeterministic operations.

Lean. `LayerEffects.Spec.bind_assoc`, `LayerEffects.Controls.put_get_not_comm`, `NondeterminismProbability.no_distributiveLaw`

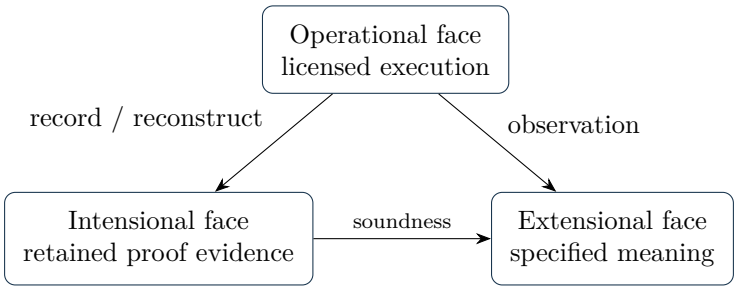
Two proved information-loss boundaries matter for inference. Charging a branch and then failing leaves no successful answer carrying that charge; the total paid work cannot in general be recovered from the surviving answers. It needs an execution account with its own observation contract. Likewise marginal positive/negative counts for A and B do not determine counts for $A \wedge B$ on the same worlds. Cartesian pairing of sample bags has a lawful product, but that product does not recover unknown dependence. A proof checker can validate either stated calculation without silently equating their meanings.

Lean. `LayerEffects.Spec.no_failed_cost_recovery`, `EvidenceSpaces.no_same_world_conjunction_from_marginals`

These models belong downstream of the rule and evidence layer. A categorical completion such as $\mathrm{Ind}(\mathcal{C})$, or a dependent type theory, requires its own objects, judgments and comparison theorems; neither follows from the displayed computation type alone. Similarly, a checked absence frontier proves a negative claim relative to its licensed future relation. Identifying that claim with a guest logic's internal negation requires a separate interpretation theorem.

4.2 Three faces across the ladder

The operational, retained-proof and semantic faces persist across the ladder. An execution explains how checking proceeded, a proof records what licensed the claim, and a semantic map explains what the claim means. A compiler or optimization must name which face it preserves, which it reflects and what information its observation forgets.



For HOL, one extensional judgment is a *semantic sequent* $\Gamma \models_M A$: every admissible assignment in model M satisfying the assumptions in Γ satisfies A. A Henkin model specifies the domains of quantification, including which function values are available. Soundness connects proof rules to this satisfaction relation; it does not identify the model with the proof's syntax. Different proofs and execution orders can have one semantic sequent. The arrows in the diagram require their stated contracts and generally cannot be inverted.

5 Rule signatures and the free cartesian multicategory

Definition 5.1 (Cartesian multicategory)**.** A cartesian multicategory over a type of sorts Σ assigns to each finite context $\Gamma : \mathrm{Fin}\, n \to \Sigma$ and sort b a type $\mathrm{Hom}(\Gamma; b)$, with variables $\mathrm{var}_i : \mathrm{Hom}(\Gamma; \Gamma_i)$ and simultaneous substitution $\mathsf{subst} : \mathrm{Hom}(\Gamma; b) \times \prod_i \mathrm{Hom}(\Delta; \Gamma_i) \to \mathrm{Hom}(\Delta; b)$, satisfying $\mathrm{var}_i[\sigma] = \sigma_i$, $t[\mathrm{var}] = t$ and $t[\sigma][\tau] = t[\sigma[\tau]]$. A multifunctor over a map of sorts $f : \Sigma \to \Sigma'$ sends $\mathrm{Hom}(\Gamma; b)$ to $\mathrm{Hom}(f \circ \Gamma; fb)$, preserving variables and substitution.

Lean. `CartesianMulticategory`, `CartesianMulticategory.Functor`

Reindexing contexts by composition makes multifunctors compose without transport; the list-indexed `MultiSortedClone` is the restriction to contexts of the form ℓ.get. This is the many-sorted abstract clone presentation; for the correspondence between clones, cartesian multicategories and their internal languages see Saville, *Clones, closed categories, and combinatory logic*[7], which also recalls the multicategories introduced by Lambek in *Deductive systems and categories II*[5]. Lambek and Scott, *Introduction to Higher Order Categorical Logic*, treat the unary deductive systems from which this generalizes.

Proposition 5.2 (Multifunctors form a category)**.** *Identity and composition of multifunctors satisfy the unit and associativity laws. A multifunctor carries closed multimorphisms to closed multimorphisms.*

Lean. `CartesianMulticategory.Functor.id_comp`, `comp_id`, `comp_assoc`, `Functor.inhabited`

Definition 5.3 (Rule signature, open derivations)**.** A rule signature S assigns to each premise list $p_1 \cdots p_k$ and conclusion c a type $S(p; c)$ of rule instances. Open derivations $\mathsf{Term}_S(\Gamma; b)$ are generated by premise holes var_i and nodes $r(t_1, \ldots, t_k)$ with $r \in S(p; c)$ and $t_j \in \mathsf{Term}_S(\Gamma; p_j)$. Substitution plugs derivations into holes. The generator of r is $\mathrm{gen}(r) = r(\mathrm{var}_1, \ldots, \mathrm{var}_k)$.

Lean. `RuleSignature`, `RuleSignature.Term`, `Term.subst`, `RuleSignature.free`, `RuleSignature.gen`

Theorem 5.4 (Freeness)**.** *Open derivations with substitution form a cartesian multicategory* $\mathsf{Free}(S)$*. For every cartesian multicategory M and map of sorts f, restriction to generators is an equivalence between multifunctors* $\mathsf{Free}(S) \to M$ *over f and interpretations*

$$\textstyle\prod_{p,c}\ S(p; c) \longrightarrow \mathrm{Hom}_M(f \circ p;\ fc).$$

Lean. `RuleSignature.universal`, `RuleSignature.functor_eq_of_gen`, `RuleSignature.lift_gen`, `Term.lift_subst`

Theorem 5.5 (Derivability)**.** *A sort is derivable in* $\mathsf{Free}(S)$*, i.e. has a closed derivation, exactly when it lies in the inductive closure of the rule predicate* $\exists r \in S(p; c)$*. Every multifunctor between free multicategories preserves derivability.*

Lean. `RuleSignature.derivable_iff_derives`, `RuleSignature.Derivable.map`

Proposition 5.6 (Structural rules)**.** *Holes may be discarded or reused, so* $\mathsf{Free}(S)$ *is cartesian; two holes of the same sort remain distinct derivations. Generators are planar (each premise used once, in order), and every closed derivation is planar.*

Lean. `Term.duplicate_holes_distinct`, `Term.gen_linear`, `Term.closed_linear`

Here the holes are metalevel proof dependencies for the fixed rule signature. This construction does not authorize copying an object-level linear resource or a capability. Such constraints must already be present in the judgment, the allowed instances or a resource-sensitive composition. Planarity of these closed terms is not an equivalence of arbitrary linear, symmetric and cartesian object logics.

Theorem 5.7 (Schematic level: instance actions)**.** *If a map of sorts carries every rule instance to an instance of the mapped premises and conclusion, it induces a multifunctor* $\mathsf{Free}(S) \to \mathsf{Free}(S)$*, and derivability is closed under the map. Actions compose coherently: if two actions compose on sorts and on instances to a third, acting by the two in turn agrees with acting by the third on every derivation, up to the identification of sorts. In particular substitution of formulas for atoms acts on the intuitionistic and classical Hilbert multicategories, coherently with composition of substitutions, so Hilbert derivability is substitution closed as a consequence of freeness.*

Lean. `RuleSignature.InstanceAction`, `RuleSignature.actFunctor`, `RuleSignature.Derivable.act`, `RuleSignature.act_comp_heq`, `Hilbert.substFunctor`, `Hilbert.substFunctor_comp_heq`, `Hilbert.source_subst`

Remark 5.8 (Contexts, binders and side conditions). Object-level contexts and binders are part of the sorts: a HOL Light theorem carries its hypothesis set and its terms their binders, and side conditions such as freshness for abstraction are part of the rule-instance predicate. They change the generators, not the kind of structure (Section 6). Rules with one conclusion give a multicategory; multi-product consuming steps are treated in Section 9.

6 Kernels as free multicategories

A kernel with a decidable rule-instance interface $(\mathrm{W}, \mathrm{isInstance})$ presents the signature whose rule instances are accepted witnesses.

Theorem 6.1 (Derivability)**.** *Kernel derivability coincides with closed derivability in the free multicategory on the kernel's accepted witnesses.*

Lean. `RuleWitness.signature`, `RuleWitness.derivable_iff_derives`

Theorem 6.2 (Certificate trees are closed morphisms)**.** *For every conclusion* b*, closed derivations of* b *correspond bijectively to certificate trees of the kernel that its tree checker accepts with conclusion* b*. Erasure to trees is injective.*

Lean. `RuleWitness.closedEquiv`, `RuleWitness.toDerivation_injective`

Theorem 6.3 (Graphs unfold)**.** *Replaying a shared derivation graph with the unfolding step yields exactly the judgments of ordinary replay, now with closed derivations. Every node of an accepted graph denotes a closed morphism of its conclusion, and therefore an accepted certificate tree.*

Lean. `RuleWitness.termStep`, `RuleWitness.unfold_judgments`, `RuleWitness.unfold_root`, `RuleWitness.unfold_tree`

Theorem 6.4 (Graphs present exactly the closed morphisms)**.** *Every closed derivation is the unfolding of the last node of some accepted derivation graph. With the previous theorem, accepted graphs present exactly the closed morphisms of the free multicategory: sharing is a property of the representation, never of the morphism.*

Lean. `RuleWitness.presented`, `RuleWitness.presented_block`, `DerivationGraph.replayWith_relocate`, `DerivationGraph.replayWith_append`

Theorem 6.5 (Authored calculi)**.** *For every validated authored calculus, the existing open derivations with ordered premise contexts are exactly the terms of the free multicategory on its rule applications, and plugging derivation vectors is substitution. Every interpretation of the rule applications in a cartesian multicategory extends uniquely to open derivations, preserving substitution. Accepted raw open proofs decode computably to typed open derivations.*

Lean. `CertificateGSLT.ruleSignature`, `OpenDerivation.termEquiv`, `OpenDerivation.toTerm_bind`, `interpret_bind`, `interpret_unique`, `decodeOpen`

Corollary 6.6. *The classifying category of every authored calculus (VibeITP, projected Metamath snapshots, HOL Light's first-order calculus) is the category of contexts of a free cartesian multicategory, with products by concatenation.*

Lean. `ClassifyingContext.instCategory`, `ClassifyingContext.concatIsLimit`, `OpenDerivation.termEquiv`

Kernel	Rule-instance interface	Free multicategory
Intuitionistic / classical Hilbert	finite codes, profile-gated excluded middle	`RuleWitness.signature(interfacep)`
Extended Hilbert	classical codes and admitted definitions	`RuleWitness.signature(interfacehs)`
HOL Light	ten rules and policy axioms	`RuleWitness.signature(ruleWitnesspolicy)`
LRAT, RUP fragment	input clauses and RUP instances	`RuleWitness.signature(interfaceF)`
Authored calculi	rule applications	`CertificateGSLT.ruleSignature`
Grounded circular truth	ten codes over signed specifications	`RuleWitness.signature(Checking.interfacesystem)`
Guarded atom rules, presentations	original rule codes	`Presentation.ruleWitness`

7 Translations as multifunctors

By Theorem 5.4, a translation is determined by the images of rule instances; derivability is preserved automatically.

Theorem 7.1 (Faithful renamings)**.** *A sort-preserving renaming of rule instances that is injective induces an injective multifunctor.*

Lean. `RuleSignature.rename`, `RuleSignature.lift_rename_injective`

Theorem 7.2 (Intuitionistic into classical)**.** *The inclusion of intuitionistic into classical Hilbert rule instances induces a faithful multifunctor that preserves but does not reflect derivability: excluded middle is classically derivable and not intuitionistically derivable.*

Lean. `Hilbert.inclusion`, `Hilbert.inclusion_faithful`, `Hilbert.inclusion_not_reflecting`

Theorem 7.3 (Classical into extended)**.** *For every list of definitions, the inclusion of classical into extended rule instances induces a faithful multifunctor. For admitted fresh definitions it is conservative on claims in the old vocabulary. It is not full: citing a definition is a derivation in the image of no classical derivation.*

Lean. `Extension.inclusion`, `Extension.inclusion_faithful`, `Extension.inclusion_conservative`, `Extension.inclusion_not_full`

Theorem 7.4 (HOL Light into its first-order calculus)**.** *Each accepted HOL Light rule instance is translated by the node step on its canonical premise context and decoded to a typed open derivation of its rendered conclusion. This determines a unique multifunctor from HOL Light's free rule multicategory to the free multicategory of its first-order calculus over the rendering of theorems. It preserves derivability, and for admitted axiom lists reflects it: the calculus derives the rendering of a sequent exactly when HOL Light derives a theorem with that sequent. Positive control:* $\vdash p = p$ *by* REFL*; negative control:* $\vdash \mathsf{F}$*.*

Lean. `HOLLight.FirstOrder.generatorImage`, `translation`, `translation_unique`, `translation_derivable`, `translation_reflects`, `Kernel.reflexivity_translated`, `Kernel.falsity_not_translated`

Theorem 7.5 (Sharing commutes with translation)**.** *For any interpretation of a kernel's rule instances in a free multicategory, translate a shared derivation graph node by node: check each node against its source premise sorts and instantiate its rule image with the translated premises. From translated entries this computes exactly the multifunctor images of the unfolded entries. Hence every accepted graph translates with one rule image per node, and the value at each node is the multifunctor image of that node's closed derivation. In particular this holds for HOL Light graphs and the translation above.*

Lean. `RuleWitness.liftStep`, `RuleWitness.liftStep_liftEntry`, `RuleWitness.replay_liftStep`, `RuleWitness.shared_translation`, `HOLLight.FirstOrder.shared_translation`

Remark 7.6. The existing compilation of first-order calculi into MM0 (the system of *Metamath Zero: The Cartesian Theorem Prover*[8]) and the existing proof-graph translation with shared lemmas complete the chain to MM0 at the level of certificates. That existing translation builds each lemma with the same node step over the cumulative context. On all 25 nodes of the three real HOL Light exports of Section 12, each of its lemmas coincides with the raw rule proof from which the multifunctor's rule image is decoded, after renaming premise positions to the lemma's parents. This is an executed comparison; the general equality is open (Section 15).

8 Costs: logical work, sharing and extension

Theorem 8.1 (Additivity)**.** *Rule nodes are additive under substitution, each hole occurrence charged once:* $\mathsf{size}(t[\sigma]) = \mathsf{size}(t) + \sum_{i \in \mathsf{occ}(t)} \mathsf{size}(\sigma_i)$*.*

Lean. `Term.size_subst`

Theorem 8.2 (Affine translations are linear on trees)**.** *If every rule image uses each premise at most once and has at most* c *rule nodes, the induced multifunctor satisfies* $\mathsf{size}(Ft) \le c \cdot \mathsf{size}(t)$*.*

Lean. `RuleSignature.lift_size_le`

Theorem 8.3 (SHARE: a contraction-induced expansion gap)**.** *Translate the chain calculus (one premise per step) into the duplication calculus by an image citing its premise twice. The* k*-chain has* $k+1$ *nodes; its translation is a tree with* $2^{k+1}-1$ *nodes; the duplication calculus has an accepted shared graph with* $k+1$ *rows for the same conclusion. The contracting image is not affine.*

Lean. `Contraction.contraction_gap`, `Contraction.image_not_affine`, `ProofSharingBounds.no_tree_sizeSimulation_dag`

Remark 8.4 (EXTEND). Extension by definitions changes the multicategory: the inclusion is faithful and conservative but not full (Section 7), and literal expansion of an admitted doubling chain has $2^{k+1} - 1$ formula nodes from encoded size at most $5(k + 1)(k + 2)$ (`PropositionalExtension.literal_expansion_gap`). Sharing never changes the multicategory: a graph and its unfolded tree are one morphism. In the sense of Cook and Reckhow, *The Relative Efficiency of Propositional Proof Systems*, a translation with polynomial size and time bounds is a p-simulation. The proved size separation rules out polynomial-size translation from the shared duplication certificates to trees; the straightforward tree-to-DAG representation comparison has no such size obstruction. An executed compiler-time bound must be supplied separately. Extended resolution's polynomial simulation of DRAT (Kiesl, Rebola-Pardo and Heule, *Extended Resolution Simulates DRAT*) is the corresponding EXTEND phenomenon for SAT certificates.

Proposition 8.5 (Time accounts)**.** *The p-simulation of HOL Light proof graphs by MM0 has a degree-ten certificate bound and a degree-ten declared time account; the declared time is at least the weight of the lemmas the translation writes. An operational simulation derives its clock from a finite-stack machine execution, and such clocks are positive.*

Lean. `HOLLight.FirstOrder.graphTranslate_weight`, `graphTime_ge_weight`, `Kernel.mm0Simulation`, `OperationalPSimulation.toPSimulation`, `OperationalPSimulation.time_positive`

Status. The HOL Light to MM0 time account is declared, bounded below by output weight; its finite-stack realization is open.

9 Atomic multi-pattern steps and schedules

A step keeps some occurrences, consumes others and produces atoms, when its rule admits them; this is the shape of multi-headed CHR simpagation (Di Giusto, Gabbrielli and Meo, *On the expressive power of multiple heads in CHR*) and of chemical-style rewriting (Berry and Boudol, *The Chemical Abstract Machine*).

Theorem 9.1 (Local characterization of replay)**.** *Sequential replay with any local step accepts exactly when the final cache extends the initial one, has one value per node, every reference points backward, and every node's step on its looked-up premises yields its value.*

Lean. `DerivationGraph.replayWith`, `DerivationGraph.replayWith_iff_local`, `DerivationGraph.replayWith_judgmentStep`

Theorem 9.2 (Schedule independence)**.** *For a step that ignores physical reference positions, reordering a graph by any permutation that keeps premises before consumers, with references relocated, accepts exactly when the original order does (for topological originals), and moves each node's value to its new position. A reordering that places a premise after its consumer is refused. The numbers of local steps and premise lookups are invariant. In particular judgment replay and unfolding are schedule independent.*

Lean. `DerivationGraph.replayWith_permute`, `replayWith_permute_reflect`, `replayWith_permute_iff`, `replayWith_permute_refused`, `permute_lookups`, `replay_permute`, `RuleWitness.unfold_permute`

Theorem 9.3 (Monotone steps are derivations)**.** *Without consumption, running single-product steps is replay of the corresponding derivation graph, and every accepted trace denotes, at every step, a closed morphism of the free multicategory on the step rules. A monotone step with several products runs exactly as the sequence of its single-product components.*

Lean. `MultiPremiseSteps.run_monotone`, `run_monotone_unfolds`, `run_split`

Theorem 9.4 (Consumption)**.** *In an accepted trace a consumed occurrence is never cited again; occurrences may be kept twice but not consumed twice, nor kept and consumed together. Two steps competing for one occurrence lead to different stores, neither of which can fire the other. Two steps with independent footprints (neither consumes what the other cites, and the second cites no product of the first) fire in either order, with the same live atoms up to the order of their products. In the formalized tree of single-wire cuts, a composite has one internal wire fewer than generators. The split/join process has two steps and two internal wires, so that compositional syntax cannot represent it while preserving those counts. This rules out that syntax as a complete account; it does not rule out encoding processes in categories or alternative polycategorical frameworks.*

Lean. `MultiPremiseSteps.consumed_not_cited`, `Controls.consumption_not_confluent`, `Controls.keep_twice`, `Controls.take_twice_refused`, `independent_commute`, `Controls.independent_steps_commute`, `PolyComposite.wires_add_one`, `process_not_polycategorical`

Remark 9.5 (What the correspondence keeps). Kept: rule witnesses, ordered and repeated premise occurrences, every conclusion, acceptance including refusal of unused bad steps, and step counts. Forgotten: the order of independent steps; sharing (a graph and its unfolding are one morphism); the identity of steps not used by the selected root. Consumption keeps resources linear and needs monoidal composition of processes; negative reads are outside this step shape.

10 Admission and revalidation of retained evidence

Theorem 10.1 (Exact revalidation)**.** *For a graph accepted under one interface, rechecking each node's rule instance against another interface, with premises read from the retained cache, succeeds exactly when the mapped graph replays under the new interface to the same cache. Admitted extensions (witness maps keeping accepted instances accepted) transport evidence; checked judgments are reusable in larger rule systems; a retained source-address schedule replays to the identical state under a new interface accepting each instance it used.*

Lean. `DerivationGraph.revalidate_iff`, `replay_extend`, `CheckedJudgment.extend`, `BorrowedCursor.run_reinterpret`

Theorem 10.2 (Propositional admission joined to retained evidence)**.** *Every accepted intuitionistic graph is accepted classically with the same cache. An accepted extended graph remains accepted when further admitted definitions are appended. Classical evidence for excluded middle is refused on intuitionistic revalidation; evidence citing a definition that was replaced by a different admitted definition is refused.*

Lean. `Revalidation.intuitionistic_to_classical`, `Revalidation.checkDAG_append`, `Controls.em_refused_intuitionistically`, `Controls.worked_extended`, `Controls.worked_changed_refused`

11 Module coverage and the LRAT action adapter

Theorem 11.1 (HOL Light kernel module)**.** *The first-order calculus of HOL Light over an axiom list renders as a portable module that the verified reader and the original validator load exactly. The loaded package derives the rendering of a sequent exactly when HOL Light derives a theorem with that sequent, for admitted axiom lists.*

Lean. `HOLLight.FirstOrder.kernelModule`, `kernelModule_read`, `kernelModule_faithful`

Status. Exported for the bare kernel: 147 rules, 991 249 bytes, read back through the actual reader and validator.

Theorem 11.2 (LRAT actions)**.** *A reverse-unit-propagation addition is a multi-premise rule instance whose premises are its hinted clauses, and it is sound. An LRAT action stream compiles, with identifiers resolved to positions and deletions enforced, to a derivation graph; acceptance of that graph with an empty clause refutes the formula, and every action denotes a closed morphism of the free multicategory on RUP instances. A resolution-asymmetric addition need not be implied, so it is not a rule instance of this calculus.*

Lean. `PropositionalLRATActions.rupCheck_sound`, `check_sound`, `check_graph`, `check_unfolds`, `rat_not_implied`, `rat_not_rule_instance`

Native profile	Faithful presentation	Portable module
VibeITP	authored calculus, `kernel_derives_iff_classifyingArrow`	yes, 123 schemas
Metamath snapshot	projected authored calculus	yes, 25 schemas
HOL Light kernel	first-order calculus, `kernelModule_faithful`	yes, 147 rules
Hilbert, extended Hilbert	exact rule codes; sorted Hilbert source presentation also exists	not qualified here
LRAT (RUP fragment)	action calculus, `check_sound`	no (RUP is a computed check)
Grounded circular truth	signed evidence rules, `judgment_coverage`	no (base admission is a parameter)
MM0, Megalodon	owned by the active MM0 lane	not covered here
Rho, MeTTa equations, GSLT recipes	retained workers (*Abstract Machines*, Section 13)	open

12 The costed HOL Light to MM0 translation

The existing receiver replays real HOL Light exports, lowers the registered receipts to proof graphs, translates each node through its first-order step, compiles the shared lemmas to MM0 saved proofs, checks them in Lean and prints specification and proof texts. The reference MM0 tools then compile the proof text and verify the binary against the specification.

Export	events	spec (MB)	MMU (MB)	emit (s)	MMB (bytes)	compile (s)	verify (s)
T_DEF	1	26.2	40.1	14.4	220 512	0.58	0.13
TRUTH	8	22.6	252.2	2166	223 536	4.08	0.14

Emission includes the Lean-side check and printing of each text: for TRUTH, lowering, node translation and shared-proof compilation take under a minute, while checking and printing the proof text and the MM1 text take 16 and 19 minutes at a peak of 2.8 GiB. The specification encodes the fixed 147-rule presentation and dominates the binary certificate; the eight-node proof adds about three kilobytes to it. The proved bounds are polynomial of degree ten with generous constants; the measured growth is far below them. The text formats and the in-process checking dominate wall-clock cost, not the reference verifier. These are executed times of the existing binary and are kept separate from the logical work and the declared time account of Section 8, which the following table reports in the measures of the proved bounds (graph size N, lemma-certificate weight, first-order rule nodes and declared time).

Export	nodes	N	certificate weight	rule nodes	declared time
T_DEF	1	371 826	185 913	1	1.9×10^{22}
TRUTH	8	3 610 421	4 562 433	1 505	1.3×10^{24}
core bool	16	43 070 295	24 292 374	1 513	5.2×10^{28}

These measurements are historical qualification runs of the receiver frozen on 13 October 2026. They have not been reproduced with a rebuilt receiver in the present source state and are not current performance guarantees.

The certificate weight is $1.26\,N$ on TRUTH, while the proved bound $(35 \cdot 10^{12} + 2)(N+1)^{10}$ exceeds 10^{78}: the degree-ten bound is sound and very loose. The declared time is dominated by its fourth-power comparison term and says nothing about executed time. On the sixteen-event core bool export, the executed compilation to MM0 saved proofs exhausted its available memory at the last of its sixteen nodes, after assembling a 156-rule presentation; memory, not time, bounds the executed translation there.

13 Dependent judgments, theorem authority and set models

Four independent questions determine a kernel interface. Keeping them separate permits the same computation to support several faithful consumers.

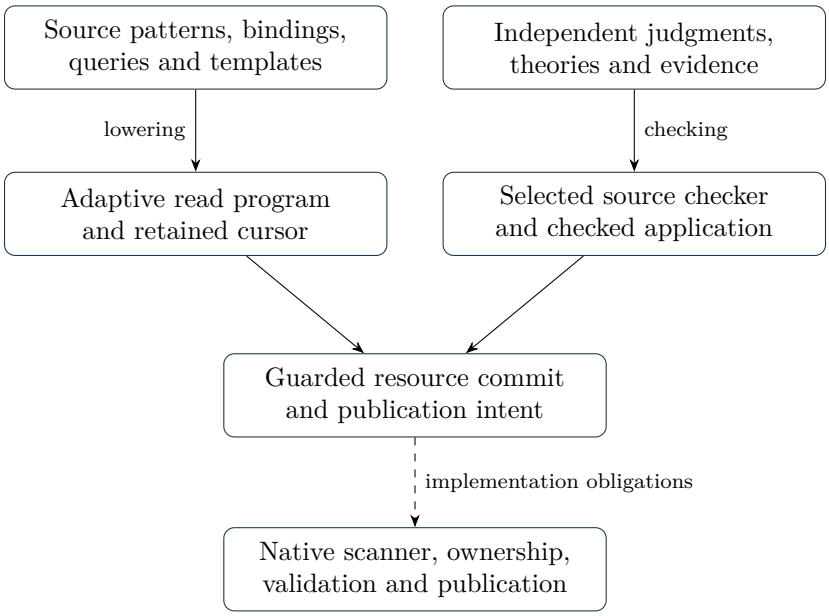
Coordinate	Declared contract
Judgment	What follows, in which context and theory: typing, equality, reachability, one answer, or a complete result observation.
Evidence	What is checked: local rule data, a tree, a shared graph, or a finite trace of licensed transitions.
Authority	How checked conclusions can be constructed and used; which profile and declaration snapshot authorize them.
Observation	What execution retains or exposes: verdict, pending work, cost, dependencies, or an optional exportable history.

An LCF interface protects theorem construction. It can also record proof terms or a journal. Certificate checking can mint the same logical kind of value without retaining the certificate. Neither choice supplies a constant checking cost, proves a native seal implementation, or recovers evidence that execution erased. A checked finite trace can be evidence when its transition rules establish the intended conclusion. A trace of an unrelated implementation is not automatically a derivation of that judgment.

The MeTTa equation kernel is sound and complete for its native reachability judgment. Full HE or PeTTa evaluation, resource consumption and mathematical multi-premise inference require their respective judgments. No single one of these observations defines every useful MeTTa kernel. The design of a MeTTa kernel integrating these judgments remains open; the following connection uses an existing mathematical calculus.

13.1 Source queries, retained execution and checked publication

The occurrence-based interaction model now connects its source grammar to the existing retained read machine. This connection concerns execution of queries and their resource footprints. A proof checker remains responsible for the logical conclusion of an inference request.



Theorem 13.1 (Source-query lowering)**.** *Suppose each region read returns exactly its finite occurrence family, in the scanner's chosen order. The constructed read program for a source list returns exactly the original compiler's row multiset after forgetting answer order. This covers reads, takes, uniqueness, aggregation, pure computed bindings, conjunction, nested absence and* `every`*. Matching alternatives, duplicate-labelled occurrences and computed-result positions are retained. The proof is by the source grammar, using the local scanner equation and the independent read interpreter; whole-program soundness is not a premise.*

Lean. `Sources.Query.scanList_complete`, `Sources.Query.run_step`, `Sources.Query.run_compile`

An `every` query returns one amalgamated row even when its body has no rows. Its enclosing absence is therefore false. With the established sound, complete and minimal matcher laws and theory-respecting pure computations, a completed empty answer is equivalent to there being no row in the independent declarative source relation. A scanner that omits an occurrence can fabricate emptiness and fails the explicit scanner premise.

Lean. `Sources.Query.run_empty_iff_no_declarative`,
`Sources.Query.Execution.completed_agreement`,
`Sources.Query.Controls.vacuous_rows_are_present`,
`Sources.Query.Controls.omitted_region_reply_is_not_complete`

Theorem 13.2 (Retained source execution)**.** *The existing cursor completes a compiled query after the number of reads on its selected finite path plus one return inspection. Splitting the budget preserves its entire saved packet and accumulated read charge. A paused packet exposes no completed answer. Successful validation and resumption against a new complete view recover that view's source rows; a changed performed read refuses resumption and retains the old packet. These are region-read counts, not bounds on matcher work or computed-binding evaluation.*

Lean. `Sources.Query.Execution.execute_complete`, `Sources.Query.Execution.execute_split`,
`Sources.Query.Execution.completed_absence`, `Sources.Query.Execution.resumed_rows`,
`Sources.Query.Controls.partial_query_is_not_absence`,
`Sources.Query.Controls.changed_prefix_retains_refused_packet`

The certificate records actual adaptive reads, including empty replies. Reordering equal-labelled occurrences preserves the row-bag observation but changes the ordered certificate. A new acknowledgement for an unread key can be observed after resumption, whereas changing the message that selected that key invalidates the continuation. The source absence constructor additionally retains its declared completeness regions, which can be stronger than the adaptive observations needed merely to establish an empty join.

Lean. `Sources.Query.validated_source_rows`,
`Sources.Query.Controls.adaptive_dependency_is_bound`,
`Sources.Query.Controls.absence_row_retains_full_footprint`,
`Sources.Query.Controls.bag_agreement_does_not_validate_reordering`,
`Sources.Query.Controls.resume_sees_unread_arrival`

Checked publication reads the selected source, consumes pending work and produces evidence only after the existing open-rule checker succeeds. Its validity theorem governs those steps from a valid initial store; arbitrary writes and native capability confinement require their own contracts. Inserting a refused evidence record violates the invariant. Adding another source object while retaining the old one does not revoke the old request: revision replacement must retire the earlier authority. A well-sorted record alone does not establish derivability.

Lean. `ProgrammableJudgmentTransforms.step_preserves_validity`,
`ProgrammableJudgmentTransforms.stored_evidence_derivation`,
`ProgrammableJudgmentTransforms.retaining_source_keeps_old_request_enabled`,
`ProgrammableJudgmentTransforms.refused_evidence_write_breaks_validity`,
`ProgrammableJudgmentServices.program_intents_preserve`

Conditional composition similarly keeps the computed guard distinct from the net update. Intermediate freshness can remain necessary when the final writes cancel. The proved mover condition licenses fusion of separated commits at the declared observation; unrestricted history equality does not follow. Both creation and removal windows have separating observers. The conditional-rewriting literature supplies the broader overlap and condition- transport programme[18, 19]; these concrete results do not yet enumerate symbolic occurrence-map overlaps or establish that general categorical programme.

Lean. `CommitPrecondition.sequential_holds_iff`, `Commit.composition_exact`,
`OccurrenceCommits.executable_fusion_iff`, `OccurrenceCommits.fusion_not_traceEq`

Status. The source-query lowering and retained-cursor comparisons are checked. The native interaction engine now retains absence and every as nested groups of jobs. Computed sources use its existing request/resume path; a reply handle identifies one request generation. Native controls exercise vacuous and nested aggregation, nonlinear matching, query-local bindings, changed dependencies, cancellation, stale replies, allocation failure and explicit post-effect failure. Both space programming and a client invoking the actual proof checker use that machinery. Complete scanner qualification, transitive dependencies of arbitrary computed code, view lifetime, full backend preparation, atomic revision validation and failure-history correspondence remain obligations. Shared execution does not identify Rho, equational reachability, HOL, MM0 or VibeITP judgments, nor construct new translations between them.

The existing correlated-reply model separately proves that a consumed or cancelled request identity stays absent through arbitrary later operations. Retargeting an old reply to the current request supplies a separating control: the value reaches the wrong continuation despite there being only one live request. The native generation check follows this contract; its memory and seal implementation retain their qualification obligations.

Lean. `Kernel.CorrelatedReplies.consumed_reply_stays_refused`,
`Kernel.CorrelatedReplies.cancelled_reply_stays_refused`,
`Kernel.CorrelatedReplies.Controls.stale_request_cannot_answer_next`,
`Kernel.CorrelatedReplies.Controls.replacing_supplied_identity_accepts_wrong_reply`

The selected native checkpoint passes 317 continuation and failure controls, including incomplete matching and refused waiter registration. Twenty mathematics programmes and twelve space-interaction fixtures agree across the current-source build, generated Cesta project and plain-C export. The three executables are byte-identical under the unchanged Cesta pin. This comparison supplies the interpreted library tree explicitly; the C export does not yet package those resources. Sanitizer qualification also identifies its SWI provider: a reviewed corrected build passes, while the installed provider retains a reproducible cleanup leak. Neither build agreement nor these finite executions constitute a whole-runtime refinement theorem.

13.2 An exact reason to require dependency

The need for dependent typing can be stated without treating every richer type theory as one system. Fix the direct Curry–Howard contract

$$A : \mathsf{Type}, \qquad P : A \to \mathsf{Type}, \qquad \prod_{x:A} P(x),$$

where the indices are the same values that programs consume, rather than codes copied into a second level. In the maintained annotated pure type system, this context and product are well formed if and only if both the ordinary function and value-to-type product rules are present. Neither rule is assumed in advance. This identifies a least formation basis for this interface, uniformly in the conversion bubble. Thus the dependent vertex admits the contract even in the weak conversion core, while the simple, polymorphic and type-operator vertices reject it under every conversion policy. The actual introduction and application rules retain the same value: given $f : \prod_{x:A} P(x)$ and $a : A$, application has type $P(a)$, with a substituted directly into the evidence type.

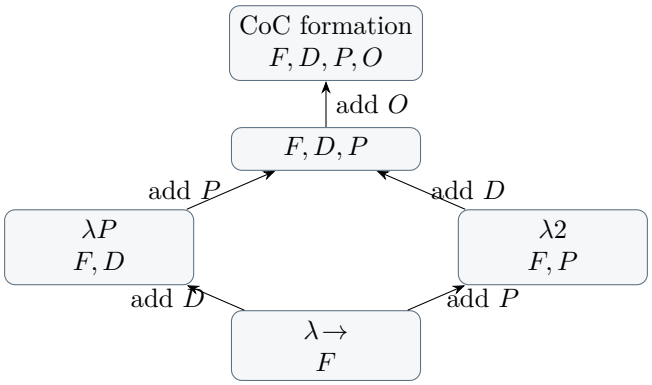
Lean. `HypercubeBubbles.directValueIndexedForall_iff_contains_dependentProducts`, `HypercubeBubbles.dependentProducts_least_directInterface`, `HypercubeBubbles.directForall_abstract`, `HypercubeBubbles.directForall_apply`, `HypercubeBubbles.directValueIndexedForall_absent_off_dependent_face`

A minimum relative to the requested interfaces. Write $F = (*, *, *)$ for ordinary functions, $D = (*, \Box, \Box)$ for value-indexed type families, $P = (\Box, *, *)$ for polymorphism, and $O = (\Box, \Box, \Box)$ for type operators. Each triple records the sort of the domain, codomain and resulting product. With the fixed sort axiom $* : \Box$, the checked formation requirements are:

Direct interface in the annotated syntax	Least product basis
$A : \mathsf{Type},\ P : A \to \mathsf{Type};\ \prod_{x:A} P(x)$	$\{F, D\}$
The preceding interface, $\prod_{X:\mathsf{Type}} X \to X$, and the kind $\mathsf{Type} \to \mathsf{Type}$	$\{F, D, P, O\}$

The second row is proved by constructing the three actual typing derivations and recovering every required rule from any such derivations. Removing any of the four rules defeats at least one requested interface, under every bubble. The polymorphic identity also has its actual program inhabitant.

Lean. `HypercubeBubbles.directFormationInterfaces_iff_contains_constructionProducts`, `HypercubeBubbles.constructionProducts_least_directInterfaces`, `HypercubeBubbles.directInterfaces_refused_of_missing_rule`, `HypercubeBubbles.polymorphicIdentity_typed`



This is one projection of the profile graph: it shows product formation with the conversion bubble held as a parameter. The four-rule endpoint is CoC's formation profile; ordinary CoC also fixes beta conversion. An arbitrary bubble at that endpoint need not be consistent or decidable. MLTT-like profiles additionally state their universes, dependent pairs, identity elimination and admitted

inductive rules. Those are further branches, not consequences of the two-rule dependency result. The cube is the classical formation framework [10]; the theorem here identifies the minimum for these particular annotated interfaces.

This is a necessity theorem for a named interface, not the claim that all logic or verification requires dependent types. HOL can quantify with logical constants, and singleton or two-level encodings can reify values before types refer to them. Those encodings change the interface: a separate route must relate a runtime value to its type-level representative. LF presents logics through signatures and judgments-as-types [11]; adequacy relates the specified object judgments to their LF evidence. The formation minimum above is not that adequacy theorem and does not imply that every encoding of higher-order logic requires a CoC host.

Which distinctions must a representation retain? The semantic results answer a complementary question, with the observation map fixed. They distinguish three contracts:

1. Literal descent through the canonical simple-family inclusion. Over an inhabited context, this holds exactly when the family is constant. A varying Boolean family and $n \mapsto \mathsf{Fin}(n)$ are outside that image.
2. Descent up to fibrewise equivalence. A factorization supplies a target family and an equivalence of each source fibre with the fibre over its observed index. Equivalent fibres preserve and reflect the existence of such a factorization. In particular, any number of the existing exact-code layers preserves this boundary. The positive constant-family and negative singleton/Boolean-family controls survive every such depth.
3. Inhabitation alone. Distinct data-bearing types can have the same inhabitation observations. Agreement here cannot justify transporting their values, equality fibres or retained proof histories.

Lean. `DependencyDescentCriterion.descends_iff_constant`, `DependencyDescentCriterion.varying_outside_image`, `DependencyDescentCriterion.finFamily_not_constant`

Lean. `DependentFamilyEquivalenceTransport.codeFamily_factorization_iff`, `DependentFamilyEquivalenceTransport.Canary.exact_representation_preserves_observer_boundary`, `DependencyDescentCriterion.readout_identifies_inhabited_types`

A stronger requirement has a sharp boundary: every source-anchored equality family can factor through an observer if and only if that observer is injective. Thus a lossy observation may support selected dependent families while failing to preserve source equality. This theorem concerns Lean's equality families, not an automatically faithful interpretation of every guest identity type. Exact coding preserves the family; it does not repair an observer that erased an index distinction needed by that family.

Lean. `EqualityFamilyObserverFactorization.EqualityFamilyFactorization.all_anchored_nonempty_iff_injective`

These semantic and syntactic criteria have different subjects. They are not a proved interpretation of every cube profile in a family model. Dependency and function extensionality are likewise separate: the existing simple and genuinely dependent beta function spaces each admit extensional and route-retaining carriers. HoTT path structure is another choice. A profile edge must therefore say whether it preserves formation, conversion, provability, data, operational observations or evidence authority; a result about one does not automatically fill the others.

Lean. `DependencyExtensionalityOrthogonality.simpleExtensional_applicationExtensional`, `DependencyExtensionalityOrthogonality.dependentExtensional_applicationExtensional`, `DependencyExtensionalityOrthogonality.simpleRouteSensitive_not_applicationExtensional`, `DependencyExtensionalityOrthogonality.dependentRouteSensitive_not_applicationExtensional`

The existing proof-level vine. The STTfa hub already connects a named object calculus to dependent evidence. It compiles the maintained HOL rules for implication, universal quantification

and core beta retyping into typed lambda-Pi-modulo terms. Normal inhabitants reflect closed implicational proofs. Reflection for core universal formulas, including free objects, additionally requires that quantified witnesses are coded translations of source terms. These are independently stated typing, normality and coding premises, not the formation requirement above.

Lean. `GSLT.Dedukti.HOLHub.sttfa_adequate`, `GSLT.Dedukti.HOLHub.sttfa_adequate_coded`, `GSLT.Dedukti.HOLHub.sttfa_adequate_coded_open`

Detour-free source proofs with core witnesses have a computed normal form; for core conclusions it decodes back to a proof of that conclusion. Every normal form of such a compiled proof equals the computed one and retains the coding condition. The checked comparison keeps the detour-free premise. It does not supply normalization for every typed hub term or reconstruct the original proof history.

Lean. `GSLT.Dedukti.HOLHub.sttfa_round_trip`, `GSLT.Dedukti.HOLHub.compile_normalForm_coded`

The negative example is useful: an unrestricted hub proof can use the bare connective `and` as a universal witness even though it is not the translation of any source HOL term. Its conclusion still has a HOL proof with another witness. Thus failure of exact evidence reconstruction is not itself failure of theorem conservativity. This existing hub uses its own declarations and rewrite theory; connecting it to every chosen formation profile, a total qualified normalizer or the native kernels requires the corresponding additional laws.

Lean. `GSLT.Dedukti.HOLHub.sttfa_not_exactly_adequate`, `GSLT.Dedukti.HOLHub.sttfa_coding_needed`

The hub at its exact formation vertex. The STTfa proof service does not use the four-rule CoC formation profile. Its declared `LFProfile.standardLF` is exactly the standard LF/λP two-rule dependent vertex $\{F, D\}$. For every profile retaining $* : \Box$, extension from that vertex is equivalent to supporting the direct interface $A : \mathsf{Type}, P : A \to \mathsf{Type}, \prod_{x:A} P(x)$, uniformly in the separately selected bubble. Once the family is available, forming the final product is indeed the short ordinary-function step. The substantive boundary is that forming the family itself requires D, and conversion cannot supply a missing formation rule.

Lean. `HypercubeHOLHub.standardLF_eq_dependentVertex`, `HypercubeHOLHub.standardLF_subsumed_iff_directInterface`

This correspondence reaches the actual hub rather than stopping at profile names. Its declarations are formed at $\{F, D\}$, its transparent-body obligation holds, every rewrite instance preserves its input type, and reduction preserves typing in formed contexts. The compiled universal- elimination control is typed there. At the simple vertex $\{F\}$ the actual decoder declaration $\eta : \mathsf{type} \to \mathsf{Type}$ cannot be formed, under any conversion theory; consequently the complete STTfa signature is not formed there.

Lean. `HypercubeHOLHub.formedSignature_at_dependentVertex`, `HypercubeHOLHub.rulesPreserveTypes_at_dependentVertex`, `HypercubeHOLHub.typePreservation_at_dependentVertex`, `HypercubeHOLHub.predInstance_typed_at_dependentVertex`, `HypercubeHOLHub.signature_not_formed_at_simpleVertex`

For any larger profile supporting the direct interface, source HOL terms stay typed in the pointwise decoded source context, and every proof of the selected implication/universal calculus compiles to typed hub evidence. The existing capture-avoiding source-substitution equation is joined to that profiled typing statement: translated substitution and LF substitution agree under the stated image premise, and the result remains typed in the translated target context. This supplies a formation/context/substitution/evidence edge. It does not change the hub's rewrite theory, add eta conversion, prove normalization of all typed terms, or make every dependent profile a model of HOL.

Lean. `HypercubeHOLHub.translate_typed_of_directInterface`,

`HypercubeHOLHub.translate_substitute_typed_of_directInterface`,
`HypercubeHOLHub.theorem_typed_of_directInterface`

CoC and a historical prototype diverge. The four-rule CoC formation vertex and the historical MettaKernel kind-result prototype are not the same top. Besides F, D, O, CoC contains

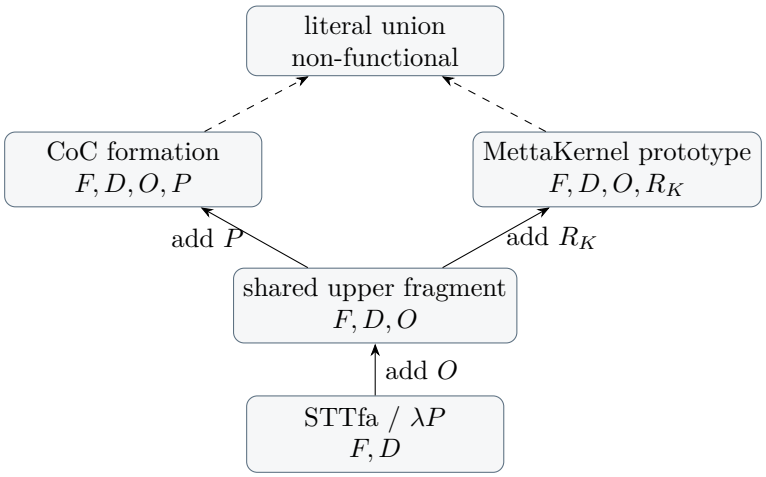
$$P = (\Box, *, *),$$

while the prototype contains the non-λ-cube rule

$$R_K = (\Box, *, \Box).$$

Thus their exact common product set is $\{F, D, O\}$. The CoC branch is the relative minimum for the three direct interfaces above. When the 31 frozen DTTBench statements are inferred at the prototype, their computed derivations exercise all four prototype rules; any profile containing that particular demand record subsumes the prototype. This is not a profile-independent minimality result. The generic, sound checker types all 31 raw statements at sort $*$ under CoC as well, without R_K: CoC assigns different result sorts to intermediate products. Standard LF remains too small. Hence the prototype-computed demand distinguishes inference profiles, while corpus typability does not distinguish these two upper branches.

Lean. `LFDTTBenchDemand.allWithinMettaKernelPrototype_eq_true`,
`LFDTTBenchDemand.allWithinStandardLF_eq_false`,
`LFDTTBenchDemand.allDemandRules_eq_mettaKernelPrototype`,
`LFDTTBenchDemand.mettaKernelPrototype_minimal_for_DTTBench`,
`LFDTTBenchDemand.allStatementsTypableInCoc_eq_true`, `LFDTTBenchDemand.coc_typable_of_mem`,
`LFDTTBenchDemand.prototype_demand_not_necessary_for_coc_typability`,
`HypercubeHOLHub.construction_mettaKernelPrototype_common_product_iff`,
`HypercubeHOLHub.coc_mettaKernelPrototype_fork`



The dashed union is a warning, not an admitted profile. It contains two rules with the same domain and codomain sorts but different result sorts, and is therefore not functional. The maintained uniqueness-of-types theorem uses functionality. Non-functionality alone is not inconsistency, but it means that blindly collecting both petals loses that metatheoretic route. A useful common extension must resolve or stratify the result-sort choice, or prove another adequate checking theory.

Lean. `HypercubeHOLHub.constructionMettaKernelPrototypeUnion_not_functional`,
`HypercubeHOLHub.mettaKernelPrototype_does_not_support_directFormationInterfaces`,
`HypercubeHOLHub.constructions_do_not_contain_DTTBench`

A coarser semantic readout can nevertheless forget the syntactic conflict. For this literal union, observed result sorts are functional exactly when the readout identifies Type and Kind. A constant

one-point readout does so; every injective readout refuses. This supplies a common semantic shadow, not a common faithful calculus or a model of typing and conversion. Adding $\mathsf{Type} : \mathsf{Type}$ is a different syntactic construction, and the maintained PTS development proves that profile inconsistent.

Lean. `HypercubeProfileStratification.constructionMettaKernelPrototypeUnion_resultFunctionalUnder_iff`, `HypercubeProfileStratification.constant_observation_reconciles`, `HypercubeProfileStratification.injective_observation_does_not_reconcile`, `TypeInTypeParadox.typeInType_inconsistent`

This is an obstruction to every untagged functional upper profile over the same two sorts, not merely to the displayed list union: any upper profile containing both branches inherits the conflicting rules. The checked common service therefore retains a branch mode. Each fibre is functional and has the exact profile checker. Evidence from the shared F, D, O fragment rechecks in both fibres. The selected direct interfaces separate one direction; the prototype-computed DTTBench demand record separates the other, although the raw statements themselves type in both upper profiles. This mode-stratified service is a common dispatch interface, not a merged calculus. Removing the tag requires resolving the result sort, extending the sort syntax or proving a replacement for the present uniqueness metatheory.

Lean. `HypercubeProfileStratification.no_functional_common_upper`, `HypercubeProfileStratification.branchProfile_functional`, `HypercubeProfileStratification.exists_checkProof_iff_deriv`, `HypercubeProfileStratification.shared_evidence_rechecks_both`, `HypercubeProfileStratification.construction_interface_separates_branches`, `HypercubeProfileStratification.prototype_computedDemand_separates_profiles`

The arbitrary-sort PTS generalization sharpens "extending the sort syntax." A sort-renaming morphism preserves axioms and product rules and transports annotated terms, beta conversion, contexts and complete typing derivations. The existing two-sort calculus is equivalent to its generic instance. Suppose the CoC and prototype maps into one functional target agree on their shared $*$ and $\Box$. Their conflicting $(\Box, *)$ rules force the two target images to coincide. The image of $* : \Box$ is then $u : u$, and the image of $(*, *, *)$ supplies the product rule used by the maintained Hurkens term. Transporting that concrete derivation inhabits $\Pi p : u.\, p$ in the target. This proves an inconsistency witness for every functional commuting cocone, rather than merely observing a non-functional two-sort union.

Lean. `GenericPureTypeSystem.Derives.map`, `GenericPureTypeSystem.TwoSort.derives_iff`, `HypercubeSortRenaming.commuting_functional_cocone_collapses`, `HypercubeSortRenaming.commuting_cocone_self_axiom`, `HypercubeSortRenaming.commuting_cocone_self_product`, `HypercubeSortRenaming.commuting_functional_cocone_inconsistent`

There is also a functional three-stratum comparison target. It sends CoC's $*, \Box$ to $\mathsf{Prop}, \mathsf{Type}_0$ and sends the prototype's $*, \Box$ to $\mathsf{Type}_0, \mathsf{Type}_1$. The seven image rules are single-valued, the displayed CoC polymorphic identity and a prototype type operator transport, and none of the three strata types itself. The maps disagree on both source sorts. Consequently this is a noncommuting span for comparison and reuse, not a join of the shared source calculus. It is not named ECC or CIC: cumulativity, a model, normalization and decidable checking remain independent obligations. The branch-tagged service above therefore remains the faithful common interface for the original syntax.

Lean. `HypercubeSortRenaming.comparisonProfile_functional`, `HypercubeSortRenaming.comparisonProfile_no_self_axiom`, `HypercubeSortRenaming.comparison_maps_disagree_on_type`, `HypercubeSortRenaming.comparison_maps_disagree_on_kind`, `HypercubeSortRenaming.coc_polymorphicIdentity_transports`, `HypercubeSortRenaming.prototype_typeOperator_transports`

The formation part of this policy is executable. A finite checker compares every pair of product rules with the same premise sorts and accepts exactly when their result sorts agree. Joining erases duplicate rules and refuses an otherwise non-functional union. Dependency plus type operators passes; CoC plus the MettaKernel prototype refuses. Every admitted join contains both input

profiles, and acceptance of a finite proof tree in either input yields finite accepted evidence in the joined profile through the existing exact profile-parametric checker.

Lean. `HypercubeHOLHub.functionalProductsCheck_eq_true_iff`, `HypercubeHOLHub.functionalProducts_iff_profileFunctional`, `HypercubeHOLHub.admitProductJoin_eq_some_iff`, `HypercubeHOLHub.admitProductJoin_eq_none_iff`, `HypercubeHOLHub.dependent_operator_join_accepted`, `HypercubeHOLHub.coc_mettaKernelPrototype_join_refused`, `HypercubeHOLHub.admitted_join_rechecks_left`, `HypercubeHOLHub.admitted_join_rechecks_right`

The corresponding MeTTa-data boundary is also checked. A tagged atom wire records both sort axioms and every ordered product rule. Decoding is a left inverse of encoding, hence the representation is injective. The atom-level join service decodes both submissions, requires identical sort axioms, then runs the formation gate. Every published result contains both input profiles and is functional. Dependency plus type operators is accepted; a conflicting sort axiom, the nonfunctional CoC/prototype union and malformed atom data are all refused. This is a source contract for a runtime consumer; it does not claim that a current executable consumes the wire or preserves its cost.

Lean. `LFProfileAtomWire.decode_encode`, `LFProfileAtomWire.encode_injective`, `LFProfileAtomWire.sameSortAxiomsCheck_eq_true_iff`, `LFProfileAtomWire.admitProfileJoin_sound`, `LFProfileAtomWire.joinEncoded_sound`, `LFProfileAtomWire.dependent_operator_join_encoded`, `LFProfileAtomWire.coc_mettaKernelPrototype_join_encoded_refused`, `LFProfileAtomWire.conflicting_sort_axiom_join_refused`, `LFProfileAtomWire.malformed_profile_refused`

This is an admission rule for formation profiles, not conversion packages. The join of two bubbles is their least upper bound as a conversion relation, and typing from each side embeds in it. Decidability and normalization of that join require the separate completion, commutation or disjointness laws already stated by the bubble theory. A runtime profile service should therefore keep formation admission and conversion-package admission as two checks.

Lean. `BubbleConversion.convertible_join_least`, `BubbleConversion.hasTypeIn_join_left`, `BubbleConversion.hasTypeIn_join_right`, `BubbleConversion.join_normalizedBy`

The conversion side has a parallel finite wire for the existing typed HOL recipe policy. A manifest selects beta and eta globally or at named simple types, and selects all or individual licensed equations. Its recursive type codec is parameterized by a lossless codec for base-type names. Decoding is a left inverse; malformed policies and out-of-range equation indices refuse. Finite manifest union denotes exactly the existing pointwise policy join, so both input local-step judgments embed in it. This gives atom-level permission data for the HOL recipe checker. It does not yet encode a dependent-calculus conversion package or establish normalization, confluence, decidability, conservativity or cost for the joined policy.

Lean. `HOLConversionPolicyAtomWire.enabled_join`, `HOLConversionPolicyAtomWire.denote_join`, `HOLConversionPolicyAtomWire.denote_extends_join_left`, `HOLConversionPolicyAtomWire.denote_extends_join_right`, `HOLConversionPolicyAtomWire.decodeType_encodeType`, `HOLConversionPolicyAtomWire.decodeFeature_encodeFeature`, `HOLConversionPolicyAtomWire.decode_encode`, `HOLConversionPolicyAtomWire.encode_injective`, `HOLConversionPolicyAtomWire.joinEncoded_sound`, `HOLConversionPolicyAtomWire.beta_eta_manifest_join`, `HOLConversionPolicyAtomWire.out_of_range_equation_refused`, `HOLConversionPolicyAtomWire.malformed_policy_refused`

The reified-index branch has its own exact price. The alternative to direct indexing can be specified rather than merely named. A beta-correct quote/splice interface gives a split readout from retained code to its body. A code-indexed family descends to that body view exactly when its fibre at every code is equivalent to its fibre at the canonical quote-after-splice code. Ordinary

source equality descends exactly when the readout is faithful, equivalently when quote after splice satisfies eta. Thus beta supplies a body view; eta is the additional condition for forgetting code identity.

Lean.
`ContextualCodeObserverFactorization.familyFactorization_iff_quoteSpliceFibreEquivalences`,
`ContextualCodeObserverFactorization.equalityFamilyFactorization_iff_quoteSplice`

The tagged-code control is the negative half. A body-constant family descends, while a family depending on a retained Boolean code tag does not; source equality does not descend either. A Tarski-style encoding has another layer: factorization of its code carrier does not imply factorization of decoding. The maintained example has one code at each of two identified levels, but that code decodes to a singleton at one level and Booleans at the other. Code syntax factors and decoding cannot. Consequently a reified route owes value/code coherence and code/meaning coherence separately. This route can avoid the direct syntactic premise only by changing the interface; it does not derive the missing formation rule.

Lean. `ContextualCodeObserverFactorization.Canary.contextual_code_observer_boundary`,
`TarskiCodeFamilyObserverFactorization.Canary.code_syntax_does_not_determine_decoding_factorization`,
`TarskiCodeFamilyObserverFactorization.Canary.constant_decoding_pi_sigma_control`

The formation branch is now concrete in the same annotated PTS as the direct necessity theorem. Direct indexing forms ($\mathrm{P\,x}$) from ordinary functions and the value-to-type rule. Type reification instead forms

$$Q : \mathrm{Type} \to \mathrm{Type} \;\vdash\; \Pi X : \mathrm{Type}.\, \Pi x : X.\, Q\, X$$

from ordinary functions, polymorphism and type operators. These rule sets are incomparable: each supports its displayed interface and refuses the other, independently of conversion. Both universal types are accepted by the sound reference checker at structural fuel five. The direct application is accepted at fuel five; the reified application first consumes a type code and then its witness, and is accepted first at fuel six. In the concrete controls its term has five syntax nodes rather than three. The direct result type contains the runtime argument; the reified result contains the type code and omits the witness. Thus witness substitution is replaced by code transport plus the separate value/code and decoding-coherence obligations above. These are exact measurements of this syntax and checker, not asymptotic complexity claims.

Lean. `ReifiedIndexElaboration.productRule_mem_of_typed_subject`,
`ReifiedIndexElaboration.reifiedTypeIndexedForall_iff`,
`ReifiedIndexElaboration.direct_reified_formation_fork`,
`ReifiedIndexElaboration.directApplicationContext_formed`,
`ReifiedIndexElaboration.reifiedApplicationContext_formed`,
`ReifiedIndexElaboration.Controls.directApplication_accepts_fuel_five`,
`ReifiedIndexElaboration.Controls.reifiedApplication_refuses_fuel_five`,
`ReifiedIndexElaboration.Controls.reifiedApplication_accepts_fuel_six`,
`ReifiedIndexElaboration.Controls.application_node_counts`,
`ReifiedIndexElaboration.Controls.result_transport_surface`,
`ReifiedIndexElaboration.Controls.formation_with_missing_decoding_coherence`

Here "quotation" and "splicing" name the abstract contextual-code operations of the modal theorem. They do not redefine the candidate runtime's surface forms. Its `quote` remains an evaluation-blocking constructor whose interior variables remain available to structural matching. The distinct `@` form retains its name-making role; matching below those names crosses the existing `lift` boundary. Relating either runtime form to the modal interface needs an adapter proving the relevant binding, beta and eta laws.

13.3 Finite dependent evidence through the common interfaces

The selected profile is the existing cumulative tower over natural-number level expressions. Its syntax has dependent functions and pairs, identity formation and reflexivity, explicit universe heads, and cumulativity. Its formation-sensitive typing relation has fourteen rules. Constants and root computations are parameterized in the general construction; this particular tower declares neither. Structural beta computation remains available. Only this declared mathematical profile is instantiated here.

Local codes contain syntax and rule selections. In the conversion rule they contain an existing finite structural conversion code. Decoding checks the local universe, declaration and head conditions and checks that conversion code; it does not receive a proof of conversion as executable input. The decoded node is an instance of the independently defined rule signature. The common tree and DAG replay algorithms therefore preserve the original premise positions and dependent substitutions. The maintained structural replay checker also publishes common checked judgments directly, and its existing telescope certificates check the ambient context.

Theorem 13.3 (Exact finite authority for the selected judgment)**.** *For the selected tower, a formation-sensitive judgment in a formed telescope holds if and only if there is an accepted finite request. Such a request checks its context, every graph row, the root address and its actual conclusion. Raw typing derivability is equivalently accepted tree evidence, accepted graph evidence, or a minted common checked judgment.*

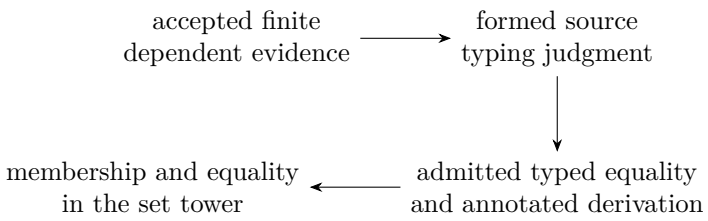
Lean. `FormationSensitiveChecking.typing_iff_checked_tree`, `FormationSensitiveChecking.typing_iff_checked_graph`, `FormationSensitiveChecking.typing_iff_minted`, `FormationSensitiveChecking.mintReplay_iff`, `FormationSensitiveChecking.judgment_iff_checked`, `Tower.Checking.coverage`

Exactness is existence of accepted evidence, not a terminating procedure for finding it. This is a connection of scoped Lean data and checked algorithms, not a portable serialization theorem or a refinement of CeTTa's C implementation. The source formation-sensitive conversion is structural conversion; the comparison into the stronger typed equality does not make every typed eta identification a source conversion.

A raw variable rule can hold in an unformed telescope. Such a raw certificate must not authorize a formed request. The controls also reject changed root claims, missing roots, forward or cyclic dependencies, invalid universe lifts, and invalid unused suffixes. Shared references retain all premise occurrences; they do not duplicate consumable execution authority.

13.4 The comparison to sets

The diagram keeps the source judgment between evidence and meaning:



Theorem 13.4 (Checked evidence has its declared set meaning)**.** *Under the explicit hypothesis* `Cofinal Inaccessibles`*, every accepted request yields an annotated derivation whose telescope, subject and displayed type erase to that request and whose statement holds in the existing set tower. Checked subject values belong to their displayed type values. Finite checked conversion between independently admitted endpoints preserves their actual model values.*

Lean. `TowerInterpretation.Checking.request_model_sound`,
`TowerInterpretation.Checking.checked_value_member`,
`TowerInterpretation.Checking.checked_type_conversion`,
`TowerInterpretation.Checking.checked_term_conversion`

For closed universe-membership queries the comparison also reflects the model: a finite request for $U_c : U_d$ exists exactly when the corresponding set universe is a member of the other. The same equivalence holds after the existing well-founded embedding into material hypersets. This is a scoped reflection result for universes, not completeness for arbitrary set truth. Under the named model hypothesis, every request for a closed inhabitant of the existing empty type is rejected.

Lean. `TowerInterpretation.Checking.universe_request_iff_set`,
`TowerInterpretation.Checking.universe_request_iff_hyperset`,
`TowerInterpretation.Checking.empty_request_refuses`

Substitution uses the actual checked component terms. Their accepted requests construct the source contextual arrow. At the semantic comparison boundary, the annotated components carry their typing and erasure witnesses; evaluation of those components constructs the satisfying-environment map. The value of a substituted checked term equals its original value in that substituted environment. Context and component compatibility are retained in the statement, rather than being replaced by an assumed semantic action.

Lean. `FormationSensitiveChecking.checked_substitute`,
`TowerInterpretation.Checking.checkedArrow`,
`TowerInterpretation.Checking.checked_value_substitution`

The model map has several distinct scopes. The cumulative tower has the set interpretation just used, with its named inaccessible-cardinal hypothesis. The HOTG-style mathematical packages add declared set operations and theory assumptions; they are not automatically translations to the native Megalodon Mathdata checker. The material hyperset family construction takes an explicit presentation and supplies contexts, dependent families, substitution and Pi/Sigma structure. Its extensional identity does not establish a faithful interpretation of arbitrary higher paths, cubical operations or every dependent profile.

A useful negative boundary is the existing HOTG-style `setTheory_not_admitted`: a package can have a set model while failing its stated computational admission discipline. The rule-constant alternative has separate preservation results. Semantic equality therefore cannot be promoted to checker conversion merely because both expressions denote the same set. Conversely, a model interpretation need not reflect all set truth back into source derivations. These comparisons preserve their assumptions; they assert neither equivalence of all foundations nor optimal native speed.

Native guarded transport. Two typed HOL schemas now transport universal and existential observations between guarded representations. Their explicit forward and backward maps preserve validity and are inverse on valid values. The observed predicate is pulled back along the chosen forward map; comparison action is retained. Both formulas hold in every Henkin model, with no full-domain or choice premise, and their Lean proofs use no host axioms. An inhabited representation of an empty carrier refutes the actual intrinsic formula obtained by dropping the target guard. A Boolean swap supplies a valid nonidentity comparison, whose action changes a selected observation.

Lean. `HOL.GuardedQuantifierTransport.models_universal`,
`HOL.GuardedQuantifierTransport.models_existential`,
`HOL.GuardedQuantifierTransport.guard_erasure_counterexample`,
`HOL.GuardedQuantifierTransport.Examples.swap_comparison`,
`HOL.GuardedQuantifierTransport.Examples.swap_changes_observation`

Independent original HOL Light proofs establish the same exact statements. The explicit-rule producer uses generalization, specialization, discharge, conjunction, equality and existential introduction/elimination. Its exported closure retains five checked logical definitions and has no AXIOM nodes; the earlier proof-search route used eta and choice. Native replay under the existing sealed

policy, which prohibits axiom introduction, checks the two exact sequents and rejects changed claims, additional axioms and a truncated stream. Packet checks also refuse a missing or repeated root and the typed guard-erasure claim. The maps' comparison certificates remain requirements of clients. This library is an interoperability and representation edge, not automatic datatype proof generation or a proof of the linked native checker.

On the same pinned checker and standard policy, eight interleaved paired fresh-process runs checked the identical two sequents. The proof-search artifact used 36,205 records and 202,056 bytes; the explicit-rule artifact used 3,023 records and 16,382 bytes. Median process time was 6.51 ms versus 2.15 ms, and median peak RSS was 9,724 versus 8,216 KiB. These are measured costs of these two proof artifacts, not an optimality theorem or a bound for other workloads. The explicit proof also qualifies under the separate sealed policy.

The exact semantic boundary for truth observations is weaker than an inverse comparison. Over the observation class containing every predicate, preserving all existential truths is equivalent to the image of valid source values being exactly the valid target values. The same equivalence holds for all universal truths. A projection from a pair of Booleans to its first component satisfies both observation laws while identifying distinct source values. Truth agreement therefore does not establish faithful transport of values, witnesses or multiplicities. The native library retains its explicit inverse-comparison hypothesis; this semantic criterion does not change its admission contract.

Lean. `HOL.GuardedQuantifierTransport.exists_observations_iff_image`, `HOL.GuardedQuantifierTransport.all_observations_iff_image`, `HOL.GuardedQuantifierTransport.truth_transport_does_not_force_injectivity`

13.5 Operational logical form and relative checking bases

The choice of a low-tier checker can follow from a richer dependent theory and the fragment requested of it. There are at least three constructions: restrict declarations and computation rules; elaborate implicit conversion into explicit evidence; or specialize a checking algorithm for a declared fragment. These give different proof formats and cost profiles. A literal syntax intersection does not decide which construction is best.

Dependent carve-outs additionally retain formed contexts, binding, substitution, universe policies and the selected identity rules. Their operational predicate comparison does not discharge these dependent laws or admit an inductive schema. A fibration organizes the obligations; an executable extraction and an optimality theorem require further evidence.

For the existing dependent calculus, every accepted formed request has a finite sufficient restriction of its declaration and root-computation package. The same context, subject and displayed type are derivable there, and every judgment of that restriction reflects to the original package. This is a constructed restriction, not a theorem that a smallest checker exists. The universe policy is retained. Computational admission and an executable root decoder for the restriction are additional obligations: pruning may remove formation or preservation support required by a chosen admission discipline.

Lean. `FormationSensitiveCheckingOSLF.checked_request_has_restriction`, `FormationSensitive.Dependencies.exists_restriction_with_reflection`, `FormationSensitive.RestrictionPreservation.oslf_observation_does_not_admit_restriction`, `FormationSensitive.RestrictionPreservation.oslf_observation_with_preserving_repair`

OSLF applies to the declared dynamics. In the conversion-evidence observation, vertices are the original scoped terms and edges are original structural steps with either orientation. The observation theory retains syntax equality as its equation policy, so conversion remains visible as finite paths. Its generated reachability native type holds exactly when the independently authored conversion holds, equivalently when some supplied finite code checks. Reverse beta is therefore a valid conversion edge; this does not declare it an evaluator reduction. A raw beta path also does not establish formed typing.

Lean. `FormationSensitiveCheckingOSLF.path_iff_conversion`, `FormationSensitiveCheckingOSLF.satisfies_iff_checked`, `Tower.Checking.Controls.reverse_beta_oslf`, `Tower.Checking.Controls.raw_beta_oslf`

Native type theory obtains richer structure through the internal languages of presheaf toposes [6]. The comparison here concerns the declared relational OSLF predicate frame and its operational routes. It does not establish that entire presheaf construction for every dependent profile.

In the general OSLF construction, semantic predicates are invariant under the selected equations. Closure under computation is a further property. The existing route interfaces yield the following separate results:

Route contract	Earned observation law
Equation-respecting map	Reindex invariant predicates. Pointwise agreement modulo target equations gives equal predicate actions.
Strict forward operational translation	Preserve existential one-step observations; additional target behavior may remain.
Locally covered translation	Also reflect those observations and finite reachability from image states.
Path-valued operational realization	Preserve finite reachability while permitting a source step to become zero or several target steps.

Lean. `EquationRespectingMap.pullback_eq_of_equiv`, `OperationalTranslation.preserves_semanticDiamond`, `CoveredTranslation.semanticDiamond_iff`, `CoveredTranslation.semanticReach_iff`, `OperationalRealization.preserves_semanticReach`

The extra-target-step control refutes unrestricted reflection. The fusion control preserves a path observation while destroying a one-step diamond. OSLF's right-adjoint box in this construction observes incoming steps; its transport cannot be inferred from outgoing coverage alone. Bounded-cost observations additionally require a declared charge and a bound on mapped paths. Predicate transport does not by itself establish those bounds, proof translation, native seal enforcement, or equality of retained proof histories. The basic behavioral GSLT morphism only promises bisimilarity preservation.

Lean. `IndexedOperational.Canary.escaping_native_type_not_reflected`, `IndexedOperational.Canary.outgoing_coverage_does_not_preserve_past_box`, `OperationalRealization.FusionCanary.fusion_changes_one_step_diamond`

HOL Light, MM0 and Megalodon consequently remain comparison points and native realizations with explicit source contracts. They do not exhaust the possible checking bases for dependent profiles. A semantic set model is another comparison edge, with its assumptions retained; it is not an executable proof translation or a reflection theorem for arbitrary set truth. Relative optimality needs the fragment, certificate representation, operations and cost model to be fixed before it can be asserted.

For the selected cumulative profile, the guarded set-model comparison also receives this OSLF conversion observation directly. Two accepted term-typing requests in a formed telescope give equal actual set values when the conversion-native observation holds. The annotated context and the named model hypothesis remain explicit.

Lean. `TowerInterpretation.Checking.oslf_term_conversion`

13.6 A computational STT carve-out, with a proved boundary

The existing one-base simply typed lambda calculus supplies a first exact comparison. Write $S(\Gamma, A)$ for its intrinsically typed terms, E for its embedding in the root-free cumulative Pi/Sigma/identity tower, and H_b for its HOL interpretation at a chosen base sort b. Simple arrows become nondependent products in E and ordinary arrows in H_b. The chosen base sort remains distinct from HOL's

proposition sort. For every context and simple type:

$$\begin{aligned}\mathrm{Join}_\beta(H_b(s), H_b(t)) &\iff s \equiv_\beta t \\ &\iff \mathrm{Conv}_{\mathrm{struct}}(E(s), E(t)) \\ &\iff \mathrm{decideConversion}(s,t) = \mathsf{true}.\end{aligned}$$

The tower conversion path may leave the simple image. Reflection follows from normalization and confluence, not an assumption that all intermediate terms remain simple. HOL uses its independently defined operational beta steps. Normal intrinsic terms remain normal there, so common reducts determine the same intrinsic normal form.

Lean. `SingleBaseSTLC.HOLComparison.holJoinable_iff_betaConv`, `SingleBaseSTLC.HOLComparison.holJoinable_iff_towerConv`, `SingleBaseSTLC.HOLComparison.decideConversion_iff_holJoinable`

The scope is one base sort, variables, abstractions and applications, with no declared constants. This compares a computational judgment, not all HOL theoremhood or arbitrary dependent inhabitants. It is a reference component for a possible native kernel, not a completed HOL bubble or a selection among the dependent language candidates.

The maps commute with simultaneous substitution and give a functor between the existing context categories. An indexed family alone is not a fibration: chosen cartesian lifts and their laws require additional constructions. Typed HOL syntax retains intermediate application types that the unannotated tower erasure can lose, even at a fixed result type. Conversion classes remain faithfully compared. Raw syntax recovery and conversion reflection are therefore different requirements on a shared representation.

Lean. `SingleBaseSTLC.HOLComparison.contextFunctor`, `SingleBaseSTLC.HOLComparison.substitute_term`, `SingleBaseSTLC.HOLComparison.Controls.hol_retains_erased_types`, `SingleBaseSTLC.ConversionDecision.betaClass_toTower_injective`

Two local policies. The common bubble interface now has a simple-term instance. Beta steps retain HOL operational evidence; a second policy additionally permits eta, with application and abstraction congruence in both policies. Free conversion trees retain their composition and intermediate states. Each tree elaborates constructively to an ordinary retained HOL equality proof with the same endpoints. The beta-only policy presents exactly the structural tower conversion above.

Lean. `SingleBaseSTLC.HOLConversionBubble.elaborate`, `SingleBaseSTLC.HOLConversionBubble.beta_path_iff_tower`

For an arrow-typed variable f, the terms f and $\lambda x.fx$ are beta-normal and not beta-convertible, although the eta policy relates them and HOL proves their equality. The tower's *typed-equality* judgment also contains eta; its *structural conversion* in the comparison does not. Even two judgments over one package cannot be interchanged silently. An equality theorem does not automatically license definitional computation. Arbitrary additional rewrite packages would need fresh admission obligations.

Lean. `SingleBaseSTLC.HOLConversionBubble.Controls.eta_strictly_adds_conversion`, `SingleBaseSTLC.HOLComparison.Controls.hol_equality_does_not_reflect_conversion`, `SingleBaseSTLC.HOLTowerModel.typed_equality_not_structural_conversion`

Operational predicates, dependent readings and models. This conversion observation and the earlier dependent checking development reuse one general construction for the equivalence closure of an authored relation. Equations of the observation theory are syntax equality; bidirectional computation edges remain visible. Its OSLF reachability predicate has exactly the support of retained conversion trees. On the beta fragment the existing total intrinsic normal-form comparison decides it. Predicate truth does not reconstruct a particular history.

Lean. `IndexedOperational.conversionTheory_path_iff`,
`IndexedOperational.satisfies_conversionNativeType_iff`,
`SingleBaseSTLC.HOLConversionBubble.satisfies_iff_path`,
`SingleBaseSTLC.HOLConversionBubble.beta_satisfies_iff_decides`

Every lawful dependent `HOLReading` reads these pure simple terms without interpreting additional constants or supplying an equality-proof algebra. Their computed codes are typed in that reading's actual selected judgment. The maintained native substitution constructor has no unread image on this fragment, and code commutes with substitution. The beta observation thus gives typed equality of the computed codes. This is a parametric inclusion result with explicit reading laws, not reflection from arbitrary dependent proofs.

Lean. `SingleBaseSTLC.HOLReadingComparison.code_typed`,
`SingleBaseSTLC.HOLReadingComparison.code_substitute`,
`SingleBaseSTLC.HOLReadingComparison.observed_typed`

For the tower, the independently formed simple context and admitted equality reach its maintained annotated set interpretation under the named `CofinalInaccessibles` hypothesis. That hypothesis belongs to the semantic edge, not the beta decision procedure. The earlier well-founded embedding into material hypersets retains its stated scope; neither result supplies arbitrary higher identities or a translation to native Mathdata.

Lean. `SingleBaseSTLC.HOLTowerModel.observed_set_equality`

From a component to a whole HOL bubble. Choice, infinity and classical principles belong to a declared theory and its interpretations; conversion policy belongs to the judgment. Option names are not independence proofs for their combinations. The existing HOL calculus, retained derivations and theory interfaces supply these logical profiles. Merely naming another kernel does not require copying their rules.

A complete proof-level carve-out needs a rule-preserving interpretation into the selected dependent judgment and a genuine reflection or conservativity argument for the declared image. Type-shape membership supplies neither. Routes to native HOL Light, Megalodon and configured MM0 must retain their own assumptions, definitions, equality policies and evidence. This comparison establishes no new complete three-way translation.

A suitable tier-zero realization can specialize a richer checker, or expose its implicit computation as explicit evidence. It need not be a literal subset of the checker's source or add an interpreter layer to a qualified native operation. Direct normalization, finite replay, optional proof recording and protected theorem authority have separate contracts. Choosing their best physical realization requires workload and cost comparisons.

13.7 Conversion policies, proof elaboration and operational fibres

A checking graph needs more than an ordering of mathematical strength. One edge may add an axiom, another may license a formerly explicit equality as conversion, and another may change only the producer of certificates. An interpretation, a conservative extension and a measured specialization carry different laws. Distinct proof and execution histories remain visible until an observation explicitly forgets them.

The current HOL reference instance fixes arbitrary base sorts and typed constants, while a policy selects beta permissions by the contracted redex's result type, eta permissions by domain and codomain, and equation permissions by declaration occurrence. The permission is checked at the actual inner redex, even when the surrounding term has another type. A cited equation carries its pattern telescope and a simultaneous substitution. Explicit typed substitution contexts and abstraction/quantifier congruence support computation under binders without dropping scope. A substitution context can use its parameter repeatedly; this derived equality action is not necessarily one evaluator contraction.

For a policy P, let R_P be the independently stated local-step relation. The maintained equality-recipe checker computes endpoints, checks every transitivity join and has exact authority:

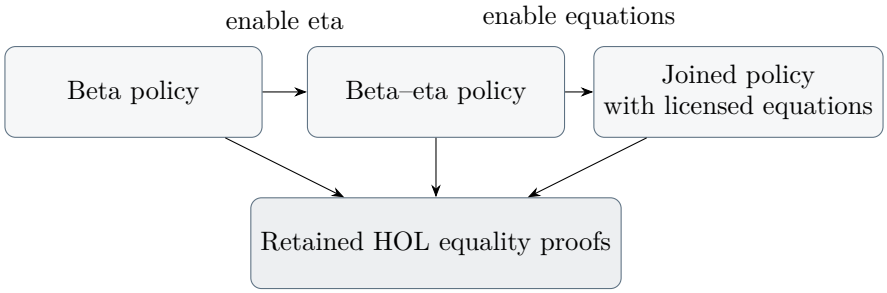
$$s\ \mathrm{EqvGen}(R_P)\ t \quad \Longleftrightarrow \quad \exists e.\ \mathrm{check}_P((s,t),e) = \mathsf{true}.$$

This is a decision about a supplied finite article. It does not require a normalizer or decide whether some article exists. The generic decoder constructs a free conversion tree from successful replay; forgetting that tree recovers the submitted recipe exactly, including its local witnesses and binary composition structure.

Lean. `EqualityRecipe.checkedPathWithRecovery`, `EqualityRecipe.forget_checkedPath`, `HOL.ConversionRecipes.leaf_authority`, `HOL.ConversionRecipes.authority`

The existing typed beta-execution carrier also compiles directly into these recipes, retaining each contracted position. A previous beta-joinability witness consequently supplies an accepted beta-only article. This is a computed forward connection to the maintained operational component; it does not assert reverse reflection for every policy or equality of proof histories.

Lean. `HOL.ConversionRecipes.executionRecipe`, `HOL.ConversionRecipes.executionRecipe_checked`, `HOL.ConversionRecipes.joinable_has_betaRecipe`



The downward edges use actual retained equality receipts for each equation, at its original telescope. They construct ordinary HOL proofs through beta, eta, congruence and equality composition. Logical retyping becomes explicit propositional equality elimination. The two-sort controls reach the same endpoint equality by primitive eta and by an instantiated eta declaration, while the submitted receipts remain distinct. Another pair has the same endpoint equality and retained proof sizes one and three. These are proof sizes, not elapsed-time comparisons.

Lean. `HOL.EquationProofs.instantiate`, `HOL.ConversionRecipes.elaborate`, `HOL.ConversionRecipes.convertProof`, `HOL.ConversionRecipes.Controls.eta_and_equation_recipes_distinct`, `HOL.ConversionRecipes.Controls.proof_costs_differ`

The existing implication/universal modulo calculus stores proposition-valued conversion. With the same equation receipts, every one of its proofs has an ordinary retained HOL proof at the original ordered sequent. This translation establishes existence; it does not reconstruct a forgotten source history. The checked recipe interface supplies that stronger reconstruction. Curry's cyclic equation cannot acquire the required HOL receipts: otherwise its existing modulo proof of $\forall p.p$ would become an ordinary HOL proof, contradicting the maintained standard model. These edges cover equations already justified by the target calculus. Fresh definitional extensions instead use the separate signature-changing unfolding and conservativity construction.

Lean. `HOL.ProofSyntaxModulo.hasHOLProof`, `HOL.ConversionElaborationControls.no_curry_equationProofs`, `HOL.ProofSyntaxModulo.definition_conservative`

What composes. Two rule packages independently admissible over the *same original* judgment system can be combined without changing derivability. Actual local certificate compilers also combine, selecting each feature's own expansion and node budget. The mixed control eliminates a

shortcut and an identity inference into a three-node base certificate; adjoining the genuinely new fact still has no such compiler. This logical composition law does not establish normalization, confluence or decidability for a union of conversion engines. Those are separate interaction obligations.

Lean. `extendRules_union_sameDerivability_iff`, `AdditionalRuleCertificateCompiler.combine`, `AdditionalRuleCertificateCompiler.combine_sameDerivability`, `ConservativeExtensionCanary.compiledMixed_count`, `ConservativeExtensionCanary.combined_newFact_forbidden`

Elaboration and execution. An elaborator may perform implicit-argument search, instance resolution, unification or tactics. Here an arbitrary producer submits finite recipes to the existing screening interface, and only accepted submissions acquire retained equality proofs. Producer completeness is unnecessary for this soundness boundary. This is a concrete consumer of the producer/checker separation, not an implementation of all those frontend algorithms.

Lean. `HOL.ConversionRecipes.elaborateSubmission`, `HeuristicEvidence.arbitrary_producer_sound`

The operational fibre over a context, type and policy has the original typed terms as states. Its OSLF reachability predicate is exactly certificate existence for $\mathrm{EqvGen}(R_P)$. Policy extensions and joins preserve this observation, including computations under binders. This predicate forgets the chosen recipe; the recipe decoder does not. Program-execution fibres additionally retain the chosen world, resources, representation and scheduling policy. Their routes use the earlier strict, covered or path-valued operational contracts as appropriate. Logical conversion is not thereby declared to be evaluator reduction, and execution costs need their own accounts.

Lean. `HOL.ConversionRecipes.observed_iff_accepted`, `HOL.ConversionRecipes.LocalStep.conversion_mono`, `HOL.ConversionRecipes.observed_mono`

Extensional comparisons remain selected bridges. Modular type-theory presentations provide an established precedent for feature-selected fragments [13]. Hofmann's comparison of specified extensional and intensional theories, with functional extensionality and UIP on the intensional side, concerns an interpretation and conservativity, rather than identical conversion algorithms. The later Morita-equivalence result compares their categories of models in its stated setting [14, 15]. These results do not establish such a bridge for every dependent candidate in this library. A semantic extensional readout can coexist with an executable intensional checker and explicit transport evidence, without making equality reflection a rule of every checker. Chosen route action, universe policy, theory assumptions and operational observations remain part of the comparison.

The new checker is a proof-producing reference on already scoped and typed syntax. It qualifies neither a raw-byte parser nor native seal enforcement, and it supplies no native performance comparison. Full dependent-fragment reflection and the routes to actual HOL Light, Mathdata and configured MM0 remain further graph edges.

13.8 Selected iota roots and checked dependent instantiation

The existing `ParameterizedPiSigmaId` rules already state the head, universe, cumulativity, constant and root-computation policies of that calculus. Its indexed-family interface separately carries constructors, an eliminator specification and typed iota schemas. The following checking connection consumes those schemas; it does not derive another eliminator or introduce a universal profile record for every dependent candidate.

An ordered schema table T stores open endpoint pairs at their respective arities. A policy $e : \mathrm{Fin}\,|T| \to \mathsf{Bool}$ selects original row occurrences. A finite root code supplies a row address and a list of scoped arguments. Replay checks the address, permission and argument count, then computes capture-safe simultaneous substitution into the selected pair. The independently defined root ev-

idence retains both the original row and the substitution. Its decoder is sound and complete for exactly those instances. Repeated equal rows remain distinct citations, and an enabled duplicate cannot rescue a disabled citation.

Lean. `SchemaTableConversionCode.retained`, `SchemaTableConversionCode.decode_sound`, `SchemaTableConversionCode.decode_complete`, `SchemaTableConversionCode.rootDecoder`

Renaming and substitution act on every submitted argument while retaining the row address. Replay commutes with these actions, including refusals. Selection by schema occurrence is stable under substitution. A selection depending on the instantiated dependent type would need its own stability law before becoming a root-computation policy.

Lean. `SchemaTableConversionCode.decode_rename`, `SchemaTableConversionCode.decode_substitute`

For a table of existing typed iota schemas, row lookup also retrieves the original telescope Γ_i, displayed type A_i and retained computation witness. The selected rule package keeps the original head, universe and constant policies and replaces its authored roots by the selected instances. Unlisted inherited and delta roots are excluded from this restriction. The signature's stated correspondence between retained witnesses and raw declared computation gives an actual rule morphism into the original declaration calculus. It preserves full formation-sensitive judgments and induces a forward OSLF conversion route. No reverse coverage, normalization or admission theorem is inferred from this inclusion.

Lean. `IndexedFamily.Checking.receiptAt`, `IndexedFamily.Checking.realizeEvidence`, `IndexedFamily.Checking.inclusion`, `IndexedFamily.Checking.judgment_inclusion`, `IndexedFamily.Checking.conversionRoute`

Typed instantiation is another finite check. At a caller's telescope Δ, a substitution request checks formation of Δ and each dependent argument obligation

$$\Delta \vdash \sigma(v) : \sigma(\Gamma_i(v)).$$

The maintained telescope and structural typing checkers perform these checks. Even a zero-argument request must check Δ: an empty argument list cannot authorize an unformed context. Every admitted formation-sensitive context morphism has such finite evidence; this is certificate existence, not an inference procedure.

Lean. `FormationSensitiveChecking.SubstitutionRequest.sound`, `FormationSensitiveChecking.SubstitutionRequest.complete`

With a permitted row, accepted arguments and the exact requested endpoints and displayed type, replay substitutes the original typed receipt. It retains both endpoint typings and the original computation witness at $\sigma(A_i)$. The same request exports a finite raw root code. The reverse implication fails: raw computation does not establish argument typing. Disabling a computation permission also does not construct a propositional identity proof in the smaller calculus.

Lean. `IndexedFamily.Checking.InstanceRequest.replay`, `IndexedFamily.Checking.InstanceRequest.formed`, `IndexedFamily.Checking.InstanceRequest.rootCode_accepted`

The existing natural/vector instance. The cumulative declaration signature has four independently defined iota cases: natural zero and successor, and vector nil and cons. Its actual four canonical schemas now give an exact decoder for that original root relation. The declaration decoder then combines inherited computation, actual delta lookup and these four cases. This supplies the missing executable parameter of common conversion and formation-sensitive judgment checking for the unchanged signature.

Lean. `NativeNaturalVectorFamilies.Checking.iotaDecoder`, `NativeNaturalVectorFamilies.Checking.decoder`,

`NativeNaturalVectorFamilies.Checking.conversion_iff_checked`,
`NativeNaturalVectorFamilies.Checking.coverage`

The controls check all four raw iota cases, an actual typed natural-eliminator instance in its dependent telescope, and its weakening to a larger context. They reject exchanged dependent arguments, a changed displayed type, incorrect arities, captured endpoints and disabled rows. An instance with unknown arguments has raw computation evidence but fails the typed instance check. The retained natural iota receipt is reconstructed by the checker, rather than supplied as part of its untrusted request.

Lean. `IndexedFamilyCheckingControls.all_verdicts`,
`IndexedFamilyCheckingControls.checkedZeroReceipt`

These results do not qualify uniform preservation of the raw signature, general positivity checking, or native execution of vector programs. The earlier root-free set comparison does not automatically interpret these added declarations. Natural infinity and no-confusion require their own constructor and induction laws; finite datatype declarations have different consequences. Translation of those laws to HOL Light, Mathdata or configured MM0, and a native realization of this checker, remain separate edges. Partial checking and operational contracts can therefore enter the graph without pretending that every node already discharges every contract.

13.9 Generated rules, bidirectional evidence and dependent append

The simple-inductive generator already produces its recursor and one iota schema per source constructor occurrence. Its computation relation is stated independently of schema replay. The generated ordered table now presents exactly that relation, and the common finite decoder checks the resulting conversion recipes. A receipt retains its constructor ordinal and complete assignment; it translates directly to a single checked root, including below binders. This connection does not identify raw conversion evidence with the generator's annotated typed receipts or establish declaration admission.

Lean. `IndexedPolynomialGenerated.Checking.presents`,
`IndexedPolynomialGenerated.Checking.table_get`,
`IndexedPolynomialGenerated.Checking.receiptCode_accepted`,
`IndexedPolynomialGenerated.Checking.conversion_iff_checked`

The bidirectional producer works against a selected existing rule package. It synthesizes types for variables, declared constants and applications, and checks lambdas and pairs against dependent products and sums. Its inputs include proposed head types, universe joins, normalizations and conversion recipes. These proposals carry no logical authority: every successful submission passes the maintained context and formation-sensitive judgment checkers. Soundness therefore holds even for incorrect producer choices. Fuel bounds the search; refusal is not a proof of nontypability.

Lean. `FormationSensitiveElaboration.replay_sound`,
`FormationSensitiveElaboration.elaborate_sound`

Restricted implicit inference reuses the existing scoped schema matcher. Argument-type patterns share one assignment, so repeated metavariables must agree. Every omitted prefix argument must be determined; no arbitrary value fills an unconstrained slot. The resulting application, with implicit arguments followed by the original explicit arguments, undergoes the same independent checking. For vector cons, the explicit head and tail determine both element type and length. Controls reject an undetermined length, inconsistent repeated element types and missing or extra arguments. This is not a complete higher-order unifier or an instance-resolution algorithm.

Lean. `FormationSensitiveElaboration.Implicit.complete_sound`,
`FormationSensitiveElaboration.Implicit.elaborate_sound`,
`InductiveElaborationControls.inferredSingleton_typed`

Conversion search traverses the maintained syntax and proposes explicit beta, projection, congruence and declaration-root recipes. Each proposed normalization is checked against its original

endpoints. Comparing two such results constructs a recipe through their common shape and checks the composite again. The soundness theorems establish conversion, not normal-form existence, completeness of search or forward evaluation of every returned conversion recipe.

Lean. `StructuralConversionSearch.normalize_sound`, `StructuralConversionSearch.compare_sound`

A checked dependent function. The independently authored cumulative Nat/Vec declaration has an element universe parameter u and a motive parameter v. The existing level-instantiation morphism specializes the latter to $\max(u, v)$, retaining both parameters and transporting declaration formation. This licenses the vector-valued motive needed by the following explicit core program:

$$\mathsf{append} : \Pi(A : U_u).\Pi(n, m : \mathsf{Nat}).\mathsf{Vec}\ A\ n \to \mathsf{Vec}\ A\ m \to \mathsf{Vec}\ A\ (\mathsf{add}\ n\ m).$$

Addition uses the declared natural eliminator, and append uses the declared vector eliminator. Neither introduces new recursion or conversion rules. The proof checks the entire closed function at this dependent type in the selected instance, with its schematic element universe and both lengths.

Lean. `NativeNaturalVectorFamilies.Programs.levelInstance`,
`NativeNaturalVectorFamilies.Programs.signature_formed`,
`NativeNaturalVectorFamilies.Programs.judgment_map`,
`NativeNaturalVectorFamilies.Programs.append_typed`

Evidence production runs outside the proof kernel. A shared constructor-data emitter records its finite submission; kernel reduction then checks only that data against the independently written program, type and decoder. The qualified append submission uses 962 shared definitions in 387836 bytes of Lean source. The checker rejects the same submission with its result index changed, its program replaced by the second input vector, or its root typing rule forged. These refusals concern the submitted evidence, rather than general unprovability of other claims.

Lean. `NativeNaturalVectorFamilies.Programs.appendChecked_true`,
`NativeNaturalVectorFamilies.Programs.changedType_refused`,
`NativeNaturalVectorFamilies.Programs.changedTerm_refused`,
`NativeNaturalVectorFamilies.Programs.forgedRoot_refused`

The nil and cons equations of this program hold under arbitrary scoped substitutions. In the cons equation the target contains append applied to the recursive tail; retaining the unprocessed tail instead is refused by the comparison check. These results establish structural conversion, rather than a forward execution strategy or a termination theorem.

Lean. `NativeNaturalVectorFamilies.Programs.Computation.append_substitute`,
`NativeNaturalVectorFamilies.Programs.Computation.append_nil`,
`NativeNaturalVectorFamilies.Programs.Computation.append_cons`

Constructor clauses and checking permissions. A scoped clause compiler consumes the admitted constructor inventory. Its current fragment has one linear, top-level clause per constructor, in source order. The compiler checks names and field counts, replaces written calls on the declared recursive children by induction hypotheses, and constructs methods for the existing eliminator. Capture-safe inverse substitution certifies that replacing those hypotheses by their original calls reconstructs the submitted clause. Whole-term elaboration still uses the independent formation-sensitive checker.

Lean. `ScopedSubstitutionInversion.invert_sound`,
`IndexedInductive.CaseElaboration.compileBranch_reconstructs`,
`IndexedInductive.CaseElaboration.compileMethods_coverage`,
`IndexedInductive.CaseElaboration.elaborate_sound`

For the admitted vector inventory, the written append clauses compile to exactly the independently authored core function above, uniformly in the payload type. Controls refuse missing, extra or reordered clauses, incorrect field counts, calls on another child or index, and a remaining recursive

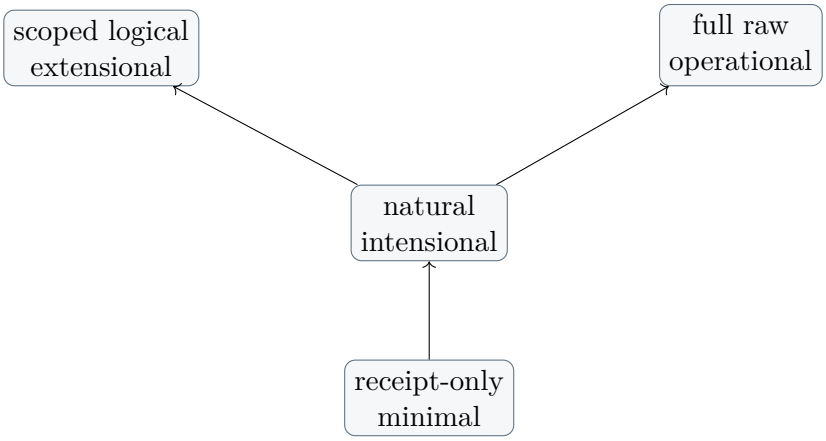
function constant. The concrete source interface includes a noncomputable interpretation of natural indices: this is a kernel-evaluated comparison, not an executable textual frontend. Nested pattern matrices and their compilation remain outside this clause fragment.

Lean. `IndexedInductive.AppendElaboration.methods_match_reference`, `IndexedInductive.AppendElaboration.compiled_matches_reference`, `IndexedInductive.AppendElaboration.compiled_typed`, `IndexedInductive.AppendElaboration.all_verdicts`

The same finite append typing submission also checks in an actual restricted calculus containing just the two natural iota rows, with unchanged heads, declarations and universe instance. Removing either row rejects that submission. The vector rows remain separate permissions for computing vector eliminator applications. Monotone permission inclusions preserve full judgments and their original contexts. Thus a smaller checking basis can be derived from a richer declared judgment, without claiming a globally smallest kernel or unprovability from the rejected submissions. These local results do not establish uniform computational admission of the package.

Lean. `NativeNaturalVectorFamilies.Programs.ComputationProfiles.permissions_mono`, `NativeNaturalVectorFamilies.Programs.ComputationProfiles.judgment_inclusion`, `NativeNaturalVectorFamilies.Programs.ComputationProfiles.append_typed_naturalOnly`, `NativeNaturalVectorFamilies.Programs.ComputationProfiles.all_verdicts`

Finite manifests for the checking-profile flower. The same admitted schema table has four explicitly separated checking nodes. The minimal node retains every typed iota receipt but licenses no iota conversion. The natural intensional node adds exactly the two natural rows. It has two incomparable children: the scoped logical-extensional node adds opaque propositional and function extensionality as object-theory constants, while the raw operational node licenses both vector rows. Neither child silently supplies the other's capability.



Inclusion and both reverse separations are checked per capability field. The logical child keeps extensionality out of judgmental equality and includes neither UIP nor equality reflection. The raw child remains an operational profile: unrestricted lifting into annotated vector conversion is refuted by the repeated-parameter counterexample.

Lean. `NativeNaturalVectorFamilies.Programs.CheckingProfiles.natural_profile_fork`, `NativeNaturalVectorFamilies.Programs.CheckingProfiles.scoped_extensionality_not_judgmental`, `NativeNaturalVectorFamilies.Programs.CheckingProfiles.scoped_has_neither_uip_nor_equalityReflection`, `NativeNaturalVectorFamilies.Programs.CheckingProfiles.fullRawOperational_not_unrestrictedAnnotatedLift`

These nodes now have a finite MeTTa `Atom` manifest. Features select tower or legacy-ground elimination, propositional or judgmental identity principles, and the receipt, propositional and judgmental observations of each actual iota-table row. Decoding checks every submitted row index. Validation rejects a judgmental iota permission unless the corresponding typed receipt is retained.

Feature union preserves this law and embeds both input checking contracts. This validates contract structure; arbitrary optional identity data, whether propositional or judgmental, still needs a profile-specific realization theorem.

Lean. `NatVecCheckingProfileAtomWire.Profiles.validCheck_eq_true_iff`,
`NatVecCheckingProfileAtomWire.Profiles.valid_join`,
`NatVecCheckingProfileAtomWire.Profiles.denote_extends_join_left`,
`NatVecCheckingProfileAtomWire.Profiles.denote_extends_join_right`,
`NatVecCheckingProfileAtomWire.Profiles.decode_encode`,
`NatVecCheckingProfileAtomWire.Profiles.joinEncoded_sound`

The four canonical manifests validate to the existing four profiles, rather than parallel reimplementations. The encoded natural profile checks the existing dependent append certificate; the encoded minimal profile refuses it. Its two licensed natural rows retrieve exactly the original typed source and target receipts, while vector nil remains non-judgmental there. A missing receipt, an out-of-range selector and malformed profile data all refuse. None of these finite checks establishes normalization, confluence, conservativity, a Hofmann translation or native runtime refinement.

Lean. `NatVecCheckingProfileAtomWire.Profiles.denote_minimal_eq`,
`NatVecCheckingProfileAtomWire.Profiles.denote_naturalIntensional_eq`,
`NatVecCheckingProfileAtomWire.Profiles.denote_scopedLogicalExtensional_eq`,
`NatVecCheckingProfileAtomWire.Profiles.denote_fullRawOperational_eq`,
`NatVecCheckingProfileAtomWire.Profiles.naturalZeroLicensedReceipt_exact`,
`NatVecCheckingProfileAtomWire.Profiles.naturalSuccLicensedReceipt_exact`,
`NatVecCheckingProfileAtomWire.Profiles.naturalIntensional_vectorNil_not_judgmental`,
`NatVecCheckingProfileAtomWire.Profiles.naturalIntensional_encoded_checks_append`,
`NatVecCheckingProfileAtomWire.Profiles.minimal_encoded_refuses_append`,
`NatVecCheckingProfileAtomWire.Profiles.missingReceipt_encoded_refused`,
`NatVecCheckingProfileAtomWire.Profiles.out_of_range_iota_refused`,
`NatVecCheckingProfileAtomWire.Profiles.malformed_profile_refused`

Interaction evidence is a separate certificate. For the natural-intensional node, the actual declaration-aware calculus is exactly the previously proved natural-only package. Its root computation is covered by the two constructor schemas, and the maintained constructor-system development proves Church–Rosser for their interaction with structural beta, pair projections and universe-head equality. The same certificate retains Pi and Sigma conversion boundaries and formation-sensitive subject reduction along arbitrary finite forward paths. The encoded natural manifest is now packaged with this separate certificate.

Lean. `CheckingProfiles.Interaction.naturalIntensional_actualRules`,
`CheckingProfiles.Interaction.naturalIntensional_roots_exact`,
`CheckingProfiles.Interaction.naturalIntensionalCertificate`,
`CheckingProfiles.Interaction.naturalIntensional_nat_not_pi`,
`NatVecCheckingProfileAtomWire.Profiles.naturalIntensional_encoded_qualified`

This proves confluence and preservation for that exact selected join. It does not prove total normalization or make those results a consequence of feature union. Nor does it qualify the full raw vector child: its repeated payload occurrences still refute unrestricted interaction with annotated conversion.

Lean. `CheckingProfiles.Interaction.fullRawOperational_not_annotatedInteraction`

General source compilation and complete generated-package admission remain separate from that checked target. Acceptance of the corresponding natural laws by native HOL Light, Mathdata and configured MM0, representation retention in CeTTa, source-bound behavioural observations and joined costs remain separate qualification links. No general conservativity, set-model interpretation of the added declarations, or native checker refinement follows from this finite typing result.

One textual program through that checker. The common module reader now consumes one source file containing the Nat and Vec declarations and addition/append constructor clauses. A finite operation adapter resolves names through the maintained nullary-constructor resolver. The separate literal-variable policy retains its own meaning; the adapter does not change general MeTTaIL rewriting. It preserves the complete selected declaration/operation blocks, refusing ignored imports, extra blocks and a different entry.

The compiler's finite plans contain constructor names, field counts and scoped recursive-child calls. Plans derived from the selected textual vector declarations agree with the independently admitted semantic inventory, uniformly in payload type. The source declarations themselves also lower to the original admitted Nat/Vec example. The same clause compiler consumes these plans without evaluating the source's noncomputable natural carrier.

Lean. `IndexedInductive.OperationSource.selectModule_exact`, `IndexedInductive.CaseElaboration.compilePlan_reconstructs`, `IndexedInductive.AppendSource.text_admitted`, `IndexedInductive.AppendSource.source_vector_plans_match`

Operational reconstruction now holds for every accepted compiler plan. Applying an emitted method to its original field variables and declared recursive-call results beta-reduces to the exact submitted clause body. For a complete accepted method list, this holds for every paired source clause in order, and the method count equals the constructor count. A successful named rule reader retains the actual lowering of its written right-hand side under the resolved source names. The combined rule compiler reconstructs that body while retaining the matched left-hand side. Both independently checked append methods instantiate the result. An unabstracted recursive constant or a call on another argument refuses. This is a computation theorem for accepted source plans; it neither authorizes a proposed recursive call nor proves arbitrary nested plan generation, source typing or termination.

Lean. `IndexedInductive.CaseElaboration.BranchPlan.method_reduces`, `IndexedInductive.CaseElaboration.compilePlans_reduces`, `IndexedInductive.CaseElaboration.compileMethods_reduces`, `IndexedInductive.OperationSource.readRule_body_exact`, `IndexedInductive.OperationSource.compileRule_reconstructs`, `IndexedInductive.AppendElaboration.methods_reconstruct_source`

Named expression expansion preserves the original lexical argument order and commutes with capture-safe substitution. Addition is compiled from its own preceding clauses before append uses it. For this actual source file, the proposed addition and complete append function equal the independent core programs above. The resulting append therefore has the full dependent type, including under the natural-only checking policy. The expected type is the explicit checking query; general dependent function-signature syntax is not supplied by this connection.

Lean. `IndexedInductive.OperationSource.Symbol.instantiate_substitute`, `IndexedInductive.OperationSource.lower_substitute`, `IndexedInductive.AppendSource.addition_matches_reference`, `IndexedInductive.AppendSource.append_matches_reference`, `IndexedInductive.AppendSource.append_typed_naturalOnly`

The producer remains untrusted. Independent replay checks its actual proposed term rather than accepting a source label or a successful parse. Controls refuse missing/reordered clauses, a call on the wrong recursive child, unbound results and omission of the recursive computation in the submitted dependent evidence. This establishes the selected input's comparison and a general formed-target soundness boundary, not correctness of this producer for every module. The grammar-driven CeTTa reader and common decoder feed the same Lean checking path; their fixture agreement is distinct from packed parser completeness and from Prime-mode execution with retained representations.

Lean. `IndexedInductive.AppendSource.checkProposal_sound`, `IndexedInductive.AppendSource.all_verdicts`

Joining the informative source package. The generated vector candidate now supplies its actual declaration inventory and two typed schemas to the append checking policy. Joining the independently checked natural prefix recovers the four-row reference table exactly. The candidate's complete declaration entries agree with the independent library; its root judgment and full formed typing judgments include into that library. This route composes with the existing maximum-level instantiation, transporting contexts, terms and result types together. The actual textual append is typed by the resulting generated policy, whose finite executable specialization is proved equal to it.

Lean. `IndexedInductive.AppendChecking.generated_table_reference`, `IndexedInductive.AppendChecking.candidate_extension`, `IndexedInductive.AppendChecking.candidate_judgment_to_program`, `IndexedInductive.AppendChecking.signature_comparison`, `IndexedInductive.AppendChecking.textual_append_typed`

The declaration-source comparison retains the complete generated library, not just the two vector equations. Live grammar-reader proposals now use the same generated checking policy, with only the two natural computation rows enabled. Independent replay checks the actual proposed term; the source fixture's term is not substituted for it. The selected positive reader gate requires exact raw and name-resolved module-data agreement and records the enabled original row ordinals. This is a selected-source admission/checking join, not general parser completeness or a completed native execution chain.

Lean. `IndexedInductive.AppendChecking.textual_candidate_library`, `IndexedInductive.AppendChecking.replayProposal_sound`, `IndexedInductive.AppendChecking.replaySource_sound`

Certificate relocation uses an injective embedding of ordered schema tables, retaining the entire schema and argument list. It checks the original source bound before changing an address, preserves acceptance and refusal under the target permissions, and composes along links. The negative example matters: an invalid local ordinal can name a valid row in the joined table, so merely copying or offsetting an unchecked integer can change its owner. Separately, the common typed-instance checker now has exact finite-certificate coverage for a fixed row and substitution, including target-context formation and independent argument typing. These results do not give uniform computation authority or identify the raw judgment with an annotated typed-equality calculus.

Lean. `SchemaTableConversionCode.Linking.relocate_comp`, `SchemaTableConversionCode.Linking.decode_relocate`, `Declaration.IndexedFamily.Checking.InstanceRequest.checked_arguments_iff`

Annotated instances and the remaining preservation boundary. The same four natural/vector rows now have independently checked annotated endpoint derivations and formed canonical telescopes. Eight declaration types, eight endpoints and four contexts are emitted as ordinary Lean proof source; the receiver checks every proof, so the producer grants no authority. The schema elaborator's inferred variable types and result types agree with these canonical contexts and types. Thus each actual right side satisfies the existing `TemplateTyped` interface.

Lean. `Programs.Annotated.endpoints_typed`, `Programs.Annotated.context_formed`, `Programs.Annotated.template_typed`

If an original row is permitted and its annotated assignment is a typed simultaneous substitution, the maintained premise interface admits the step. Substituting the independent endpoint proofs gives a typed equality at the instantiated result type. Erasure preserves that equality in the selected raw typed-equality calculus. This does not identify that judgment with the separate formation-sensitive structural-conversion judgment used for append. Each row has a canonical equality witness; altered result-type metadata is refused.

Lean. `Programs.Annotated.instantiated_equal`, `Programs.Annotated.erased_instantiated_equal`, `Programs.Annotated.canonical_equal`, `Programs.Annotated.all_metadataVerdicts`

For this actual package, injectivity and discrimination of annotated type formers suffice for uniform

annotated root preservation and admission. Those former facts remain an explicit, undischarged package obligation; the result does not assert unconditional preservation. The natural-only policy also satisfies the existing first-order, left-linear lifting theorem at every annotated input. Vector patterns repeat the element parameter and length index. Their typed instances are covered above, while unrestricted lifting requires a different annotation-coherence argument. Failure of this one syntactic condition is neither a failure of vector checking nor a normalization counterexample.

Lean. `Programs.Annotated.root_preserving_of_formerFacts`, `Programs.Annotated.root_admitted_of_formerFacts`, `Programs.Annotated.lift_natural_root`, `Programs.Annotated.vector_cons_repeats_index`

Actual function annotations and typed natural computation. The natural addition used inside append's result index now has a complete Church-annotated function. Its motive carries the actual natural domain; the recursive method retains the dependent domain obtained by applying that motive. Typed beta equalities supply the conversions in both the method and recursor application. The existing declaration telescope types all arguments independently. Erasure yields the original M1 addition, and the maintained bidirectional annotator reconstructs this independently typed term exactly.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Functions.addFunction_typed`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.producedAddFunction_exact`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_erases`

Its typed zero and successor equations cite the original permitted rows and then compose the actual method's beta equalities. The closed one-plus-zero case is a nonempty witness. With every iota root disabled the function is still typed, while the annotated root relation is empty: typing grants no new computation permission. Two genuinely typed motives with different domains share one erasure. These facts neither infer annotated typing from raw erasure nor discharge general former injectivity.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_zero`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_successor`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.typed_without_iota`, `NativeNaturalVectorFamilies.Programs.Annotated.Functions.domain_erasure_distinction`

The complete dependent append consumes this annotated addition. Its original vector constructor and eliminator declarations supply the six dependent arguments; the eliminator is not re-derived. The constructor method retains four explicit argument domains and uses the permitted natural successor equation to convert its constructed vector to the required length. The nil branch uses the permitted zero equation. Abstraction over the independently formed original context gives a closed function at its actual dependent function type, with no package-wide former-facts assumption.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Append.method_body_typed`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.method_typed`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunction_typed`

Erasure identifies both that function and its type with the original raw M1 program. The maintained bidirectional annotator reconstructs the independently typed function exactly when supplied its expected type. Without that guidance, its output still erases to the source but has different domain annotations. The natural-only policy suffices for these finite function derivations; uniform richer-package admission, former facts and unrestricted annotation coherence for the repeated-parameter vector roots remain distinct obligations.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunction_erases`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunctionType_erases`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.producedAppendFunction_typed`, `NativeNaturalVectorFamilies.Programs.Annotated.Append.unconstrained_erasure_distinction`

The actual annotated append now also has typed nil and cons computation equations. Each cites its original permitted vector row and supplies an independently typed simultaneous assignment. The

cons method computes by a general beta law for relative annotated telescopes: its ambient assignment is restricted from the complete typed assignment, while the four method arguments are applied in declaration order. The result retains the substituted dependent type. Closed nil and nonempty-cons examples witness the equations; erasure retains the original raw typed-equality judgment and recursive cons target, without identifying it with formation-sensitive structural conversion.

Lean. `TypedEquality.Annotated.CTele.lams_apply`,
`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_nil`,
`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_cons`,
`NativeNaturalVectorFamilies.Programs.Annotated.Append.closed_cons_erased`

The actual function remains typed under the natural-only policy, which disables both vector rows. Its concrete vector computation certificates are accepted by the full policy and refused by that smaller policy. Function typing, licensed computation and uniform package admission therefore remain separate graph contracts even for the same term and declarations.

Lean.
`NativeNaturalVectorFamilies.Programs.Annotated.Append.typed_with_vector_rows_disabled`,
`NativeNaturalVectorFamilies.Programs.Annotated.Append.all_computationVerdicts`

These actual checking policies are connected by the maintained annotated package-morphism interface. A schema-family inclusion with the same declaration table preserves the independently supplied universe-policy map, root instances and their exact premise lists. Morphism composition follows the ancestry of every required premise through both edges. It therefore transports complete typing, equality and subtyping statements, formed contexts and typed substitutions; no proof-size or native-cost conclusion is built into that law.

Lean. `TypedEquality.Annotated.ChurchRules.ofSchemas_morphism`,
`TypedEquality.Annotated.ChurchRules.Morphism.comp`,
`NativeNaturalVectorFamilies.Programs.Annotated.judgment_mono`,
`NativeNaturalVectorFamilies.Programs.Annotated.context_mono`,
`NativeNaturalVectorFamilies.Programs.Annotated.substitution_mono`

For the actual append, the natural-only policy includes into an intermediate policy that additionally licenses vector nil, and thence into all four original rows. The independently typed function crosses this composed route. Its closed nil equation already holds at the intermediate node; its actual cons certificate refuses there and accepts at the full node. These are computation policies over the same declared calculus, not complete Lean or extensional profiles and not a reverse theorem-transport result.

Lean. `NativeNaturalVectorFamilies.Programs.Annotated.Append.function_after_two_extensions`,
`NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_before_cons`,
`NativeNaturalVectorFamilies.Programs.Annotated.Append.cons_certificate_strict`

The authored natural carrier and HOL formulas. The sequentially admitted natural declaration has an independently established representation by ordinary naturals. A new consumer of its original polynomial eliminator gives the function satisfying

$$f(\mathsf{Zero}) = z, \qquad f(\mathsf{Succ}\ n) = s\ n\ (f\ n),$$

and proves uniqueness and full predicate induction. Its actual zero and successor therefore supply the existing Peano structure, including injectivity, disjointness and an infinite carrier. Lifting that carrier to the universe of the maintained HOL model constructs a full-domain interpretation of the existing Peano formulas. For every target simple type, an explicit HOL formula for the existence and uniqueness of the recursive function is true in this model. The proof uses the source eliminator rather than postulating a recursion axiom.

Lean. `IndexedInductiveNaturalHOL.recursion_exists_unique`,
`IndexedInductiveNaturalHOL.models_axioms`, `IndexedInductiveNaturalHOL.models_recursion`,
`IndexedInductiveNaturalHOL.infinite_carrier`

This connects the declared source carrier to actual HOL formulas, not merely to a list of names for natural-number facts. It is a particular semantic interpretation: it neither extracts proofs from truth in one model nor identifies the model's constants with a serialized HOL Light closure. The selected native HOL Light law interface below now supplies exact statement comparison; Mathdata/MM0 transport and full source definition-history refinement retain their own obligations. The source's noncomputable semantic recursor supplies no executable numeric representation or performance result.

The selected native HOL Light natural-law interface. Five independently typed intrinsic formulas now retain the actual source presentation: numeral wrapping, the result-before-predecessor recursion method, Boolean equality in successor injectivity, and an explicit injective endomorphism with an omitted value for carrier infinity. The recursion root is existential; uniqueness belongs to the stronger source theorem above. The actual authored carrier interprets all five formulas at arbitrary result carriers using its original eliminator. A Boolean-valued instance computes distinct results at zero and successor. A finite collapsing model refutes no-confusion, and removing the source's numeral wrapper changes the native syntax even though its constructed interpretation reads that wrapper as identity.

Lean. `HOL.NaturalFacts.recursion_meaning`, `HOL.NaturalFacts.infinity_meaning`, `HOL.NaturalFacts.collapsing_noConfusion_refused`, `IndexedInductiveNaturalHOL.NativeFacts.all_models`, `IndexedInductiveNaturalHOL.NativeFacts.boolean_recursion_witness`, `HOLLight.NaturalFacts.bare_zero_statement_distinct`

The existing source exporter produces the four retained natural roots and a new carrier-infinity theorem derived from successor injectivity and no-confusion. Native CeTTa replay checks the 23225-node closure as 67384 records and returns all five statements exactly as printed from the independent typed terms. The declared standard HOL policy retains its three axioms; no source axiom is added. Altered claims, an additional axiom and truncation refuse natively, while missing, changed and repeated statement packets refuse at the intake gate. Production, adapter and replay costs remain distinct. The source producers are untrusted. This qualifies the selected law-signature interface; full polymorphic source definition-history refinement and linked-C correctness are not inferred.

Lean. `HOLLight.HOLSyntaxWire.encodeTerm_rename`, `HOL.Names.lift_comp`

An axiom-free native closure component. The Megalodon definition

$$N_{z,s}(n) \;:=\; \forall P.\; P(z) \to (\forall k.\; P(k) \to P(s(k))) \to P(n)$$

uses only simple types, implication and quantification. Its native Mathdata body and the zero, successor and induction statements erase exactly from independently typed HOL terms, after the declared definition is unfolded. The finite zero and successor proof objects also pass the existing executable Mathdata checker in Lean. A separate Megalodon source file proves these laws, including induction with a closure hypothesis in the step. Native CeTTa replay accepts all six source proofs with an empty imported-axiom allowlist and compares the exact definition and three principal statements. Both CeTTa dialects agree on these observations. Altered conclusions and unbound hypotheses refuse; failed claims are not published.

Lean. `Megalodon.NaturalClosure.body_erases`, `Megalodon.NaturalClosure.induction_statement_erases`, `Megalodon.NaturalClosure.zero_proof_checked`, `Megalodon.NaturalClosure.successor_proof_checked`

The selected authored natural carrier satisfies this very predicate at every value. Its existing induction removes the extra closure hypothesis without excluding any source values. Interpreting successor as constant zero instead excludes a genuine source successor, while still admitting zero;

this is a nonempty negative comparison. Closure and induction alone do not imply successor injectivity, zero separation, unique recursion or infinity. Those native-route obligations remain explicit, and no proof is extracted from truth in the particular source model.

Lean. `IndexedInductiveNaturalMegalodon.source_member`, `IndexedInductiveNaturalMegalodon.source_native_formulas`, `IndexedInductiveNaturalMegalodon.collapsing_successor_refused`

The same closure through MM0's HOL presentation. The typed closure syntax and its Henkin meaning now have a common HOL owner. An MM0 syntax producer retains types and explicit binder names; it commutes with intrinsic variable renaming. The natural-closure definition body printed from that typed term supplies a second specification for the actual MM1 proofs. Native MM0 accepts zero closure, successor closure and ordinary closure elimination against both specifications, with their original object-typing hypotheses. The latter rule uses an unrestricted predicate step; it is distinct from the strengthened induction statement above.

Lean. `MM0.HOLSyntaxWire.encodeTerm_rename`, `MM0.NaturalClosure.definition_meaning`

CeTTa's HE, PeTTa and Prime modes each pass fourteen exact-statement, retained-proof and refusal controls on the same artifact. A separately built upstream `mm0-c` also accepts it and refuses changed definitions and conclusions. The source extension adds no axioms. The producer's dependency audit reports only logical, typing and equality rules, without choice, eta, type-definition or infinity axioms. Those axioms remain available in the loaded full HOL presentation; extracting a smaller base is separate work. A finite collapsing interpretation admits zero but excludes another inhabitant, showing why closure alone does not supply infinity. General HOL-presentation adequacy and full native natural-number fact transport remain open.

Lean. `MM0.NaturalClosure.finite_zero`, `MM0.NaturalClosure.finite_other_refused`

Closure-premised induction in the same MM0 route. The next source extension proves induction with the generated-number guard retained in its step premise. It strengthens the property to the conjunction of closure membership and the property, applies the existing ordinary elimination rule, then extracts the property. This is a derived proof using the existing logical core, with one auxiliary definition and no new axioms. An independent public statement and definition specification are emitted through the maintained MM0 wire machinery. The compiler and format producer remain outside the theorem authority.

The corresponding intrinsic formula holds in every chosen Henkin model. For admitted zero, successor and property, the strengthened predicate is the denotation of an actual lambda term. Henkin term closure therefore admits this particular predicate without a full-domain assumption. In a three-point interpretation, zero maps to itself while an unreachable point satisfying the property maps to a point that fails it. Its guarded step and induction hold, while the unguarded step fails. This separates the rule premises, not the strength of any dependent candidate.

Lean. `HOL.NaturalClosure.parameterValuation_admissible`, `HOL.NaturalClosure.denote_strengthening`, `HOL.NaturalClosure.strengthening_admitted`, `HOL.NaturalClosure.closedUnder_induction_admitted`, `HOL.NaturalClosure.models_induction_henkin`, `HOL.NaturalClosure.InductionControls.closed_iff_zero`, `HOL.NaturalClosure.InductionControls.guarded_step`, `HOL.NaturalClosure.InductionControls.unguarded_step_refused`, `HOL.NaturalClosure.InductionControls.guarded_induction`

Upstream verification accepts the compiled artifact against both source and independent specifications. Native HE, PeTTa and Prime modes each pass twenty controls in both the selected optimized and instrumented configurations, using the same 65,328-byte artifact. Replayed induction retains all seven premises; changed guards, dropped membership and exchanged hypotheses are refused.

Changed auxiliary definitions, changed conclusions and truncation are refused as well. The dependency inventory uses logical, typing and equality rules without choice, eta or infinity. The original closure profile still passes fourteen controls per mode through the same gate. These are qualified proof/statement routes; full natural recursion, no-confusion, infinity and source-presentation adequacy remain separate obligations.

Conditional native unique recursion. The extension defines the least graph containing (z, a) and closed under $(n, v) \mapsto (s(n), f(n, v))$. Its independent inductive reading is functional when s is injective and never returns z. Every generated number has a graph value. A selector satisfying its explicit selection law therefore defines a recursive function, unique on the natural closure. The native statement retains all three interfaces; none is an imported natural-number axiom.

Lean. `Megalodon.NaturalRecursion.Graph.functional`,
`Megalodon.NaturalRecursion.Graph.exists_of_generated`,
`Megalodon.NaturalRecursion.selected_unique`

In full-domain interpretations, the intrinsic HOL graph denotes exactly this inductive graph. The independent typed recursion statement has the stated zero/successor equations and pointwise uniqueness. Its native erasure is well formed, and the retained native proof is checked against that erasure. The combined source supplies 24 proofs and seven definitions, with no imported axioms; replay, portable evidence and altered-claim controls pass in both CeTTa dialects. Compiled Lean checking also accepts the complete ordered source batch. This last execution result is not a kernel-reduced proof of the finite batch-acceptance proposition.

Lean. `Megalodon.NaturalRecursion.denote_graph_iff`,
`Megalodon.NaturalRecursion.models_recursion`,
`Megalodon.NaturalRecursion.native_statement_erases`,
`Megalodon.NaturalRecursion.native_statement_formed`

On the authored carrier, every graph value equals the original eliminator's value. All valid selectors agree there, and a classical interpretation supplies a selector at every inhabited target. This is a semantic choice witness, not a runtime provider or a global dependent-type-theory option. A constant-zero selector fails a genuine successor computation. The full native no-confusion/infinity interface and MM0 transport remain open.

Lean. `IndexedInductiveNaturalMegalodon.source_graph_iff`,
`IndexedInductiveNaturalMegalodon.source_selectors_agree`,
`IndexedInductiveNaturalMegalodon.source_recursion_with_choice`,
`IndexedInductiveNaturalMegalodon.constant_selector_wrong_result`

The selected structural-conversion node. The actual natural-only checking policy retains original schema occurrences 0 and 1. Its two independent constructor schemas agree exactly with that selection. Their left sides are linear constructor patterns, their right-side variables are covered, and two matching left sides determine the same rule. The maintained constructor-system theorem therefore proves confluence of the policy's raw beta/iota conversion, including pair projections and its declared head equality. It also supplies the Pi/Sigma conversion boundaries used by formation-sensitive preservation. Addition at zero witnesses actual computation. Conversely, no conversion identifies the declared natural type with a Pi, including paths outside its simple image. These results do not supply termination, annotated typed-eta former facts, or a constructor-system presentation for the repeated-parameter vector schemas.

Lean. `ComputationProfiles.NaturalConversion.schema_iff`,
`ComputationProfiles.NaturalConversion.roots_exact`,
`ComputationProfiles.NaturalConversion.churchRosser`,
`ComputationProfiles.NaturalConversion.pi_boundary`,
`ComputationProfiles.NaturalConversion.sigma_boundary`,
`ComputationProfiles.NaturalConversion.addition_zero`,
`ComputationProfiles.NaturalConversion.natural_not_pi`

Uniform preservation in the checking policy. The declaration-telescope interface recovers the motive, zero method, successor method and input from any typing of the actual recursor call. Recovery retains the original formed declarations and replays its complete conversion/cumulativity tail. At zero, the recovered zero method already has the principal result type. At successor, constructor-argument recovery types the predecessor; renewing the input component of the typed substitution constructs the recursive call. The same successor method then supplies the contractum. The shared telescope interface now proves construction as well as recovery, uniformly in its finite dependent arity.

Lean. `FormationSensitive.DeclarationSpine.applyTelescope`, `ComputationProfiles.NaturalConversion.Preservation.recover`, `ComputationProfiles.NaturalConversion.Preservation.zero_preserves`, `ComputationProfiles.NaturalConversion.Preservation.successor_preserves`

These root proofs and the earned conversion boundaries discharge contextual preservation for this checking policy. The universe and head laws are inherited from the tower. Beta reduction, declared natural computation and projections preserve the formed context and exact displayed type under every context. Concrete checked zero and successor redexes witness the premises; four actual steps compute the latter to the successor of zero. Reusing its submitted evidence with a Pi result is refused. The policy still disables vector roots. Richer annotated typed-eta former facts remain distinct from the actual whole-function annotation already checked above.

Lean. `ComputationProfiles.NaturalConversion.Preservation.roots`, `ComputationProfiles.NaturalConversion.Preservation.steps_preserve`, `ComputationProfiles.NaturalConversion.Controls.zero_typed`, `ComputationProfiles.NaturalConversion.Controls.successor_path`, `ComputationProfiles.NaturalConversion.Controls.successor_preserved`, `ComputationProfiles.NaturalConversion.Controls.altered_type_refused`

Operational roots with a smaller checking basis. All four original operational iota roots preserve this same smaller checking judgment. Recovering the vector eliminator's six dependent arguments retains its original declaration and displayed-type adjustment. The nil target is its recovered branch. In the cons case, a second telescope inversion recovers the constructor's length, head and tail. They extend the recovered parameter substitution into the original canonical context. The canonical contractum uses the existing eliminator and retains its recursive call; substitution and replay then give the exact original displayed type. Closed formation of that eliminator is recovered from the actual source derivation, without assuming preservation for the larger operational relation.

Lean. `FormationSensitive.Typing.applyClosed_head`, `FormationSensitive.DeclarationSpine.ofObservedTelescope`, `FormationSensitive.DeclarationSpine.constant_formed`, `ComputationProfiles.NaturalConversion.Preservation.Vector.recover`, `ComputationProfiles.NaturalConversion.Preservation.Vector.constructor_arguments`, `ComputationProfiles.NaturalConversion.Preservation.Vector.canonical_cons_target`, `ComputationProfiles.NaturalConversion.Preservation.Vector.nil_preserves`, `ComputationProfiles.NaturalConversion.Preservation.Vector.cons_preserves`, `ComputationProfiles.NaturalConversion.Preservation.operational_roots`, `ComputationProfiles.NaturalConversion.Preservation.operational_rule_roots`

The canonical nil and cons sources are independently checked under the smaller policy, and the new theorem types their operational targets. Nevertheless the cons rule is not a root conversion of that checker: its concrete certificate is refused there and accepted with the original full permissions. This distinguishes execution from checking authority. The theorem covers roots; lifting a larger operational relation through arbitrary dependent contexts requires an additional transport law, since its steps are not thereby available as checking conversions.

Lean. `ComputationProfiles.NaturalConversion.Controls.vector_nil_typed`, `ComputationProfiles.NaturalConversion.Controls.vector_cons_typed`, `ComputationProfiles.NaturalConversion.Controls.vector_nil_preserved`,

ComputationProfiles.NaturalConversion.Controls.vector_cons_preserved,
ComputationProfiles.NaturalConversion.Controls.vector_cons_root_disabled,
ComputationProfiles.NaturalConversion.Controls.vector_cons_citation_refused,
ComputationProfiles.NaturalConversion.Controls.vector_cons_citation_enabled

The opaque annotated baseline. Disabling every declared iota permission leaves all eight original Nat/Vec declarations and their level/head policy intact. The maintained theorem for packages without root computation gives the richer annotated former injectivity and no-confusion laws. In particular, the actual addition function's product type cannot equal a universe. The same function is typed with those permissions disabled; its actual first beta application is an equality at its displayed function type.

Lean. NativeNaturalVectorFamilies.Programs.Annotated.inactive_raw_no_root,
NativeNaturalVectorFamilies.Programs.Annotated.inactive_formerFacts,
NativeNaturalVectorFamilies.Programs.Annotated.inactive_pi_not_head, NativeNaturalVectorF
amilies.Programs.Annotated.Functions.inactive_function_type_not_universe,
NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_function_beta

The cumulative head-preservation argument has a common owner and serves both this declaration policy and the existing set-profile client. Along with the earned opaque former facts, it discharges the maintained contextual preservation theorem: every finite beta/projection/head reduction of a term typed in a formed context gives a typed equality, and a term at the exact original dependent type. No termination is assumed or concluded. The independent annotator still reconstructs the real function. Its natural-zero root citation is refused here and accepted when the original row is enabled. The absence of declared roots is explicit; this baseline does not admit an enabled Nat/Vec computation or establish annotation coherence.

Lean. Annotated.CHeadPreserving.ofLevelTower,
NativeNaturalVectorFamilies.Programs.Annotated.head_preserving,
NativeNaturalVectorFamilies.Programs.Annotated.inactive_reduction_equal,
NativeNaturalVectorFamilies.Programs.Annotated.inactive_reduction_typed,
NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_zero_citation_refused,
NativeNaturalVectorFamilies.Programs.Annotated.Functions.full_zero_citation_accepted

The natural annotation has a nontrivial reading. The actual natural type, zero, successor and dependent recursor now use the existing domain numeral ideals and type-projected constants. Staging constant lookup avoids a recursive definition of the interpretation. The four original declared types agree with the maintained numeral interface. All eight declarations remain present; the other constants are opaque here and their computation is disabled.

Lean. NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.nat_declared,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.zero_declared,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.successor_declared,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.recursor_declared,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.constants_valid

Every enabled root is an instance of one of the original two natural rows. Its actual annotated argument spine supplies the dependent semantic telescope premises; the interpretation then uses the existing projected zero and successor computation theorems. Result-type and spine premises are retained. Together with the original head policy this gives a valid reading of the selected annotated package. The existing closed proof of one plus zero denotes successor of zero. That value is distinct from zero, and soundness refutes a purported annotated equation between them. These are concrete semantic and negative controls for this package, not a new strength separation or a general claim about dependent candidates. Semantic validity does not supply logical-relation adequacy, enabled former injectivity, unrestricted annotation coherence or native refinement.

Lean. NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.recursor_spine,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.zero_valid,

NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.successor_valid,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.roots_valid,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.valid,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.one_plus_zero_value,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.zero_ne_successor_zero,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.zero_equal_successor_refused

An opaque decoder extension that retains computation. The maintained package sum adds the existing fresh proposition/decoder declarations to the enabled natural annotation. Spine facts are contravariant in declaration inclusion: a larger signature's facts retain each original declaration's type. Consequently a valid reading of the original package and a valid reading of an opaque second package give a valid reading of their sum. Original constants keep their types, and every root comes from the original package; the second has none. This common semantic law also handles shadowed second declarations.

Lean. Annotated.SpineFacts.of_decl_inclusion, Annotated.ReadingValid.sum_opaque

The concrete decoder names are fresh, and its two declarations have their stated types. Both original natural roots remain with precisely their original required evidence. An actual zero computation and the typed one-plus-zero proof survive the existing forward judgment map, while the false zero/successor equation is still refuted. The opaque decoder constants have no observations in this reading; the natural constructors and recursor retain their nontrivial interpretation. This is neither the no-root extension used for opaque base packages. The forward semantic law concerns validity; the return comparison below supplies conservativity for its stated fragment. Recursor adequacy and enabled former facts remain further obligations.

Lean. NaturalDomain.Decoder.fresh_prop, NaturalDomain.Decoder.fresh_holds,
NaturalDomain.Decoder.prop_declared, NaturalDomain.Decoder.holds_declared,
NaturalDomain.Decoder.declarations_valid, NaturalDomain.Decoder.valid,
NaturalDomain.Decoder.roots_iff, NaturalDomain.Decoder.premises_iff,
NaturalDomain.Decoder.zero_root, NaturalDomain.Decoder.one_plus_zero,
NaturalDomain.Decoder.zero_equal_successor_refused

The return route preserves the required evidence. The existing constant-instance interface maps the fresh proposition to its universe and the decoder to the identity at that universe. Original declared names and all eight closed declared types stay fixed. The templates, inferred variable types and reflexivity equations of all four original rows satisfy the corresponding support conditions. Hence the comparison is uniform over their permission selections; repeated vector parameters do not require an annotation-lifting claim for this operation.

Lean.
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.original_names_fixed,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.declarationTypeFixe
dCheck_true,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.original_types_fixed,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.row_left_fixed,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.row_right_fixed, Nati
veNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.row_variable_type_fixed,
NativeNaturalVectorFamilies.Programs.Annotated.NaturalDomain.Decoder.schema_support

The common schema law lets arbitrary instance arguments change under the constant interpretation. Both endpoints remain instances of the original schema, and the entire required-premise list becomes precisely the list for the changed assignment. Root stability alone would not establish that law. Original constant formation, these schema laws and the maintained decoder-definition typing proofs construct the full return interpretation.

Lean. Annotated.patternPremises_instConsts_of_fixed, Annotated.SchemaConstantsFixed.root,
Annotated.SchemaConstantsFixed.requires, Annotated.ChurchRules.ofSchemas_constInstance,

NaturalDomain.Decoder.original_interpretation,
NaturalDomain.Decoder.decoder_type_expansion, NaturalDomain.Decoder.return_interpretation,
NaturalDomain.Decoder.returned_judgment

For every statement fixed by this interpretation, derivability in the extended package is equivalent to derivability in the original package. The fixed-statement condition covers context, displayed type and all terms. The actual addition statement reflects at both natural-only and full permissions. Conversely the fresh proposition's typing statement changes to the typing of the universe, so the interpretation is not the identity. The false natural constructor equation also refuses via the return route. These are theorem-level judgment maps. No equality of retained proof data, serialized certificate translator, annotation-coherence result or execution cost bound follows from them.

Lean. NaturalDomain.Decoder.fixed_statement_iff, NaturalDomain.Decoder.proposition_typed,
NaturalDomain.Decoder.returned_proposition_typed,
NaturalDomain.Decoder.proposition_statement_changes,
NaturalDomain.Decoder.original_addition_fixed,
NaturalDomain.Decoder.original_addition_reflected,
NaturalDomain.Decoder.full_policy_addition_reflected,
NaturalDomain.Decoder.false_equation_refused_by_return

Typing the supplied HOL type-definition equation. For the displayed declarations $\mathsf{abs} : A \to N$, $\mathsf{rep} : N \to A$ and $a : N$, the term $\mathsf{abs}(\mathsf{rep}(a))$ has type N. The maintained OpenTheory source checker accepts its equality with a at N, and rejects an equality indexed by A whenever $A \neq N$. This is proved for arbitrary types, with concrete distinct-type controls.

Lean. MM0.HOLTypeDefinitionBoundary.roundtrip_typed,
MM0.HOLTypeDefinitionBoundary.defined_equality_typed,
MM0.HOLTypeDefinitionBoundary.representation_equality_refused,
MM0.HOLTypeDefinitionBoundary.defined_equality_checked,
MM0.HOLTypeDefinitionBoundary.representation_equality_check_refused,
MM0.HOLTypeDefinitionBoundary.distinct_types, MM0.HOLTypeDefinitionBoundary.all_verdicts

The pinned supplied `hol.mm0`/`hol.mm1` example had the representation-type index in `abs\_rep`; its native declaration confirmed that exact source formula. Both local files now use the defined-type index. Qualification compares the complete native axiom declaration with an independent reference and refuses the reverted specification using both the separate verifier and native service. The argument and hypothesis inventories are retained; no axiom is added. The qualified closure/induction proofs do not depend on this axiom. This repair addresses source typing, without inferring inconsistency of the earlier untyped presentation or full HOL-presentation adequacy.

Authorization retains the selected judgment. The legacy indexed-family authorization interface requires uniform `HasType` preservation. The current formation-sensitive checker uses a judgment containing its formed ambient context and additional formation premises; the annotation has a further typing/equality interface. Forgetting refined derivations gives raw ones. A converse upgrade from arbitrary raw derivations is impossible even with a formed empty context: the raw identity rule permits an undeclared domain, whereas regularity and product inversion of the refined judgment require that domain's declaration. This comparison is proved generically and checked for every original Nat/Vec permission selection. The actual append remains a positive refined derivation with its raw view.

Lean. FormationSensitive.raw_identity_at_any_domain,
FormationSensitive.judgment_identity_domain_declared,
FormationSensitive.raw_identity_missing_domain, FormationSensitive.no_uniform_raw_upgrade

These are deliberately different specified judgments. This does not refute legacy preservation or expose a strength limitation of a dependent candidate. It identifies the remaining interface obligation: the proved formed operational preservation and annotated former-fact conditional cannot alone fill the legacy raw-authorization field. The selected natural-only field is discharged below by

its own raw proof. The full four-row family candidates retain their separate preservation obligation; no weakening or identification of those interfaces is assumed.

A concrete pointed injection in the native MM0 route. The next proof extension selects an injective, non-surjective endomorphism of the supplied HOL individual type and a point outside its image. Its six public results prove the successor and zero typings, successor injectivity and non-surjectivity, and omitted-zero/no-confusion statements. They use only the existing logical rules, choice, eta and infinity; no natural-number fact is supplied as a new axiom. Intrinsically typed definition bodies and an independent statement emitter bind this extension to the maintained MM0 specification. The omitted-point formula has its stated Henkin meaning without a full-function-domain hypothesis.

Lean. `MM0.NaturalCarrier.noPreimage_meaning`, `MM0.NaturalCarrier.all_verdicts`

The separate verifier and native service accept the same compiled extension against the independent specification. Altering the injection/zero constructions, reversing no-confusion, changing zero's type, or removing the typed argument from proof replay is rejected. Both selected optimized and sanitizer configurations pass 36 controls in each dialect on the same 73,808-byte artifact. The unchanged closure/induction profiles retain their 16/22 controls and original artifact identities. The source's declaration order is part of this checked interface. This establishes a pointed infinite carrier; forming its natural subtype and proving primitive recursion and complete source-to-native natural facts remain required. Neither the MM1 compiler nor the entire supplied HOL presentation is thereby formally refined.

The native natural subtype. The existing nonempty-predicate type-definition principles form the subtype of the selected zero/successor closure. Representation and abstraction transport constructor typing, zero/successor separation, successor injectivity, induction and Dedekind infinity. The induction proof reuses the established closure theorem through an explicit predicate pullback; no new natural-number axiom or authored eliminator is introduced. The abstraction/representation equality is used at the actual defined type. Its exact native proof dependencies include the existing four type-definition principles. The induction proof does not depend on the infinity axiom; constructor separation and infinity retain that dependency. These are proof-local dependency statements, not an automatic weakening of the ambient specification.

Lean. `MM0.NaturalType.all_verdicts`

An independent reference specifies the subtype, constructors, pullback and six public statements. The shared gate rejects altered membership, pullback, equality indices, missing induction premises and switched replay inputs. Both selected native configurations pass 50 controls per dialect on the same 87,400-byte artifact. The native Boolean-equality abbreviation is retained in exact statement comparison. This is a checked natural-subtype construction, not yet primitive recursion, a complete comparison with the authored Nat interpretation, or a refinement of the compiler/native C.

Native primitive recursion. The MM0 extension defines the least binary graph for a start value and a result-before-predecessor step method. Existing natural induction proves existence; constructor separation and injectivity give zero uniqueness, successor inversion and functionality. The existing choice rule then selects a graph value and yields the zero and successor equations. The universally quantified recursion theorem retains an arbitrary result type. No natural-number fact is added as an axiom. This proof uses the existing choice principles during totality as well as during final selection; its existence proof is not claimed choice-free.

An independent intrinsic formula prints the final statement. Its standard interpretation agrees with the shared natural fact, whose numeral wrapper is interpreted as identity. This is a proved comparison of those stated interpretations, not a syntactic identification or an arbitrary-Henkin-model

comparison. A nonempty natural-number instance satisfies the formula; a fixed-point zero/successor carrier refutes it.

Lean. `MM0.NaturalRecursion.formula_meaning`, `MM0.NaturalRecursion.shared_fact_meaning`, `MM0.NaturalRecursion.standard_natural_recursion`, `MM0.NaturalRecursion.fixed_zero_refuses_recursion`, `MM0.NaturalRecursion.all_verdicts`

The maintained native gate compares twelve added public statements and two definitions against the independent reference. Both optimized and ASan/UBSan configurations pass 74 controls per dialect on the same 195,296-byte artifact. Altered graph output, reversed method arguments, changed selection, missing functionality evidence and forged replay conclusions refuse. The older closure, induction, carrier and subtype profiles retain their artifact identities and controls. Sanitizer leak detection was disabled. The complete theorem retains the existing HOL choice, infinity and type-definition assumptions; dependency inspection does not establish compiler/native-C refinement or complete authored-source transport to this native carrier.

Authored-carrier comparison for MM0. The independent intrinsic graph prints the exact native definition and denotes the finite inductive recursion graph in the declared standard interpretation. Its value is exactly the existing authored recursor's value, with result/predecessor order transposed once and natural indices transported through the carrier comparison. Any selector satisfying the explicit selection law returns that value. An incorrect successor output is rejected on the same well-formed carrier. The guarded native induction sequent retains its two typing hypotheses and its two logical premises. The constructor, induction, recursion and infinity readings agree with the five shared natural laws on the authored interpretation. Named injection and surjectivity predicates are accounted for by genuine closed-definition reducts, whose satisfaction comparison holds in arbitrary Henkin models. The graph and shared-law comparisons retain their stated standard-model scope. This connects selected native claim forms to source meaning; full proof extraction, soundness of the entire supplied HOL presentation and linked-C refinement remain separate.

Lean. `MM0.NaturalFactsMeaning.graph_definition_prints`, `MM0.NaturalFactsMeaning.graph_meaning`, `MM0.NaturalFactsMeaning.induction_statement_prints`, `MM0.NaturalFactsMeaning.infinity_definitions_meaning`, `MM0.NaturalFactsMeaning.shared_fact_meaning`, `IndexedInductiveNaturalMM0.source_graph_iff`, `IndexedInductiveNaturalMM0.source_selector_agrees`, `IndexedInductiveNaturalMM0.wrong_successor_output_refused`

An explicit Megalodon natural profile. The axiom-free Mathdata development proves recursion conditionally from constructor separation, injectivity and a selector law. A separate optional profile discharges those interfaces with one polymorphic selector parameter and exactly three named axioms: its selection law, double-negation elimination and Dedekind infinity. These are admitted by an explicit import policy; the Egal profile tag contributes no axioms. Removing any one refuses import.

The profile derives selected zero and successor operations, constructor separation, injectivity, guarded induction, non-surjectivity and polymorphic recursion on the generated carrier. Its independent Lean interpretation selects the existing authored successor as the infinite injection and the existing zero as the omitted point. Those selected operations therefore agree with the authored carrier, and the graph-selected function agrees with its maintained eliminator at every result type. Singleton infinity and a constant successor provide negative controls.

The source exporter is separated from optional Proofgold metadata: polymorphic Mathdata definitions and explicitly identified polymorphic parameters export when only the native source stream is requested, while Proofgold or HTML requests retain their prior refusal. Earlier natural proof streams remain byte-identical, malformed proofs refuse, and native HE/PeTTa replay checks every exported theorem. This is one optional classical theory node, not a default kernel extension, HOTG import, parametricity result or whole-compiler refinement.

Lean. `Megalodon.NaturalTheoryProfile.successor_ne_zero`,
`Megalodon.NaturalTheoryProfile.successor_injective`,
`Megalodon.NaturalTheoryProfile.recursion`,
`Megalodon.NaturalTheoryProfile.unit_refuses_infinity`,
`IndexedInductiveNaturalMegalodonProfile.selected_successor_eq`,
`IndexedInductiveNaturalMegalodonProfile.selected_zero_eq`,
`IndexedInductiveNaturalMegalodonProfile.source_recursor_agrees`,
`IndexedInductiveNaturalMegalodonProfile.constant_successor_refused`

Direct raw authority for the selected checking policy. The existing raw generation and Pi conversion boundary recover dependent arguments at a declaration-rooted spine. The exact conversion/cumulativity tail transports a replacement at its principal type to the displayed type. No formation premise or raw-to-formed upgrade is needed. The formed spine now shares this analysis through its raw view, retaining formation-sensitive replay for its stronger judgment.

Lean. `HasType.DeclarationSpine.recoverApplication`,
`HasType.DeclarationSpine.recoverTelescope`, `FormationSensitive.DeclarationSpine.toRaw`

For the two original natural iota rows, that recovery proves uniform raw preservation in arbitrary contexts, retaining all eight declarations. The actual recursor methods and recursive call are typed by the existing telescope-application theorem. Separately, the maintained elaborator and replay checker certify all eight declaration types with every iota rule disabled. Their formation evidence transports to each original permission selection. Formation plus raw preservation constructs the existing checked signature for the natural-only policy; the original dependent append is well typed there. Actual zero and successor redexes supply positive witnesses, while missing declarations and a lower claimed universe refuse.

Lean. `NaturalConversion.RawPreservation.declared_preserves`,
`NaturalConversion.RawPreservation.Formation.signature_formed`,
`NaturalConversion.RawPreservation.checkedSignature`,
`NaturalConversion.RawPreservation.checked_append`,
`NaturalConversion.RawPreservation.actual_zero_preserved`,
`NaturalConversion.RawPreservation.actual_successor_target`,
`NaturalConversion.RawPreservation.missing_spine_refused`,
`NaturalConversion.RawPreservation.all_verdicts`

This authority concerns the selected source signature. It does not authorize the original four-row family candidates, arbitrary vector contextual computation, richer annotated equality or a native C checker. The earlier missing-domain distinction between raw and formed typing remains valid.

Committed-opt qualification of the selected Nat/Vec chain. The unchanged joined driver now passes all twenty stages with no open qualification obligations at its recorded artifact cohort. Normal and ASan/UBSan executables come from committed opt revision `dcb7dffad6bd`. Both 1,024-round array-lifetime panels and injected allocation-failure panels pass the stated fail-stop contract. All three native-fact routes, changed-evidence refusals, length/counter observations and three fixed-point runs pass. The complete frozen Lean import cohort remains unchanged through the run; its audit checks 164,556 safe declarations with standard axioms only. This qualifies the selected declared chain. General native checker refinement and fact generation for every admitted signature remain separate obligations.

13.10 A Lean-family branch: exact checking, reconstruction and model soundness

Lean compatibility does not form one vertex. The official Lean kernel, Lean4Lean, con-leche and a Lean4Less translation expose different judgments and different theorems about them. They belong to a connected family whose edges state their actual strength.

The local authority interface distinguishes four public outcomes: acceptance with a retained output, rejection of an invalid proposal, decline on an unsupported feature, and operational error. Only acceptance carries an authority theorem. A model-sound service maps every accepted output to a semantic realization. Therefore a model-forbidden input cannot be accepted, without implying whether it is rejected, declined or encounters an error. An acceptance refinement is one-way: every optimized acceptance maps to a pure-checker acceptance. Such refinements compose, and pure-checker model soundness transports through them. Neither direction supplies completeness.

Lean. `LeanCheckerAuthorityFlower.Service.ModelSound.not_accepts_forbidden`, `LeanCheckerAuthorityFlower.Service.AcceptanceRefinement.modelSound`, `LeanCheckerAuthorityFlower.Service.AcceptanceRefinement.comp`

Checked reconstruction gives the elaboration edge its authority. An untrusted producer may propose a core certificate; publication occurs only after the destination checker recomputes it. The resulting four-outcome service accepts exactly the certificates published by the existing reconstruction combinator, and destination soundness alone transports each accepted result to its meaning or model. No theorem about the producer is needed. A failed producer is a decline and a checker refusal is a rejection.

Lean. `LeanCheckerAuthorityFlower.Service.checkedReconstruction_accepts_iff`, `LeanCheckerAuthorityFlower.Service.checkedReconstruction_sound`, `LeanCheckerAuthorityFlower.Service.checkedReconstruction_modelSound`

The distinctions are strict. A nonempty sound service may decline a true input, so soundness does not give completeness. Two implementations may agree on every result while both accept an invalid claim. Refusing one chosen contradiction also does not imply soundness for all claims. Conversely, a finite nonvacuous model-sound service accepts its modeled claim and refuses a model-forbidden one. These controls prevent differential testing or a single consistency canary from being promoted to semantic authority.

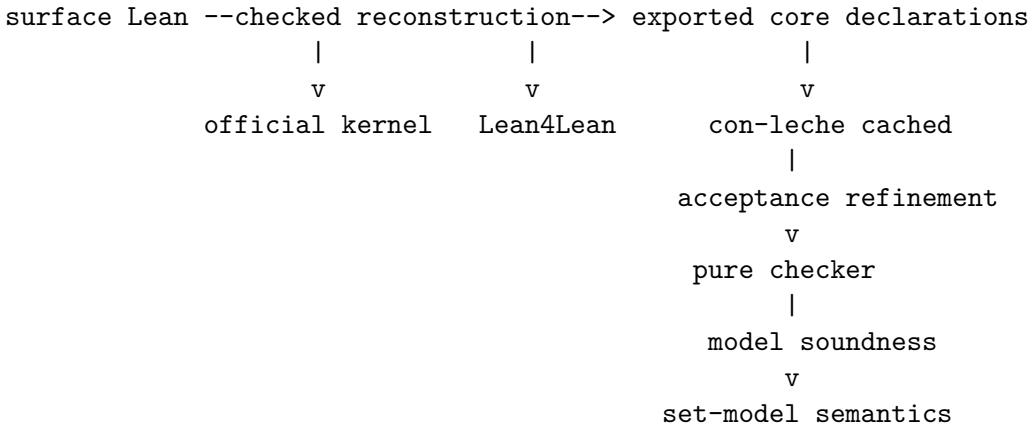
Lean. `LeanCheckerAuthorityFlower.Service.Controls.sound_does_not_imply_completeness`, `LeanCheckerAuthorityFlower.Service.Controls.agreement_does_not_imply_soundness`, `LeanCheckerAuthorityFlower.Service.Controls.false_refusal_does_not_imply_soundness`, `LeanCheckerAuthorityFlower.Service.Controls.finiteModelService_refuses_forbidden`

The already implemented finite-catalog Lean fragment supplies an exact lower node: executable beta, delta and head-application acceptance agrees with its authored raw GSLT and the generated OSLF exact-target predicate. The existing typed-boundary counterexample still shows that this raw node is not a Lean typing checker. Thus the flower has a proved exact operational edge without calling it the full Lean kernel.

Lean. `LeanCheckerAuthorityFlower.Service.catalogHeadService_accepts_iff_ntt`, `Lean4LeanRungTwoTypedBoundary.raw_accepts_while_typed_ntt_rejects`

Con-leche supplies a stronger external semantic pattern [17]. Its verified mode checks exported Lean declarations using a cached implementation; accepted cached runs simulate a pure checker, whose accepted environments are proved to have a model. The model interprets the pinned `False` as the empty set and equality as set equality, yielding the checked refusal of an export containing a theorem of `False`. The result is relative to the project's `SetTheory` interface: ZF without infinity and choice, together with an omega-chain of Grothendieck universes; the Lean metalogic supplies choice. Lean 4.35 distributes con-leche as an external checker. The exported model theorem is deliberately one-way. It does not prove equality with the official kernel's acceptance relation, and its simplified public model omits delta and iota equations. Computational adequacy and official-kernel parity therefore remain separate edges.

This suggests the following Lean branch rather than a single "Lean profile":



Lean4Lean remains useful for the independently stated typing and definitional equality judgments and for implementation comparison [16]. Its locally pinned metatheory still leaves whole-inductive implementation verification unfinished, so no full-Lean exactness edge is inferred here. Con-leche instead gives a complete acceptance-to-model route for its own verified checker, including a uniform positivity analysis for mutual and nested inductives and checker-generated recursors. For the present M2 goal, that design motivates four separately checked components: block recognition, positivity and uniform-parameter admission, recursor generation, and a least-fixed-point semantic law. Reusing its architecture does not import its acceptance relation or select it as a foundation.

The release-receipt boundary now binds the complete export stream and import closure, toolchain and exporter revisions, checker family, source revision and mode, selected profile and assumptions, primitive pins and executable digest. The adapter recomputes the observed coordinate. Invalid coordinates are errors; a different valid coordinate declines as inapplicable. Only an underlying accepting outcome produces a receipt, and ordinary or model soundness reaches it solely through the underlying checker's theorem. The pinned con-leche decoder retains exit zero with its declaration count as acceptance, exit one as rejection, exit two as decline, and malformed or error results as operational errors. Verified and trusted modes cannot reuse one another's receipts, nor can the official kernel reuse a con-leche receipt. Matching digests therefore make an already checked theorem applicable; they do not create a proof, establish official-kernel parity or prove that an input observer hashed the intended bytes. That last statement is the concrete adapter's correspondence obligation. Portable subfragments may additionally be rechecked by HOL Light, Megalodon or MM0, while full Lean terms remain native to a Lean-family checker.

Lean. `LeanCheckerReleaseReceipt.ReleaseBoundService.accepts_iff`,
`LeanCheckerReleaseReceipt.ReleaseBoundService.sound`,
`LeanCheckerReleaseReceipt.ReleaseBoundService.modelSound`,
`LeanCheckerReleaseReceipt.ReleaseBoundService.run_eq_declined_of_valid_mismatch`,
`LeanCheckerReleaseReceipt.conLecheVerifiedChecker_valid_iff`,
`LeanCheckerReleaseReceipt.Controls.trusted_mode_cannot_reuse_verified_receipt`,
`LeanCheckerReleaseReceipt.Controls.checker_family_cannot_reuse_receipt`

13.11 Four inductive petals and agent-facing data

The M2 inductive path now separates the four components suggested by the external Lean-family comparison. First, an executable recognizer retains an ordered constructor block. Second, successful recognition produces strict positivity and uniform-parameter evidence: each direct or nested recursive occurrence repeats the canonical block parameter vector. A nested occurrence is accepted only beneath a named positive container supplied by the selected registry. Function fields, unknown containers, changed parameter vectors and duplicate constructor names refuse.

Lean. `PositiveInductiveBlock.RawCode.positive_of_decode`,
`PositiveInductiveBlock.recognize_positive`,
`PositiveInductiveBlock.RawCode.functionField_refused`,

PositiveInductiveBlock.RawCode.changedParameters_refused,
PositiveInductiveBlock.RawCode.unknownContainer_refused

Third, the recognized block compiles to an indexed polynomial. Its ordinary recursor and dependent eliminator are generated from the polynomial fixed family; they are not accepted from the declaration. Both compute on every recognized constructor layer. Fourth, initiality proves every constructor-preserving map equal to the generated recursor, and displayed initiality gives the corresponding uniqueness theorem for dependent elimination. These are semantic computation laws. A selected DTT profile must still form the declarations and decide which iota equations are judgmental.

Lean. PositiveInductiveBlock.Admitted.recursor_construct,
PositiveInductiveBlock.Admitted.recursor_unique,
PositiveInductiveBlock.Admitted.eliminator_construct,
PositiveInductiveBlock.Admitted.eliminator_unique

Guarded representations and recursive spaces. A source family need not inhabit every index. An unindexed raw tree therefore retains its original index and constructor shape, with a recursive validity predicate. A partial position code also checks canonical padding outside its image: valid required children do not justify an extra stored child. Encoding and proof-directed decoding are inverse, and validity is exactly least constructor-closed membership. Direct positions and finite-to-natural slots instantiate one construction.

Lean. IndexedPolynomial.GuardedRaw.algebraRepresentation,
IndexedPolynomial.GuardedRaw.member_iff_valid,
IndexedPolynomial.SlotEncoding.Controls.tail_refused

The common recursive-space presentation takes an indexed polynomial, natural slot codes and a constructor-tag map. Its image retains the complete shape, result index and child function. Both inverse laws and constructor coherence hold. First-order inventories use the source-order finite codec; registered codes and complete registered blocks reuse their existing ordered-position equivalence. Natural, Vector and SearchTree controls retain values, order and arity. Changed tags, absent payloads, hidden children and changed Vector arity have separating refusals. These are semantic representations; registered ordered finitarity does not provide an executable decoder.

Lean. IndexedPolynomial.RecursiveSpaceRepresentation.recursiveAlgebraRepresentation,
IndexedPositiveContainer.RecursiveSpaceRepresentation.blockRepresentation,
IndexedInductive.RecursiveSpaceControls.Registered.relational_order_retained,
IndexedInductive.RecursiveSpaceControls.Registered.vector_three_third_present

The relational construction underlying HOL Light's recursive space is also formalized. Its numerical pairing and relational injections construct the least generated subtype, retaining the arbitrary bottom filler selected by the source choice term. Abstraction/representation laws, constructor separation, injectivity and full predicate induction supply the recursive-space algebra; its least membership agrees with the existing HOL fixed-point syntax in a full-function-space model. Those algebraic laws yield the polynomial comparison. Constructor-only recursion leaves its bottom result unspecified; a supplied bottom gives a unique recursion function.

Lean. HOLLight.RecursiveSpaceSemantics.represent_abstract,
HOLLight.RecursiveSpaceSemantics.model,
HOLLight.RecursiveSpaceSemantics.predicateMember_iff_rawMember, RecursiveSpace.equivalence,
RecursiveSpace.recurse_unique, RecursiveSpace.constructor_recursion_not_unique

First-order constructor data has a fixed product type independent of the parameter environment. The dependent sum of its indexed shapes is equivalent to that product, and the original expression interpreter computes the result and child indices. The same target recursive-space algebra can consequently serve every environment. A changed environment may require a different result index for the same data tuple: the context remains part of the validity contract. This semantic equivalence concerns host equality in the declaration description; it supplies no erasure theorem for object proof

syntax.

Lean. `IndexedPolynomial.Description.constructorDataEquiv`,
`IndexedPolynomial.Description.recursive_index_from_data`,
`IndexedPolynomial.Description.DataRepresentation.encode_constructor`,
`IndexedInductive.ConstructorDataControls.wrong_parameter_index_refused`

Separately, the actual HOL Light recursive-space certificates replay on the selected native checker: 78,963 retained theorem outputs pass exact conclusion and axiom-dependency checks. Standard-policy acceptance, sealed-policy refusal and a forged-conclusion refusal pass. The retained history explicitly declares `ETA_AX`, `SELECT_AX` and `INFINITY_AX`; individual proof masks do not erase surrounding declaration commitments. A Prime curriculum consumer retrieves the four opaque constructor/induction/recursion facts through the kernel service and rejects forged conclusions and printed authority seals. The complete closure also passes the typed registered source receiver: 78,962 source events give 78,960 registered commands and 78,963 theorem outputs, including the actual `prod`, `num` and `recspace` admissions. A separately selected profile fixes the primitive basis and the three permitted axioms. Changed type conclusions, missing existence parents, repeated names, unlisted axioms and malformed source inputs are refused. The indexed adapter remains an untrusted source producer. Typed reception does not itself prove original-source reading adequacy, identify the interpretation with the relational model or supply Prime-space type-definition transport.

For constant-only reference definitions, the actual receiving computation now constructs the original chronological model history. Its ordered outputs are the admitted definition equations, and the constructed model satisfies every equation at every source type instance. Carriers and full function domains are preserved. A closed two-definition document supplies nonempty applications; a changed output and a repeated source name are refused. This construction validates admitted definition equations, not the truth of a profile's extra axiom commitments. Carrier-changing type definitions, the full recursive-space model correspondence, complete native proof production and implementation refinement remain separate obligations.

Lean. `HOLLight.DefinitionReplayModels.replay_constant_history`,
`HOLLight.DefinitionReplayModels.replay_constant_models`,
`HOLLight.DefinitionReplayModels.loaded_reference_models`,
`HOLLight.DefinitionReplayModels.Controls.profile_definitions_have_model`,
`HOLLight.DefinitionReplayModels.Controls.forged_reference_refused`

The source receiver's existing schema matcher also supplies complete type instantiation certificates. Consistent repeated parameters and identity entries removed during normalization retain the required substitution action. The independent final instance check rejects incompatible repetitions. Annotation, sequent, theorem and inference-parameter formation checks are exact with this executable finder, without assuming a certificate-selection family. These are formation results, not completeness of proof search or implementation refinement.

Lean. `HOLLight.TypeInstantiationCertificates.instanceCertificates_complete`,
`HOLLight.TypeInstantiationCertificates.constantInstanceB_exact`,
`HOLLight.TypeInstantiationCertificates.inferenceFormationCheck_exact`,
`HOLLight.TypeInstantiationCertificates.Controls.inconsistent_not_instance`

A generic retained pointed witness now specializes through the original native `INST_TYPE` and `INST` operations. For arbitrary source carrier types and predicates, the computed conclusion is the exact pointed-predicate application, with empty hypotheses and no native logical axiom dependencies. The original predicate need not be inhabited. Empty-predicate and function-carrier instances each export 569 primitive nodes; open and sealed native replay accepts them and refuses changed conclusions. General registered type introduction retains its separate formation, closedness, freshness and definition-equation obligations.

Lean. `HOLLight.PointedPredicateInstantiation.specializedProof_accepted`,
`HOLLight.PointedPredicateInstantiation.specialized_conclusion`,
`HOLLight.PointedPredicateInstantiation.specialized_hypotheses`,
`HOLLight.PointedPredicateInstantiation.specialized_axioms`,

HOLLight.PointedPredicateInstantiation.Controls.changed_conclusion_refused

The registered substitution producer checks every stored maplet, including an unused entry. From a formed input theorem and formed stored maplets, the actual receiver constructs its two-command execution, appending two theorem outputs and two receipts. Earlier proof addresses and source derivations are preserved. A closed example changes both a carrier type and a free term; unused undeclared type and term maplets are refused. Its execution has a path in the existing registered GSLT and the existing OSLF theorem-reachability readout. This observation concerns support; occurrence-sensitive observations, scheduling and native implementation costs remain separate.

Lean. HOLLight.DefinitionAdapters.TheoremFormed.substitute,
HOLLight.RegisteredSubstitution.apply_available,
HOLLight.RegisteredSubstitution.preserves_lookup,
HOLLight.RegisteredSubstitution.preserves_derivations,
HOLLight.RegisteredSubstitution.execution_path,
HOLLight.RegisteredSubstitution.Controls.final_observed

One cached generic proof now has independent primitive replay, exact endpoint and registered-formation proofs. The shared proof-graph reception construction also proves the converse: successful reception gives the same primitive graph and its actual formation obligations. An undeclared constant separates the two checks. Signature extension transports the cached formation results through the complete executable instance finder. Consequently the registered witness producer requires only a formed carrier and a formed expanded predicate in a signature extending the primitive core. It returns the original compiler theorem at its exact address, without a per-node witness-formation premise or inhabitance premise for the original predicate. Registered introduction now composes this reception with actual source type admission. The admitted signature's instance certificates check both the original inputs and the newly registered maps; successful admission constructs the same definition for either successful provider. The existing structural formation theorem constructs both pointwise laws. Fresh names and predicate closedness remain explicit inputs, while equation formation is not a caller premise. The final command retains two new theorem addresses and one ordered receipt, preserving source derivations.

Lean. HOLLight.PointedPredicateTemplate.accepted,
HOLLight.PointedPredicateTemplate.exact_conclusion,
HOLLight.PointedPredicateTemplate.nodes_formed,
HOLLight.ProofExecution.Registered.replayProofs_iff,
HOLLight.RegisteredPointedPredicate.replay_available,
HOLLight.RegisteredPointedPredicate.replay_preserves_derivations

Lean. HOLLight.RegisteredPointedTypeDefinition.introduce_available,
HOLLight.RegisteredPointedTypeDefinition.introduce_roots,
HOLLight.RegisteredPointedTypeDefinition.introduce_derived_laws,
HOLLight.RegisteredPointedTypeDefinition.finish_counts

Constructed mixed and universal-predecessor histories already supply the carrier-changing model installation. Registered replay now connects each actual proof, constant and type command to that semantic family machinery. The type existence licence follows from the earlier cached theorem's proof. Each declaration constructs its new interpretation and schemas and proves preservation of the preceding licensed schemas. The empty-policy replay theorem supplies its own initial frame and constructs a full-domain model of the received cache, retaining each sequent's hypotheses. The actual pointed producer has one such model in which both returned hypothesis-free laws hold. The retained η/choice/infinity profile now has an interpreted reference initialization. One coherent primitive family fixes the individual carrier to lifted natural numbers, follows the nine original Boolean definition admissions, and validates the actual formed, closed source commitments. The reference replay retains nine cached equations and nine command receipts. Adjoining the three interpreted schemas enlarges the selected policy while retaining the model. Every accepted registered proof, constant or type-definition replay after this reference state therefore has a constructed full-domain model, with source sequent hypotheses retained. A finite individual carrier cannot sup-

ply the infinity witness. The complete original interleaved source history still needs its own model correspondence; the nine-definition reference route is not that original trace. Identification with the particular recursive-space relational model and native implementation refinement also remain separate.

Lean. `HOLLight.StandardTheoryInterpretation.referenceHistory_model`, `HOLLight.SourceProfileModels.reference_replayed`, `HOLLight.SourceProfileModels.reference_models_eta`, `HOLLight.SourceProfileModels.reference_models_selection`, `HOLLight.SourceProfileModels.reference_models_infinity`, `HOLLight.SourceProfileModels.replay_models`

Lean. `HOLLight.InferenceInterpretation.inference`, `HOLLight.RegisteredReplayModels.replay_models`, `HOLLight.RegisteredReplayModels.Frame.cached_falsity_refused`, `HOLLight.RegisteredPointedModels.introduce_laws_models`, `HOLLight.RegisteredPointedModels.introduce_available_models`

The relative transitivity control concludes $x = z$ from $x = y$ and $y = z$ while retaining both source assumptions. Its native certificate is accepted under the standard policy, but the same conclusion with those hypotheses dropped is refused. Standard-policy acceptance is consequently distinct from an empty source hypothesis context.

Strict positivity does not imply finite execution. The positive Reader container has a natural-number position at one shape and is not finitary. Accordingly a separate registry capability supplies a finite position type at every registered shape. The list registry used below has that capability. This distinction keeps infinitary semantics from silently becoming a finite MeTTa traversal.

Lean. `PositiveInductiveBlock.Code.positionFintype`, `IndexedContainer.Examples.reader_not_finitary`

Finite cardinality alone still does not select an execution order. The indexed container layer now separates an ‘OrderedFinitary‘ capability: each shape has an explicit complete, duplicate-free position list. List and Vector supply their ordinary ‘Fin‘ order; Reader cannot, because ordered finitarity implies finitarity. Registered nested code inherits the capability. Its position enumerator follows constructor-field order: a direct recursive field first, then each nested container in its declared order, then the remaining fields. Every semantic position occurs and the resulting list has no duplicates. This enumeration’s length is therefore exactly the finite cardinality of the semantic position fibre, giving the theory-side work count. This is the interface needed before an emitted traversal may claim source ordering or a per-position cost.

Lean. `IndexedContainer.Examples.listOrderedFinitary`, `IndexedContainer.Examples.vectorOrderedFinitary`, `IndexedContainer.Examples.reader_not_orderedFinitary`, `IndexedPositiveContainer.Registry.OrderedFinitary.toFinitary`, `IndexedPositiveContainer.Code.mem_orderedPositions`, `IndexedPositiveContainer.Code.orderedPositions_nodup`, `IndexedPositiveContainer.Code.orderedPositions_length`, `IndexedPositiveContainer.ListInstance.orderedFinitary`, `IndexedPositiveContainer.VectorControl.orderedFinitary`

The established unindexed registered-container grammar now embeds into the indexed registry with a complete signature comparison. Constructor data, recursive-position fibres and each required child-family index are preserved; the selected ordered position lists agree occurrence by occurrence. The comparison applies to every registry-coherent source block and retains its constructor occurrence ordinals. A generic polynomial theorem transports constructor layers naturally in the recursive carrier, induces an equivalence of the complete fixed families, and proves that folds commute with both directions under transported constructor methods. For the admitted SearchTree block, every tree retains its answer-collecting recursion, and the images of solved false and solved true remain distinct. A mutual Workflow control checks a child index different from its root. These are seman-

tic transport laws; open-term guest interpretation and complete runtime recursor compilation are separate obligations.

Lean. `IndexedPolynomial.SignatureEquiv.extensionEquiv_natural`, `IndexedPolynomial.SignatureEquiv.fixEquiv`, `IndexedPolynomial.SignatureEquiv.fold_toFix`, `IndexedPolynomial.SignatureEquiv.fold_fromFix`, `IndexedPositiveContainer.Unindexed.signatureEquiv`, `IndexedPositiveContainer.Unindexed.ordered_observations_preserved`, `IndexedPositiveContainer.Unindexed.blockRecursor_preserved`, `ModuleSource.PositiveInductiveLoweringControls.WholeFamily.answers_preserved`, `ModuleSource.PositiveInductiveLoweringControls.WholeFamily.distinct_solved_answers`, `ModuleSource.PositiveInductiveLoweringControls.MutualIndex.child_index_ne_root`

The indexed machine formation/focus regression now has an explicit compiled Lean command, `nativeIndexedMachineFormationCheck`. It executes the maintained formation action on the original PartrecMachine and checks all 110 generated declaration rows, including the existing rejection controls. Library compilation retains the action without executing this runtime panel. The recorded operational qualification is separate from the kernel-checked theorems about the generated judgments.

The authored cube panel likewise has an explicit compiled command, `nativeAuthoredCubeCheck`. The twenty existing batch modules retain their original actions. The command checks their declared index order and executes the complete schedule, preserving the fixture-coverage checks and the actual native certificate checks. At the recorded cohort, the three fixtures contain 175 formers in total; all twenty batches and sixty coverage checks pass. These runtime actions are retained by library imports and executed at qualification, separately from mathematical proof checking.

The selected order now reaches a quoted MeTTa traversal program. The program walks a captured constructor list and emits one `Visited` record per input occurrence. Its execution theorem in the existing PeTTa machine covers arbitrary payload atoms, retaining executable-looking syntax as data. Every completed run returns exactly the one ordered result, with no additional answers, store changes or I/O. The source adapter supplies the exact position list from the selected indexed-container code. Output length equals the semantic position cardinality; for a fixed shape, equality of complete outputs is equivalent to pointwise equality of the payload observations. A control uses the actual semantic erasure of the resolved textual VectorTree branch. Missing repeated occurrences, reordered outputs and zero-fuel completion are negative controls. This realizes traversal of already represented positions; interpreting arbitrary open guest terms and compiling arbitrary authored recursor clauses remain further edges.

Lean. `IndexedContainerTraversal.returns`, `IndexedContainerTraversal.completed_iff`, `IndexedContainerTraversal.ordered_returns`, `IndexedContainerTraversal.ordered_record_count`, `IndexedContainerTraversal.ordered_output_eq_iff`, `IndexedContainerTraversal.Controls.textualVectorInput_two`, `IndexedContainerTraversal.Controls.model_checks`

List and Vector container values now have a shared physical algebraic-list carrier, with exact decoding in both directions relative to the element codec. The Vector decoder retains the declared output length: a changed length, a dropped child, an ill-typed payload or malformed list syntax refuses. Equal payloads at different positions remain separate entries. The execution adapter sends these actual container extensions to the source-pinned visitor; it does not merely accept an unrelated list with the same cardinality.

Lean. `IndexedContainerWire.decodeListValue_encodeListValue`, `IndexedContainerWire.encodeListValue_decodeListValue`, `IndexedContainerWire.decodeVectorValue_encodeVectorValue`, `IndexedContainerWire.encodeVectorValue_decodeVectorValue`, `IndexedContainerWire.wrong_vector_length_refused`, `IndexedContainerTraversal.ContainerValues.vector_value_completed_iff`

The next source program performs a complete bottom-up fold. Executable structural encoding of

an indexed-polynomial fixed family agrees with the independent generated semantic fold. Faithful shape encoding and complete position coverage make the tree encoding injective. The execution theorem lifts correctness of a method on one constructor layer to the complete fixed family, folding every child in the selected order before calling the method. Its premise describes that one layer, not an already-correct recursive call. The admitted SearchTree instance discharges the premise for the existing answer-collecting algebra. Thus every semantic SearchTree has the same ordered solved answers in the quoted PeTTa program; any completed run has exactly that one result, with no store or I/O effects. Distinct input trees remain distinguishable at the quotation boundary even when their answer lists coincide.

Lean. `IndexedContainerWire.Recursive.encodeFix_eq_fold`, `IndexedContainerWire.Recursive.encodeFix_injective`, `IndexedContainerTraversal.Folding.fold_returns`, `IndexedContainerTraversal.Folding.Search.answers_return`, `IndexedContainerTraversal.Folding.Search.answers_completed_iff`, `IndexedContainerTraversal.Folding.Search.quote_injective`

An execution probe exposed a source-emission error at computed calls. The formal evaluator treats a variable-headed tuple as data, whereas the selected native candidate dispatches it as an application. Following such a tuple with `eval` therefore executed a returned code payload twice in the native candidate. Emission now constructs the application explicitly with `cons-atom` and executes it once. Both the method proof and the diagnostic replay retain the original code-shaped result; neither runtime quotation policy nor evaluator implementation was changed. This is a qualified compilation fragment, not a claim of full correspondence between the formal evaluator and every native PeTTa expression.

The qualification driver distinguishes observed values from completed runs. Six Lean controls emit exact source and record minimal evaluator fuel. The retained SWI-PeTTa reference run emits all six expected packets, and the oracle refuses missing packets, duplicate packets, a dropped repeated occurrence, a reordered payload and unexpected additional stdout. Five reference cases complete. The 32-position case publishes its exact packet but does not terminate within the observation window. This is a reference-run completion divergence; production qualification requires a separate replay on a pinned CeTTa artifact in PeTTa mode. The source-machine counts are 17 transitions for the empty case, 125 for the two-position textual branch, 179 for either three-position case, 71 for a retained syntax payload and 1,745 for 32 positions. These are measured source-machine counts, not a universal runtime-cost theorem or a C performance comparison.

The pinned frozen-opt diagnostic now completes all six visitor controls and eight fold controls. The fold controls cover solved and empty constructors, order, repetition, nesting, depth sixteen, width thirty-two and a method returning executable-looking data. Their exact minimal source-machine counts are respectively 129, 126, 664, 933, 1,465, 4,385, 8,734 and 104. Five visitor output mutations and six fold mutations are refused. Each mutation starts from an independently accepted model-output baseline, so a wrong native baseline cannot make the negative controls vacuous. The candidate's executable hash is retained, but its committed-source build provenance has not been established; the driver therefore keeps these runs diagnostic and does not claim production release qualification or C implementation correctness.

The selected formed List and Vector providers now feed this traversal through an exact source-evidence comparison. Registered List-bundle lookup is proved equal to direct source-spine lookup, retaining the actual relation witness at each position. Vector entries retain an element and its induction witness; List entries retain source, target and relation evidence. The scoped binary term codec is injective, and the physical decoder retains the scope parameter. Decoding syntax alone grants no typing judgment. The observation list sent to the visitor is proved equal to the actual selected semantic bundle, and every completed model run returns exactly that ordered list.

Lean. `IndexedContainer.listInductionBundle_exact_cast`, `FormationSensitiveRegisteredListGuestLift.canonicalLiftRegisteredPositionEquiv_eq`, `FormationSensitiveBundleTraversal.selectedEntries_eq_entries`,

FormationSensitiveBundleTraversal.entries_completed_iff,
FormationSensitiveBundleTraversal.ListProvider.encodeFrame_injective,
FormationSensitiveBundleTraversal.ListProvider.selectedEntries_eq_entries,
FormationSensitiveBundleTraversal.ListProvider.entries_completed_iff

The List control uses a formed context containing two evidence variables of the same relation type. Its two source and target lists agree, but exchanging the evidence variables changes the exact output. Thus this adapter does not erase proof-relevant evidence under the guise of representation transport. All three native diagnostic cases complete, with six packet mutations refused. Each takes 125 source-machine transitions. The binary codec reduces the Vector control's serialized call from 526,927 to 2,241 characters and each List call from 880 to 380; the transition count is unchanged and does not measure the internal work of processing an atom. These are canonical finite source spines with scoped elements, not an interpretation of an arbitrary open List or Vector variable.

Lean. FormationSensitiveBundleTraversal.Canonical.formed,
FormationSensitiveBundleTraversal.Canonical.refuses_wrong_scope,
FormationSensitiveBundleTraversal.ListProvider.Controls.context_formed,
FormationSensitiveBundleTraversal.ListProvider.Controls.formed,
FormationSensitiveBundleTraversal.ListProvider.Controls.swapped_formed,
FormationSensitiveBundleTraversal.ListProvider.Controls.same_values_different_output

Two checked scenarios give the petals agent-facing content. The first is a nested search tree with either a solved Boolean answer or a registered list of recursive branches. Its generated recursor collects answers in source order; a two-child branch exposes two distinct recursive positions. The second is a Boolean-indexed mutual family of workflows and actions. Workflows may schedule actions; actions may contain subworkflows. Its generated recursor counts actions across both directions of the dependency.

Lean. PositiveInductiveScenarios.Search.source_positive,
PositiveInductiveScenarios.Search.answers_solved,
PositiveInductiveScenarios.Search.answers_branch,
PositiveInductiveScenarios.Search.nested_positions_distinct,
PositiveInductiveScenarios.Workflow.source_positive,
PositiveInductiveScenarios.Workflow.actionCount_schedule,
PositiveInductiveScenarios.Workflow.actionCount_action,
PositiveInductiveScenarios.Workflow.actionCount_subworkflow

The search tree now has an exact lowering from the shared ‘.module‘ syntax data produced by both readers. The selected fragment retains every field and constructor name, requires a registry witness for ‘List‘, and round-trips telescopes, the family and the one-family module shell. Its canonical text is:

```
Module AgentSearch {
  Theory Data() {
    Inductives {
      SearchTree parameters (Answer : Type) indices () : Type {
        solved (answer : Answer) : SearchTree Answer;
        branch (children : List (SearchTree Answer)) : SearchTree Answer;
      }
    }
  }
  theory Data()
}
```

The erased result is definitionally the independently specified nested search block above. An unknown container, a function field, a changed parameter header, duplicate constructor names and an extra module block all refuse.

Lean. `PositiveInductiveLowering.lowerCode_quote`, `PositiveInductiveLowering.quote_lowerCode`, `PositiveInductiveLowering.lowerFamily_quote`, `PositiveInductiveLowering.quote_lowerFamily`, `PositiveInductiveLowering.lowerModule_quote`, `PositiveInductiveLowering.quote_lowerModule`, `PositiveInductiveLoweringControls.Search.family_roundtrip`, `PositiveInductiveLoweringControls.Search.module_roundtrip`, `PositiveInductiveLoweringControls.Search.erased_family_eq_existing`, `PositiveInductiveLoweringControls.Search.unknown_container_refused`, `PositiveInductiveLoweringControls.Search.negative_field_refused`, `PositiveInductiveLoweringControls.Search.extra_block_refused`

The textual mutual route is now checked for the existing workflow/action scenario. Separate 'Workflow' and 'Action' headers share one resolver into the Boolean-indexed family: scheduling crosses from plans to actions and subworkflows cross back. Lowering additionally checks that every constructor returns the family named by its header and that constructor names are unique across the whole block. The accepted two-family module round-trips exactly and erases definitionally to the independently specified mutual polynomial. Swapped family order, a Workflow constructor returning Action, a name collision across headers and a negative function field are refused. This closes the source-to-semantic mutual join.

Lean. `MutualWorkflowActionLowering.lowerFamilies_quote`, `MutualWorkflowActionLowering.lowerModule_quote`, `MutualWorkflowActionLowering.Controls.erased_eq_existing`, `MutualWorkflowActionLowering.Controls.wrong_workflow_result_refused`, `MutualWorkflowActionLowering.Controls.duplicate_across_families_refused`, `MutualWorkflowActionLowering.Controls.swapped_families_refused`, `MutualWorkflowActionLowering.Controls.negative_field_refused`

The object guest consumes exactly that erasure through the already proved direct-polynomial bridge. It generates one Boolean-indexed family, four constructors and a dependent eliminator. Family tags use the existing natural zero/successor encoding; the authored permission payload remains a separate opaque object type, so the index representation is not mistaken for the data interpretation. The complete declaration library forms, and all four computation endpoints form before their own iota rules are enabled. The generated schema list retains four typed proof-relevant receipts in source order. 'done', 'schedule (subworkflow done)', 'subworkflow done', and 'action' at a typed permission variable check. 'schedule done' refuses because it supplies a Workflow where the constructor requires an Action. This establishes typed canonical computation receipts, not a semantic equivalence between the opaque permission type and Lean's Boolean type.

Lean. `PositiveInductiveWorkflowActionGuest.Source.textual_erasure_eq_direct_bridge`, `PositiveInductiveWorkflowActionGuest.signature_formed`, `PositiveInductiveWorkflowActionGuest.rows_checked`, `PositiveInductiveWorkflowActionGuest.schemas_length`, `PositiveInductiveWorkflowActionGuest.typedReceiptAt`, `PositiveInductiveWorkflowActionGuest.iota_rule_names`, `PositiveInductiveWorkflowActionGuest.verdicts_all`

Publication after direct structural admission is shared rather than rebuilt by each source route. The common producer elaborates the complete declaration prefix, checks global name separation and checks every generated computation row before returning a candidate. Its soundness theorem identifies the candidate's checker table with the complete generated table, while companion theorems retain the exact declaration signature and proof-relevant computation root. Textual source publication is admission followed by this producer. The independently admitted Vector family and the textual two-family Workflow/Action family both instantiate the boundary, and the latter's typed object guest consumes the published candidate's schemas. Zero fuel and aliased mutual constructor names are negative controls. This is a consolidation of publication and evidence flow; it is not a new positivity or normalization theorem.

Lean. `IndexedInductive.Guest.Package.fromAdmitted_sound`, `IndexedInductive.Guest.Package.fromAdmitted_signature`,

IndexedInductive.Guest.Package.fromAdmitted_computation,
IndexedInductive.Guest.PackageControls.admitted_vector_publication,
IndexedInductive.Guest.PackageControls.candidate_vector_table,
IndexedInductive.Guest.PackageControls.candidate_vector_signature,
IndexedInductive.Guest.PackageControls.candidate_vector_computation,
IndexedInductive.Guest.PackageControls.publishMutual_eq_admitted,
IndexedInductive.Guest.PackageControls.admitted_mutual_publication_zero,
IndexedInductive.Guest.PackageControls.admitted_aliased_names_not_published,
PositiveInductiveWorkflowActionGuest.candidate_table

Every admitted indexed family now has a generic selected HOL fact boundary. The selection is an equivalence from the actual tagged total fixed-point family to one HOL base carrier, retaining each family index as data. The generated HOL signature separates the carrier, a complete source constructor layer, the same layer with recursive children mapped to an arbitrary result carrier, and that result carrier. Its recursion sentence says that every mapped-layer algebra has a fold. Its no-confusion sentence says that the complete constructor-layer map is injective. Its induction sentence says that every predicate closed under complete constructor layers covers the carrier. The generated fold computes on each layer and is unique, while unequal authored constructor occurrences remain disjoint. All three structural sentences are valid in the selected full-domain model for every admitted family. Dedekind infinity enters the export manifest only with a separate injective endomorphism having an omitted point. The authored natural family supplies one and selects all four capabilities; no finite carrier, including lifted Boolean, can supply one. For Nat the manifest projects exactly to the existing five names and statement wires: recursion, induction, zero/successor separation, successor injectivity and infinity. Their exact formulas retain the existing authored natural interpretation. This generates semantic HOL formulas and reaches one maintained native statement producer. Producing native proofs for arbitrary admitted signatures remains a distinct stage.

Lean. IndexedInductive.HOLFacts.Interpretation.construct_ne_of_occurrence_ne,
IndexedInductive.HOLFacts.Interpretation.recurse_construct,
IndexedInductive.HOLFacts.Interpretation.recurse_unique,
IndexedInductive.HOLFacts.models_recursion, IndexedInductive.HOLFacts.models_induction,
IndexedInductive.HOLFacts.models_noConfusion,
IndexedInductive.HOLFacts.Selection.carrierInfinity_mem_iff,
IndexedInductive.HOLFacts.Selection.models_of_mem,
IndexedInductive.HOLFacts.Selection.infinity_not_selected_of_finite,
IndexedInductive.HOLFacts.NaturalControl.all_selected_models,
IndexedInductive.HOLFacts.NaturalControl.projectedNativeFacts_eq,
IndexedInductive.HOLFacts.NaturalControl.projectedStatements_eq,
IndexedInductive.HOLFacts.NaturalControl.projectedNames_eq,
IndexedInductive.HOLFacts.NaturalControl.projected_models,
IndexedInductive.HOLFacts.bool_has_no_infinityWitness

Constructor-specific no-confusion is generated without erasing constructor arguments. The projection enumerates every ordered pair of distinct authored constructor occurrences. Its structured pair name accompanies a HOL sentence quantifying two complete layers: if their tags are the selected occurrences, their constructed carrier values differ. Every generated sentence is valid under the selected carrier equivalence. The Nat control generates exactly `(Zero, Succ)` and `(Succ, Zero)`. A pair with equal occurrences is excluded at the generator type. Rendering this generic signature and producing native proofs for it remain separate from the exact maintained Nat statement route.

Lean. IndexedInductive.HOLFacts.ConstructorProjection.separation_meaning,
IndexedInductive.HOLFacts.ConstructorProjection.models_separation,
IndexedInductive.HOLFacts.ConstructorProjection.namedNoConfusions_models,
IndexedInductive.HOLFacts.NaturalControl.zeroSuccPair_models,
IndexedInductive.HOLFacts.NaturalControl.all_named_noConfusion_names

The structural and constructor-pair signatures now have typed HOL Light wire encoders. Explicit name records provide every base type and typed constant; emitted records retain the source for-

mula beside its wire. Mapping back to those retained sources recovers the full manifest in order, and each retained formula is valid in the selected model. The non-Nat mutual even/odd control emits recursion, induction and complete-layer no-confusion, omits infinity, and emits all six ordered separation pairs among `zero`, `odd` and `even`. This establishes generic claim emission. It does not yet prove acceptance by a native kernel.

The shared wire carrier now has executable structural equality. A finite catalog decoder is sound without a naming assumption: every returned source belongs to the catalog and re-encodes to the input wire. Roundtrip in the other direction requires injective statement encoding. All mutual structural and constructor-pair statements pass that roundtrip. A negative control maps every constructor occurrence to one name; the zero/odd and zero/even statements then collapse, and decoding the latter returns the earlier source. Thus collision freedom is a checked premise rather than an informal naming convention. Native proof production remains open.

Lean. `IndexedInductive.HOLFacts.NativeWire.Selection.emit_sources`,
`IndexedInductive.HOLFacts.NativeWire.Selection.emit_source_models`,
`IndexedInductive.HOLFacts.WireCatalog.decode_encode`,
`IndexedInductive.HOLFacts.WireCatalog.decode_sound`,
`IndexedInductive.HOLFacts.NativeWire.Selection.decode_statement`,
`IndexedInductive.HOLFacts.ConstructorProjection.emitAll_sources`,
`IndexedInductive.HOLFacts.ConstructorProjection.emitAll_source_models`,
`IndexedInductive.HOLFacts.ConstructorProjection.decodePair_statement`,
`IndexedInductive.HOLFacts.ConstructorProjection.decodePair_sound`,
`IndexedInductive.HOLFacts.MutualControl.emittedStructural_sources`,
`IndexedInductive.HOLFacts.MutualControl.emittedStructural_refuses_infinity`,
`IndexedInductive.HOLFacts.MutualControl.emittedStructural_source_models`,
`IndexedInductive.HOLFacts.MutualControl.emitted_names`,
`IndexedInductive.HOLFacts.MutualControl.emitted_source_models`,
`IndexedInductive.HOLFacts.MutualControl.structural_roundtrips`,
`IndexedInductive.HOLFacts.MutualControl.pair_roundtrips`,
`IndexedInductive.HOLFacts.MutualControl.colliding_occurrence_names_collapse_statements`,
`IndexedInductive.HOLFacts.MutualControl.colliding_occurrence_names_break_roundtrip`

A finite non-Nat family now has a proof-producing native route. The admitted two-nullary-constructor declaration is equivalent to Boolean data; its actual generated fold transports through an explicit classical equivalence to HOL's logical Boolean carrier. Independently typed claims state recursion, induction, constructor separation, cases and finiteness. The source family interprets all five. In arbitrary Henkin models of this signature, an explicit semantic Hilbert-choice family suffices to make the transported source fold admissible; the other four facts require no choice premise in this model interface. This is a sufficient condition, not a necessity or independence claim. The ordinary full-domain reading also covers an empty result carrier, where the recursion sentence's outer quantification is vacuous. The structural manifest omits infinity, and no infinity witness can exist for its finite carrier.

Lean. `IndexedInductiveBooleanHOL.ofBool_toBool`, `IndexedInductiveBooleanHOL.false_generated`,
`IndexedInductiveBooleanHOL.true_generated`,
`IndexedInductiveBooleanHOL.logicalRecursor_commutes`,
`IndexedInductiveBooleanHOL.NativeMeaning.all_models`,
`IndexedInductiveBooleanHOL.NativeMeaning.logicalRecursor_admissible`,
`IndexedInductiveBooleanHOL.NativeMeaning.all_models_of_choice`,
`IndexedInductiveBooleanHOL.infinity_not_selected`,
`IndexedInductiveBooleanHOL.no_infinity_witness`

Independent HOL Light proofs replay all five exact claims through the native service: 8,274 source proof nodes become 23,298 replay records in a 129,527-byte artifact. The closure uses only the existing eta and choice axioms; it adds no axiom and contains no infinity axiom. Changed claims, an unlicensed axiom and a truncated stream refuse natively, while missing, changed and repeated statement packets refuse at intake. The same shared gate retains the existing natural-law acceptance target. This qualifies the selected finite law interface against the pinned executable, not generic

proof production for every admitted signature, full source definition-history refinement, linked-C correctness or committed-source release provenance. No MeTTa proof evaluator substitutes for the native HOL checker.

The selected carrier comparison has an additional boundary: structural admission does not imply inhabitance. The actual admitted declaration $\mathsf{Loop} : T \to T$, with no base constructor, has an empty least fixed family. Its generated fold into the empty type proves that no source value exists. A faithful direct carrier equivalence into an inhabited simple type is therefore impossible. In particular, a semantic Hilbert-choice family supplies an inhabitant of every base domain and rules out such a direct interpretation. This obstruction concerns the selected carrier comparison, not positivity or consistency of the admitted declaration.

Lean. `IndexedInductive.HOLFacts.InhabitanceBoundary.source_admitted`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.source_empty`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.source_nonempty_of_choice`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.no_direct_choice_model`

A guarded reading handles this source boundary explicitly. For every admitted family, its tagged source carrier is equivalent to the guarded part of $\mathsf{Option}(\mathsf{ULift}(\mathit{Source}))$. The total carrier has a `none` value, while the guard accepts exactly the represented source values. Typed HOL universal and existential formulas, with their respective implication and conjunction guards, agree exactly with source quantification. The full-domain model admits a semantic choice family. For the recursive-only source, guarded existence of a true property is false, while unguarded existence is true; guarded universal falsity is true, while its unguarded version is false. Thus removing the guard changes the represented logic. Source functions have an exact comparison as well. Every total function between two admitted source carriers has one strict, guard-preserving extension between their Option carriers, and every such extension restricts back to that source function. Encoding preserves identity and composition. Typed HOL universal and existential quantifiers over the qualified function image agree with source-function quantification for an arbitrary host predicate pulled back along this encoding. Strictness specifies the selected behavior at the added invalid input; it is not asserted necessary for every interpretation. An explicit non-strict extension agrees with identity on all guarded values yet differs at `none`, so that behavior may be hidden only by an interface that excludes the corresponding observation.

Lean. `IndexedInductive.HOLFacts.InhabitanceBoundary.functionEquiv`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.mapSource_identity`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.mapSource_composition`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.functionsAll_iff`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.functionsEx_iff`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.fillMissing_agrees_on_guard`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.fillMissing_differs`

These are formula-level semantic comparisons in the declared choice fragment, not a complete native-theory model, automatic guarded proof emission or a runtime adapter.

Lean. `IndexedInductive.HOLFacts.InhabitanceBoundary.guardedEquiv`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.guardedChoice`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.guardedAll_iff`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.guardedEx_iff`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.guarded_exists_refuses`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.unguarded_exists_accepts`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.guarded_universal_vacuous`,
`IndexedInductive.HOLFacts.InhabitanceBoundary.unguarded_universal_refuses`

A selected HOL interpretation may change carrier representation along an explicit equivalence. Construction and mapped recursive layers commute with the carrier map, and the generated fold commutes on every source value. An infinity witness transports its injective endomorphism and omitted point, so the selected manifest is invariant. Carrier changes obey identity, composition and inverse laws, as does the retained infinity evidence. The Nat control transports the authored

polynomial carrier to lifted host naturals, material von Neumann naturals and natural surreals. The lifted material-to-surreal route closes the same triangle as the initial-algebra comparisons; the two routes select the same interpretation and fact manifest. Every fold agrees and recursion, induction, no-confusion and infinity remain valid. These are the natural subtypes, not arbitrary hypersets or all surreals. This is semantic transport: the chosen inverses need not be executable adapters, and no cost theorem follows.

Lean. `IndexedInductive.HOLFacts.Interpretation.transport_construct`, `IndexedInductive.HOLFacts.Interpretation.transport_mapLayer`, `IndexedInductive.HOLFacts.Interpretation.transport_recurse`, `IndexedInductive.HOLFacts.Interpretation.transport_refl`, `IndexedInductive.HOLFacts.Interpretation.transport_trans`, `IndexedInductive.HOLFacts.Interpretation.transport_symm`, `IndexedInductive.HOLFacts.InfinityWitness.transport_refl`, `IndexedInductive.HOLFacts.InfinityWitness.transport_trans`, `IndexedInductive.HOLFacts.InfinityWitness.transport_symm`, `IndexedInductive.HOLFacts.Selection.transport_refl_witness`, `IndexedInductive.HOLFacts.Selection.transport_trans_witness`, `IndexedInductive.HOLFacts.Selection.transport_symm_witness`, `IndexedInductive.HOLFacts.Selection.transport_facts`, `IndexedInductive.HOLFacts.NaturalControl.hostSelection_facts`, `IndexedInductive.HOLFacts.NaturalControl.host_recurse_commutes`, `IndexedInductive.HOLFacts.NaturalControl.host_all_selected_models`, `IndexedInductive.HOLFacts.NaturalControl.lifted_comparison_triangle`, `IndexedInductive.HOLFacts.NaturalControl.material_surreal_interpretation_route`, `IndexedInductive.HOLFacts.NaturalControl.material_surreal_selection_facts_route`, `IndexedInductive.HOLFacts.NaturalControl.materialSelection_facts`, `IndexedInductive.HOLFacts.NaturalControl.surrealSelection_facts`, `IndexedInductive.HOLFacts.NaturalControl.material_recurse_commutes`, `IndexedInductive.HOLFacts.NaturalControl.surreal_recurse_commutes`, `IndexedInductive.HOLFacts.NaturalControl.material_all_selected_models`, `IndexedInductive.HOLFacts.NaturalControl.surreal_all_selected_models`

The guarded word backend reaches the same selected HOL carrier through its exact observation. Both a successful `UInt64` branch and the unbounded fallback decode to the authored successor. A finite word alone cannot supply the infinity witness. The tagged word-or-Nat result is not injective as a carrier representation because its two branches may denote the same number; unchecked word successor also fails at the boundary. Separately, any exact-integer provider satisfying the existing roundtrip, successful-soundness and fallback-availability contracts computes natural addition by one at that host carrier. Its representation stays abstract. These results establish the theory contract, not a C/GMP proof or a cost comparison.

Lean. `NaturalBackendHOLFacts.successorBackend_host_exact`, `NaturalBackendHOLFacts.successorBackend_authored_exact`, `NaturalBackendHOLFacts.fast_branch_host_exact`, `NaturalBackendHOLFacts.fallback_branch_host_exact`, `NaturalBackendHOLFacts.finiteWord_has_no_infinityWitness`, `NaturalBackendHOLFacts.observeBackend_not_injective`, `NaturalBackendHOLFacts.unchecked_word_successor_not_host_exact`, `NaturalBackendHOLFacts.IntegerProvider.addOne_host_exact`

Typed nested elimination needs an additional edge. The current direct guest compiler allocates one recursive argument and induction hypothesis per direct recursive field. Treating the whole list as such a field would therefore give one recursive position. The genuine two-child nested branch has two positions; the two fibres are not equivalent, and neither are the complete constructor polynomials. The missing interface is a registered-container induction bundle: one motive witness for every actual container position. The List instance is proved equivalent to a dependent list of hypotheses in source order. For canonical typed List spines, the proof-relevant native `mapRel` family now realizes the bundle in object syntax: a retained target/evidence spine is equivalent to the actual registered List

induction bundle and hence to one typed pair per source position. Its encoded term forms in the existing guest calculus, and the resulting judgment is stable under every typed context substitution. The already formed native List eliminator handles arbitrary open List traversal.

The first complete nested guest slice now applies both pieces to the exact textual `SearchTree` source. Source-ordered compilation produces an opaque family, solved and branch constructors, and a dependent eliminator. The branch method receives a dependent sum containing a List of packed recursive results and a proof-relevant native relation spine connecting each packed result to its exact child. The native List eliminator constructs that bundle for an arbitrary open child List. Both solved and branch endpoints are independently replayed at their displayed dependent types before becoming proof-relevant iota receipts; typed substitution transports the complete branch receipt. A negative control removes the bundle and is refused. The signature remains a candidate: the canonical typed receipts are established, while a preservation theorem for every arbitrary raw iota instance is a separate authorization obligation. Accordingly, the safe receipt-only profile installs no judgmental roots. Raw iota evidence can still be built from five copies of an undeclared constant, while the corresponding canonical branch substitution is provably not typed. This witness makes the authority boundary executable: retaining a checked schema does not license every raw substitution.

Lean. `IndexedContainer.listAllEquiv`, `IndexedContainer.listInductionBundle`, `NestedInductiveGuestBoundary.Search.nested_position_card`, `NestedInductiveGuestBoundary.Search.direct_position_card`, `NestedInductiveGuestBoundary.Search.nested_positions_not_direct`, `NestedInductiveGuestBoundary.Search.nested_polynomial_not_direct_surrogate`, `NestedInductiveGuestBoundary.Search.listPositionBundle`, `FormationSensitiveRegisteredListGuestLift.canonicalLiftEquiv`, `FormationSensitiveRegisteredListGuestLift.canonicalLiftPositionEquiv`, `FormationSensitiveRegisteredListGuestLift.canonicalLiftSemanticBundleEquiv`, `FormationSensitiveRegisteredListGuestLift.canonicalLiftRegisteredPositionEquiv`, `FormationSensitiveRegisteredListGuestLift.QualifiedLift.judgment_and_positionEvidence`, `FormationSensitiveRegisteredListGuestLift.QualifiedLift.substitute`, `FormationSensitiveRegisteredListGuestLift.Controls.singleton_position_target`, `FormationSensitiveRegisteredListGuestLift.Controls.empty_target_refused`, `PositiveInductiveSearchTreeGuest.sourceConstructorEntries_eq`, `PositiveInductiveSearchTreeGuest.textual_source_eq`, `PositiveInductiveSearchTreeGuest.inventory_formed`, `PositiveInductiveSearchTreeGuest.rename_liftChildren`, `PositiveInductiveSearchTreeGuest.subst_liftChildren`, `PositiveInductiveSearchTreeGuest.endpointVerdicts_eq_true`, `PositiveInductiveSearchTreeGuest.branch_bundle_formed`, `PositiveInductiveSearchTreeGuest.solvedReceipt`, `PositiveInductiveSearchTreeGuest.branchReceipt`, `PositiveInductiveSearchTreeGuest.branchReceipt_substitute`, `PositiveInductiveSearchTreeGuest.receiptOnly_no_root`, `PositiveInductiveSearchTreeGuest.untypedAssignment_not_typed`, `PositiveInductiveSearchTreeGuest.rawEvidence_not_typedInstance`

Authored nested folds and their result view. A separate source file now presents solved and branch fold clauses over that same admitted SearchTree declaration. Finite plans are derived from its one-data-field and registered-List constructor forms. The existing clause compiler abstracts the declared nested recursive map, preserving constructor order, and both emitted methods beta-reconstruct their original clauses. The selected source-library expansion uses the intrinsic List eliminator; it is not an opaque host map.

The actual dependent branch receives a List of child/result pairs and the source-correspondence evidence above. Its adapter traverses that List and projects each result. For every finite source child spine, including open child terms, this result view and the source recursive map reach the same ordered List of object-level recursive calls. The theorem does not evaluate those calls in the host

or erase their source association. The complete emitted term, including the adapter, independently passes formation-sensitive replay. Applying the emitted closed function to its actual arguments beta-reduces to the same eliminator body used by the bundle theorem. A changed source body that uses the original children as its result List can pass production but fails that replay. Missing, extra and reordered clauses, a wrong nested recursive input, an unbundled call, unknown names and unresolved imports also refuse.

This is a selected textual source-to-typed-term and finite-computation edge. It is not a full source-parser adequacy theorem, a generic plan generator for every registered container, a semantic decoding of an arbitrary open List variable or authorization of all raw SearchTree iota instances.

Lean. `PositiveInductiveSearchTreeElaboration.fold_typed`,
`PositiveInductiveSearchTreeElaboration.liftChildren_fst_encode`,
`PositiveInductiveSearchTreeElaboration.bundleResults_common_output`,
`PositiveInductiveSearchTreeElaboration.selected_plans_derived`,
`PositiveInductiveSearchTreeElaboration.source_methods_reconstruct`,
`PositiveInductiveSearchTreeElaboration.source_typed`,
`PositiveInductiveSearchTreeElaboration.source_application`,
`PositiveInductiveSearchTreeElaboration.checkProposal_sound`,
`PositiveInductiveSearchTreeElaboration.all_verdicts`

Checked names and explicit computation. The authored fold name receives a fresh transparent declaration with the actual checked compiler output as its body. The declaration type is formed independently and no recursive source constant remains in that body. Ordinary delta unfolding preserves every displayed refined type, including the original conversion and cumulativity tail. Applying the named function then reaches the compiled eliminator body by delta/beta computation. The named source map and the dependent bundle result view consequently reach the same ordered calls on every finite child spine. Reusing the existing family name and installing a bare self-delta definition are formally refused.

Lean. `PositiveInductiveSearchTreeElaboration.Definitions.formed`,
`PositiveInductiveSearchTreeElaboration.Definitions.typedUnfolding`,
`PositiveInductiveSearchTreeElaboration.Definitions.namedApplication`,
`PositiveInductiveSearchTreeElaboration.Definitions.namedBundleCommonOutput`,
`PositiveInductiveSearchTreeElaboration.Definitions.shadow_refused`,
`PositiveInductiveSearchTreeElaboration.Definitions.self_refused`

The fold also instantiates the existing branch schema by an independently checked formation-sensitive context substitution. Its five components retain the actual payload, constant motive, leaf method, bundle-consuming branch method and child List. Substitution transports both fully formed endpoints; the displayed motive application converts to the result type. The resulting proof-relevant iota receipt retains the exact fold source, target and result type. A method using children as results fails the independent component check. This is explicit typed computation evidence; global raw tree-iota preservation remains a separate obligation.

Lean. `PositiveInductiveSearchTreeElaboration.Branch.contextMorphism`,
`PositiveInductiveSearchTreeElaboration.Branch.result_endpoints`,
`PositiveInductiveSearchTreeElaboration.Branch.receipt`,
`PositiveInductiveSearchTreeElaboration.Branch.children_as_results_refused`

Open declaration generation for registered containers. Constructor and method types are now generated over the complete admitted registered-container code grammar. A nested field retains both its container output expression and its recursive child's family index. Its induction hypothesis is the whole dependent bundle, after all original fields and in recursive-field order. Generation commutes with scoped substitution. On every ordinary indexed code, the extended generator agrees with the existing constructor and method generator; the actual Nat-indexed Vec instantiates this comparison.

The admitted SearchTree and VectorTree codes reproduce their independently checked declaration inventories. A mixed Vector source places a payload between its Nat arity and nested field, with direct recursive children before and after that field. Its generated method independently forms and keeps all three induction hypotheses in order. A faulty output reader commutes with substitution and agrees on the simple branch, but picks the intervening payload as a length in the mixed source; independent typing refuses it. Registry syntax laws therefore grant no formation or computation authority. Generic formation completeness, source-program joins and semantic decoding of arbitrary open containers remain separate obligations.

Lean. `IndexedPositiveContainerGuestTypes.constructorType_substitute`, `IndexedPositiveContainerGuestTypes.methodBody_substitute`, `IndexedPositiveContainerGuestTypes.constructorType_ordinary`, `IndexedPositiveContainerGuestTypes.methodBody_ordinary`, `IndexedPositiveContainerGuestControls.Lists.declarations_admitted`, `IndexedPositiveContainerGuestControls.Vectors.declarations_admitted`, `IndexedPositiveContainerGuestControls.Vectors.Mixed.method_formed`, `IndexedPositiveContainerGuestControls.Vectors.Mixed.wrong_length_refused`

Constructor formation from primitive contracts. A structural theorem now establishes formation of the generated constructor type for every code in the declared grammar. Its premises concern the primitive syntax: typed data sorts, the family kind, typed index expressions in the data environment, and the registered container's field-type operation. These primitive domains and family fibres are formed in one selected universe. An actual product rule and an exact or cumulative bound supply product closure there. The proof preserves the typed data environment through every data, direct-recursive and registered-nested binder. It takes no formation proof for the generated constructor type as an input.

Lean. `IndexedPositiveContainerGuestTypes.constructorType_formed`

Closing the actual parameter telescope forms the generated `constructorEntry`. Freshness of its published name then yields ordinary opaque admission against the actual prior signature. The concrete List source instantiates these contracts in a prefix containing only its family declaration over the existing native List library. Every constructor code over that registry forms there. The parsed branch's generated entry is admitted through the generic theorem and independently passes the existing executable producer and replay. Omitting its family prefix is refused, and an undeclared family refutes the primitive contract. Other universe policies, representation adequacy and computation authority retain their separate obligations.

Lean. `IndexedInductive.Guest.Declarations.Registered.constructorEntry_formed`, `IndexedInductive.Guest.Declarations.Registered.constructorEntry_admitted`, `IndexedPositiveContainerGuestControls.Lists.Formation.primitives`, `IndexedPositiveContainerGuestControls.Lists.Formation.every_constructor_formed`, `IndexedPositiveContainerGuestControls.Lists.Formation.family_branch_admitted`, `IndexedPositiveContainerGuestControls.Lists.Formation.source_branch_checked`, `IndexedPositiveContainerGuestControls.Lists.Formation.missing_family_refused`, `IndexedPositiveContainerGuestControls.Lists.Formation.missing_prefix_refused`

The actual Vector registry discharges the same contracts in its family-only prefix. Both the branch and mixed source constructors are admitted and pass independent checking, while interpreting the intervening payload as a Vector length refuses. The native $\mathsf{All}\ P\ xs$ type now forms without a pointwise witness function: its context contains only the element type, predicate, length and Vector. Finite structural evidence independently replays this formation. Typed substitution carries the result to arbitrary typed inputs under every signature extension. The registered Vector bundle therefore forms without assuming its requested evidence. This qualifies the selected primitive basis for the method-formation theorem below. Arbitrary open-container semantic decoding retains its separate interpretation obligation.

Lean. `NativeVectorAll.TypeFormation.allType_typed`,

IndexedPositiveContainerGuestControls.Vectors.Formation.primitives,
IndexedPositiveContainerGuestControls.Vectors.Formation.bundle_formed,
IndexedPositiveContainerGuestControls.Vectors.Formation.branch_admitted,
IndexedPositiveContainerGuestControls.Vectors.Formation.mixed_admitted,
IndexedPositiveContainerGuestControls.Vectors.Formation.wrong_mixed_refused

Source-local index admission. The structural formation criterion also accepts an explicit predicate on source index expressions. Every result index, direct-recursive index and nested-child index in the constructor must satisfy that predicate; container output expressions keep their separate registry contract. Primitive index typing is required on these admitted expressions under every typed context substitution. Constructor, dependent-method, method-inventory and complete eliminator formation follow by the same structural proofs. Unrestricted index typing is recovered as the universal-admission specialization.

Lean. IndexedPositiveContainerGuestTypes.indexTypingOn_unrestricted_iff,
IndexedPositiveContainerGuestTypes.constructorType_formedOn,
IndexedPositiveContainerGuestTypes.methodBody_formedOn,
IndexedPositiveContainerGuestTypes.methodType_formedOn,
IndexedPositiveContainerGuestTypes.methodsType_formedOn,
IndexedPositiveContainerGuestTypes.eliminatorType_formedOn

The original Nat-indexed Vec supplies the separating instance. Its reference interpreter contains a deliberately undeclared addition symbol, so its unrestricted index-typing contract is impossible. The zero, variable and successor fragment is typed from the existing natural primitives, and all indices of both original constructors belong to that fragment. The original Cons constructor type forms through the scoped theorem in a prefix containing Nat and the vector family, with neither vector constructor nor eliminator present. Closing its parameter and fresh opaque admission use the generic constructor-entry theorem. Its generated entry equals the original reference entry, which the independent executable admission checker accepts; missing Nat and undeclared addition refuse. This qualifies the constructor instance. Semantic interpretation and computation authority remain separate obligations.

Lean. IndexedInductive.Guest.Declarations.Registered.constructorEntry_admittedOn,
IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.native_total_contract_refused, IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.supported_index_typed,
IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_cons_admitted,
IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_generated_cons_checked, IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_missing_nat_refused, IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_addition_refused

Independent data and result universes. The structural method and eliminator criterion now separates the data/family level α, motive-result level β and method/bundle bound γ. Explicit product and cumulative contracts close fields and induction hypotheses at γ, without assuming either ordering between α and β. Binding the motive also requires a bound containing the sort of U_β; this obligation is distinct from forming a motive application. The original Nat-indexed Vec supplies the concrete instance: methods use $\max(\alpha, \beta)$, and the motive-binding bound contains $\beta+1$. Its generated eliminator type is structurally formed and ordinarily admitted in the original prefix containing both constructors and no vector eliminator. It equals the existing reference type. The existing level-instantiation morphism transports that formation. Independent checks accept both $(\alpha, \beta) = (0, 2)$ and $(2, 0)$. The former accepts the full type at U_3 and refuses U_2; missing constructors also refuse. These are formation and opaque-admission results. Elimination lift coherence, semantic interpretation, raw iota preservation and native realization remain separate obligations.

Lean. IndexedPositiveContainerUniverseControls.generated_type_formed,
IndexedPositiveContainerUniverseControls.generated_type_exact,

IndexedPositiveContainerUniverseControls.eliminator_admitted,
IndexedPositiveContainerUniverseControls.instantiated_type_formed,
IndexedPositiveContainerUniverseControls.higher_result_checked,
IndexedPositiveContainerUniverseControls.lower_result_checked,
IndexedPositiveContainerUniverseControls.motive_successor_bound_checked,
IndexedPositiveContainerUniverseControls.missing_motive_successor_refused

The registered VectorTree closes the corresponding nested instance. Its original child-vector fields keep their length expressions and data level. A universe-instantiated template of the existing native `All` program forms its induction-evidence bundle at $\max(\alpha, \beta)$ with a predicate returning into U_β; no pointwise witness function is required. The template changes the result-universe annotation while retaining the child universe in the native Vector declarations. Its checked open judgment and typed substitution qualify the bundle operation in extended signatures. The original constructor prefix, motive and bundle operation assemble at these same coordinates, and the structural theorem forms and ordinarily admits the complete generated eliminator before its name is present. Its independently written interface checks. Missing constructors refuse; the original child vector checks at its declared Vector type but cannot replace the required induction-evidence bundle. Semantic interpretation, elimination lift coherence, raw iota preservation and native realization remain separate obligations.

Lean. NativeVectorAll.Universes.allType_typed,
NativeVectorAll.Universes.substitute_allType,
IndexedPositiveContainerUniverseTreeControls.generated_type_formed,
IndexedPositiveContainerUniverseTreeControls.source_type_exact,
IndexedPositiveContainerUniverseTreeControls.source_inventory_admitted,
IndexedPositiveContainerUniverseTreeControls.source_bundle_formed,
IndexedPositiveContainerUniverseTreeControls.original_children_typed,
IndexedPositiveContainerUniverseTreeControls.original_children_not_bundle

Canonical computation across universe annotations. For every finite constructor-vector input, the same native `All` type program reaches a quoted dependent-pair spine. The quotation retains every original position and the application of the predicate to its element. Instantiating only the program's raw universe annotation at two different levels produces the same quoted result. The selected checked universe profile is one instance of this law. A two-position reflexive-equality predicate supplies the concrete source control; its quoted spine differs from the one-position spine. The quotation may still contain reducible predicate applications, and an explicit beta step witnesses that distinction. These are native raw-computation laws on canonical inputs. They do not establish arbitrary neutral-term typed lift coherence, global normalization or new computation authority.

Lean. NativeVectorAll.Universes.Canonical.allTypeAtUniverse_steps,
NativeVectorAll.Universes.Canonical.universe_annotation_common_result,
NativeVectorAll.Universes.Canonical.allType_steps,
NativeVectorAll.Universes.Canonical.two_positions_reduce,
NativeVectorAll.Universes.Canonical.two_positions_not_one,
NativeVectorAll.Universes.Canonical.quoted_predicate_can_reduce

Typed cumulative elimination and instantiation. One enlarged native elimination profile keeps the element and predicate universes fixed. Both the original and enlarged `All` result annotations form at its common upper bound, including on a neutral vector. On constructor vectors the common quoted computation gives conversion between these types. Every inhabitant judgment therefore transports in either direction while retaining the same term. A checked two-position proof supplies a nonconstant predicate instance. The open programs remain syntactically distinct; this does not assert their nonconvertibility. Level instantiation transports both formed open programs and commutes with capture-avoiding input substitution. Instantiated extensions supply their own computation transport, and exact declaration lookup prevents shadowing. An insufficient symbolic result bound is refused; collapsing level parameters nevertheless admits the corresponding

instantiated judgment. These results give typed compatibility on arbitrary inputs and cumulative conversion on constructor vectors. Equality between the two type programs at an arbitrary neutral vector remains open.

Lean. `NativeVectorAll.Universes.CumulativeElimination.both_typed`, `NativeVectorAll.Universes.CumulativeElimination.canonical_typing_iff`, `NativeVectorAll.Universes.CumulativeElimination.two_positions_lift`, `NativeVectorAll.Universes.CumulativeElimination.instantiated_both_typed`, `NativeVectorAll.Universes.CumulativeElimination.collapsed_upper_fits_lower`, `NativeVectorAll.Universes.Canonical.instantiation_substitution_square`

Evidence maps at neutral vectors. The same elimination profile supplies explicit maps $\mathsf{All}_\gamma(P, n, xs) \to \mathsf{All}_\delta(P, n, xs)$ and back, where $\gamma = \max(\alpha, \beta)$ and $\delta = \max(\gamma, \kappa)$. The vector xs may be neutral. The motive returns the function type between the two annotations; the nil method is identity, and the cons method sends evidence p to $\langle \pi_1 p, ih(\pi_2 p)\rangle$. Shared finite certificates undergo independent replay against the original open programs. Typed substitution and application then prove equivalence of inhabitation under signature extensions. The existing two-position identity predicate supplies concrete forward and return applications. A projection shortcut and a reversed certificate cannot reuse the forward receipt. This is an explicit comparison by object programs. Inverse laws, preservation of witness identity and equality of the neutral type programs remain open; no new identity eliminator or computation rule is added.

Lean. `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.forward_typed`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.backward_typed`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.inhabited_iff`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.two_maps_typed`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.two_positions_inhabited`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.projection_certificate_refused`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.swapped_certificate_refused`

Constructor computation and pair observations. For a constructor vector of length n, the same map accepts an arbitrary supplied evidence term q. Its source reduction reaches the pair expansion $E_0(q) = q$ and $E_{n+1}(q) = \langle \pi_1 q, E_n(\pi_2 q)\rangle$. Every head projection below n, and the final tail after n projections, reduce to the corresponding observation of q. These observations also commute with a round trip through the two annotations. The complete round trip reduces to one expansion $E_n(q)$: projection beta contracts a repeated expansion to a single one. On an explicitly assembled pair spine the map reduces to that same spine. The generic pair-spine laws quantify over the parameterized scoped syntax; the native computation theorems use the actual Vector iota rules. The vector argument is a constructor spine. No equality of an arbitrary neutral vector's mapped proof, eta contraction, unique execution strategy or global raw-iota preservation follows from these finite source paths.

Lean. `ParameterizedPiSigmaId.Presentation.PairSpine.head_expand`, `ParameterizedPiSigmaId.Presentation.PairSpine.tail_expand`, `ParameterizedPiSigmaId.Presentation.PairSpine.expand_idempotent`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.canonical_steps`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.canonical_head`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.canonical_tail`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.roundtrip_steps`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.roundtrip_head`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.roundtrip_tail`

Typed pair-eta reconstruction. In the separate typed equality judgment with dependent pair eta, formation data for each successive projection fibre suffice to prove $\Gamma \vdash E_n(q) \equiv q : T$. The terminal fibre and its inhabitant remain arbitrary; unit eta and uniqueness of identity proofs are not used. The reconstruction also supplies the object proof $\mathsf{refl}(E_n(q)) : \mathsf{Id}_T(E_n(q), q)$ by typed

conversion. Typed substitution and rule-package morphisms preserve the dependent spine. A second formulation reconstructs equality from typed comparisons of all head projections and the final tail, using their actual dependent fibres. The concrete unknown bundle $q : \Sigma(A : \mathcal{U}_\alpha).\Sigma(x : A).A$ exercises two dependent layers and transports its object identity witness through every substitution of the universe parameter. The same expansion is provably not raw beta/projection-convertible to q in the root-free tower. Thus the eta judgment supplies an additional, explicit equality principle. These results do not yet transfer the actual Vector maps into that judgment or prove their inverse law on a neutral vector.

Lean. `ParameterizedPiSigmaId.Presentation.PairSpine.TypedSpine.expand_typed_and_equal`, `ParameterizedPiSigmaId.Presentation.PairSpine.TypedSpine.expand_identity`, `ParameterizedPiSigmaId.Presentation.PairSpine.TypedSpine.substitute`, `ParameterizedPiSigmaId.Presentation.PairSpine.TypedSpine.mapHead`, `ParameterizedPiSigmaId.Presentation.PairSpine.equal_of_readout`, `ParameterizedPiSigmaId.Presentation.TypedEquality.DependentPairSpine.neutral_identity_levels`, `ParameterizedPiSigmaId.Presentation.TypedEquality.DependentPairSpine.neutral_expansion_not_raw_convertible`

Independent position witnesses and result views. The maintained constant-motive result program now reads an arbitrary supplied bundle on a constructor vector. Its target is the Vector encoding of the ordered head projections. An assembled list of independent witnesses gives those exact witnesses, with no universal witness function on element values and no restriction on the final tail. The former pointwise computation is a corollary. A further instance consumes the actual registered Vector induction bundle at the same source value, using its exact finite-position interface and an explicit witness quotation. This semantic quotation is separate from decoding an arbitrary open source container or implementing a runtime adapter. Repeated source values may carry distinct supplied position witnesses. The result view forgets the final tail: two independently checked inputs with distinct raw tails have source paths to the same result vector. In contrast, the universe map retains that tail. Raw pair expansion also differs syntactically from an opaque input; this is not a nonconvertibility claim.

Lean. `NativeVectorAll.ResultView.canonical_steps_observe`, `NativeVectorAll.ResultView.canonical_steps_assembled`, `NativeVectorAll.ResultView.semanticBundle_steps`, `NativeVectorAll.ResultView.independent_formed`, `NativeVectorAll.ResultView.first_tail_formed`, `NativeVectorAll.ResultView.second_tail_formed`, `NativeVectorAll.ResultView.shared_result_distinct_inputs`, `NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.expansion_syntax_distinct`

Independent caller bounds in an enlarged Vector profile. One further profile uses native Vector at $\gamma = \max(\alpha, \beta)$ and elimination into $\mathsf{succ}\,\gamma$. The existing result program is universe-instantiated in that signature, while its requested function type retains caller bounds $A : U_\alpha$ and $R : U_\beta$. Both its input and output vectors belong to this specified γ instance. The signature is formed and the complete finite certificate replays against the actual program and function type. Replaying that same certificate in the original native All profile is refused. This is a checked comparison point, not transport of an original lower-universe Vector or a non-derivability theorem for that profile. The maintained evidence writer reproduces the original result certificate byte for byte and emits this independently replayed profile through the same source-emission owner. The separately named result instance below retains the original input universe; general registered-container adaptation remains open.

Lean. `NativeVectorAll.ResultView.Universes.program_formed`, `NativeVectorAll.ResultView.Universes.original_certificate_refused`

Computation under profile instantiation. An explicit level instance of the original native All profile has exactly this enlarged signature. Writing θ for its level substitution, formed source judgments and finite raw computation paths transport as follows:

$$\Gamma \vdash_{\mathrm{All}} t : T \Longrightarrow \theta\Gamma \vdash_{\mathrm{enlarged}} \theta t : \theta T, \qquad t \longrightarrow^{*}_{\mathrm{All}} u \Longrightarrow \theta t \longrightarrow^{*}_{\mathrm{enlarged}} \theta u.$$

The actual result program therefore computes the ordered mapped witnesses on mapped constructor inputs, with an arbitrary final evidence tail. The two-position control retains independent witnesses at repeated source values. Its conversion code is accepted by the enlarged profile and refused when the proposed result swaps those witnesses. A separate raw-syntax control distinguishes the program's universe annotation. Contexts and inputs follow θ. The following construction instead retains the original input universe. They supply neither unrestricted subject reduction nor normalization, and refusal of the supplied code is not a nonconvertibility theorem.

Lean. `NativeVectorAll.ResultView.Universes.judgment_map`,
`NativeVectorAll.ResultView.Universes.program_computes_mapped_witnesses`,
`NativeVectorAll.ResultView.Universes.mapped_conversion_accepted`,
`NativeVectorAll.ResultView.Universes.mapped_swapped_refused`,
`NativeVectorAll.ResultView.Universes.program_annotation_changes`

Original input and a separately named higher result. The native Vector at α and a separately named instance at $\gamma = \max(\alpha, \beta)$ now coexist in one formed declaration signature. The four shared natural-number entries agree exactly; the Vector type, constructors and eliminator use distinct names. Formation of their compatible union reuses the existing signature-extension theorem. Independent finite replay checks the actual object program at the type

$$\Pi(A : U_\alpha)(R : U_\beta)(n : \mathsf{Nat})(xs : \mathsf{Vec}_\alpha\, A\, n).$$
$$\mathsf{All}_{\alpha,\beta}(\lambda_.R)\, n\, xs \;\to\; \mathsf{Vec}^{+}_{\gamma}\, R\, n.$$

The source Vector keeps its original family at α; its All evidence still depends on that exact source Vector. The program reads position evidence and builds the higher-universe result family. The same finite certificate is refused when its proposed result family is changed. The unrenamed signatures fail the exact entry compatibility condition; this is not a non-derivability or inconsistency claim. A two-position conversion code retains independent witnesses at repeated source values and refuses the result with those witnesses swapped.

Constant-name maps commute with universe-head maps. The joined root relation is stable under level substitution, so both signature formation and the complete program judgment transport through the actual level instance. A closed instance has data at level zero and result types at level one. These results do not establish general inverse laws, global raw-iota preservation, normalization or a native runtime refinement.

Lean. `NativeVectorAll.ResultView.OriginalInputUniverses.program_formed`,
`NativeVectorAll.ResultView.OriginalInputUniverses.signature_formed`,
`NativeVectorAll.ResultView.OriginalInputUniverses.changed_result_certificate_refused`,
`NativeVectorAll.ResultView.OriginalInputUniverses.unrenamed_signatures_incompatible`,
`NativeVectorAll.ResultView.OriginalInputUniverses.independent_conversion_accepted`,
`NativeVectorAll.ResultView.OriginalInputUniverses.swapped_conversion_refused`,
`NativeVectorAll.ResultView.OriginalInputUniverses.program_level_transport`,
`NativeVectorAll.ResultView.OriginalInputUniverses.result_signature_level_formed`

Complete original-input readout and a Vector of types. A shared elimination law derives a complete ordered readout from the actual Vector iota roots and the local nil and cons computations. Its destination constructors are explicit parameters. It supplies both the existing result program and the original-input higher-result program. In the latter profile, for finite lists of scoped source terms xs, witnesses ws and final tail t,

$$\mathsf{results}\, A\, R\, |xs|\, \ulcorner xs \urcorner_\alpha\, \mathsf{assemble}(ws, t)$$
$$\longrightarrow^{*} \mathsf{encode}^{+}_{R}(ws), \qquad |xs| = |ws|.$$

Here results is the actual five-argument source program. The result reads one witness per source position and does not inspect the final evidence tail. Its construction changes the destination constructor prefix while retaining the result-type and witness arguments. A checked raw-syntax example distinguishes this from globally renaming every constant inside a nested result-type argument. The registered Vector induction bundle specializes the computation under an explicit witness quotation. The whole path also transports through the actual universe-level instance. These raw computation statements do not supply an arbitrary open-term decoder.

Lean. `NativeVectorAll.ResultView.EvidenceElimination.fold_reads`, `NativeVectorAll.ResultView.OriginalInputUniverses.Computation.program_witnesses`, `NativeVectorAll.ResultView.OriginalInputUniverses.Computation.registered_bundle_steps`, `NativeVectorAll.ResultView.OriginalInputUniverses.Computation.program_level_witnesses`, `NativeVectorAll.ResultView.OriginalInputUniverses.Computation.constructor_prefix_renaming_is_distinct`

At $\alpha = 0$ and $\beta = 1$, take $A = \mathsf{Nat}$ and $R = U_0$. The two repeated input values $[0, 0]$ carry the independent result witnesses Nat and $\mathsf{Id}_{\mathsf{Nat}}(0, 0)$. A checked transparent definition exposes the application, whose output is a Vector of these two types. The application and returned Vector have separate formation-sensitive typing proofs. A finite conversion certificate checks this output and refuses the same certificate with the two witnesses swapped. Qualified unfolding preserves the caller's displayed type. General subject reduction, normalization and native runtime refinement remain separate obligations.

Lean. `NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.formed`, `NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.typed_unfolding`, `NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.application_typed`, `NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.output_typed`, `NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.application_conversion`, `NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.swapped_refused`

Naturality of complete generated declarations. A head map commuting with the actual index interpretation and registered field and bundle syntax commutes with generated constructors, ordered dependent methods and the complete eliminator. These primitive contracts extend through every source binder and induction hypothesis. The existing native `All` template supplies the registered Vector bundle at an arbitrary result annotation. Its universe-instantiation law therefore extends to the actual indexed Vec and nested VectorTree eliminators. A concrete level collapse witnesses the whole generated VectorTree law; retaining its original annotation yields different syntax. This is syntactic coherence, with typing transport and computation authority retaining their separate contracts.

Lean. `IndexedPositiveContainerGuestTypes.constructorType_mapHead`, `IndexedPositiveContainerGuestTypes.methodBody_mapHead`, `IndexedPositiveContainerGuestTypes.eliminatorType_mapHead`, `IndexedPositiveContainerUniverseControls.generated_type_substLevels`, `IndexedPositiveContainerUniverseTreeControls.generated_type_substLevels`, `IndexedPositiveContainerUniverseTreeControls.generated_collapse_exact`, `IndexedPositiveContainerUniverseTreeControls.generated_annotation_changes`

Dependent method formation. A structural theorem forms generated dependent methods throughout the declared code grammar. Beyond constructor primitives, the registry supplies typing for its whole-field bundle operation. The actual motive and constructor must be typed, and pending induction hypotheses must be formed. The proof retains the typed data/index environment through every binder and appends each direct hypothesis or whole nested bundle in source order. No premise assumes formation of the generated method itself.

Lean. `IndexedPositiveContainerGuestTypes.methodBody_formed`,

IndexedPositiveContainerGuestTypes.methodType_formed

The actual Vector branch supplies a concrete witness before the tree eliminator is declared. Its prefix contains only the family declarations and branch constructor, whose type matches the authored source. The method forms through the generic theorem, closes over its motive and is admitted as an ordinary opaque declaration. Independent checking accepts that entry and refuses a missing or undeclared constructor. This is formation under explicit primitive contracts and the common universe bound. The complete eliminator result below additionally uses explicit domain and universe contracts; other universe cases, semantic decoding and computation authority remain separate.

Lean. IndexedPositiveContainerGuestControls.Vectors.MethodFormation.no_eliminator, IndexedPositiveContainerGuestControls.Vectors.MethodFormation.source_method_formed, IndexedPositiveContainerGuestControls.Vectors.MethodFormation.method_admitted, IndexedPositiveContainerGuestControls.Vectors.MethodFormation.source_method_checked, IndexedPositiveContainerGuestControls.Vectors.MethodFormation.missing_constructor_refused, IndexedPositiveContainerGuestControls.Vectors.MethodFormation.undeclared_constructor_ref used

Complete eliminator formation. The full eliminator forms structurally from those method contracts, typed constructors, a typed optional index domain and explicit universe/product rules. The proof forms the motive in its upper universe, the final dependent result and the complete ordered method inventory, then binds the motive. It assumes no formation of the generated eliminator.

Lean. IndexedPositiveContainerGuestTypes.motiveType_formed, IndexedPositiveContainerGuestTypes.resultType_formed, IndexedPositiveContainerGuestTypes.methodsType_formed, IndexedPositiveContainerGuestTypes.eliminatorType_formed

The actual VectorTree instance uses both authored constructors at their original prefixes, with no tree eliminator present. Generic constructor formation supplies their typings; the generated eliminator type matches the original source declaration. Ordinary opaque admission reconstructs the full authored inventory. Independent checking accepts the final entry and refuses missing constructors and the tested lower-universe claim. The sufficient common-level criterion keeps its upper/output universe contracts explicit. Further universe configurations, general indexed-eliminator qualification, semantic interpretation, raw ι preservation and native implementation refinement remain separate.

Lean. IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.prefix_exact, IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.constructors_typed, Inde xedPositiveContainerGuestControls.Vectors.EliminatorFormation.source_eliminator_formed, IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.full_inventory_admitted, IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.lower_universe_refused

The complete dependent eliminator interface is now generated from the same source inventory: its motive, ordered methods and final index/value binders. Only an indexed family supplies an index-domain term; the unindexed interface has no such input or binder. Complete generation commutes with substitution and preserves the previous ordinary indexed interface. The actual List SearchTree and VectorTree declaration inventories use this generator, and their entries retain independently checked whole-inventory comparisons. Source-prefix replay supplies admission and formation as described below. The Nat-indexed Vec instantiates the indexed comparison. Reversing the constructor inventory changes the selected nested interfaces, exposing an order error that a constructor-by-constructor check would miss. These are generation and comparison results; further universe configurations, general admission and computation authority remain separate obligations.

Lean. IndexedPositiveContainerGuestTypes.eliminatorType_substitute, IndexedPositiveContainerGuestTypes.eliminatorType_ordinary, IndexedPositiveContainerGuestControls.Lists.eliminator_exact, IndexedPositiveContainerGuestControls.Vectors.eliminator_exact, IndexedPositiveContainerGuestControls.OrdinaryVectors.eliminator_exact,

IndexedPositiveContainerGuestControls.Lists.reversed_order_not_exact,
IndexedPositiveContainerGuestControls.Vectors.reversed_order_not_exact

Registered-container declaration generation also closes the family, every constructor and the complete eliminator under one parameter telescope. One name assignment supplies both the published names and their internal type prefixes. The complete ordinary indexed inventory is preserved; the actual Nat-indexed Vec supplies the comparison. The existing independent declaration checker replays each generated type at its source prefix. Admission of the dependency prefix composes with admission of the generated suffix, giving a formed opaque signature without treating the dependencies as formed merely because a suffix check passed. Prefix evidence also proves name separation; an eliminator cannot alias its family at any producer budget over an admitted prefix. Both nested-family clients use this path. A family/eliminator name collision and a Vector length/data-sort mismatch refuse. This is checked inventory integration, not generic formation completeness or new computation authority.

Lean. DeclarationAdmissionReplay.Admitted.append,
DeclarationAdmissionReplay.Admitted.names_nodup,
IndexedInductive.Guest.Declarations.Registered.admitInventory_names,
IndexedInductive.Guest.Declarations.Registered.aliased_endpoints_refused,
IndexedInductive.Guest.Declarations.Registered.entries_ordinary,
IndexedInductive.Guest.Declarations.Registered.admitInventory_sound,
IndexedInductive.Guest.Declarations.Registered.admitInventory_formed,
IndexedPositiveContainerGuestControls.Lists.declarations_formed,
IndexedPositiveContainerGuestControls.Vectors.declarations_formed,
IndexedPositiveContainerGuestControls.OrdinaryVectors.entries_exact,
IndexedPositiveContainerGuestControls.Lists.collision_refused,
IndexedPositiveContainerGuestControls.Vectors.wrong_data_refused

Production and independent replay compose over consecutive declaration blocks, retaining exactly the same concatenated certificates. Successful production supplies one certificate per declaration, and whole-inventory acceptance is the conjunction of the two prefix-aligned acceptance results. The existing VectorTree inventory proof uses this equation to share checked declaration facts. The producer, checker, complete source inventory and budgets are unchanged; this factors the proof of their result.

Lean. IndexedInductive.Guest.Declarations.produce_append,
IndexedInductive.Guest.Declarations.elaborate_append,
IndexedInductive.Guest.Declarations.elaborate_isSome_append,
IndexedInductive.Guest.Declarations.elaborate_cons_of,
PositiveInductiveVectorTreeGuest.inventoryAccepted_eq_true

The same admitted code now generates source-call plans. A scan binds earlier data fields before interpreting later container outputs and child-family indices, retaining direct and nested call order. Its call list commutes with scoped substitution and has one entry per direct induction hypothesis or complete nested bundle. This argument count differs from the number of semantic child positions inside a container. The authored SearchTree producer uses the generic scan and preserves its independently checked output.

For the mixed Vector source, five original fields yield three call entries. Three separate source bodies compile to the first, middle and last method hypothesis; the first has an explicit beta reconstruction. A call on the payload or a changed nested length refuses during call abstraction. A further substitution makes the two direct calls identical as terms while retaining all three slots. Direct value-only deduplication leaves two slots and cannot serve as the original method interface; sharing computation while retaining argument positions remains possible. These are source-call occurrences, not independent statistical samples. These laws concern the raw clause compiler. Generic whole-program formation, complete-bundle/result-view adapters and their semantic qualifications remain separate obligations.

Lean. IndexedPositiveContainerGuestPlans.calls_substitute,

`IndexedPositiveContainerGuestPlans.plan_call_count`,
`IndexedPositiveContainerGuestPlans.compile_reconstructs`,
`IndexedPositiveContainerGuestPlansControls.VectorCalls.compilation_exact`,
`IndexedPositiveContainerGuestPlansControls.VectorCalls.first_beta`,
`IndexedPositiveContainerGuestPlansControls.VectorCalls.wrong_child_refused`,
`IndexedPositiveContainerGuestPlansControls.VectorCalls.wrong_length_refused`,
`IndexedPositiveContainerGuestPlansControls.VectorCalls.coincident_children_keep_slots`,
`IndexedPositiveContainerGuestPlansControls.VectorCalls.value_dedup_drops_slot`

Named clauses at generated dependent interfaces. The shared header generator now retains every original constructor field and its ambient parameters. Constructor parameters select their existing ambient slots. The actual SearchTree reader uses this generator. The five-field mixed source also passes through the named reader; a recursive use of its payload and an aliased source pattern refuse. On the ordinary fragment, a direct-call template preserves the existing index/child call list, with the actual Nat-indexed Vec append plan as a comparison witness. A generic service then compiles a named clause and independently replays its emitted method at the dependent type generated from the same constructor code. Accepted output retains both that typing judgment and the actual source compilation result.

Lean. `IndexedInductive.OperationSource.Header.fromPlan_slot_count`,
`IndexedPositiveContainerGuestPlans.calls_ordinary`,
`IndexedPositiveContainerGuestPlans.elaborateClause_sound`,
`IndexedPositiveContainerGuestPlansControls.OrdinaryAppend.cons_plan_exact`,
`IndexedPositiveContainerGuestPlansControls.VectorCalls.named_before_compiled`

The authored VectorTree module supplies the two-field dependent branch. Its actual declaration block lowers to the qualified inventory, and both compiled methods check at their generated interfaces. The branch keeps the earlier length and the complete Vector-position induction bundle. Its compiled method beta-reconstructs the parsed right-hand side; the application typing is reconstructed from the checked method and its typed arguments. This does not assume that the bidirectional producer can synthesize every unannotated lambda application. Both endpoints are typed, and the exact parsed target receives the existing VectorTree proof-relevant receipt. Replacing the induction bundle by the child Vector survives raw compilation but refuses dependent checking. These are clause-stage statements, with no new datatype, raw Tree iota authority, whole-program adequacy or native realization claim.

Lean. `PositiveInductiveVectorTreeElaboration.source_inventory_exact`,
`PositiveInductiveVectorTreeElaboration.solved_typed`,
`PositiveInductiveVectorTreeElaboration.branch_typed`,
`PositiveInductiveVectorTreeElaboration.branch_beta`,
`PositiveInductiveVectorTreeElaboration.branch_endpoints_typed`,
`PositiveInductiveVectorTreeElaboration.sourceBranchReceipt`,
`PositiveInductiveVectorTreeElaboration.unbundled_compiles`,
`PositiveInductiveVectorTreeElaboration.unbundled_refused`

Constant-motive Vector results. The native Vector `All` bundle now has an object-level result projection. At a constant motive $\lambda x.\, R$, its complete program has type

$$\Pi A\, R\, n\, xs.\ \mathrm{All}(A, \lambda x.\, R, n, xs) \to \mathrm{Vec}(R, n).$$

Independent finite evidence replay establishes this polymorphic judgment. For every finite constructor spine, the projection reduces to one result per source position, in order; the element terms may be open. Substitution commutes with the projection. A version that uses original children in place of recursive results refuses the executable checking control.

Lean. `NativeVectorAll.ResultView.program_formed`, `NativeVectorAll.ResultView.subst_project`,
`NativeVectorAll.ResultView.canonical_steps`

The VectorTree specialization calls that actual projection through a fresh ordinary transparent declaration. Its body and declared type are checked, the extended library is formed, and delta unfolding preserves the caller's displayed type. The whole fold independently replays at its closed polymorphic interface. Its branch method receives the original arity, original child vector and a separate result vector of that same arity.

Lean. `PositiveInductiveVectorTreeGuest.ResultFold.checking_formed`,
`PositiveInductiveVectorTreeGuest.ResultFold.execution_formed`,
`PositiveInductiveVectorTreeGuest.ResultFold.fold_typed`,
`PositiveInductiveVectorTreeGuest.ResultFold.projection_typedUnfolding`

All five components of the substitution into the original branch context replay independently. This transports the existing parsed source receipt at the actual result type. Subsequent beta steps reconstruct the projected node call, whose result typing is checked independently. On a finite child spine, its result vector reduces to the ordered recursive fold calls, retaining repeated occurrences. The selected witness has three positions, with the first child repeated in the third. A children-as-results method is formally refused by the checker. These statements qualify this source specialization and its explicit computation path; arbitrary open-container semantic decoding, generic whole-program adequacy and universal raw-iota preservation remain separate obligations.

Lean. `PositiveInductiveVectorTreeGuest.ResultFold.contextMorphism`,
`PositiveInductiveVectorTreeGuest.ResultFold.sourceReceipt`,
`PositiveInductiveVectorTreeGuest.ResultFold.source_path`,
`PositiveInductiveVectorTreeGuest.ResultFold.source_target_typed`,
`PositiveInductiveVectorTreeGuest.ResultFold.childrenResults_canonical`,
`PositiveInductiveVectorTreeGuest.ResultFold.canonical_witness`,
`PositiveInductiveVectorTreeGuest.ResultFold.wrongBranch_refused`

Registered-container outputs are now explicit in the semantic interface. An entry owns its output-expression language, its evaluation in the constructor environment, and the indexed container at that output. The List entry retains the established container and constant unit output. The Vector entry evaluates a length expression and exposes exactly $\mathrm{Fin}\, n$ recursive positions. Its induction bundle is represented by a dependent list and proved equivalent to one motive witness at every length-indexed position. The position fibres at lengths two and three are not equivalent, so output-index erasure is observably wrong at the recursion interface.

The position bundle is now generic over the indexed registry. Semantic induction qualification remains separate from executable lookup: every successful qualified lookup supplies a representation equivalent to one motive witness at each position of that container value. A recursive construction combines those representations with direct recursive fields and later fields. Its exactness theorem identifies the resulting bundle with one witness at every position of the complete constructor polynomial, including mixed direct and registered-nested occurrences. List and Vector instantiate the same theorem. A positive Reader container also has a semantic bundle, but its natural-number position fibre has no finite traversal capability. Thus strict positivity, dependent induction evidence and executable finitarity are three distinct petals.

Lean. `IndexedPositiveContainer.Code.bundleExact`,
`IndexedPositiveContainer.ListInstance.inductionQualified`,
`IndexedPositiveContainer.VectorControl.inductionQualified`,
`IndexedPositiveContainer.VectorControl.genericBundleExact`,
`IndexedPositiveContainer.ReaderControl.inductionQualified`,
`IndexedPositiveContainer.ReaderControl.not_finitary`

The textual source boundary has a corresponding indexed-output specialization. A constructor may bind a natural length and use that name in a later `Vector n VectorTree` field. Lowering preserves the scoped length expression and erases exactly to the independently defined Vector semantic code. Data-binder freshness makes length-name lookup and quotation inverse; unbound lengths, shadowing and negative function fields are refused. This qualifies indexed source admission and semantic erasure. The object-language bundle and its first generated guest are described below.

Lean. `IndexedContainer.vectorInductionBundle`, `IndexedContainer.vector_position_card`, `IndexedContainer.vector_two_not_equiv_three`, `IndexedPositiveContainer.ListInstance.entry_container_eq`, `IndexedPositiveContainer.ListInstance.entry_output_eq`, `IndexedPositiveContainer.VectorControl.position_card`, `IndexedPositiveContainer.VectorControl.two_not_equiv_three`, `IndexedPositiveContainer.VectorControl.bundleAt`, `IndexedPositiveVectorLowering.Scope.lookup_quoteLength`, `IndexedPositiveVectorLowering.Scope.quote_lookupLength`, `IndexedPositiveVectorLowering.lowerModule_quote`, `IndexedPositiveVectorLowering.Controls.branch_erases_to_indexed_vector`, `IndexedPositiveVectorLowering.Controls.shadowedLength_refused`

The object-language Vector bundle now has a reusable core. The existing native Vec eliminator is instantiated with element universe α and elimination universe $\alpha + 1$. A first elimination computes $\mathsf{All}\ P\ n\ xs : U_\alpha$: nil gives the singleton identity type $0 =_{\mathsf{Nat}} 0$, while cons gives a dependent pair containing the head witness and the recursively computed tail evidence. A second elimination constructs that exact type from a pointwise witness function. Both terms commute with renaming and substitution. Formation-sensitive replay accepts the type and its inhabitant, rejects the inhabitant at 'Nat', and rejects an undeclared witness function. Because 'All' retains the actual source Vector in its motive, the construction does not confuse exact position evidence with an unrelated Vector of the same length. The generated 'VectorTree' declaration below consumes this reusable core.

The semantic and object accounts now meet on every canonical constructor spine. A source List selects the actual length-indexed Vector extension. Its registered-container induction bundle is equivalent to the source-ordered dependent witness spine, with the expected witness at every exact $\mathrm{Fin}\, n$ position. Native 'All' evidence reduces by the existing Vec iota and beta rules to the nested-pair quotation of that same semantic bundle. A two-position control retains both witness applications in order; swapping distinguishable positions changes the quotation. Thus formation, native computation and semantic position coverage agree on this fragment without granting any new judgmental rule.

Lean. `NativeVectorAll.signature_formed`, `NativeVectorAll.rename_allType`, `NativeVectorAll.subst_allType`, `NativeVectorAll.rename_allEvidence`, `NativeVectorAll.subst_allEvidence`, `NativeVectorAll.Controls.allType_accepted`, `NativeVectorAll.Controls.allEvidence_accepted`, `NativeVectorAll.Controls.wrong_type_refused`, `NativeVectorAll.Controls.undeclared_witness_refused`, `NativeVectorAll.Controls.allEvidence_typed`, `NativeVectorAll.CanonicalAdequacy.semanticBundleEquiv`, `NativeVectorAll.CanonicalAdequacy.pointwiseBundle_exact`, `NativeVectorAll.CanonicalAdequacy.allEvidence_steps_semanticBundle`, `NativeVectorAll.Controls.CanonicalAdequacyControls.two_position_reduces`, `NativeVectorAll.Controls.CanonicalAdequacyControls.swapped_positions_refused`

These two instances now share one selected object-realization contract. The exact registry lookup determines the semantic induction bundle. A realization retains an inhabitant of that bundle, a formation-checked object term, a quotation of semantic bundles, and a structural computation from the object term to the quotation of the selected inhabitant. Applying the container's existing exactness equivalence yields one motive witness at every recursive position. The canonical List relation spine and the closed two-element Vector reflexivity bundle instantiate the contract through their real registry entries. As a negative control, the positive but infinitary Reader at the impossible motive has no semantic bundle and therefore admits no selected object realization or object value. This interface connects evidence already produced; source syntax cannot create a missing motive witness.

Lean. `FormationSensitiveIndexedContainerBundleRealization.SelectedObjectBundle.positionWitnesses`, `FormationSensitiveIndexedContainerBundleRealization.ListInstance.registeredRealization`, `FormationSensitiveIndexedContainerBundleRealization.VectorInstance.registeredRealization`,

FormationSensitiveIndexedContainerBundleRealization.VectorInstance.covers_two_positions, FormationSensitiveIndexedContainerBundleRealization.ReaderNegative.semanticBundle_empty, FormationSensitiveIndexedContainerBundleRealization.ReaderNegative.noSelectedObjectBundle, FormationSensitiveIndexedContainerBundleRealization.ReaderNegative.noObjectValue

The interface now lifts through complete constructor code. A generic quotation fold follows result, data, direct-recursive and registered-nested nodes in source order. Its nested callback is indexed by the actual lookup proof, container output, shape and child family. A code-wide selected realization then binds one 'Code.Bundle', a formed object term and its computation to that quotation; the existing 'bundleExact' theorem recovers every direct and nested position. The nontrivial control contains one direct recursive child followed by a length-two Vector field. Its object term pairs the direct reflexivity witness with native 'All' evidence, forms at the corresponding nested Sigma type, computes through the existing Vec rules to the generic quotation, and exposes all three witnesses. The generic quotation is now available, while automatic formation for arbitrary open source inputs remains to be generated.

Lean. FormationSensitiveIndexedContainerBundleRealization.Code.quoteBundle, FormationSensitiveIndexedContainerBundleRealization.Code.SelectedObjectBundle.positionWitnesses, FormationSensitiveIndexedContainerBundleRealization.MixedControl.quote_bundle, FormationSensitiveIndexedContainerBundleRealization.MixedControl.object_formed, FormationSensitiveIndexedContainerBundleRealization.MixedControl.object_steps_quote, FormationSensitiveIndexedContainerBundleRealization.MixedControl.allPositionWitnesses

Formation assembly is generic once the actual leaves are available. 'LeafRealizations' requests only the direct and nested leaves occurring in one selected constructor shape. A pair algebra combines their formed object terms, semantic values, quotations and computations; 'realizeFromLeaves' recurses over the same constructor code and returns a complete formed object value for 'Code.Bundle'. For the mixed direct/Vector control, the compiled semantic value, term, nested Sigma type and quotation agree exactly with the independently constructed reference. This removes handwritten assembly while leaving the production of each open direct or nested leaf as an explicit source-specific obligation.

Lean. FormationSensitiveIndexedContainerBundleRealization.Code.realizeFromLeaves, FormationSensitiveIndexedContainerBundleRealization.Code.ObjectValue.toSelectedCode, FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiled_value_eq, FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiled_term_eq, FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiled_type_eq, FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiled_quote_eq, FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiledPositionWitnesses

Provider selection is now part of that recursion. A partial provider table offers formation-qualified object values for direct leaves and for nested leaves indexed by the actual registry lookup, output, shape and children. The compiler walks arbitrary recognized code, combines accepted leaves with the pair algebra and refuses the entire realization if any leaf provider is absent. Every successful result contains the exact complete semantic bundle, so it yields a motive witness at every actual position. Total producers embed as always-accepting tables. On the mixed direct/Vector control, a uniformly inhabited motive makes both providers available; the partial compiler agrees exactly with the total reference and covers all three positions. Removing the direct provider or the Vector provider refuses independently. Automatic routing and assembly are therefore generic; producing formation evidence for arbitrary open source values remains a source-specific elaboration obligation.

Lean.
FormationSensitiveIndexedContainerBundleRealization.Code.PartialLeafProviders.ofTotal, FormationSensitiveIndexedContainerBundleRealization.Code.realizeWithProviders?, FormationSensitiveIndexedContainerBundleRealization.Code.positionWitnessesOfProviderSuccess, FormationSensitiveIndexedContainerBundleRealization.MixedControl.completeProviders_compile, FormationSensitiveIndexedContainerBundleRealization.MixedControl.providerPositionWitnesses, FormationSensitiveIndexedContainerBundleRealization.MixedControl.missingNestedProvider_refused, FormationSensitiveIndexedContainerBundleRealization.MixedControl.missingDirectProv

ider_refused

The same routing now lifts over ordered constructor inventories. Each constructor request retains its own environment, result index, shape and child family; the pointwise result retains the corresponding code and refusal. The actual resolved textual VectorTree family supplies two requests from the exact erasures of its ordered ‘solved‘ and ‘branch‘ constructors. With complete providers both accept. Removing the Vector provider leaves ‘solved‘ accepted and refuses ‘branch‘, demonstrating that the inventory adapter neither loses constructor identity nor turns a family-wide capability into authority for an unrelated occurrence. The control uses a uniformly inhabited motive to test routing; its open object-language branch evidence remains the separate formation/logical-relation obligation.

Lean. `FormationSensitiveIndexedContainerBundleRealization.Code.realizeRequest?`, `FormationSensitiveIndexedContainerBundleRealization.Code.realizeInventory?`, `FormationSensitiveIndexedContainerBundleRealization.Code.realizeInventory?_at`, `PositiveInductiveVectorTreeGuest.ProviderInventory.codes_eq`, `PositiveInductiveVectorTreeGuest.ProviderInventory.solved_provider_accepted`, `PositiveInductiveVectorTreeGuest.ProviderInventory.branch_provider_accepted`, `PositiveInductiveVectorTreeGuest.ProviderInventory.missingNested_verdicts`

That final indexed guest edge is checked. The accepted textual constructor order generates the solved and branch declaration rows. The complete ordered inventory adds the external ‘Answer‘ type, ‘VectorTree‘, both constructors and the dependent eliminator. Its branch method receives the exact All P n *children*. The branch iota target builds that value by native Vec elimination, applying the tree eliminator recursively to every child. Source, target and bundle terms recheck independently before becoming proof-relevant receipts; the branch receipt transports through every typed substitution. Supplying the raw child Vector where its ‘All‘ evidence is required is refused. This establishes typed canonical branch computation, not global preservation of every raw schema instance.

Lean. `PositiveInductiveVectorTreeGuest.sourceConstructorEntries_eq`, `PositiveInductiveVectorTreeGuest.textual_source_roundtrip`, `PositiveInductiveVectorTreeGuest.rename_liftChildren`, `PositiveInductiveVectorTreeGuest.subst_liftChildren`, `PositiveInductiveVectorTreeGuest.inventory_formed`, `PositiveInductiveVectorTreeGuest.endpointVerdicts_eq_true`, `PositiveInductiveVectorTreeGuest.branch_bundle_formed`, `PositiveInductiveVectorTreeGuest.solvedReceipt`, `PositiveInductiveVectorTreeGuest.branchReceipt`, `PositiveInductiveVectorTreeGuest.branchReceipt_substitute`

Such families support several concrete agent workloads. A proof-search agent can retain a tree of solved answers and unresolved subgoals. A planner can keep pure workflow structure distinct from effect-bearing actions. A theorem agent can use a mutual goal/evidence family so evidence construction may expose new typed obligations without confusing the two states. An interaction agent can index protocol traces by initial and final state so only composable turns are combined. Syntax, explanation and provenance trees can keep ordered child lists while obtaining recursion and induction once from the declaration. These types organize evidence; they do not themselves supply a search policy, fair scheduler, effect authorization or truth.

The existing direct vector and even/odd blocks enter the richer recognizer through an explicit bridge, preserving constructor order and names. Direct constructor shapes have the same interpretation. A reusable polynomial- signature equivalence additionally retains an equivalence of every shape fibre, an equivalence of recursive positions over the corresponding shape, and equality of the child index selected by each position. The direct-code translation satisfies this law constructor by constructor and for the whole ordered inventory. The vector and even/odd inventories instantiate it. If the odd constructor's only child index is changed, no such equivalence exists. Nested guest declaration formation, typed generated recursors and native execution remain separate M2 edges.

Lean. `IndexedPolynomial.SignatureEquiv`, `PositiveInductiveBlockBridge.Direct.shape_eq`,

PositiveInductiveBlockBridge.Direct.polynomialEquiv,
PositiveInductiveBlockBridge.Direct.admittedPolynomialEquiv,
PositiveInductiveBlockBridge.Direct.admitted_names,
PositiveInductiveBlockBridge.Direct.admitted_recognized,
PositiveInductiveBlockBridge.Controls.vectorPolynomialEquiv,
PositiveInductiveBlockBridge.Controls.mutualPolynomialEquiv,
PositiveInductiveBlockBridge.Controls.translated_odd_not_equiv_wrongChild

14 Status ledger

Entries marked "proved" are checked without `sorry`, custom axioms or native decision. Entries marked "open" are obligations, not proved claims. Axioms are those reported by `##print axioms` on the listed declaration: P = `propext`, C = `Classical.choice`, Q = `Quot.sound`; "none" means no axioms.

Part	Claim	Lean declaration	Axioms	Status
A1	Freeness: multifunctors out of open derivations are interpretations of rule instances	`RuleSignature.universal`	Q	proved
A1	A multifunctor is determined by its generators	`RuleSignature.functor_eq_of_gen`	Q	proved
A1	Extension agrees with the interpretation on generators	`RuleSignature.lift_gen`	none	proved
A1	Extension preserves substitution	`RuleSignature.Term.lift_subst`	Q	proved
A1	Right unit law	`RuleSignature.Term.subst_vars`	Q	proved
A1	Associativity of substitution	`RuleSignature.Term.subst_subst`	Q	proved
A1	Closed derivability is the inductive rule closure	`RuleSignature.derivable_iff_derives`	PQ	proved
A1	Closed derivations are planar	`RuleSignature.Term.closed_linear`	P	proved
A1	Generators are planar	`RuleSignature.Term.gen_linear`	PQ	proved
A1	Holes of equal sort stay distinct	`RuleSignature.Term.duplicate_holes_distinct`	P	proved
A1	Stable instance actions induce multifunctors	`RuleSignature.actFunctor`	PQ	proved
A1	Derivability is closed under instance actions	`RuleSignature.Derivable.act`	PQ	proved
A1	Substitution acts on Hilbert multicategories	`Hilbert.substFunctor`	PQ	proved
A1	Hilbert derivability is substitution closed	`Hilbert.source_subst`	PQ	proved
A1	Instance actions compose coherently	`RuleSignature.act_comp_heq`	PQ	proved
A1	Substitution multifunctors compose coherently	`Hilbert.substFunctor_comp_heq`	PCQ	proved
A1	Multifunctors compose associatively	`CartesianMulticategory.Functor.comp_assoc`	Q	proved
A1	Multifunctors preserve closed morphisms	`CartesianMulticategory.Functor.inhabited`	none	proved
A1	Kernel derivability is closed derivability	`RuleWitness.derivable_iff_derives`	PQ	proved
A1	Closed morphisms are accepted certificate trees	`RuleWitness.closedEquiv`	PQ	proved
A1	Erasure to trees is injective	`RuleWitness.toDerivation_injective`	PQ	proved
A1	Unfolding replay agrees with judgment replay	`RuleWitness.unfold_judgments`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
A1	Every accepted graph node unfolds to a closed morphism	`RuleWitness.unfold_root`	PQ	proved
A1	Every accepted graph node yields an accepted tree	`RuleWitness.unfold_tree`	PQ	proved
A1	Every closed morphism is the unfolding of an accepted graph root	`RuleWitness.presented`	PCQ	proved
A1	Presented blocks combine into one graph	`RuleWitness.presented_block`	PCQ	proved
A1	Authored open derivations are free-multicategory terms	`CertificateGSLT.OpenDerivation.termEquiv`	PQ	proved
A1	Plugging is substitution	`CertificateGSLT.OpenDerivation.toTerm_bind`	PQ	proved
A1	Interpretation of open derivations preserves substitution	`CertificateGSLT.interpret_bind`	PQ	proved
A1	Interpretation of open derivations is unique	`CertificateGSLT.interpret_unique`	PQ	proved
A1	Computable decoding of accepted raw open proofs	`CertificateGSLT.decodeOpen`	PCQ	proved
A2	Translations preserve derivability	`RuleSignature.Derivable.map`	Q	proved
A2	Injective renamings induce faithful multifunctors	`RuleSignature.lift_rename_injective`	Q	proved
A2	Intuitionistic into classical: faithful	`Hilbert.inclusion_faithful`	PQ	proved
A2	Intuitionistic into classical: not reflecting	`Hilbert.inclusion_not_reflecting`	PCQ	proved
A2	Classical into extended: faithful	`Extension.inclusion_faithful`	PQ	proved
A2	Classical into extended: conservative	`Extension.inclusion_conservative`	PCQ	proved
A2	Classical into extended: not full	`Extension.inclusion_not_full`	PCQ	proved
A2	HOL Light into its first-order calculus: multifunctor	`HOLLight.FirstOrder.translation`	PCQ	proved
A2	That multifunctor is determined by rule images	`HOLLight.FirstOrder.translation_unique`	PCQ	proved
A2	It preserves derivability	`HOLLight.FirstOrder.translation_derivable`	PCQ	proved
A2	It reflects derivability	`HOLLight.FirstOrder.translation_reflects`	PCQ	proved
A2	Positive control: REFL translated	`HOLLight.FirstOrder.Kernel.reflexivity_translated`	PCQ	proved
A2	Negative control: F not derivable	`HOLLight.FirstOrder.Kernel.falsity_not_translated`	PCQ	proved
A2	Node-by-node translation of one node is the multifunctor image	`RuleWitness.liftStep_liftEntry`	PCQ	proved
A2	Shared translation commutes with unfolding	`RuleWitness.replay_liftStep`	PCQ	proved
A2	Accepted graphs translate with sharing, coherently	`RuleWitness.shared_translation`	PCQ	proved
A2	HOL Light graphs translate with sharing, coherently	`HOLLight.FirstOrder.shared_translation`	PCQ	proved
A3	Logical work is additive under substitution	`RuleSignature.Term.size_subst`	PQ	proved
A3	Affine translations are linear on trees	`RuleSignature.lift_size_le`	PCQ	proved
A3	Contraction: linear source, exponential tree, linear graph	`Contraction.contraction_gap`	PCQ	proved
A3	The contracting image is not affine	`Contraction.image_not_affine`	PQ	proved
A3	Trees do not size-simulate graphs (existing)	`ProofSharingBounds.no_tree_sizeSimulation_dag`	PCQ	proved
A3	Literal expansion of definitions is exponential (existing)	`PropositionalExtension.literal_expansion_gap`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
A3	HOL graph to MM0 certificate bound, degree 10 (existing)	`HOLLight.FirstOrder.graphTranslate_weight`	PCQ	proved
A3	Declared time is at least the written lemma weight	`HOLLight.FirstOrder.graphTime_ge_weight`	PCQ	proved
A3	MM0 p-simulates HOL Light graphs, declared time (existing)	`HOLLight.FirstOrder.Kernel.mm0Simulation`	PCQ	proved
A3	Operational clocks are positive (existing)	`ProofSystems.OperationalPSimulation.time_positive`	PCQ	proved
A4	Sequential replay is local	`DerivationGraph.replayWith_iff_local`	PCQ	proved
A4	Schedule independence (forward)	`DerivationGraph.replayWith_permute`	PCQ	proved
A4	Schedule independence (reflection)	`DerivationGraph.replayWith_permute_reflect`	PCQ	proved
A4	Schedule independence (both directions)	`DerivationGraph.replayWith_permute_iff`	PCQ	proved
A4	Dependency-violating schedules are refused	`DerivationGraph.replayWith_permute_refused`	PCQ	proved
A4	Lookup count is schedule invariant	`DerivationGraph.permute_lookups`	PCQ	proved
A4	Replayed blocks relocate into larger caches	`DerivationGraph.replayWith_relocate`	PCQ	proved
A4	Unfoldings are schedule invariant	`RuleWitness.unfold_permute`	PCQ	proved
A4	Monotone single-product steps are graph replay	`MultiPremiseSteps.run_monotone`	PQ	proved
A4	Monotone traces denote closed morphisms	`MultiPremiseSteps.run_monotone_unfolds`	PQ	proved
A4	Monotone multi-product steps split	`MultiPremiseSteps.run_split`	PQ	proved
A4	Consumption is linear	`MultiPremiseSteps.consumed_not_cited`	PQ	proved
A4	Consumption is not confluent	`MultiPremiseSteps.Controls.consumption_not_confluent`	PQ	proved
A4	Independent consuming steps commute up to product order	`MultiPremiseSteps.independent_commute`	PQ	proved
A4	Control: two independent consuming steps commute	`MultiPremiseSteps.Controls.independent_steps_commute`	PQ	proved
A4	Split/join exceeds tree-shaped single-wire-cut composition	`MultiPremiseSteps.process_not_polycategorical`	PQ	proved
A4	Ordered occurrence scanner is complete	`Sources.Query.scanList_complete`	PCQ	proved
A4	Primitive source query agrees with source rows	`Sources.Query.run_step`	PCQ	proved
A4	Recursive query lowering preserves row bags	`Sources.Query.run_compile`	PCQ	proved
A4	Completed emptiness is declarative absence	`Sources.Query.run_empty_iff_no_declarative`	PCQ	proved
A4	Validated adaptive reads preserve source rows	`Sources.Query.validated_source_rows`	PCQ	proved
A4	Completed cursor returns source rows and reads	`Sources.Query.Execution.completed_rows`	PCQ	proved
A4	Compiled query completes at its read-path bound	`Sources.Query.Execution.execute_complete`	PCQ	proved
A4	Budget splitting retains the complete packet	`Sources.Query.Execution.execute_split`	PCQ	proved
A4	Completed negative query has source meaning	`Sources.Query.Execution.completed_absence`	PCQ	proved
A4	Completed query is sound and covers alternatives	`Sources.Query.Execution.completed_agreement`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
A4	Validated resumption recovers current source rows	`Sources.Query.Execution.resumed_rows`	PCQ	proved
A4	Unfinished query is distinct from absence	`Sources.Query.Controls.partial_query_is_not_absence`	PCQ	proved
A4	Empty conjunction and vacuous every have rows	`Sources.Query.Controls.vacuous_rows_are_present`	PCQ	proved
A4	Omitted region replies violate completeness	`Sources.Query.Controls.omitted_region_reply_is_not_complete`	PCQ	proved
A4	Adaptive reads retain the selected binding	`Sources.Query.Controls.adaptive_dependency_is_bound`	PCQ	proved
A4	Absence retains its full completeness footprint	`Sources.Query.Controls.absence_row_retains_full_footprint`	PCQ	proved
A4	Bag agreement does not validate ordered reads	`Sources.Query.Controls.bag_agreement_does_not_validate_reordering`	PCQ	proved
A4	Resumption observes an unread arrival	`Sources.Query.Controls.resume_sees_unread_arrival`	PCQ	proved
A4	Changed performed reads retain a refused packet	`Sources.Query.Controls.changed_prefix_retains_refused_packet`	PCQ	proved
A4	Checked publication preserves evidence validity	`ProgrammableJudgmentTransforms.step_preserves_validity`	PCQ	proved
A4	Stored evidence reconstructs a source derivation	`ProgrammableJudgmentTransforms.stored_evidence_derivation`	PCQ	proved
A4	Retained old authority still enables old work	`ProgrammableJudgmentTransforms.retaining_source_keeps_old_request_enabled`	PQ	proved
A4	Unchecked refused evidence breaks store validity	`ProgrammableJudgmentTransforms.refused_evidence_write_breaks_validity`	PCQ	proved
A4	Untrusted service replies preserve checked intents	`ProgrammableJudgmentServices.program_intents_preserve`	PCQ	proved
A4	Finite initial guards characterize sequential commits	`CommitPrecondition.sequential_holds_iff`	PCQ	proved
A4	Guarded net update exactly composes two commits	`Commit.composition_exact`	PCQ	proved
A4	Independent intervening events permit guarded fusion	`executable_fusion_iff`	PCQ	proved
A4	Fusion does not preserve the complete event trace	`fusion_not_traceEq`	PCQ	proved
A4	Matching precedes linear admission	`Sources.Query.Controls.nonlinear_match_is_still_a_row`	PCQ	proved
A4	Nested every filters the aggregate row	`Sources.Query.Controls.nested_every_filters_the_aggregate`	PCQ	proved
A4	Consumed reply identities remain refused	`Kernel.CorrelatedReplies.consumed_reply_stays_refused`	PCQ	proved
A4	Cancelled reply identities remain refused	`Kernel.CorrelatedReplies.cancelled_reply_stays_refused`	PCQ	proved
A4	Old request cannot answer the next one	`Kernel.CorrelatedReplies.Controls.stale_request_cannot_answer_next`	P	proved
A4	Retargeting a reply loses correlation	`Kernel.CorrelatedReplies.Controls.replacing_supplied_identity_accepts_wrong_reply`	P	proved
C	HOL Light kernel module loads exactly	`HOLLight.FirstOrder.kernelModule_read`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
C	Loaded HOL Light module is faithful	`HOLLight.FirstOrder.kernelModule_faithful`	PCQ	proved
C	VibeITP module loads exactly (existing)	`VibeITP.CheckingMachine.kernelModule_read`	PCQ	proved
C	RUP instances are sound	`PropositionalLRATActions.rupCheck_sound`	PQ	proved
C	LRAT action checker is sound	`PropositionalLRATActions.check_sound`	PQ	proved
C	LRAT actions denote closed morphisms	`PropositionalLRATActions.check_unfolds`	PQ	proved
C	RAT additions are not RUP rule instances	`PropositionalLRATActions.rat_not_rule_instance`	PQ	proved
C	Positive control: RUP refutation	`PropositionalLRATActions.Controls.xor_accepted`	PQ	proved
C	Negative control: deleted premise	`PropositionalLRATActions.Controls.deleted_premise_refused`	PQ	proved
C	RAT proof classified as extension	`PropositionalLRATActions.Controls.rat_refused`	PQ	proved
D	Revalidation is exact	`DerivationGraph.revalidate_iff`	PCQ	proved
D	Admitted extensions transport evidence	`DerivationGraph.replay_extend`	PCQ	proved
D	Checked judgments extend to larger rule systems	`DerivationGraph.CheckedJudgment.extend`	none	proved
D	Retained schedules reinterpret under accepting interfaces	`DerivationGraph.BorrowedCursor.run_reinterpret`	PQ	proved
D	Intuitionistic evidence is classical evidence	`Revalidation.intuitionistic_to_classical`	PCQ	proved
D	Evidence survives appended admitted definitions	`Revalidation.checkDAG_append`	PCQ	proved
D	Negative control: excluded middle refused	`Revalidation.Controls.em_refused_intuitionistically`	P	proved
D	Positive control: extended definitions	`Revalidation.Controls.worked_extended`	PCQ	proved
D	Negative control: changed definition	`Revalidation.Controls.worked_changed_refused`	PQ	proved
G	Ten source evidence rules have an exact executable interface	`KripkeTruth.Checking.interface`	P	proved
G	Accepted certificates yield original source evidence	`KripkeTruth.Checking.check_sound`	PQ	proved
G	Certificate coverage is exactly source evidence	`KripkeTruth.Checking.coverage`	PCQ	proved
G	Certificates characterize least-fixed-point membership	`KripkeTruth.Checking.judgment_coverage`	PCQ	proved
G	Closed categorical proofs characterize grounded judgments	`KripkeTruth.Checking.closed_proof_iff_grounded`	PQ	proved
G	Finite decision and certificate authority coincide	`KripkeTruth.Checking.finite_decision_iff_certificate`	PCQ	proved
G	Completed unknown excludes both certificate polarities	`KripkeTruth.Checking.finite_undetermined_iff_no_certificates`	PCQ	proved
G	Positive control: admitted fact	`KripkeTruth.Checking.Controls.fact_accepts`	P	proved
G	Negative control: forged negative conclusion	`KripkeTruth.Checking.Controls.forged_false_refuses`	P	proved
G	Negative control: unsupported truth reference	`KripkeTruth.Checking.Controls.absent_premise_refuses`	P	proved
G	Liar has no certificate at either polarity	`KripkeTruth.Checking.Controls.liar_has_no_certificate`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
G	Negative control: unadmitted negative base leaf	`KripkeTruth.Checking.Controls.false_base_leaf_refuses`	P	proved
G	Negative control: cyclic proof-graph reference	`KripkeTruth.Checking.Controls.cyclic_graph_refuses`	P	proved
E	Weighted state and branch-cost sequencing is associative	`LayerEffects.Spec.bind_assoc`	PCQ	proved
E	A write and a read need not commute	`LayerEffects.Controls.put_get_not_comm`	PCQ	proved
E	The specified distribution-over-finite-sets law is impossible	`NondeterminismProbability.no_distributiveLaw`	PCQ	proved
E	Surviving answers cannot recover every failed-branch charge	`LayerEffects.Spec.no_failed_cost_recovery`	PCQ	proved
E	Marginals cannot determine same-world conjunction	`EvidenceSpaces.no_same_world_conjunction_from_marginals`	PQ	proved
D	Characterize direct value-indexed predicate contexts	`HypercubeBubbles.directPredicateContext_formed_iff`	P	proved
D	Characterize direct value-indexed Curry–Howard	`HypercubeBubbles.directValueIndexedForall_iff`	P	proved
D	Form direct value-indexed universals at the dependent vertex	`HypercubeBubbles.directValueIndexedForall_at_dependent`	P	proved
D	Refuse direct dependency at the other three cube vertices	`HypercubeBubbles.directValueIndexedForall_absent_off_dependent_face`	P	proved
D	Introduce direct value-indexed evidence	`HypercubeBubbles.directForall_abstract`	P	proved
D	Substitute the actual argument into the evidence type	`HypercubeBubbles.directForall_apply`	P	proved
D	Recover both rules of the direct formation basis	`HypercubeBubbles.directValueIndexedForall_iff_contains_dependentProducts`	P	proved
D	Keep direct formation independent of the bubble	`HypercubeBubbles.directValueIndexedForall_bubble_iff`	P	proved
D	Identify the least basis for the direct universal interface	`HypercubeBubbles.dependentProducts_least_directInterface`	P	proved
D	Type the polymorphic identity at every sufficient profile	`HypercubeBubbles.polymorphicIdentity_typed`	P	proved
D	Form the kind of unary type operators	`HypercubeBubbles.typeOperatorKind_formed`	P	proved
D	Characterize the basis for three direct formation interfaces	`HypercubeBubbles.directFormationInterfaces_iff_contains_constructionProducts`	P	proved
D	Identify the relative four-rule formation minimum	`HypercubeBubbles.constructionProducts_least_directInterfaces`	P	proved
D	Refuse the interfaces after any required rule is omitted	`HypercubeBubbles.directInterfaces_refused_of_missing_rule`	P	proved
D	Characterize literal descent into simple families	`DependencyDescentCriterion.descends_iff_constant`	Q	proved
D	Keep the varying Boolean family outside the simple image	`DependencyDescentCriterion.varying_outside_image`	PCQ	proved
D	Keep finite index types outside the constant-family image	`DependencyDescentCriterion.finFamily_not_constant`	PCQ	proved
D	Separate inhabitation from data-preserving semantics	`DependencyDescentCriterion.readout_identifies_inhabited_types`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Preserve and reflect family descent under exact coding	`DependentFamilyEquivalenceTransport.codeFamily_factorization_iff`	Q	proved
D	Retain both observer controls at every exact-code depth	`DependentFamilyEquivalenceTransport.Canary.exact_representation_preserves_observer_boundary`	PCQ	proved
D	Characterize observers preserving every anchored equality family	`EqualityFamilyObserverFactorization.EqualityFamilyFactorization.all_anchored_nonempty_iff_injective`	PCQ	proved
D	Supply the simple extensional function-space control	`DependencyExtensionalityOrthogonality.simpleExtensional_applicationExtensional`	none	proved
D	Supply the dependent extensional function-space control	`DependencyExtensionalityOrthogonality.dependentExtensional_applicationExtensional`	none	proved
D	Retain routes at the simple function-space control	`DependencyExtensionalityOrthogonality.simpleRouteSensitive_not_applicationExtensional`	none	proved
D	Retain routes at the dependent function-space control	`DependencyExtensionalityOrthogonality.dependentRouteSensitive_not_applicationExtensional`	none	proved
D	Reflect normal evidence for the closed implicational fragment	`GSLT.Dedukti.HOLHub.sttfa_adequate`	PCQ	proved
D	Reflect coded normal evidence for core universal proofs	`GSLT.Dedukti.HOLHub.sttfa_adequate_coded`	PCQ	proved
D	Retain free objects in coded normal proof reflection	`GSLT.Dedukti.HOLHub.sttfa_adequate_coded_open`	PCQ	proved
D	Compute the normal roundtrip for detour-free core-witness proofs	`GSLT.Dedukti.HOLHub.sttfa_round_trip`	PCQ	proved
D	Retain coding in every normal form of a detour-free core-witness proof	`GSLT.Dedukti.HOLHub.compile_normalForm_coded`	PCQ	proved
D	Separate evidence reconstruction from theoremhood with an uncoded witness	`GSLT.Dedukti.HOLHub.sttfa_not_exactly_adequate`	PCQ	proved
D	Reject the uncoded normal universal witness	`GSLT.Dedukti.HOLHub.sttfa_coding_needed`	PCQ	proved
D	Characterize extension above the direct-dependency formation vertex	`HypercubeHOLHub.standardLF_subsumed_iff_directInterface`	P	proved
D	Form the actual STTfa declarations at the dependency vertex	`HypercubeHOLHub.formedSignature_at_dependentVertex`	PCQ	proved
D	Type the STTfa bodies at the dependency vertex	`HypercubeHOLHub.typedBodies_at_dependentVertex`	PCQ	proved
D	Preserve types for STTfa rewrites at the dependency vertex	`HypercubeHOLHub.rulesPreserveTypes_at_dependentVertex`	PCQ	proved
D	Retain typing under STTfa reduction at the dependency vertex	`HypercubeHOLHub.typePreservation_at_dependentVertex`	PCQ	proved
D	Type translated HOL terms above the direct-dependency vertex	`HypercubeHOLHub.translate_typed_of_directInterface`	PQ	proved
D	Join source substitution coherence with profiled target typing	`HypercubeHOLHub.translate_substitute_typed_of_directInterface`	PCQ	proved
D	Type compiled HOL evidence above the direct-dependency vertex	`HypercubeHOLHub.theorem_typed_of_directInterface`	PCQ	proved
D	Type the universal-elimination control at the exact vertex	`HypercubeHOLHub.predInstance_typed_at_dependentVertex`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Refuse a value-indexed decoder declaration at the simple vertex	`HypercubeHOLHub.eta_declaration_not_formed_at_simpleVertex`	PCQ	proved
D	Refuse the STTfa signature at the simple vertex	`HypercubeHOLHub.signature_not_formed_at_simpleVertex`	PCQ	proved
D	Form every frozen DTTBench statement at the indexed profile	`LFDTTBenchDemand.allWithinMettaKernelPrototype_eq_true`	P	proved
D	Refuse the full frozen DTTBench corpus at the basic profile	`LFDTTBenchDemand.allWithinStandardLF_eq_false`	P	proved
D	Compute the complete indexed product demand of the frozen corpus	`LFDTTBenchDemand.allDemandRules_eq_mettaKernelPrototype`	P	proved
D	Identify the indexed profile as least for the frozen corpus demand	`LFDTTBenchDemand.mettaKernelPrototype_minimal_for_DTTBench`	P	proved
D	Characterize the common rules of CoC and the MettaKernel kind-result prototype	`HypercubeHOLHub.construction_mettaKernelPrototype_common_product_iff`	P	proved
D	Separate CoC formation from the MettaKernel kind-result prototype	`HypercubeHOLHub.constructions_not_subsumed_mettaKernelPrototype`	P	proved
D	Separate the MettaKernel kind-result prototype from CoC formation	`HypercubeHOLHub.mettaKernelPrototype_not_subsumed_constructions`	P	proved
D	Reject functionality of the literal upper-profile union	`HypercubeHOLHub.constructionMettaKernelPrototypeUnion_not_functional`	P	proved
D	Keep the CoC direct interfaces outside the MettaKernel kind-result prototype	`HypercubeHOLHub.mettaKernelPrototype_does_not_support_directFormationInterfaces`	P	proved
D	Keep the indexed corpus demand outside the CoC branch	`HypercubeHOLHub.constructions_do_not_contain_DTTBench`	P	proved
D	Record the two useful upper branches and their nonfunctional union	`HypercubeHOLHub.coc_mettaKernelPrototype_fork`	P	proved
D	Characterize code-indexed family descent through quote and splice	`ContextualCodeObserverFactorization.familyFactorization_iff_quoteSpliceFibreEquivalences`	Q	proved
D	Characterize preservation of code equality by quote-splice eta	`ContextualCodeObserverFactorization.equalityFamilyFactorization_iff_quoteSplice`	PQ	proved
D	Separate body-constant from tag-sensitive code families	`ContextualCodeObserverFactorization.Canary.contextual_code_observer_boundary`	PCQ	proved
D	Separate code-carrier descent from decoding descent	`TarskiCodeFamilyObserverFactorization.Canary.code_syntax_does_not_determine_decoding_factorization`	PCQ	proved
D	Compose coherent code decoding with dependent products and sums	`TarskiCodeFamilyObserverFactorization.Canary.constant_decoding_pi_sigma_control`	PCQ	proved
D	Retain every annotated product rule occurring in a typed subject	`ReifiedIndexElaboration.productRule_mem_of_typed_subject`	PQ	proved
D	Characterize the exact three-rule type-reified index vertex	`ReifiedIndexElaboration.reifiedTypeIndexedForall_iff`	PQ	proved
D	Separate the direct and type-reified formation vertices	`ReifiedIndexElaboration.direct_reified_formation_fork`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Check the direct and reified applications in formed contexts	`ReifiedIndexElaboration.Controls.applications_have_formed_contexts`	PCQ	proved
D	Measure the extra reified application in the sound reference checker	`ReifiedIndexElaboration.Controls.reifiedApplication_accepts_fuel_six`	C	proved
D	Separate direct witness transport from reified code transport	`ReifiedIndexElaboration.Controls.result_transport_surface`	PQ	proved
D	Keep decoding coherence separate from reified formation	`ReifiedIndexElaboration.Controls.formation_with_missing_decoding_coherence`	PCQ	proved
D	Decide compatibility of two formation rules exactly	`HypercubeHOLHub.compatibleResultsCheck_eq_true_iff`	PCQ	proved
D	Decide finite formation-profile functionality exactly	`HypercubeHOLHub.functionalProductsCheck_eq_true_iff`	PCQ	proved
D	Identify the executable gate with the uniqueness premise	`HypercubeHOLHub.functionalProducts_iff_profileFunctional`	none	proved
D	Characterize accepted formation-profile joins	`HypercubeHOLHub.admitProductJoin_eq_some_iff`	PCQ	proved
D	Characterize refused formation-profile joins	`HypercubeHOLHub.admitProductJoin_eq_none_iff`	PCQ	proved
D	Retain every left formation rule in an admitted join	`HypercubeHOLHub.admitted_join_contains_left`	PCQ	proved
D	Retain every right formation rule in an admitted join	`HypercubeHOLHub.admitted_join_contains_right`	PCQ	proved
D	Accept the dependency and type-operator formation join	`HypercubeHOLHub.dependent_operator_join_accepted`	none	proved
D	Refuse the blind CoC and prototype formation join	`HypercubeHOLHub.coc_mettaKernelPrototype_join_refused`	none	proved
D	Embed the left profile into an admitted join	`HypercubeHOLHub.admitted_join_subsumes_left`	PCQ	proved
D	Embed the right profile into an admitted join	`HypercubeHOLHub.admitted_join_subsumes_right`	PCQ	proved
D	Recheck left-profile evidence in an admitted join	`HypercubeHOLHub.admitted_join_rechecks_left`	PCQ	proved
D	Recheck right-profile evidence in an admitted join	`HypercubeHOLHub.admitted_join_rechecks_right`	PCQ	proved
D	Exclude every untagged functional upper profile containing both branches	`HypercubeProfileStratification.no_functional_common_upper`	P	proved
D	Characterize coarse result-sort reconciliation of the CoC/prototype union	`HypercubeProfileStratification.constructionMettaKernelPrototypeUnion_resultFunctionalUnder_iff`	P	proved
D	Exhibit the one-point result-sort shadow of the conflicting profiles	`HypercubeProfileStratification.constant_observation_reconciles`	P	proved
D	Exclude reconciliation by any faithful result-sort observation	`HypercubeProfileStratification.injective_observation_does_not_reconcile`	P	proved
D	Keep the Type-in-Type upper profile explicitly inconsistent	`TypeInTypeParadox.typeInType_inconsistent`	PQ	proved
D	Retain functionality separately in each upper-profile fibre	`HypercubeProfileStratification.branchProfile_functional`	none	proved
D	Embed the exact shared formation fragment in either branch	`HypercubeProfileStratification.shared_subsumed_branch`	P	proved
D	Check each selected upper-profile fibre exactly	`HypercubeProfileStratification.exists_checkProof_iff_deriv`	P	proved

Part	Claim	Lean declaration	Axioms	Status
D	Recheck common-fragment evidence in both upper fibres	`HypercubeProfileStratification.shared_evidence_rechecks_both`	P	proved
D	Separate the construction fibre by its direct formation interface	`HypercubeProfileStratification.construction_interface_separates_branches`	P	proved
D	Separate the profiles by the prototype-computed demand record	`HypercubeProfileStratification.prototype_computedDemand_separates_profiles`	P	proved
D	Decode the canonical MeTTa atom profile wire losslessly	`LFProfileAtomWire.decode_encode`	PQ	proved
D	Reflect equality of formation profiles through their atom encodings	`LFProfileAtomWire.encode_injective`	PQ	proved
D	Decide exact agreement of the finite sort-axiom tables	`LFProfileAtomWire.sameSortAxiomsCheck_eq_true_iff`	P	proved
D	Publish only functional profile joins containing both inputs	`LFProfileAtomWire.admitProfileJoin_sound`	PCQ	proved
D	Commute atom-level joining with canonical profile encoding	`LFProfileAtomWire.joinEncoded_encode`	PQ	proved
D	Recover both decoded inputs and the admitted functional upper profile	`LFProfileAtomWire.joinEncoded_sound`	PCQ	proved
D	Accept the encoded dependency and type-operator join	`LFProfileAtomWire.dependent_operator_join_encoded`	PQ	proved
D	Refuse the encoded nonfunctional CoC and prototype union	`LFProfileAtomWire.coc_mettaKernelPrototype_join_encoded_refused`	P	proved
D	Refuse a join with conflicting sort axioms	`LFProfileAtomWire.conflicting_sort_axiom_join_refused`	P	proved
D	Refuse malformed atom data before profile admission	`LFProfileAtomWire.malformed_profile_refused`	P	proved
D	Characterize bubble union as the least conversion extension	`BubbleConversion.convertible_join_least`	none	proved
D	Embed left-bubble typing into the conversion join	`BubbleConversion.hasTypeIn_join_left`	P	proved
D	Embed right-bubble typing into the conversion join	`BubbleConversion.hasTypeIn_join_right`	P	proved
D	Normalize a bubble join under the stated interaction law	`BubbleConversion.join_normalizedBy`	none	proved
D	Compute finite conversion-feature union exactly	`HOLConversionPolicyAtomWire.enabled_join`	PQ	proved
D	Identify finite manifest union with the existing typed policy join	`HOLConversionPolicyAtomWire.denote_join`	PQ	proved
D	Embed the left typed conversion policy in the manifest join	`HOLConversionPolicyAtomWire.denote_extends_join_left`	PQ	proved
D	Embed the right typed conversion policy in the manifest join	`HOLConversionPolicyAtomWire.denote_extends_join_right`	PQ	proved
D	Decode recursively encoded HOL types losslessly	`HOLConversionPolicyAtomWire.decodeType_encodeType`	PQ	proved
D	Decode each typed conversion feature losslessly	`HOLConversionPolicyAtomWire.decodeFeature_encodeFeature`	PQ	proved
D	Decode the complete conversion-policy atom wire losslessly	`HOLConversionPolicyAtomWire.decode_encode`	PQ	proved
D	Reflect manifest equality through atom encoding	`HOLConversionPolicyAtomWire.encode_injective`	PQ	proved
D	Commute encoded policy joining with finite manifest union	`HOLConversionPolicyAtomWire.joinEncoded_encode`	PQ	proved
D	Recover exact submitted manifests from every published policy join	`HOLConversionPolicyAtomWire.joinEncoded_sound`	P	proved

Part	Claim	Lean declaration	Axioms	Status
D	Recover the existing global beta-eta policy from two finite features	`HOLConversionPolicyAtomWire.beta_eta_manifest_join`	PQ	proved
D	Accept the encoded beta and eta policy join	`HOLConversionPolicyAtomWire.beta_eta_encoded_join`	PQ	proved
D	Refuse an equation selector outside the declared table	`HOLConversionPolicyAtomWire.out_of_range_equation_refused`	PCQ	proved
D	Refuse malformed conversion-policy atom data	`HOLConversionPolicyAtomWire.malformed_policy_refused`	P	proved
D	Decode every encoded bounded policy index exactly	`FinitePolicyAtomWire.decodeFin_encodeFin`	P	proved
D	Refuse a policy index at or above its declared table bound	`FinitePolicyAtomWire.out_of_range_refused`	P	proved
D	Decide the typed-receipt law for finite Nat/Vec profile manifests	`NatVecCheckingProfileAtomWire.Profiles.validCheck_eq_true_iff`	PCQ	proved
D	Preserve the typed-receipt law under manifest union	`NatVecCheckingProfileAtomWire.Profiles.valid_join`	PCQ	proved
D	Validate exactly the Nat/Vec manifests satisfying the receipt law	`NatVecCheckingProfileAtomWire.Profiles.validate_isSome_iff`	PCQ	proved
D	Embed the left Nat/Vec checking contract in a valid manifest join	`NatVecCheckingProfileAtomWire.Profiles.denote_extends_join_left`	PCQ	proved
D	Embed the right Nat/Vec checking contract in a valid manifest join	`NatVecCheckingProfileAtomWire.Profiles.denote_extends_join_right`	PCQ	proved
D	Decode every finite Nat/Vec checking capability exactly	`NatVecCheckingProfileAtomWire.Profiles.decodeFeature_encodeFeature`	PCQ	proved
D	Decode complete Nat/Vec checking manifests losslessly	`NatVecCheckingProfileAtomWire.Profiles.decode_encode`	PCQ	proved
D	Reflect Nat/Vec manifest equality through atom encoding	`NatVecCheckingProfileAtomWire.Profiles.encode_injective`	PCQ	proved
D	Recover exact valid inputs from every published Nat/Vec manifest join	`NatVecCheckingProfileAtomWire.Profiles.joinEncoded_sound`	PCQ	proved
D	Recover the existing receipt-only checking profile from its manifest	`NatVecCheckingProfileAtomWire.Profiles.denote_minimal_eq`	PCQ	proved
D	Recover the existing natural-intensional checking profile from its manifest	`NatVecCheckingProfileAtomWire.Profiles.denote_naturalIntensional_eq`	PCQ	proved
D	Recover the existing scoped logical-extensional profile from its manifest	`NatVecCheckingProfileAtomWire.Profiles.denote_scopedLogicalExtensional_eq`	PCQ	proved
D	Recover the existing full raw operational profile from its manifest	`NatVecCheckingProfileAtomWire.Profiles.denote_fullRawOperational_eq`	PCQ	proved
D	Validate the encoded natural-intensional profile exactly	`NatVecCheckingProfileAtomWire.Profiles.naturalIntensional_encoded_validated`	PCQ	proved
D	Retrieve the canonical typed natural-zero receipt from its licensed row	`NatVecCheckingProfileAtomWire.Profiles.naturalZeroLicensedReceipt_exact`	PCQ	proved
D	Retrieve the canonical typed natural-successor receipt from its licensed row	`NatVecCheckingProfileAtomWire.Profiles.naturalSuccLicensedReceipt_exact`	PCQ	proved
D	Keep vector-nil computation outside natural-intensional conversion	`NatVecCheckingProfileAtomWire.Profiles.naturalIntensional_vectorNil_not_judgmental`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Check dependent append through the encoded natural-intensional profile	`NatVecCheckingProfileAtomWire.Profiles.naturalIntensional_encoded_checks_append`	PCQ	proved
D	Refuse append evidence under the encoded receipt-only profile	`NatVecCheckingProfileAtomWire.Profiles.minimal_encoded_refuses_append`	PCQ	proved
D	Refuse judgmental iota authority without its retained receipt	`NatVecCheckingProfileAtomWire.Profiles.missingReceipt_encoded_refused`	PCQ	proved
D	Refuse an iota capability outside the admitted schema table	`NatVecCheckingProfileAtomWire.Profiles.out_of_range_iota_refused`	PCQ	proved
D	Refuse malformed Nat/Vec checking-profile data	`NatVecCheckingProfileAtomWire.Profiles.malformed_profile_refused`	PCQ	proved
D	Run all finite Nat/Vec profile-wire controls	`NatVecCheckingProfileAtomWire.Profiles.all_verdicts`	PCQ	proved
D	Identify the natural profile with its actual declaration-aware rule package	`CheckingProfiles.Interaction.naturalIntensional_actualRules`	PCQ	proved
D	Expose exactly the two selected Nat iota schemas at the profile boundary	`CheckingProfiles.Interaction.naturalIntensional_roots_exact`	PCQ	proved
D	Package Church–Rosser, Pi/Sigma boundaries and formed preservation for the natural profile	`CheckingProfiles.Interaction.naturalIntensionalCertificate`	PCQ	proved
D	Separate the natural carrier from dependent function types through full conversion	`CheckingProfiles.Interaction.naturalIntensional_nat_not_pi`	PCQ	proved
D	Retain the annotated-interaction obstruction for the full raw vector profile	`CheckingProfiles.Interaction.fullRawOperational_not_annotatedInteraction`	PCQ	proved
D	Attach the interaction certificate to the encoded natural profile	`NatVecCheckingProfileAtomWire.Profiles.naturalIntensional_encoded_qualified`	PCQ	proved
D	Exact finite local-rule witnesses	`FormationSensitiveChecking.interface`	PQ	proved
D	Original typing iff common tree acceptance	`FormationSensitiveChecking.typing_iff_checked_tree`	PCQ	proved
D	Original typing iff common shared-graph acceptance	`FormationSensitiveChecking.typing_iff_checked_graph`	PCQ	proved
D	Original typing iff logical theorem minting	`FormationSensitiveChecking.typing_iff_minted`	PCQ	proved
D	Maintained replay publishes common authority exactly	`FormationSensitiveChecking.mintReplay_iff`	PQ	proved
D	Full formed judgment iff finite request acceptance	`FormationSensitiveChecking.judgment_iff_checked`	PCQ	proved
D	Admitted substitution preserves checkable source evidence	`FormationSensitiveChecking.checked_substitute`	PCQ	proved
D	Checked request reconstructs the original formed judgment	`FormationSensitiveChecking.Request.sound`	PQ	proved
D	Cumulative tower finite evidence coverage	`Tower.Checking.coverage`	PCQ	proved
D	All positive and negative cumulative controls pass	`Tower.Checking.Controls.all_pass`	PQ	proved
D	Accepted requests hold in the named set-tower model	`TowerInterpretation.Checking.request_model_sound`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Every closed empty-type request refuses under the model hypothesis	`TowerInterpretation.Checking.empty_request_refuses`	PCQ	proved
D	Checked subjects denote members of their displayed types	`TowerInterpretation.Checking.checked_value_member`	PCQ	proved
D	Checked type conversion preserves actual set values	`TowerInterpretation.Checking.checked_type_conversion`	PCQ	proved
D	Checked subject conversion preserves actual set values	`TowerInterpretation.Checking.checked_term_conversion`	PCQ	proved
D	Checked components construct the original contextual arrow	`TowerInterpretation.Checking.checkedArrow`	PQ	proved
D	Checked values commute with admitted annotated substitution	`TowerInterpretation.Checking.checked_value_substitution`	PCQ	proved
D	Finite closed-universe requests exactly characterize set membership	`TowerInterpretation.Checking.universe_request_iff_set`	PCQ	proved
D	The same requests characterize membership in well-founded hypersets	`TowerInterpretation.Checking.universe_request_iff_hyperset`	PCQ	proved
O	Finite reachability is an equation-invariant native predicate	`GSLTTypeSynthesis.semanticReach`	none	proved
O	Equation-wise route agreement gives equal predicate actions	`EquationRespectingMap.pullback_eq_of_equiv`	PQ	proved
O	Forward strict routes preserve existential next-step observations	`OperationalTranslation.preserves_semanticDiamond`	PQ	proved
O	Operational route agreement preserves semantic reindexing	`OperationalTranslation.pullback_eq_of_equiv`	PQ	proved
O	Local coverage reflects existential next-step observations	`CoveredTranslation.semanticDiamond_iff`	PQ	proved
O	Covered routes preserve and reflect finite reachability	`CoveredTranslation.semanticReach_iff`	PQ	proved
O	Path-valued lowering preserves finite reachability	`OperationalRealization.preserves_semanticReach`	PQ	proved
O	Fusion need not preserve a one-step diamond	`OperationalRealization.FusionCanary.fusion_changes_one_step_diamond`	P	proved
O	Actual covered growth preserves the modal comparison	`IndexedOperational.Canary.covered_growth_semanticDiamond`	PQ	proved
O	Extra target behavior refutes modal reflection	`IndexedOperational.Canary.escaping_native_type_not_reflected`	PQ	proved
O	Outgoing coverage does not preserve the incoming box	`IndexedOperational.Canary.outgoing_coverage_does_not_preserve_past_box`	PQ	proved
O	Conversion paths equal the original structural conversion relation	`FormationSensitiveCheckingOSLF.path_iff_conversion`	P	proved
O	Conversion-native OSLF truth iff accepted finite evidence	`FormationSensitiveCheckingOSLF.satisfies_iff_checked`	PCQ	proved
O	Dependency restriction yields a genuine forward conversion route	`FormationSensitiveCheckingOSLF.restrictionRoute`	PQ	proved
O	Checked higher requests admit a finite sufficient source-reflecting restriction	`FormationSensitiveCheckingOSLF.checked_request_has_restriction`	PQ	proved
O	Reverse beta inhabits the conversion native observation	`Tower.Checking.Controls.reverse_beta_oslf`	PCQ	proved
O	Raw beta observation carries no typing premise	`Tower.Checking.Controls.raw_beta_oslf`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
O	Conversion-native truth does not admit a restricted computation package	`FormationSensitive.RestrictionPreservation.oslf_observation_does_not_admit_restriction`	PCQ	proved
O	The repaired smaller package retains conversion and root preservation	`FormationSensitive.RestrictionPreservation.oslf_observation_with_preserving_repair`	PCQ	proved
O	Guarded OSLF term conversion preserves actual set values	`TowerInterpretation.Checking.oslf_term_conversion`	PCQ	proved
S	Generic conversion observation presents the authored relation	`IndexedOperational.conversionTheory_path_iff`	none	proved
S	Native reachability has exactly the declared conversion support	`IndexedOperational.satisfies_conversionNativeType_iff`	PCQ	proved
S	Simple type interpretation retains the atomic sort	`SingleBaseSTLC.HOLComparison.type_injective`	none	proved
S	Typed HOL syntax retains source terms	`SingleBaseSTLC.HOLComparison.term_injective`	none	proved
S	Renaming commutes with the HOL interpretation	`SingleBaseSTLC.HOLComparison.rename_term`	Q	proved
S	Simultaneous substitution commutes with the HOL interpretation	`SingleBaseSTLC.HOLComparison.substitute_term`	PQ	proved
S	The simple context interpretation is a functor	`SingleBaseSTLC.HOLComparison.contextFunctor`	PQ	proved
S	HOL beta joinability reflects intrinsic beta conversion	`SingleBaseSTLC.HOLComparison.holJoinable_iff_betaConv`	PCQ	proved
S	The HOL and structural tower conversions agree on the image	`SingleBaseSTLC.HOLComparison.holJoinable_iff_towerConv`	PCQ	proved
S	The intrinsic checker decides the HOL observation	`SingleBaseSTLC.HOLComparison.decideConversion_iff_holJoinable`	PCQ	proved
S	Beta conversion supplies retained HOL equality evidence	`SingleBaseSTLC.HOLComparison.equality_proof_of_conversion`	PCQ	proved
S	HOL equality need not reflect into structural conversion	`SingleBaseSTLC.HOLComparison.Controls.hol_equality_does_not_reflect_conversion`	PCQ	proved
S	Typed HOL terms distinguish a raw tower erasure collision	`SingleBaseSTLC.HOLComparison.Controls.hol_retains_erased_types`	none	proved
S	Conversion trees elaborate to retained HOL proofs	`SingleBaseSTLC.HOLConversionBubble.elaborate`	PCQ	proved
S	The beta bubble is exact for structural tower conversion	`SingleBaseSTLC.HOLConversionBubble.beta_path_iff_tower`	PCQ	proved
S	Native predicate truth is the support of retained paths	`SingleBaseSTLC.HOLConversionBubble.satisfies_iff_path`	PCQ	proved
S	The beta native predicate has a total decision procedure	`SingleBaseSTLC.HOLConversionBubble.beta_satisfies_iff_decides`	PCQ	proved
S	Eta strictly enlarges the local conversion policy	`SingleBaseSTLC.HOLConversionBubble.Controls.eta_strictly_adds_conversion`	PCQ	proved
S	The native observation distinguishes beta and eta policies	`SingleBaseSTLC.HOLConversionBubble.Controls.eta_observation_depends_on_policy`	PCQ	proved
S	Pure simple terms require no interpreted constants	`SingleBaseSTLC.HOLReadingComparison.reads`	PQ	proved
S	The actual dependent reader constructs typed codes	`SingleBaseSTLC.HOLReadingComparison.code_typed`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
S	Computed native codes commute with the maintained substitution	`SingleBaseSTLC.HOLReadingComparison.code_substitute`	PQ	proved
S	The beta native observation gives dependent typed equality	`SingleBaseSTLC.HOLReadingComparison.observed_typed`	PCQ	proved
S	Observed equality reaches the explicit set-tower model	`SingleBaseSTLC.HOLTowerModel.observed_set_equality`	PCQ	proved
S	Typed eta equality is distinct from structural conversion	`SingleBaseSTLC.HOLTowerModel.typed_equality_not_structural_conversion`	PCQ	proved
E	Joint admissibility over one base	`extendRules_union_sameDerivability_iff`	P	proved
E	Combine actual certificate compilers	`AdditionalRuleCertificateCompiler.combine`	PCQ	proved
E	Mixed certificate elimination is conservative	`AdditionalRuleCertificateCompiler.combine_sameDerivability`	PCQ	proved
E	Mixed-feature expansion has three base nodes	`ConservativeExtensionCanary.compiledMixed_count`	PCQ	proved
E	Admissible package cannot license a new fact	`ConservativeExtensionCanary.combined_newFact_forbidden`	PCQ	proved
E	Compute retained evidence and recovery receipt	`EqualityRecipe.checkedPathWithRecovery`	PQ	proved
E	Recover the complete submitted recipe	`EqualityRecipe.forget_checkedPath`	PQ	proved
E	Retained evidence replays at its endpoints	`EqualityRecipe.evaluate_forgetPath`	PQ	proved
H	Instantiate licensed equations with scope	`HOL.EquationProofs.instantiate`	PQ	proved
H	Combine equation proof receipts	`HOL.EquationProofs.append`	P	proved
H	Licensed compatible source steps have HOL proofs	`HOL.SourceStep.hasHOLProof`	PCQ	proved
H	Licensed source articles have HOL proofs	`HOL.CoreConversion.hasHOLProof`	PCQ	proved
H	Logical retyping elaborates to HOL evidence	`HOL.ProofSyntaxModulo.hasHOLProof`	PCQ	proved
H	Curry equation cannot obtain HOL receipts	`HOL.ConversionElaborationControls.no_curry_equationProofs`	PCQ	proved
H	Exact local conversion-policy replay	`HOL.ConversionRecipes.leaf_authority`	PCQ	proved
H	Exact authority for selected finite recipes	`HOL.ConversionRecipes.authority`	PCQ	proved
H	Permissions follow the contracted redex type	`HOL.ConversionRecipes.Policy.betaAllowed_mono`	none	proved
H	Policy routes preserve conversion support	`HOL.ConversionRecipes.LocalStep.conversion_mono`	none	proved
H	Construct retained HOL equality proofs	`HOL.ConversionRecipes.elaborate`	PCQ	proved
H	Replace logical casts with equality elimination	`HOL.ConversionRecipes.convertProof`	PCQ	proved
H	Native observation is certificate existence	`HOL.ConversionRecipes.observed_iff_accepted`	PCQ	proved
H	Operational observation follows policy edges	`HOL.ConversionRecipes.observed_mono`	PCQ	proved
H	Untrusted proposals produce checked HOL evidence	`HOL.ConversionRecipes.elaborateSubmission`	PCQ	proved
H	Convergent equality retains different receipts	`HOL.ConversionRecipes.Controls.eta_and_equation_recipes_distinct`	none	proved

Part	Claim	Lean declaration	Axioms	Status
H	Equal endpoints retain different proof sizes	`HOL.ConversionRecipes.Controls.proof_costs_differ`	PCQ	proved
H	Two-sort conversion and elaboration controls	`HOL.ConversionRecipes.Controls.all_verdicts`	PCQ	proved
H	Least upper policy for joined generators	`HOL.ConversionRecipes.Policy.join_extends_iff`	P	proved
H	Compile actual typed beta executions	`HOL.ConversionRecipes.executionRecipe`	PCQ	proved
H	The execution compiler produces accepted recipes	`HOL.ConversionRecipes.executionRecipe_checked`	PCQ	proved
H	Previous beta observation reaches the recipe checker	`HOL.ConversionRecipes.joinable_has_betaRecipe`	PCQ	proved
I	Retain selected schema positions and substitutions	`SchemaTableConversionCode.retained`	PQ	proved
I	Finite schema decoding is sound	`SchemaTableConversionCode.decode_sound`	PQ	proved
I	Finite schema decoding covers every selected instance	`SchemaTableConversionCode.decode_complete`	PQ	proved
I	Qualify selected schema-root replay	`SchemaTableConversionCode.rootDecoder`	PQ	proved
I	Root replay commutes with renaming	`SchemaTableConversionCode.decode_rename`	PQ	proved
I	Root replay commutes with substitution	`SchemaTableConversionCode.decode_substitute`	PQ	proved
I	Check formed dependent substitutions	`FormationSensitiveChecking.SubstitutionRequest.sound`	PQ	proved
I	Admitted substitutions have finite evidence	`FormationSensitiveChecking.SubstitutionRequest.complete`	PCQ	proved
I	Compute the original typed schema receipt	`IndexedFamily.Checking.receiptAt`	PQ	proved
I	Reconstruct the original iota witness	`IndexedFamily.Checking.realizeEvidence`	PQ	proved
I	Selected computation includes into the declaration calculus	`IndexedFamily.Checking.inclusion`	PQ	proved
I	Inclusion retains full formed judgments	`IndexedFamily.Checking.judgment_inclusion`	PQ	proved
I	Selected conversion has a forward operational route	`IndexedFamily.Checking.conversionRoute`	PQ	proved
I	Checked instantiation reconstructs typed iota evidence	`IndexedFamily.Checking.InstanceRequest.replay`	PQ	proved
I	Typed iota publication checks its target context	`IndexedFamily.Checking.InstanceRequest.formed`	PQ	proved
I	Typed requests export accepted root codes	`IndexedFamily.Checking.InstanceRequest.rootCode_accepted`	PQ	proved
I	Exact decoding of all four Nat/Vec iota cases	`NativeNaturalVectorFamilies.Checking.iotaDecoder`	PCQ	proved
I	Join actual Nat/Vec roots with declaration replay	`NativeNaturalVectorFamilies.Checking.decoder`	PCQ	proved
I	Actual Nat/Vec conversion has exact finite evidence	`NativeNaturalVectorFamilies.Checking.conversion_iff_checked`	PCQ	proved
I	Connect the Nat/Vec formation-sensitive checker	`NativeNaturalVectorFamilies.Checking.coverage`	PCQ	proved
I	Typed iota and dependent-substitution controls	`IndexedFamilyCheckingControls.all_verdicts`	PCQ	proved
E	Replay a generated row at its original ordinal	`SchemaTableConversionCode.decode_at`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
E	All selected schema roots equal the original schema support	`SchemaTableConversionCode.all_roots_iff`	PQ	proved
E	Use independent source presentations in common replay	`SchemaTableConversionCode.presentedDecoder`	PQ	proved
E	Generated simple iota rules have the exact finite presentation	`IndexedPolynomialGenerated.Checking.presents`	PQ	proved
E	Connect generated roots to the common conversion decoder	`IndexedPolynomialGenerated.Checking.decoder`	PQ	proved
E	Recover the schema of the original constructor occurrence	`IndexedPolynomialGenerated.Checking.table_get`	PQ	proved
E	Replay generated receipts with their complete assignments	`IndexedPolynomialGenerated.Checking.receiptCode_accepted`	PQ	proved
E	Generated conversion has exact finite evidence	`IndexedPolynomialGenerated.Checking.conversion_iff_checked`	PQ	proved
E	Generated receipts compile to accepted recipes	`IndexedPolynomialGenerated.Checking.receiptRecipe_accepted`	PQ	proved
E	Accepted submissions give original formed judgments	`FormationSensitiveElaboration.replay_sound`	PQ	proved
E	Bidirectional evidence production is sound after replay	`FormationSensitiveElaboration.elaborate_sound`	PQ	proved
E	Inferred implicit assignments retain all determined slots	`FormationSensitiveElaboration.Implicit.complete_sound`	PQ	proved
E	Implicit insertion checks the actual rendered application	`FormationSensitiveElaboration.Implicit.elaborate_sound`	PQ	proved
E	Generated replay and elaboration controls	`InductiveElaborationControls.all_verdicts`	PCQ	proved
E	Infer both omitted vector-cons arguments and check the result	`InductiveElaborationControls.inferredSingleton_typed`	PCQ	proved
E	Proposed normalization carries a checked conversion recipe	`StructuralConversionSearch.normalize_sound`	PQ	proved
E	Bounded comparison produces sound conversion evidence	`StructuralConversionSearch.compare_sound`	PQ	proved
E	Instantiate the motive universe at the element/motive maximum	`NativeNaturalVectorFamilies.Programs.levelInstance`	PQ	proved
E	The selected Nat/Vec universe instance is formed	`NativeNaturalVectorFamilies.Programs.signature_formed`	PCQ	proved
E	Transport full judgments through that universe instance	`NativeNaturalVectorFamilies.Programs.judgment_map`	PQ	proved
E	Independent replay accepts the finite append evidence	`NativeNaturalVectorFamilies.Programs.appendChecked_true`	PCQ	proved
E	The complete closed dependent append function is typed	`NativeNaturalVectorFamilies.Programs.append_typed`	PCQ	proved
E	Refuse changed append result indices in the submitted evidence	`NativeNaturalVectorFamilies.Programs.changedType_refused`	PCQ	proved
E	Refuse a changed append program with the same submission	`NativeNaturalVectorFamilies.Programs.changedTerm_refused`	PCQ	proved
E	Refuse a forged append root typing rule	`NativeNaturalVectorFamilies.Programs.forgedRoot_refused`	PCQ	proved
E	Dependent append acceptance and refusal controls	`NativeNaturalVectorFamilies.Programs.all_verdicts`	PCQ	proved
I	Compute a retained natural-eliminator receipt	`IndexedFamilyCheckingControls.checkedZeroReceipt`	PCQ	proved
K	Append commutes with capture-safe substitution	`NativeNaturalVectorFamilies.Programs.Computation.append_substitute`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
K	Append nil conversion holds for arbitrary scoped arguments	`NativeNaturalVectorFamilies.Programs.Computation.append_nil`	PCQ	proved
K	Append cons conversion retains the recursive append result	`NativeNaturalVectorFamilies.Programs.Computation.append_cons`	PCQ	proved
K	Refuse omission of append recursive computation	`NativeNaturalVectorFamilies.Programs.Computation.wrongBranch_refused`	PCQ	proved
K	Symbolic append computation controls	`NativeNaturalVectorFamilies.Programs.Computation.all_verdicts`	PCQ	proved
K	Inverse substitution reconstructs its original scoped source	`ScopedSubstitutionInversion.invert_sound`	PQ	proved
K	Recursive-call abstraction preserves binder scope	`ScopedSubstitutionInversion.Controls.all_verdicts`	PQ	proved
K	Constructor clauses retain their actual names and arities	`IndexedInductive.CaseElaboration.adaptClause_name`	none	proved
K	Compiled methods reconstruct written recursive child calls	`IndexedInductive.CaseElaboration.compileBranch_reconstructs`	PQ	proved
K	Constructor clauses cover the exact ordered admitted inventory	`IndexedInductive.CaseElaboration.compileMethods_coverage`	PQ	proved
K	Constructor-case elaboration checks its actual eliminator application	`IndexedInductive.CaseElaboration.elaborate_sound`	PQ	proved
K	Written append clauses generate the independent reference methods	`IndexedInductive.AppendElaboration.methods_match_reference`	PQ	proved
K	Scoped append compilation gives the checked closed program	`IndexedInductive.AppendElaboration.compiled_matches_reference`	PQ	proved
K	The compiled append function retains its dependent type	`IndexedInductive.AppendElaboration.compiled_typed`	PCQ	proved
K	The source-compiled append is typed by the natural-only policy	`IndexedInductive.AppendElaboration.compiled_typed_naturalOnly`	PCQ	proved
K	Uniform scoped append clause acceptance and refusal controls	`IndexedInductive.AppendElaboration.all_verdicts`	PQ	proved
K	Retain schema evidence when permissions increase	`SchemaTableConversionCode.Evidence.weakenPermissions`	P	proved
K	Schema-root permissions include monotonically	`SchemaTableConversionCode.root_mono`	PQ	proved
K	Restricted Nat/Vec computation includes into the original signature	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.includes_original`	PCQ	proved
K	Nat/Vec declaration permissions include monotonically	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.permissions_mono`	PCQ	proved
K	Restricted checking preserves full original judgments	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.judgment_inclusion`	PCQ	proved
K	Existing append evidence checks with the two natural iota rows	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.append_checked_naturalOnly`	PCQ	proved
K	The smaller policy types the full dependent append function	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.append_typed_naturalOnly`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
K	Checking and computation permission controls	`NativeNaturalVectorFamilies.Programs.ComputationProfiles.all_verdicts`	PCQ	proved
K	The admitted natural eliminator computes at zero	`IndexedInductiveNaturalHOL.recurse_zero`	P	proved
K	The admitted natural eliminator computes at successor	`IndexedInductiveNaturalHOL.recurse_succ`	P	proved
K	The natural recursion equations determine their actual function	`IndexedInductiveNaturalHOL.recurse_unique`	PQ	proved
K	The authored natural carrier has unique recursion	`IndexedInductiveNaturalHOL.recursion_exists_unique`	PQ	proved
K	The authored natural carrier has full predicate induction	`IndexedInductiveNaturalHOL.induction`	PQ	proved
K	The authored carrier supplies the existing Peano structure	`IndexedInductiveNaturalHOL.peanoStructure`	PQ	proved
K	The source-carrier HOL interpretation has full domains	`IndexedInductiveNaturalHOL.fullDomains`	PQ	proved
K	The source carrier satisfies the existing HOL Peano formulas	`IndexedInductiveNaturalHOL.models_axioms`	PQ	proved
K	The HOL interpretation retains natural no-confusion	`IndexedInductiveNaturalHOL.models_noConfusion`	PQ	proved
K	The HOL interpretation satisfies second-order induction	`IndexedInductiveNaturalHOL.models_induction`	PQ	proved
K	The source eliminator satisfies the HOL unique-recursion formula schema	`IndexedInductiveNaturalHOL.models_recursion`	PQ	proved
K	The authored natural carrier is infinite	`IndexedInductiveNaturalHOL.infinite_carrier`	PCQ	proved
K	The actual interpreted successor cannot collapse to zero	`IndexedInductiveNaturalHOL.successor_zero_refused`	PQ	proved
K	Finite case plans reconstruct the written recursive calls	`IndexedInductive.CaseElaboration.compilePlan_reconstructs`	PQ	proved
K	Finite case plans retain exact ordered clause coverage	`IndexedInductive.CaseElaboration.compilePlans_coverage`	PQ	proved
K	The selected direct module reconstructs every source block	`IndexedInductive.OperationSource.selectModule_exact`	P	proved
K	Lexical symbol expansion commutes with substitution	`IndexedInductive.OperationSource.Symbol.instantiate_substitute`	PQ	proved
K	Named symbol applications retain substituted arguments	`IndexedInductive.OperationSource.applySymbol_substitute`	PQ	proved
K	Named operation lowering commutes with substitution	`IndexedInductive.OperationSource.lower_substitute`	PQ	proved
K	Named operation expansion and scope controls	`IndexedInductive.OperationSource.Controls.all_verdicts`	PCQ	proved
K	Named constructor-clause and resolution controls	`IndexedInductive.OperationSource.Controls.all_clauseVerdicts`	PCQ	proved
K	The actual textual program retains its complete module data	`IndexedInductive.AppendSource.source_reconstructed`	P	proved
K	Textual declarations lower to the independent admitted vectors	`IndexedInductive.AppendSource.text_admitted`	P	proved
K	Executable source plans agree with the admitted semantic inventory	`IndexedInductive.AppendSource.source_vector_plans_match`	PCQ	proved
K	Textual addition compiles from its own source clauses	`IndexedInductive.AppendSource.addition_matches_reference`	PCQ	proved
K	Textual append compiles to the independent checked core	`IndexedInductive.AppendSource.append_matches_reference`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
K	The actual textual append has the full dependent function type	`IndexedInductive.AppendSource.append_typed`	PCQ	proved
K	Textual append retains its type in the natural-only policy	`IndexedInductive.AppendSource.append_typed_naturalOnly`	PCQ	proved
K	Independent replay checks the actual textual producer output	`IndexedInductive.AppendSource.checkProposal_sound`	PCQ	proved
K	Textual append acceptance and malformed-source controls	`IndexedInductive.AppendSource.all_verdicts`	PCQ	proved
L	The exact closure body denotes quantified predicate closure	`HOL.NaturalClosure.denote_body`	none	proved
L	Applied native closure retains its selected model meaning	`HOL.NaturalClosure.denote_predicate`	none	proved
L	Every base point satisfies its closure predicate	`HOL.NaturalClosure.closedUnder_zero`	none	proved
L	Admitted successors preserve predicate closure	`HOL.NaturalClosure.closedUnder_successor`	none	proved
L	Strengthened closure gives induction in full-domain models	`HOL.NaturalClosure.closedUnder_induction`	none	proved
L	The exact zero statement is valid in the selected Henkin semantics	`HOL.NaturalClosure.models_zero`	none	proved
L	The exact successor statement is valid in Henkin semantics	`HOL.NaturalClosure.models_successor`	none	proved
L	The exact induction statement holds in full-domain models	`HOL.NaturalClosure.models_induction`	none	proved
N4	Binder naming commutes with lifted variable renaming	`HOL.Names.lift_comp`	Q	proved
N4	Intrinsic HOL renaming commutes with MM0 HOL syntax production	`MM0.HOLSyntaxWire.encodeTerm_rename`	PQ	proved
N4	Printed natural-closure body retains the shared Henkin meaning	`MM0.NaturalClosure.definition_meaning`	none	proved
N4	Concrete MM0 closure names retain binder separation	`MM0.NaturalClosure.reference_names_distinct`	none	proved
N4	Zero is generated in the finite collapsing interpretation	`MM0.NaturalClosure.finite_zero`	none	proved
N4	Another finite inhabitant need not belong to natural closure	`MM0.NaturalClosure.finite_other_refused`	none	proved
N4	MM0 HOL closure reference controls pass	`MM0.NaturalClosure.all_verdicts`	P	proved
L	Independent intrinsic HOL syntax erases to the native closure body	`Megalodon.NaturalClosure.body_erases`	P	proved
L	The actual native closure definition has its declared type	`Megalodon.NaturalClosure.native_body_typed`	P	proved
L	The three native closure statements are formed	`Megalodon.NaturalClosure.native_statements_formed`	P	proved
L	Native zero unfolding agrees with the intrinsic statement	`Megalodon.NaturalClosure.zero_statement_erases`	PCQ	proved
L	Native successor unfolding agrees with the intrinsic statement	`Megalodon.NaturalClosure.successor_statement_erases`	PCQ	proved
L	Native induction unfolding agrees with the intrinsic statement	`Megalodon.NaturalClosure.induction_statement_erases`	PCQ	proved
L	The finite native zero proof checks against its exact claim	`Megalodon.NaturalClosure.zero_proof_checked`	PCQ	proved
L	The finite native successor proof checks against its exact claim	`Megalodon.NaturalClosure.successor_proof_checked`	PCQ	proved
L	Native closure acceptance and altered-evidence controls	`Megalodon.NaturalClosure.all_verdicts`	PCQ	proved
L	Every authored natural satisfies the exact closure predicate	`IndexedInductiveNaturalMegalodon.source_member`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
L	The source interpretation satisfies the closure membership formula	`IndexedInductiveNaturalMegalodon.source_membership_formula`	PQ	proved
L	The three native formulas hold on the authored carrier	`IndexedInductiveNaturalMegalodon.source_native_formulas`	PQ	proved
L	Closure-restricted induction retains every authored source value	`IndexedInductiveNaturalMegalodon.source_induction`	PQ	proved
L	A collapsed successor excludes a genuine source successor	`IndexedInductiveNaturalMegalodon.collapsing_successor_refused`	PQ	proved
L	The collapsed-operation negative has nonempty closure	`IndexedInductiveNaturalMegalodon.collapsing_zero_accepted`	PQ	proved
L	Composed certificate links agree with their address composition	`SchemaTableConversionCode.Linking.relocate_comp`	P	proved
L	Schema relocation preserves both acceptance and refusal	`SchemaTableConversionCode.Linking.decode_relocate`	PQ	proved
L	A schema embedding preserves the selected root judgment	`SchemaTableConversionCode.Linking.root_inclusion`	PQ	proved
L	Package linking and permission controls	`SchemaTableConversionCode.Linking.Controls.all_verdicts`	PQ	proved
L	The informative source vector table joins the exact natural prefix	`IndexedInductive.AppendChecking.generated_table_reference`	PCQ	proved
L	The generated library matches the independent declaration inventory	`IndexedInductive.AppendChecking.generated_library_reference`	PCQ	proved
L	The informative candidate retains the reference declaration entries	`IndexedInductive.AppendChecking.candidate_entries_reference`	PCQ	proved
L	Candidate computation includes into the independent Nat/Vec roots	`IndexedInductive.AppendChecking.candidate_root_inclusion`	PCQ	proved
L	Both declarations and roots extend from the generated candidate	`IndexedInductive.AppendChecking.candidate_extension`	PCQ	proved
L	The generated source calculus preserves full reference judgments	`IndexedInductive.AppendChecking.candidate_judgment_inclusion`	PCQ	proved
L	The candidate route composes with the selected universe instance	`IndexedInductive.AppendChecking.candidate_judgment_to_program`	PCQ	proved
L	Generated and executable checking policies agree exactly	`IndexedInductive.AppendChecking.signature_comparison`	PCQ	proved
L	The actual textual declarations select the informative candidate table	`IndexedInductive.AppendChecking.textual_candidate_table`	PCQ	proved
L	The actual textual append is typed by the generated checking policy	`IndexedInductive.AppendChecking.textual_append_typed`	PCQ	proved
L	Finite replay checks the term in the generated package policy	`IndexedInductive.AppendChecking.replay_sound`	PCQ	proved
L	Vector receipts retain their source bounds and global permissions	`IndexedInductive.AppendChecking.linked_vector_replay`	PCQ	proved
L	Package linking and permission controls	`IndexedInductive.AppendChecking.all_verdicts`	PCQ	proved
L	The natural-only package policy checks the actual textual append	`IndexedInductive.AppendChecking.replay_naturalOnly`	PCQ	proved
L	Fixed-row typed substitutions have exact finite certificate coverage	`Declaration.IndexedFamily.Checking.InstanceRequest.checked_arguments_iff`	PCQ	proved
L	Selected original rows present exactly the selected roots	`SchemaTableConversionCode.selected_roots_iff`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
L	Opaque declaration extensions expose only their declared roots	`DeclarationConversionCode.opaque_roots_iff`	PQ	proved
L	The finite pattern check establishes the existing lifting condition	`Annotated.SchemaTable.linearCheck_sound`	PQ	proved
L	Linear selected patterns lift every raw root at an annotated input	`Annotated.SchemaTable.root_lift`	PQ	proved
L	Root permissions retain the annotated declarations	`Programs.Annotated.constantType_eq`	PCQ	proved
L	The actual natural-only roots lift uniformly with annotations	`Programs.Annotated.lift_natural_root`	PCQ	proved
L	The vector nil pattern repeats its parameter	`Programs.Annotated.vector_nil_repeats_parameter`	P	proved
L	The vector cons pattern repeats its length index	`Programs.Annotated.vector_cons_repeats_index`	P	proved
L	All four canonical endpoints have independent annotated typing	`Programs.Annotated.endpoints_typed`	PCQ	proved
L	The actual annotated canonical telescopes are formed	`Programs.Annotated.context_formed`	PCQ	proved
L	Canonical right sides typecheck against left-inferred templates	`Programs.Annotated.template_typed`	PCQ	proved
L	Annotated root preservation conditional on former injectivity	`Programs.Annotated.root_preserving_of_formerFacts`	PCQ	proved
L	Former inversion discharges the actual schema premises	`Programs.Annotated.root_premised_of_formerFacts`	PCQ	proved
L	Former facts suffice for the actual annotated root equalities	`Programs.Annotated.root_admitted_of_formerFacts`	PCQ	proved
L	Licensed rows give typed equalities under typed assignments	`Programs.Annotated.instantiated_equal`	PCQ	proved
L	Erasure preserves the selected typed instance equality	`Programs.Annotated.erased_instantiated_equal`	PCQ	proved
L	Each original row has a canonical typed equality witness	`Programs.Annotated.canonical_equal`	PCQ	proved
L	Canonical typing metadata and changed-type refusal controls	`Programs.Annotated.all_metadataVerdicts`	PCQ	proved
L	Natural lifting and repeated vector-parameter controls	`Programs.Annotated.all_verdicts`	PCQ	proved
L	The full textual declaration source supplies the complete generated library	`IndexedInductive.AppendChecking.textual_candidate_library`	P	proved
L	Actual submitted proposals replay in the generated policy	`IndexedInductive.AppendChecking.replayProposal_sound`	PCQ	proved
L	Live source proposals retain their generated checking policy	`IndexedInductive.AppendChecking.replaySource_sound`	PCQ	proved
I	The recursion graph is functional under successor separation	`NaturalRecursion.Graph.functional`	none	proved
I	Graph values agree with every recursive solution	`NaturalRecursion.Graph.agrees`	none	proved
I	Every generated number has a graph value	`NaturalRecursion.Graph.exists_of_generated`	none	proved
I	A lawful selector selects a graph value	`NaturalRecursion.selected_graph`	none	proved
I	The selected recursor satisfies its zero equation	`NaturalRecursion.selected_zero`	none	proved
I	The selected recursor satisfies its successor equation	`NaturalRecursion.selected_successor`	none	proved
I	Selected recursion is unique on the generated carrier	`NaturalRecursion.selected_unique`	none	proved
I	Full-domain Leibniz equality has the declared meaning	`NaturalRecursion.denote_equality`	none	proved

Part	Claim	Lean declaration	Axioms	Status
I	Full-domain encoded existence has the declared meaning	`NaturalRecursion.denote_existential`	none	proved
I	The typed graph denotes the independent inductive graph	`NaturalRecursion.denote_graph_iff`	none	proved
I	The typed uniqueness clause has its pointwise meaning	`NaturalRecursion.denote_uniqueness`	PQ	proved
I	The typed zero-separation interface retains its meaning	`NaturalRecursion.denote_zeroExcluded`	PQ	proved
I	The typed injectivity interface retains its meaning	`NaturalRecursion.denote_successorInjective`	PQ	proved
I	The typed choice interface retains its selection law	`NaturalRecursion.denote_choiceCondition`	PQ	proved
I	The typed recursion clauses retain their equations	`NaturalRecursion.denote_recursionProperty`	PQ	proved
I	Full-domain models satisfy conditional unique recursion	`NaturalRecursion.models_recursion`	PQ	proved
I	The independent typed graph matches its exported definition	`NaturalRecursion.graph_source_definition`	P	proved
I	Typed Leibniz equality matches the exported definition	`NaturalRecursion.equality_source_definition`	P	proved
I	Typed existence matches the exported definition	`NaturalRecursion.existential_source_definition`	P	proved
I	The independent typed recursion statement erases exactly	`NaturalRecursion.native_statement_erases`	P	proved
I	The independent native recursion statement is formed	`NaturalRecursion.native_statement_formed`	P	proved
I	The authored eliminator supplies a recursion graph value	`IndexedInductiveNaturalMegalodon.source_recursion_graph`	PQ	proved
I	The graph is exactly the authored eliminator graph	`IndexedInductiveNaturalMegalodon.source_graph_iff`	PQ	proved
I	Every lawful selector agrees with the authored eliminator	`IndexedInductiveNaturalMegalodon.source_selector_agrees`	PQ	proved
I	Lawful selectors agree on every authored natural	`IndexedInductiveNaturalMegalodon.source_selectors_agree`	PQ	proved
I	The classical interpretation supplies scoped inhabited choice	`IndexedInductiveNaturalMegalodon.source_recursion_with_choice`	PCQ	proved
I	The authored full-domain model satisfies the native schema	`IndexedInductiveNaturalMegalodon.source_recursion_statement`	PQ	proved
I	Source successor discharges both typed interfaces	`IndexedInductiveNaturalMegalodon.source_successor_formulas`	PQ	proved
I	The recursion graph contains a genuine successor result	`IndexedInductiveNaturalMegalodon.source_recursion_nonempty`	none	proved
I	A constant-zero selector fails a genuine source computation	`IndexedInductiveNaturalMegalodon.constant_selector_wrong_result`	PQ	proved
I	Original selected natural rows are exactly the constructor schemas	`ComputationProfiles.NaturalConversion.schema_iff`	PCQ	proved
I	Actual natural-only roots have exact schema coverage	`ComputationProfiles.NaturalConversion.roots_exact`	PCQ	proved
I	The actual natural schemas form a constructor system	`ComputationProfiles.NaturalConversion.system`	PCQ	proved
I	Natural-only structural conversion is confluent	`ComputationProfiles.NaturalConversion.churchRosser`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Natural-only conversion has the Pi boundary	`ComputationProfiles.NaturalConversion.pi_boundary`	PCQ	proved
I	Natural-only conversion has the Sigma boundary	`ComputationProfiles.NaturalConversion.sigma_boundary`	PCQ	proved
I	Natural-only roots require the original four-argument eliminator	`ComputationProfiles.NaturalConversion.root_head`	PCQ	proved
I	Bare declared constants do not compute in this policy	`ComputationProfiles.NaturalConversion.constant_normal`	PCQ	proved
I	Actual addition at zero is a permitted conversion	`ComputationProfiles.NaturalConversion.addition_zero`	PCQ	proved
I	Structural conversion separates the natural type from Pi	`ComputationProfiles.NaturalConversion.natural_not_pi`	PCQ	proved
I	Typed substitutions construct arbitrary dependent declaration spines	`FormationSensitive.DeclarationSpine.applyTelescope`	PQ	proved
I	Observed recursor typing retains its genuine declaration formation	`ComputationProfiles.NaturalConversion.Preservation.recursor_spine`	PCQ	proved
I	Recover actual dependent recursor arguments and result adjustment	`ComputationProfiles.NaturalConversion.Preservation.recover`	PCQ	proved
I	The actual selected zero root uniformly preserves typing	`ComputationProfiles.NaturalConversion.Preservation.zero_preserves`	PCQ	proved
I	Recover a successor predecessor at its declared domain	`ComputationProfiles.NaturalConversion.Preservation.successor_argument`	PCQ	proved
I	The actual selected successor root uniformly preserves typing	`ComputationProfiles.NaturalConversion.Preservation.successor_preserves`	PCQ	proved
I	All actual natural-only roots uniformly preserve typing	`ComputationProfiles.NaturalConversion.Preservation.roots`	PCQ	proved
I	Natural-only contextual computation preserves full formed judgments	`ComputationProfiles.NaturalConversion.Preservation.steps_preserve`	PCQ	proved
I	An actual checked zero redex witnesses preservation	`ComputationProfiles.NaturalConversion.Controls.zero_typed`	PCQ	proved
I	An actual checked successor redex witnesses preservation	`ComputationProfiles.NaturalConversion.Controls.successor_typed`	PCQ	proved
I	The uniform zero theorem applies to a real checked redex	`ComputationProfiles.NaturalConversion.Controls.zero_preserved`	PCQ	proved
I	Four actual steps compute the selected successor example	`ComputationProfiles.NaturalConversion.Controls.successor_path`	PCQ	proved
I	The full successor computation retains its exact type	`ComputationProfiles.NaturalConversion.Controls.successor_preserved`	PCQ	proved
I	The same natural proof submission refuses an altered Pi result	`ComputationProfiles.NaturalConversion.Controls.altered_type_refused`	PCQ	proved
I	Natural preservation controls accept and refuse their intended inputs	`ComputationProfiles.NaturalConversion.Controls.all_verdicts`	PCQ	proved
N5	The original Nat declaration retains its universe typing	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.natural_typed`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
N5	The original zero declaration has type Nat	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.zero_typed`	PCQ	proved
N5	The original successor has its dependent-function type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.successor_constant_typed`	PCQ	proved
N5	Successor preserves the actual annotated Nat type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.successor_typed`	PCQ	proved
N5	Successor transports actual typed natural equalities	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.successor_equal`	PCQ	proved
N5	The natural type lies in the selected motive universe	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.natural_in_universe`	PCQ	proved
N5	The selected motive universe has its successor typing	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.universe_typed`	PCQ	proved
N5	The natural motive function type is formed	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.motive_type`	PCQ	proved
N5	The actual annotated natural motive has its declared type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.motive_typed`	PCQ	proved
N5	The applied actual motive retains its universe typing	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.motive_applied`	PCQ	proved
N5	The actual motive computes by a finite typed beta equality	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.motive_beta`	PCQ	proved
N5	The recursive natural-method domain is formed	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_inner_type`	PCQ	proved
N5	The full natural method function type is formed	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_type`	PCQ	proved
N5	The actual method body retains its indexed motive result	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.method_body_typed`	PCQ	proved
N5	The actual recursive method abstraction is typed	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_inner_typed`	PCQ	proved
N5	The complete annotated natural method has its declared type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_typed`	PCQ	proved
N5	Two typed beta steps compute the actual natural method	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.step_apply_equal`	PCQ	proved
N5	The original natural recursor has its closed telescope type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.recursor_typed`	PCQ	proved
N5	Actual recursor arguments form a typed simultaneous substitution	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.arguments_typed`	PCQ	proved
N5	Recursor application retains the actual indexed motive result	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.eliminate_typed`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
N5	Annotated addition is typed for arbitrary typed natural arguments	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_typed`	PCQ	proved
N5	Annotated addition erases to the independently defined M1 code	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_erases`	P	proved
N5	The complete annotated addition function has its curried type	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.addFunction_typed`	PCQ	proved
N5	The closed annotated function erases to actual closed M1 addition	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.addFunction_erases`	P	proved
N5	The existing bidirectional annotator reconstructs typed addition	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.producedAddFunction_exact`	PCQ	proved
N5	The actual produced addition has an independent typing derivation	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.producedAddFunction_typed`	PCQ	proved
N5	The actual typed zero equation retains original row permission	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_zero`	PCQ	proved
N5	The actual typed successor equation retains original row permission	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_successor`	PCQ	proved
N5	The inactive annotated profile has no root computation	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_no_root`	PCQ	proved
N5	Closed addition stays typed with every iota root disabled	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.typed_without_iota`	PCQ	proved
N5	A second actual motive has its own genuine domain typing	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.otherMotive_typed`	PCQ	proved
N5	Different typed motive domains share one erasure	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.domain_erasure_distinction`	none	proved
N5	Actual annotated one plus zero equals one at Nat	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.one_plus_zero`	PCQ	proved
N5	Annotated natural-program reference controls pass	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.all_verdicts`	PCQ	proved
I	Actual annotated addition commutes with substitution	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_substitute`	P	proved
I	Actual annotated addition commutes with renaming	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.add_rename`	P	proved
I	Abstract an independently typed body over formed dependent domain and codomain	`TypedEquality.Annotated.CTyped.abstractTerm`	P	proved
I	Actual dependent append: elementSort typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.elementSort_typed`	PCQ	proved
I	Actual dependent append: in motive universe	`NativeNaturalVectorFamilies.Programs.Annotated.Append.in_motive_universe`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Actual dependent append: vector constant typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.vector_constant_typed`	PCQ	proved
I	Actual dependent append: vector typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.vector_typed`	PCQ	proved
I	Actual dependent append: vector index equal	`NativeNaturalVectorFamilies.Programs.Annotated.Append.vector_index_equal`	PCQ	proved
I	Actual dependent append: nil constant typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_constant_typed`	PCQ	proved
I	Actual dependent append: nil typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_typed`	PCQ	proved
I	Actual dependent append: cons constant typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.cons_constant_typed`	PCQ	proved
I	Actual dependent append: cons typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.cons_typed`	PCQ	proved
I	Actual dependent append: result typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.result_typed`	PCQ	proved
I	Actual dependent append: result substitute	`NativeNaturalVectorFamilies.Programs.Annotated.Append.result_substitute`	P	proved
I	Actual dependent append: innerType open	`NativeNaturalVectorFamilies.Programs.Annotated.Append.innerType_open`	PQ	proved
I	Actual dependent append: family applied	`NativeNaturalVectorFamilies.Programs.Annotated.Append.family_applied`	PCQ	proved
I	Actual dependent append: recursor constant typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.recursor_constant_typed`	PCQ	proved
I	Actual dependent append: arguments typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.arguments_typed`	PCQ	proved
I	Actual dependent append: eliminate typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.eliminate_typed`	PCQ	proved
I	Actual dependent append: innerType typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.innerType_typed`	PCQ	proved
I	Actual dependent append: familyType typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.familyType_typed`	PCQ	proved
I	Actual dependent append: motiveInner typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motiveInner_typed`	PCQ	proved
I	Actual dependent append: motive typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motive_typed`	PCQ	proved
I	Actual dependent append: motiveInner substitute	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motiveInner_substitute`	P	proved

Part	Claim	Lean declaration	Axioms	Status
I	Actual dependent append: motiveInner rename	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motiveInner_rename`	P	proved
I	Actual dependent append: motive rename	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motive_rename`	P	proved
I	Actual dependent append: motive beta	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motive_beta`	PCQ	proved
I	Actual dependent append: method body typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.method_body_typed`	PCQ	proved
I	Actual dependent append: method typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.method_typed`	PCQ	proved
I	Actual dependent append: nil branch typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_branch_typed`	PCQ	proved
I	Actual dependent append: append typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_typed`	PCQ	proved
I	Actual dependent append: functionContext formed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.functionContext_formed`	PCQ	proved
I	Actual dependent append: functionBody typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.functionBody_typed`	PCQ	proved
I	Actual dependent append: functionResult typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.functionResult_typed`	PCQ	proved
I	Actual dependent append: appendFunction typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunction_typed`	PCQ	proved
I	Actual dependent append: motive erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.motive_erases`	PQ	proved
I	Actual dependent append: method erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.method_erases`	PQ	proved
I	Actual dependent append: append erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_erases`	PQ	proved
I	Actual dependent append: appendFunction erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunction_erases`	PQ	proved
I	Actual dependent append: appendFunctionType erases	`NativeNaturalVectorFamilies.Programs.Annotated.Append.appendFunctionType_erases`	P	proved
I	Actual dependent append: producedAppendFunction exact	`NativeNaturalVectorFamilies.Programs.Annotated.Append.producedAppendFunction_exact`	PCQ	proved
I	Actual dependent append: producedAppendFunction typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.producedAppendFunction_typed`	PCQ	proved
I	Actual dependent append: naturalOnly typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.naturalOnly_typed`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Actual dependent append: unconstrained erasure distinction	`NativeNaturalVectorFamilies.Programs.Annotated.Append.unconstrained_erasure_distinction`	PCQ	proved
I	Actual dependent append: all verdicts	`NativeNaturalVectorFamilies.Programs.Annotated.Append.all_verdicts`	PCQ	proved
I	Typed append computation: methodContext formed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.methodContext_formed`	PCQ	proved
I	Typed append computation: methodResult typed	`NativeNaturalVectorFamilies.Programs.Annotated.Append.methodResult_typed`	PCQ	proved
I	Typed append computation: method apply equal	`NativeNaturalVectorFamilies.Programs.Annotated.Append.method_apply_equal`	PCQ	proved
I	Typed append computation: append nil	`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_nil`	PCQ	proved
I	Typed append computation: append cons	`NativeNaturalVectorFamilies.Programs.Annotated.Append.append_cons`	PCQ	proved
I	Typed append computation: natural as element	`NativeNaturalVectorFamilies.Programs.Annotated.Append.natural_as_element`	PCQ	proved
I	Typed append computation: closed nil	`NativeNaturalVectorFamilies.Programs.Annotated.Append.closed_nil`	PCQ	proved
I	Typed append computation: closed cons	`NativeNaturalVectorFamilies.Programs.Annotated.Append.closed_cons`	PCQ	proved
I	Typed append computation: closed cons erased	`NativeNaturalVectorFamilies.Programs.Annotated.Append.closed_cons_erased`	PCQ	proved
I	Typed append computation: typed with vector rows disabled	`NativeNaturalVectorFamilies.Programs.Annotated.Append.typed_with_vector_rows_disabled`	PCQ	proved
I	Typed append computation: all computationVerdicts	`NativeNaturalVectorFamilies.Programs.Annotated.Append.all_computationVerdicts`	PCQ	proved
I	Restrict a typed telescope assignment to its ambient context	`TypedEquality.Annotated.CTele.ambientSub_typed`	P	proved
I	Opening after ambient restriction reconstructs the full assignment	`TypedEquality.Annotated.CTele.inst_ambientSub`	PQ	proved
I	Typed dependent telescope application preserves equality	`TypedEquality.Annotated.CTele.apply_equal`	PQ	proved
I	Relative telescope abstractions compute under a full independently typed assignment	`TypedEquality.Annotated.CTele.lams_apply`	PQ	proved
I	Selected annotated policy transport: opaque values	`NativeNaturalVectorFamilies.Programs.Annotated.opaque_values`	PCQ	proved
I	Selected annotated policy transport: presentation	`NativeNaturalVectorFamilies.Programs.Annotated.presentation`	PCQ	proved
I	Selected annotated policy transport: permissions morphism	`NativeNaturalVectorFamilies.Programs.Annotated.permissions_morphism`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Selected annotated policy transport: judgment mono	`NativeNaturalVectorFamilies.Programs.Annotated.judgment_mono`	PCQ	proved
I	Selected annotated policy transport: context mono	`NativeNaturalVectorFamilies.Programs.Annotated.context_mono`	PCQ	proved
I	Selected annotated policy transport: substitution mono	`NativeNaturalVectorFamilies.Programs.Annotated.substitution_mono`	PCQ	proved
I	Selected annotated policy transport: erase root	`NativeNaturalVectorFamilies.Programs.Annotated.erase_root`	PCQ	proved
I	Selected annotated policy transport: natural patterns linear	`NativeNaturalVectorFamilies.Programs.Annotated.natural_patterns_linear`	PCQ	proved
I	Selected annotated policy transport: vector cons repeats parameter	`NativeNaturalVectorFamilies.Programs.Annotated.vector_cons_repeats_parameter`	P	proved
I	Selected annotated policy transport: naturalOnly included	`NativeNaturalVectorFamilies.Programs.Annotated.Append.naturalOnly_included`	PCQ	proved
I	Selected annotated policy transport: function after two extensions	`NativeNaturalVectorFamilies.Programs.Annotated.Append.function_after_two_extensions`	PCQ	proved
I	Selected annotated policy transport: nil before cons	`NativeNaturalVectorFamilies.Programs.Annotated.Append.nil_before_cons`	PCQ	proved
I	Selected annotated policy transport: cons certificate strict	`NativeNaturalVectorFamilies.Programs.Annotated.Append.cons_certificate_strict`	PCQ	proved
I	Selected annotated policy transport: all profileVerdicts	`NativeNaturalVectorFamilies.Programs.Annotated.Append.all_profileVerdicts`	PCQ	proved
I	Premise translation commutes with head-map composition	`TypedEquality.Annotated.CPremise.mapHead_comp`	P	proved
I	Compose premise-preserving annotated package morphisms	`TypedEquality.Annotated.ChurchRules.Morphism.comp`	P	proved
I	Transport a presented annotated package under an enlarged schema family	`TypedEquality.Annotated.ChurchRules.ofSchemas_morphism`	PQ	proved
I	Typed natural fact interface: fullDomains	`HOL.NaturalFacts.fullDomains`	none	proved
I	Typed natural fact interface: recursion meaning	`HOL.NaturalFacts.recursion_meaning`	none	proved
I	Typed natural fact interface: induction meaning	`HOL.NaturalFacts.induction_meaning`	none	proved
I	Typed natural fact interface: noConfusion meaning	`HOL.NaturalFacts.noConfusion_meaning`	none	proved
I	Typed natural fact interface: successorInjective meaning	`HOL.NaturalFacts.successorInjective_meaning`	none	proved
I	Typed natural fact interface: infinity meaning	`HOL.NaturalFacts.infinity_meaning`	none	proved
I	Typed natural fact interface: collapsing noConfusion refused	`HOL.NaturalFacts.collapsing_noConfusion_refused`	P	proved
I	Typed natural fact interface: encodeTerm rename	`HOLLight.HOLSyntaxWire.encodeTerm_rename`	PQ	proved
I	Typed natural fact interface: names distinct	`HOLLight.NaturalFacts.names_distinct`	none	proved

Part	Claim	Lean declaration	Axioms	Status
I	Typed natural fact interface: binder names distinct	`HOLLight.NaturalFacts.binder_names_distinct`	P	proved
I	Typed natural fact interface: bare zero statement distinct	`HOLLight.NaturalFacts.bare_zero_statement_distinct`	P	proved
I	Authored carrier at the native fact signature: models recursion	`IndexedInductiveNaturalHOL.NativeFacts.models_recursion`	PQ	proved
I	Authored carrier at the native fact signature: models induction	`IndexedInductiveNaturalHOL.NativeFacts.models_induction`	PQ	proved
I	Authored carrier at the native fact signature: models noConfusion	`IndexedInductiveNaturalHOL.NativeFacts.models_noConfusion`	PQ	proved
I	Authored carrier at the native fact signature: models successorInjective	`IndexedInductiveNaturalHOL.NativeFacts.models_successorInjective`	PQ	proved
I	Authored carrier at the native fact signature: models infinity	`IndexedInductiveNaturalHOL.NativeFacts.models_infinity`	PQ	proved
I	Authored carrier at the native fact signature: all models	`IndexedInductiveNaturalHOL.NativeFacts.all_models`	PQ	proved
I	Authored carrier at the native fact signature: boolean recursion witness	`IndexedInductiveNaturalHOL.NativeFacts.boolean_recursion_witness`	P	proved
I	Observed telescope retains a typed application head	`FormationSensitive.Typing.applyClosed_head`	P	proved
I	Observed telescope recovers its original declaration spine	`FormationSensitive.DeclarationSpine.ofObservedTelescope`	P	proved
I	A declaration head retains its closed formation	`FormationSensitive.DeclarationSpine.constant_formed`	P	proved
I	Original vector eliminator declaration	`ComputationProfiles.NaturalConversion.Preservation.Vector.recursor_declared`	PCQ	proved
I	Vector telescope application is the original eliminator call	`ComputationProfiles.NaturalConversion.Preservation.Vector.application_shape`	P	proved
I	Vector telescope recovers the dependent principal result	`ComputationProfiles.NaturalConversion.Preservation.Vector.principal_result`	P	proved
I	Observed vector application retains its original declaration	`ComputationProfiles.NaturalConversion.Preservation.Vector.recursor_spine`	PCQ	proved
I	All dependent vector arguments and displayed-type replay	`ComputationProfiles.NaturalConversion.Preservation.Vector.recover`	PCQ	proved
I	Operational vector nil preserves smaller-policy typing	`ComputationProfiles.NaturalConversion.Preservation.Vector.nil_preserves`	PCQ	proved
I	Original vector cons declaration	`ComputationProfiles.NaturalConversion.Preservation.Vector.constructor_declared`	PCQ	proved
I	Observed constructor recovers its four dependent arguments	`ComputationProfiles.NaturalConversion.Preservation.Vector.constructor_arguments`	PCQ	proved
I	Canonical vector cons target from original eliminator formation	`ComputationProfiles.NaturalConversion.Preservation.Vector.canonical_cons_target`	PCQ	proved
I	Operational vector cons preserves smaller-policy typing	`ComputationProfiles.NaturalConversion.Preservation.Vector.cons_preserves`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	All original operational roots preserve smaller checking judgments	`ComputationProfiles.NaturalConversion.Preservation.operational_roots`	PCQ	proved
I	Existing producer yields the canonical nil submission	`ComputationProfiles.NaturalConversion.Controls.vector_nil_produced`	PCQ	proved
I	Existing producer yields the canonical cons submission	`ComputationProfiles.NaturalConversion.Controls.vector_cons_produced`	PCQ	proved
I	Canonical vector nil source checks under natural-only policy	`ComputationProfiles.NaturalConversion.Controls.vector_nil_typed`	PCQ	proved
I	Canonical vector cons source checks under natural-only policy	`ComputationProfiles.NaturalConversion.Controls.vector_cons_typed`	PCQ	proved
I	Canonical operational nil target retains its type	`ComputationProfiles.NaturalConversion.Controls.vector_nil_preserved`	PCQ	proved
I	Canonical operational cons target retains its type	`ComputationProfiles.NaturalConversion.Controls.vector_cons_preserved`	PCQ	proved
I	Operational cons is absent from smaller checking roots	`ComputationProfiles.NaturalConversion.Controls.vector_cons_root_disabled`	PCQ	proved
I	Smaller checker refuses the operational cons citation	`ComputationProfiles.NaturalConversion.Controls.vector_cons_citation_refused`	PCQ	proved
I	Full checker accepts the exact same cons citation	`ComputationProfiles.NaturalConversion.Controls.vector_cons_citation_enabled`	PCQ	proved
I	Actual original declaration roots preserve smaller checking judgments	`ComputationProfiles.NaturalConversion.Preservation.operational_rule_roots`	PCQ	proved
I	Actual closure parameters form an admitted Henkin valuation	`HOL.NaturalClosure.parameterValuation_admissible`	none	proved
I	Actual lambda denotes the strengthened closure predicate	`HOL.NaturalClosure.denote_strengthening`	none	proved
I	Henkin term closure admits the particular induction predicate	`HOL.NaturalClosure.strengthening_admitted`	none	proved
I	Closure-premised induction from local parameter admission	`HOL.NaturalClosure.closedUnder_induction_admitted`	none	proved
I	Guarded induction holds in every selected Henkin model	`HOL.NaturalClosure.models_induction_henkin`	none	proved
I	Three-point interpretation has exactly its generated zero	`HOL.NaturalClosure.InductionControls.closed_iff_zero`	P	proved
I	Three-point guarded step holds	`HOL.NaturalClosure.InductionControls.guarded_step`	P	proved
I	Three-point unguarded step fails	`HOL.NaturalClosure.InductionControls.unguarded_step_refused`	P	proved
I	Three-point actual guarded induction	`HOL.NaturalClosure.InductionControls.guarded_induction`	P	proved
I	Native guarded-step and strengthened-definition controls	`MM0.NaturalInduction.all_verdicts`	P	proved
I	Common cumulative head preservation for annotated packages	`Annotated.CHeadPreserving.ofLevelTower`	P	proved

Part	Claim	Lean declaration	Axioms	Status
I	Original disabled table has no raw root	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_raw_no_root`	PCQ	proved
I	Actual declaration policies reuse the tower head reading	`NativeNaturalVectorFamilies.Programs.Annotated.headReading`	PCQ	proved
I	Actual declaration policies retain ground-head equality	`NativeNaturalVectorFamilies.Programs.Annotated.ground_head_eq`	PCQ	proved
I	Opaque annotated declarations have former injectivity and no-confusion	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_formerFacts`	PCQ	proved
I	Opaque annotated product cannot equal a universe head	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_pi_not_head`	PCQ	proved
I	Cumulative heads preserve typing under every actual permission selection	`NativeNaturalVectorFamilies.Programs.Annotated.head_preserving`	PCQ	proved
I	Opaque contextual reductions are equal at the original dependent type	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_reduction_equal`	PCQ	proved
I	Opaque contextual reductions retain the original dependent type	`NativeNaturalVectorFamilies.Programs.Annotated.inactive_reduction_typed`	PCQ	proved
I	Actual opaque natural function product is not a universe	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_function_type_not_universe`	PCQ	proved
I	Actual opaque natural function takes a typed beta step	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_function_beta`	PCQ	proved
I	Opaque declarations refuse the actual natural zero citation	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.inactive_zero_citation_refused`	PCQ	proved
I	Enabling rows accepts that same natural zero citation	`NativeNaturalVectorFamilies.Programs.Annotated.Functions.full_zero_citation_accepted`	PCQ	proved
M1	Nontrivial interpretation of the original natural type	`NaturalDomain.reading_nat`	PCQ	proved
M1	Original zero is the numeral zero ideal	`NaturalDomain.reading_zero`	PCQ	proved
M1	Original successor uses the projected numeral constructor	`NaturalDomain.reading_successor`	PCQ	proved
M1	Original recursor uses the projected dependent recursor	`NaturalDomain.reading_recursor`	PCQ	proved
M1	Actual natural declaration matches the numeral interface	`NaturalDomain.nat_declared`	PCQ	proved
M1	Actual zero declaration matches the numeral interface	`NaturalDomain.zero_declared`	PCQ	proved
M1	Actual successor declaration matches the numeral interface	`NaturalDomain.successor_declared`	PCQ	proved
M1	Actual dependent recursor declaration matches the numeral interface	`NaturalDomain.recursor_declared`	PCQ	proved
M1	Every original constant is fixed by its declared projection	`NaturalDomain.constants_valid`	PCQ	proved
M1	Original recursor lookup recovers the semantic dependent spine	`NaturalDomain.recursor_spine`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
M1	Natural zero computation with original semantic spine premises	`NaturalDomain.zero_valid`	PCQ	proved
M1	Natural successor computation with original semantic spine premises	`NaturalDomain.successor_valid`	PCQ	proved
M1	Both actual selected annotated rows have valid interpretations	`NaturalDomain.roots_valid`	PCQ	proved
M1	Nontrivial valid reading of the natural-only annotation	`NaturalDomain.valid`	PCQ	proved
M1	Existing actual addition proof has equal denotations	`NaturalDomain.one_plus_zero`	PCQ	proved
M1	Actual successor of zero denotes its numeral ideal	`NaturalDomain.successor_zero`	PCQ	proved
M1	Actual one plus zero denotes successor of zero	`NaturalDomain.one_plus_zero_value`	PCQ	proved
M1	The natural interpretation separates zero and successor	`NaturalDomain.zero_ne_successor_zero`	PCQ	proved
M1	Soundness refutes the false annotated constructor equation	`NaturalDomain.zero_equal_successor_refused`	PCQ	proved
M1	Larger declaration spine facts retain original types	`SpineFacts.of_decl_inclusion`	PQ	proved
M1	Semantic validity of an opaque package sum retaining computation	`ReadingValid.sum_opaque`	PQ	proved
M1	Decoder proposition name is fresh for the original signature	`NaturalDomain.Decoder.fresh_prop`	PCQ	proved
M1	Decoder name is fresh for the original signature	`NaturalDomain.Decoder.fresh_holds`	PCQ	proved
M1	Opaque proposition declaration has its original type	`NaturalDomain.Decoder.prop_declared`	PCQ	proved
M1	Opaque decoder declaration has its original type	`NaturalDomain.Decoder.holds_declared`	PCQ	proved
M1	Original nontrivial reading validates opaque decoder declarations	`NaturalDomain.Decoder.declarations_valid`	PCQ	proved
M1	Semantic validity with original natural computation and fresh decoder	`NaturalDomain.Decoder.valid`	PCQ	proved
M1	Decoder extension retains exactly the original natural root cases	`NaturalDomain.Decoder.roots_iff`	PCQ	proved
M1	Decoder extension retains original root evidence requirements	`NaturalDomain.Decoder.premises_iff`	PCQ	proved
M1	Actual original typed addition proof persists into decoder extension	`NaturalDomain.Decoder.one_plus_zero`	PCQ	proved
M1	Actual natural zero computation remains enabled	`NaturalDomain.Decoder.zero_root`	PCQ	proved
M1	Decoder extension still refutes the false constructor equation	`NaturalDomain.Decoder.zero_equal_successor_refused`	PCQ	proved
M1	Abstraction/representation roundtrip has the defined type	`HOLTypeDefinitionBoundary.roundtrip_typed`	PQ	proved
M1	Equality at the defined type passes source typing	`HOLTypeDefinitionBoundary.defined_equality_typed`	PQ	proved
M1	Distinct representation equality index is refused	`HOLTypeDefinitionBoundary.representation_equality_refused`	PQ	proved
M1	Correctly indexed equation passes source ingress	`HOLTypeDefinitionBoundary.defined_equality_checked`	PCQ	proved
M1	Wrong equality index refuses source ingress	`HOLTypeDefinitionBoundary.representation_equality_check_refused`	PCQ	proved
M1	Concrete representation and defined types differ	`HOLTypeDefinitionBoundary.distinct_types`	none	proved

Part	Claim	Lean declaration	Axioms	Status
M1	Type-definition boundary controls compute	`HOLTypeDefinitionBoundary.all_verdicts`	PQ	proved
M1	Constant expansion retains the complete schema-premise list	`patternPremises_instConsts_of_fixed`	PQ	proved
M1	Supported schema roots survive constant expansion	`SchemaConstantsFixed.root`	PQ	proved
M1	Required evidence follows the same constant expansion	`SchemaConstantsFixed.requires`	PQ	proved
M1	Supported declarations and schemas give a full constant interpretation	`ChurchRules.ofSchemas_constInstance`	PQ	proved
M1	Fresh proposition expands to its universe	`NaturalDomain.Decoder.interpret_prop`	PQ	proved
M1	Fresh decoder expands to the universe identity	`NaturalDomain.Decoder.interpret_holds`	PQ	proved
M1	Every original declared name is fixed	`NaturalDomain.Decoder.original_names_fixed`	PCQ	proved
M1	All eight original declaration types pass finite support checking	`NaturalDomain.Decoder.declarationTypeFixedCheck_true`	PCQ	proved
M1	Every actual declared type is fixed by the return map	`NaturalDomain.Decoder.original_types_fixed`	PCQ	proved
M1	All four original left templates are fixed	`NaturalDomain.Decoder.row_left_fixed`	PCQ	proved
M1	All four original right templates are fixed	`NaturalDomain.Decoder.row_right_fixed`	PCQ	proved
M1	All four rows retain their inferred variable types	`NaturalDomain.Decoder.row_variable_type_fixed`	PCQ	proved
M1	Every selected original row has full constant-interpretation support	`NaturalDomain.Decoder.schema_support`	PCQ	proved
M1	Original computation and evidence survive the interpretation	`NaturalDomain.Decoder.original_interpretation`	PCQ	proved
M1	Decoder type expansion reuses the maintained definitions	`NaturalDomain.Decoder.decoder_type_expansion`	PQ	proved
M1	Full return interpretation for every original permission selection	`NaturalDomain.Decoder.return_interpretation`	PCQ	proved
M1	Return preserves complete judgments and contexts	`NaturalDomain.Decoder.returned_judgment`	PCQ	proved
M1	Equal derivability for statements fixed by the return interpretation	`NaturalDomain.Decoder.fixed_statement_iff`	PCQ	proved
M1	The fresh proposition is typed in each extension	`NaturalDomain.Decoder.proposition_typed`	PCQ	proved
M1	Its changed universe image is typed in the original package	`NaturalDomain.Decoder.returned_proposition_typed`	PCQ	proved
M1	Fresh proposition typing is not fixed by the return map	`NaturalDomain.Decoder.proposition_statement_changes`	PQ	proved
M1	The actual original addition statement is fixed	`NaturalDomain.Decoder.original_addition_fixed`	PQ	proved
M1	Actual natural-only addition reflects through the return route	`NaturalDomain.Decoder.original_addition_reflected`	PCQ	proved
M1	Actual full-policy addition reflects through the return route	`NaturalDomain.Decoder.full_policy_addition_reflected`	PCQ	proved
M1	Return interpretation refutes the false natural constructor equation	`NaturalDomain.Decoder.false_equation_refused_by_return`	PCQ	proved
M1	Legacy identity typing omits domain formation	`FormationSensitive.raw_identity_at_any_domain`	P	proved
M1	Refined identity typing retains its domain declaration	`FormationSensitive.judgment_identity_domain_declared`	PQ	proved
M1	Raw/refined identity boundary at a missing domain	`FormationSensitive.raw_identity_missing_domain`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
M1	Formed contexts alone do not upgrade every legacy derivation	`FormationSensitive.no_uniform_raw_upgrade`	PQ	proved
M1	Selected carrier no-preimage formula has the omitted-point meaning	`MM0.NaturalCarrier.noPreimage_meaning`	none	proved
M1	Selected carrier reference-format controls compute	`MM0.NaturalCarrier.all_verdicts`	P	proved
M1	Natural-subtype statement/domain controls compute	`MM0.NaturalType.all_verdicts`	P	proved
M1	Raw declared constant keeps its exact type adjustment	`HasType.constantAdjustment`	P	proved
M1	Missing declarations cannot create raw spines	`HasType.DeclarationSpine.missingConstant`	P	proved
M1	Raw declared spine reconstructs its principal typing	`HasType.DeclarationSpine.typing`	P	proved
M1	Raw spine recovers observed type adjustment	`HasType.DeclarationSpine.adjustment`	PQ	proved
M1	Raw dependent argument and displayed-type replay	`HasType.DeclarationSpine.recoverApplication`	PQ	proved
M1	Raw spine replays exact conversion/cumulativity tails	`HasType.DeclarationSpine.replay`	PQ	proved
M1	Raw declared telescope recovers every dependent argument	`HasType.DeclarationSpine.recoverTelescope`	PQ	proved
M1	Formed declaration spines share the raw analysis	`FormationSensitive.DeclarationSpine.toRaw`	P	proved
M1	Actual original natural recursor supplies its raw declared spine	`NaturalConversion.RawPreservation.recursor_spine`	PCQ	proved
M1	Actual natural telescope recovers every raw argument	`NaturalConversion.RawPreservation.recover`	PCQ	proved
M1	Natural zero root preserves arbitrary raw observed types	`NaturalConversion.RawPreservation.zero_preserves`	PCQ	proved
M1	Declared successor recovers its raw predecessor typing	`NaturalConversion.RawPreservation.successor_argument`	PCQ	proved
M1	Natural successor root preserves arbitrary raw observed types	`NaturalConversion.RawPreservation.successor_preserves`	PCQ	proved
M1	Actual selected roots preserve the legacy raw judgment	`NaturalConversion.RawPreservation.roots_preserve`	PCQ	proved
M1	Exact selected signature raw-preservation field is discharged	`NaturalConversion.RawPreservation.declared_preserves`	PCQ	proved
M1	All eight declaration-formation certificates check with iota disabled	`NaturalConversion.RawPreservation.Formation.all_produced`	PCQ	proved
M1	Accepted declaration certificates yield the formed judgment	`NaturalConversion.RawPreservation.Formation.request_formed`	PCQ	proved
M1	Formation requests cover every actual declaration lookup	`NaturalConversion.RawPreservation.Formation.lookup_covered`	PCQ	proved
M1	All selections retain formed original declarations	`NaturalConversion.RawPreservation.Formation.signature_formed`	PCQ	proved
M1	Existing checked-signature carrier has formation and raw preservation	`NaturalConversion.RawPreservation.checkedSignature`	PCQ	proved
M1	Actual checked append retains the selected checked signature	`NaturalConversion.RawPreservation.checked_append`	PCQ	proved
M1	The actual zero computation is a permitted root	`NaturalConversion.RawPreservation.addition_zero_root`	PCQ	proved
M1	Actual checked zero redex uses uniform raw preservation	`NaturalConversion.RawPreservation.actual_zero_preserved`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
M1	Actual successor redex has its original dependent raw type	`NaturalConversion.RawPreservation.actual_successor_target`	PCQ	proved
M1	Selected actual signature refuses a missing declaration	`NaturalConversion.RawPreservation.missing_lookup`	PCQ	proved
M1	Selected raw spine refuses a missing declaration	`NaturalConversion.RawPreservation.missing_spine_refused`	PCQ	proved
M1	Eight formation and two negative controls compute	`NaturalConversion.RawPreservation.all_verdicts`	PCQ	proved
I	Read ordinary root preservation as a cross-policy contract	`FormationSensitive.RootPreservation.toAgainst`	P	proved
I	Preserve natural checking under every original operational root	`NaturalConversion.Preservation.operational_preservation_against`	PCQ	proved
I	Admit a repeated-payload vector step after annotation erasure	`Annotated.RepeatedParameterCanary.raw_step`	PCQ	proved
I	Refuse the same source as an annotated schema instance	`Annotated.RepeatedParameterCanary.no_annotated_step`	PCQ	proved
I	Refute unrestricted four-row annotation lifting	`Annotated.RepeatedParameterCanary.unrestricted_lifting_fails`	PCQ	proved
I	Separate natural lifting from the full repeated-parameter table	`Annotated.natural_lifts_but_full_does_not`	PCQ	proved
I	Compose checking-profile capability inclusions	`IndexedFamily.Checking.Profile.Extends.trans`	PQ	proved
I	Transport selected conversion through profile inclusion	`IndexedFamily.Checking.Profile.computationMorphism`	PQ	proved
I	Transport formed typing through profile inclusion	`IndexedFamily.Checking.Profile.judgment_mono`	PQ	proved
I	Transport a profile judgment into a stated realization	`IndexedFamily.Checking.Profile.Realization.includeJudgment`	PQ	proved
I	Include the retained-receipt node in natural conversion	`CheckingProfiles.minimal_extends_naturalIntensional`	PCQ	proved
I	Add scoped logical extensionality without changing iota	`CheckingProfiles.naturalIntensional_extends_scopedLogicalExtensional`	PCQ	proved
I	Extend natural conversion by the original raw operational rows	`CheckingProfiles.naturalIntensional_extends_fullRawOperational`	PCQ	proved
I	Separate retained receipts from judgmental conversion	`CheckingProfiles.not_naturalIntensional_extends_minimal`	PCQ	proved
I	Separate intensional and scoped logical profiles	`CheckingProfiles.not_scopedLogicalExtensional_extends_naturalIntensional`	PCQ	proved
I	Separate logical principles from the raw operational branch	`CheckingProfiles.not_scopedLogicalExtensional_extends_fullRawOperational`	PCQ	proved
I	Separate raw vector conversion from the logical branch	`CheckingProfiles.not_fullRawOperational_extends_scopedLogicalExtensional`	PCQ	proved
I	Construct the incomparable logical and operational children	`CheckingProfiles.natural_profile_fork`	PCQ	proved
I	Connect the operational child to natural checking	`CheckingProfiles.fullRawOperational_preserves_naturalChecking`	PCQ	proved
I	Keep the full raw child outside unrestricted annotated conversion	`CheckingProfiles.fullRawOperational_not_unrestrictedAnnotatedLift`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Embed selected natural checking in the layered calculus	`CheckingProfiles.naturalToScopedMorphism`	PCQ	proved
I	Type propositional extensionality in the layered calculus	`CheckingProfiles.scoped_propositionExtensionality_typed`	PCQ	proved
I	Type dependent append in the scoped logical node	`CheckingProfiles.append_typed_scopedLogicalExtensional`	PCQ	proved
I	Check positive and negative profile requests	`CheckingProfiles.all_verdicts`	PCQ	proved
M1	Intrinsic native recursion formula retains its mathematical meaning	`MM0.NaturalRecursion.formula_meaning`	none	proved
M1	Bare-zero and numeral-wrapped recursion agree in the stated standard model	`MM0.NaturalRecursion.shared_fact_meaning`	none	proved
M1	Recursion formula has the nonempty natural-number instance	`MM0.NaturalRecursion.standard_natural_recursion`	none	proved
M1	A collapsed constructor refutes the recursion formula	`MM0.NaturalRecursion.fixed_zero_refuses_recursion`	none	proved
M1	Native recursion finite format controls compute	`MM0.NaturalRecursion.all_verdicts`	P	proved
I	Print the intrinsic graph as the exact native definition	`MM0.NaturalFactsMeaning.graph_definition_prints`	P	proved
I	Interpret native graph syntax by the independent inductive graph	`MM0.NaturalFactsMeaning.graph_meaning`	none	proved
I	Preserve all guarded native induction premises	`MM0.NaturalFactsMeaning.induction_statement_prints`	P	proved
I	Print the native constructor-separation formula	`MM0.NaturalFactsMeaning.separation_prints`	none	proved
I	Account for native Boolean-equality abbreviation	`MM0.NaturalFactsMeaning.injectivity_prints`	none	proved
I	Print the exact alias-bearing native infinity formula	`MM0.NaturalFactsMeaning.infinity_prints`	none	proved
I	Expand named infinity definitions beneath binders	`MM0.NaturalFactsMeaning.infinity_expands`	none	proved
I	Preserve infinity through genuine Henkin definition reducts	`MM0.NaturalFactsMeaning.infinity_definitions_meaning`	PQ	proved
I	Interpret the curried native induction law	`MM0.NaturalFactsMeaning.induction_meaning`	none	proved
I	Retain open induction conclusion and premise evidence	`MM0.NaturalFactsMeaning.induction_rule`	none	proved
I	Preserve native constructor-separation orientation	`MM0.NaturalFactsMeaning.no_confusion_meaning`	none	proved
I	Interpret native nonsurjectivity by an omitted point	`MM0.NaturalFactsMeaning.infinity_meaning`	PCQ	proved
I	Compare five native readings with shared natural laws	`MM0.NaturalFactsMeaning.shared_fact_meaning`	PCQ	proved
I	Qualify typed native reference and refusal controls	`MM0.NaturalFactsMeaning.all_verdicts`	P	proved
I	Validate five native readings on the authored carrier	`IndexedInductiveNaturalMM0.all_models`	PCQ	proved
I	Validate named native infinity on the source definition reduct	`IndexedInductiveNaturalMM0.native_infinity_model`	PCQ	proved
I	Retain open induction conclusion and premise evidence	`IndexedInductiveNaturalMM0.induction_rule`	PCQ	proved
I	Identify exact native graph values with the source recursor	`IndexedInductiveNaturalMM0.source_graph_iff`	PQ	proved
I	Construct a native graph witness from source recursion	`IndexedInductiveNaturalMM0.source_graph`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Prove selector agreement under its explicit law	`IndexedInductiveNaturalMM0.source_selector_agrees`	PQ	proved
I	Reject an incorrect output at a genuine source successor	`IndexedInductiveNaturalMM0.wrong_successor_output_refused`	PQ	proved
I	Accept the nonconstant source successor output	`IndexedInductiveNaturalMM0.successor_output_accepted`	PQ	proved
I	Transport recursion graph indices through structured maps	`Megalodon.NaturalRecursion.Graph.mapIndex`	none	proved
I	Select an explicitly witnessed infinite self-map	`Megalodon.NaturalTheoryProfile.selected_infinite`	none	proved
I	Derive an omitted point using the selected classical option	`Megalodon.NaturalTheoryProfile.not_surjective_has_omitted`	none	proved
I	Select an omitted point under the stated selection law	`Megalodon.NaturalTheoryProfile.zero_omitted`	none	proved
I	Separate every selected successor from selected zero	`Megalodon.NaturalTheoryProfile.successor_ne_zero`	none	proved
I	Retain injectivity of the selected infinite function	`Megalodon.NaturalTheoryProfile.successor_injective`	none	proved
I	Generate selected zero	`Megalodon.NaturalTheoryProfile.generated_zero`	none	proved
I	Close generation under selected successor	`Megalodon.NaturalTheoryProfile.generated_successor`	none	proved
I	Construct the selected recursion graph on generated inputs	`Megalodon.NaturalTheoryProfile.recursor_graph`	none	proved
I	Establish the selected recursor zero equation	`Megalodon.NaturalTheoryProfile.recursor_zero`	none	proved
I	Establish the selected recursor successor equation	`Megalodon.NaturalTheoryProfile.recursor_successor`	none	proved
I	Prove generated-carrier uniqueness of selected recursion	`Megalodon.NaturalTheoryProfile.recursor_unique`	none	proved
I	Interpret the selected native recursion theorem	`Megalodon.NaturalTheoryProfile.recursion`	none	proved
I	Refuse the infinity option on a singleton carrier	`Megalodon.NaturalTheoryProfile.unit_refuses_infinity`	none	proved
I	Refuse a function that lacks the selection law	`Megalodon.NaturalTheoryProfile.constant_false_refuses_selection`	none	proved
I	Check the optional theory inventory	`Megalodon.NaturalTheoryProfile.all_verdicts`	P	proved
I	Construct a lawful preferred-witness selector	`IndexedInductiveNaturalMegalodonProfile.choosePreferred_selects`	PCQ	proved
I	Retain a preferred witness when it satisfies the predicate	`IndexedInductiveNaturalMegalodonProfile.choosePreferred_of_holds`	PCQ	proved
I	Witness authored successor as a Dedekind injection	`IndexedInductiveNaturalMegalodonProfile.successorIsInfinite`	PQ	proved
I	Identify selected successor with authored successor	`IndexedInductiveNaturalMegalodonProfile.selected_successor_eq`	PCQ	proved
I	Identify selected zero with authored zero	`IndexedInductiveNaturalMegalodonProfile.selected_zero_eq`	PCQ	proved
I	Generate every authored natural value	`IndexedInductiveNaturalMegalodonProfile.source_generated`	PCQ	proved
I	Identify selected recursion with the authored eliminator	`IndexedInductiveNaturalMegalodonProfile.source_recursor_agrees`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
I	Derive total authored recursion and uniqueness	`IndexedInductiveNaturalMegalodonProfile.source_recursion`	PQ	proved
I	Refuse a constant successor as the infinite witness	`IndexedInductiveNaturalMegalodonProfile.constant_successor_refused`	P	proved
D	Exclude every model-forbidden input from model-sound acceptance	`LeanCheckerAuthorityFlower.Service.ModelSound.not_accepts_forbidden`	none	proved
D	Transport model soundness through one-way acceptance refinement	`LeanCheckerAuthorityFlower.Service.AcceptanceRefinement.modelSound`	none	proved
D	Compose acceptance refinements without asserting equivalence	`LeanCheckerAuthorityFlower.Service.AcceptanceRefinement.comp`	none	proved
D	Characterize checked reconstruction by destination acceptance	`LeanCheckerAuthorityFlower.Service.checkedReconstruction_accepts_iff`	PQ	proved
D	Transport destination soundness through untrusted reconstruction	`LeanCheckerAuthorityFlower.Service.checkedReconstruction_sound`	PQ	proved
D	Transport destination model soundness through reconstruction	`LeanCheckerAuthorityFlower.Service.checkedReconstruction_modelSound`	PQ	proved
D	Expose exact step decisions as exact acceptance services	`LeanCheckerAuthorityFlower.Service.ofStepDecision_accepts_iff`	P	proved
D	Identify finite-catalog Lean head acceptance with its generated OSLF target type	`LeanCheckerAuthorityFlower.Service.catalogHeadService_accepts_iff_ntt`	PCQ	proved
D	Separate sound acceptance from completeness by a nonempty service	`LeanCheckerAuthorityFlower.Service.Controls.sound_does_not_imply_completeness`	P	proved
D	Show that exact checker agreement alone supplies no soundness	`LeanCheckerAuthorityFlower.Service.Controls.agreement_does_not_imply_soundness`	none	proved
D	Separate one contradiction refusal from general soundness	`LeanCheckerAuthorityFlower.Service.Controls.false_refusal_does_not_imply_soundness`	PQ	proved
D	Refuse a model-forbidden claim in a nonvacuous model-sound service	`LeanCheckerAuthorityFlower.Service.Controls.finiteModelService_refuses_forbidden`	P	proved
D	Characterize release-bound acceptance by exact coordinates and underlying acceptance	`LeanCheckerReleaseReceipt.ReleaseBoundService.accepts_iff`	P	proved
D	Retain the exact release while transporting checker soundness	`LeanCheckerReleaseReceipt.ReleaseBoundService.sound`	P	proved
D	Transport model soundness without deriving it from receipt metadata	`LeanCheckerReleaseReceipt.ReleaseBoundService.modelSound`	P	proved
D	Decline every valid mismatching release coordinate	`LeanCheckerReleaseReceipt.ReleaseBoundService.run_eq_declined_of_valid_mismatch`	P	proved
D	Require a concrete executable digest for the verified con-leche source pin	`LeanCheckerReleaseReceipt.conLecheVerifiedChecker_valid_iff`	P	proved
D	Refuse reuse across verified/trusted modes and checker families	`LeanCheckerReleaseReceipt.Controls.verdicts_all`	P	proved

Part	Claim	Lean declaration	Axioms	Status
D	Compose direct and registered-nested hypotheses into the exact complete position family	`IndexedPositiveContainer.Code.bundleExact`	Q	proved
D	Instantiate the generic induction bundle for the indexed Vector registry	`IndexedPositiveContainer.VectorControl.genericBundleExact`	PCQ	proved
D	Separate semantic Reader induction from impossible finite traversal	`IndexedPositiveContainer.ReaderControl.not_finitary`	PCQ	proved
D	Select complete duplicate-free source order for List positions	`IndexedContainer.Examples.listOrderedFinitary`	PCQ	proved
D	Select complete duplicate-free source order for Vector positions	`IndexedContainer.Examples.vectorOrderedFinitary`	PCQ	proved
D	Refuse ordered traversal for the infinitary Reader container	`IndexedContainer.Examples.reader_not_orderedFinitary`	PCQ	proved
D	Enumerate every mixed direct and nested code position in selected order	`IndexedPositiveContainer.Code.mem_orderedPositions`	PCQ	proved
D	Keep the selected mixed position enumeration duplicate-free	`IndexedPositiveContainer.Code.orderedPositions_nodup`	PCQ	proved
D	Count exactly one ordered traversal entry per semantic position	`IndexedPositiveContainer.Code.orderedPositions_length`	PCQ	proved
D	Execute the quoted position traversal while retaining arbitrary captured payloads	`IndexedContainerTraversal.returns`	PCQ	proved
D	Reflect completed traversal to one exact result with no effects or extra answers	`IndexedContainerTraversal.completed_iff`	PCQ	proved
D	Connect selected constructor-position order to emitted PeTTa execution	`IndexedContainerTraversal.ordered_returns`	PCQ	proved
D	Count the output records by the exact semantic position cardinality	`IndexedContainerTraversal.ordered_record_count`	PCQ	proved
D	Reflect equal complete observations to equality at every fixed-shape position	`IndexedContainerTraversal.ordered_output_eq_iff`	PCQ	proved
D	Generate the two-position input from the resolved textual VectorTree branch	`IndexedContainerTraversal.Controls.textualVectorInput_two`	PCQ	proved
D	Round-trip an exact two-family Workflow/Action source block	`MutualWorkflowActionLowering.lowerModule_quote`	PCQ	proved
D	Identify the textual mutual block with the independent Boolean-indexed polynomial	`MutualWorkflowActionLowering.Controls.erased_eq_existing`	Q	proved
D	Refuse wrong result families, cross-header duplicates, swaps and negative fields	`MutualWorkflowActionLowering.Controls.verdicts_all`	P	proved
D	Connect the textual mutual block exactly to the direct guest polynomial	`PositiveInductiveWorkflowActionGuest.Source.textual_erasure_eq_direct_bridge`	Q	proved
D	Form the generated Workflow/Action family, constructors and eliminator	`PositiveInductiveWorkflowActionGuest.signature_formed`	PCQ	proved
D	Retain four typed proof-relevant iota receipts in source order	`PositiveInductiveWorkflowActionGuest.iota_rule_names`	PCQ	proved
D	Refuse a Workflow child at the Action-typed schedule field	`PositiveInductiveWorkflowActionGuest.verdicts_all`	P	proved
D	Identify every published admitted-family candidate with its complete generated table	`IndexedInductive.Guest.Package.fromAdmitted_sound`	Q	proved
D	Retain the exact generated signature after admitted-family publication	`IndexedInductive.Guest.Package.fromAdmitted_signature`	Q	proved
D	Retain the proof-relevant root computation after admitted-family publication	`IndexedInductive.Guest.Package.fromAdmitted_computation`	Q	proved

Part	Claim	Lean declaration	Axioms	Status
D	Publish the independently admitted Vector through the shared candidate producer	`IndexedInductive.Guest.PackageControls.admitted_vector_publication`	PCQ	proved
D	Identify the admitted Vector candidate table with the generated table	`IndexedInductive.Guest.PackageControls.candidate_vector_table`	PCQ	proved
D	Identify the admitted Vector candidate signature with the generated signature	`IndexedInductive.Guest.PackageControls.candidate_vector_signature`	PCQ	proved
D	Identify the admitted Vector computation with the generated proof-relevant root	`IndexedInductive.Guest.PackageControls.candidate_vector_computation`	PCQ	proved
D	Factor textual mutual publication through the admitted-family producer	`IndexedInductive.Guest.PackageControls.publishMutual_eq_admitted`	Q	proved
D	Refuse zero-fuel admitted mutual publication	`IndexedInductive.Guest.PackageControls.admitted_mutual_publication_zero`	P	proved
D	Refuse aliased constructor names after mutual admission	`IndexedInductive.Guest.PackageControls.admitted_aliased_names_not_published`	P	proved
D	Make the Workflow/Action object guest consume the published candidate table	`PositiveInductiveWorkflowActionGuest.candidate_table`	PCQ	proved
D	Keep different authored constructor occurrences disjoint in every selected HOL carrier	`IndexedInductive.HOLFacts.Interpretation.construct_ne_of_occurrence_ne`	PQ	proved
D	Compute the selected structural fold on every complete constructor layer	`IndexedInductive.HOLFacts.Interpretation.recurse_construct`	PQ	proved
D	Characterize the selected structural fold by its constructor equations	`IndexedInductive.HOLFacts.Interpretation.recurse_unique`	Q	proved
D	Validate the generated recursion sentence for every admitted selected interpretation	`IndexedInductive.HOLFacts.models_recursion`	PQ	proved
D	Validate structural induction for every admitted selected interpretation	`IndexedInductive.HOLFacts.models_induction`	PQ	proved
D	Validate complete-layer no-confusion for every admitted selected interpretation	`IndexedInductive.HOLFacts.models_noConfusion`	PQ	proved
D	Select carrier infinity exactly when its separate witness is present	`IndexedInductive.HOLFacts.Selection.carrierInfinity_mem_iff`	P	proved
D	Validate every fact named by the selected HOL manifest	`IndexedInductive.HOLFacts.Selection.models_of_mem`	PQ	proved
D	Refuse infinity selection on every finite selected carrier	`IndexedInductive.HOLFacts.Selection.infinity_not_selected_of_finite`	P	proved
D	Select and validate recursion, induction, no-confusion and infinity for the authored natural family	`IndexedInductive.HOLFacts.NaturalControl.all_selected_models`	PCQ	proved
D	Project the four natural capabilities to exactly the existing five named facts	`IndexedInductive.HOLFacts.NaturalControl.projectedNativeFacts_eq`	Q	proved
D	Project the selected natural facts to exactly the maintained HOL Light wires	`IndexedInductive.HOLFacts.NaturalControl.projectedStatements_eq`	Q	proved

Part	Claim	Lean declaration	Axioms	Status
D	Recover the five established native theorem names in order	`IndexedInductive.HOLFacts.NaturalControl.projectedNames_eq`	Q	proved
D	Retain the authored natural meaning for every projected formula	`IndexedInductive.HOLFacts.NaturalControl.projected_models`	PQ	proved
D	Refute a Dedekind-infinity witness on lifted Boolean	`IndexedInductive.HOLFacts.bool_has_no_infinityWitness`	P	proved
D	Characterize the HOL separation formula for two constructor occurrences	`IndexedInductive.HOLFacts.ConstructorProjection.separation_meaning`	Q	proved
D	Validate constructor separation whenever the authored occurrences differ	`IndexedInductive.HOLFacts.ConstructorProjection.models_separation`	PQ	proved
D	Validate every automatically generated named no-confusion formula	`IndexedInductive.HOLFacts.ConstructorProjection.namedNoConfusions_models`	PQ	proved
D	Validate the named Zero/Succ separation projection	`IndexedInductive.HOLFacts.NaturalControl.zeroSuccPair_models`	PCQ	proved
D	Generate exactly both ordered Nat constructor-separation names	`IndexedInductive.HOLFacts.NaturalControl.all_named_noConfusion_names`	none	proved
D	Retain every structural source fact through typed HOL Light wire emission	`IndexedInductive.HOLFacts.NativeWire.Selection.emit_sources`	Q	proved
D	Retain semantic validity for every emitted structural source formula	`IndexedInductive.HOLFacts.NativeWire.Selection.emit_source_models`	PQ	proved
D	Retain every named constructor-pair formula through wire emission	`IndexedInductive.HOLFacts.ConstructorProjection.emitAll_sources`	Q	proved
D	Retain semantic validity for every emitted constructor-pair formula	`IndexedInductive.HOLFacts.ConstructorProjection.emitAll_source_models`	PQ	proved
D	Emit exactly the three structural mutual-family facts without infinity	`IndexedInductive.HOLFacts.MutualControl.emittedStructural_sources`	PCQ	proved
D	Generate exactly six ordered mutual-family constructor pairs	`IndexedInductive.HOLFacts.MutualControl.emitted_names`	PCQ	proved
D	Retain semantic validity for the non-Nat emitted manifest	`IndexedInductive.HOLFacts.MutualControl.emittedStructural_source_models`	PQ	proved
D	Decode every injectively encoded finite-catalog wire back to its source	`IndexedInductive.HOLFacts.WireCatalog.decode_encode`	PQ	proved
D	Ensure every successful catalog decode re-encodes to the input wire	`IndexedInductive.HOLFacts.WireCatalog.decode_sound`	PQ	proved
D	Round-trip selected structural statements under injective encoding	`IndexedInductive.HOLFacts.NativeWire.Selection.decode_statement`	PQ	proved
D	Round-trip constructor-pair statements under injective encoding	`IndexedInductive.HOLFacts.ConstructorProjection.decodePair_statement`	PQ	proved
D	Round-trip every mutual structural statement	`IndexedInductive.HOLFacts.MutualControl.structural_roundtrips`	PCQ	proved
D	Round-trip all six mutual constructor-pair statements	`IndexedInductive.HOLFacts.MutualControl.pair_roundtrips`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Expose statement collapse when occurrence names collide	`IndexedInductive.HOLFacts.MutualControl.colliding_occurrence_names_collapse_statements`	P	proved
D	Refuse the later-source roundtrip after a naming collision	`IndexedInductive.HOLFacts.MutualControl.colliding_occurrence_names_break_roundtrip`	P	proved
D	Commute complete constructor layers with selected carrier transport	`IndexedInductive.HOLFacts.Interpretation.transport_construct`	Q	proved
D	Commute recursive-layer mapping with selected carrier transport	`IndexedInductive.HOLFacts.Interpretation.transport_mapLayer`	Q	proved
D	Commute the generated fold with selected carrier transport	`IndexedInductive.HOLFacts.Interpretation.transport_recurse`	PQ	proved
D	Leave selected interpretations unchanged along identity transport	`IndexedInductive.HOLFacts.Interpretation.transport_refl`	Q	proved
D	Compose selected carrier transports exactly	`IndexedInductive.HOLFacts.Interpretation.transport_trans`	Q	proved
D	Recover selected interpretations after inverse transport	`IndexedInductive.HOLFacts.Interpretation.transport_symm`	Q	proved
D	Leave infinity evidence unchanged along identity transport	`IndexedInductive.HOLFacts.InfinityWitness.transport_refl`	Q	proved
D	Compose transported infinity evidence along carrier equivalences	`IndexedInductive.HOLFacts.InfinityWitness.transport_trans`	Q	proved
D	Recover infinity evidence after inverse transport	`IndexedInductive.HOLFacts.InfinityWitness.transport_symm`	Q	proved
D	Preserve selected infinity evidence along identity transport	`IndexedInductive.HOLFacts.Selection.transport_refl_witness`	PQ	proved
D	Compose the infinity evidence retained by a selected manifest	`IndexedInductive.HOLFacts.Selection.transport_trans_witness`	PQ	proved
D	Recover selected infinity evidence after inverse transport	`IndexedInductive.HOLFacts.Selection.transport_symm_witness`	PQ	proved
D	Preserve the exact selected fact manifest across carrier equivalence	`IndexedInductive.HOLFacts.Selection.transport_facts`	PQ	proved
D	Preserve the Nat manifest when transported to lifted host naturals	`IndexedInductive.HOLFacts.NaturalControl.hostSelection_facts`	PCQ	proved
D	Commute authored and host-Nat structural folds	`IndexedInductive.HOLFacts.NaturalControl.host_recurse_commutes`	PQ	proved
D	Validate all transported structural facts over host naturals	`IndexedInductive.HOLFacts.NaturalControl.host_all_selected_models`	PQ	proved
D	Commute the lifted host/material/surreal comparison triangle	`IndexedInductive.HOLFacts.NaturalControl.lifted_comparison_triangle`	PQ	proved
D	Identify direct and material-mediated natural-surreal interpretations	`IndexedInductive.HOLFacts.NaturalControl.material_surreal_interpretation_route`	PQ	proved
D	Preserve the selected fact manifest on the material-surreal route	`IndexedInductive.HOLFacts.NaturalControl.material_surreal_selection_facts_route`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Preserve the selected manifest over material naturals	`IndexedInductive.HOLFacts.NaturalControl.materialSelection_facts`	PCQ	proved
D	Preserve the selected manifest over natural surreals	`IndexedInductive.HOLFacts.NaturalControl.surrealSelection_facts`	PCQ	proved
D	Commute the structural fold with material-natural transport	`IndexedInductive.HOLFacts.NaturalControl.material_recurse_commutes`	PQ	proved
D	Commute the structural fold with natural-surreal transport	`IndexedInductive.HOLFacts.NaturalControl.surreal_recurse_commutes`	PQ	proved
D	Validate all selected HOL facts over material naturals	`IndexedInductive.HOLFacts.NaturalControl.material_all_selected_models`	PQ	proved
D	Validate all selected HOL facts over natural surreals	`IndexedInductive.HOLFacts.NaturalControl.surreal_all_selected_models`	PQ	proved
D	Decode guarded word-or-fallback successor at the selected HOL carrier	`NaturalBackendHOLFacts.successorBackend_host_exact`	PCQ	proved
D	Identify guarded backend successor with the authored polynomial successor	`NaturalBackendHOLFacts.successorBackend_authored_exact`	PCQ	proved
D	Retain exact selected observations on both fast and fallback branches	`NaturalBackendHOLFacts.fast_branch_host_exact`	PCQ	proved
D	Refuse a selected infinity witness on the finite word carrier	`NaturalBackendHOLFacts.finiteWord_has_no_infinityWitness`	P	proved
D	Separate exact observation from carrier equivalence for the tagged backend	`NaturalBackendHOLFacts.observeBackend_not_injective`	P	proved
D	Refute unchecked word successor at the selected carrier boundary	`NaturalBackendHOLFacts.unchecked_word_successor_not_host_exact`	P	proved
D	Connect every sound available exact-integer provider to natural addition by one	`NaturalBackendHOLFacts.IntegerProvider.addOne_host_exact`	PCQ	proved
D	Recover every registry-coherent recognized constructor from its canonical raw spelling	`PositiveInductiveBlock.RawCode.decode_quote`	PQ	proved
D	Derive positivity and uniform-parameter evidence from successful recognition	`PositiveInductiveBlock.RawCode.positive_of_decode`	P	proved
D	Derive finite recursive positions from a finitary container registry	`PositiveInductiveBlock.Code.positionFintype`	PCQ	proved
D	Retain positivity for every constructor in an accepted block	`PositiveInductiveBlock.recognize_positive`	PQ	proved
D	Recover an exact ordered block after registry-coherent quotation	`PositiveInductiveBlock.recognize_quote`	PQ	proved
D	Compute the generated recursor on every admitted constructor layer	`PositiveInductiveBlock.Admitted.recursor_construct`	none	proved
D	Characterize the generated recursor by initiality	`PositiveInductiveBlock.Admitted.recursor_unique`	Q	proved
D	Compute the generated dependent eliminator on every constructor layer	`PositiveInductiveBlock.Admitted.eliminator_construct`	none	proved
D	Characterize the generated dependent eliminator by displayed initiality	`PositiveInductiveBlock.Admitted.eliminator_unique`	Q	proved
D	Admit the registered-list nested search-tree block positively	`PositiveInductiveScenarios.Search.source_positive`	PQ	proved
D	Give every nested search branch a finite recursive-position type	`PositiveInductiveScenarios.Search.branchPositionFintype`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Collect the answer of a solved search node	`PositiveInductiveScenarios.Search.answers_solved`	P	proved
D	Collect nested branch answers in source order	`PositiveInductiveScenarios.Search.answers_branch`	P	proved
D	Retain distinct recursive positions for two nested search children	`PositiveInductiveScenarios.Search.nested_positions_distinct`	P	proved
D	Admit the mutual workflow/action block positively	`PositiveInductiveScenarios.Workflow.source_positive`	PQ	proved
D	Compute the generated recursor at a finished workflow	`PositiveInductiveScenarios.Workflow.actionCount_done`	P	proved
D	Follow workflow-to-action recursion	`PositiveInductiveScenarios.Workflow.actionCount_schedule`	P	proved
D	Count a primitive action	`PositiveInductiveScenarios.Workflow.actionCount_action`	P	proved
D	Follow action-to-workflow recursion	`PositiveInductiveScenarios.Workflow.actionCount_subworkflow`	P	proved
D	Refuse a nested recursive occurrence with changed parameters	`PositiveInductiveScenarios.Refusals.changedParameters_refused`	none	proved
D	Refuse recursion under an unregistered container	`PositiveInductiveScenarios.Refusals.unknownContainer_refused`	none	proved
D	Refuse a negative recursive function field	`PositiveInductiveScenarios.Refusals.negativeFunction_refused`	none	proved
D	Refuse duplicate constructor names	`PositiveInductiveScenarios.Refusals.duplicateConstructor_refused`	none	proved
D	Preserve direct constructor shapes in the richer grammar	`PositiveInductiveBlockBridge.Direct.shape_eq`	PQ	proved
D	Preserve direct admitted constructor order and names	`PositiveInductiveBlockBridge.Direct.admitted_names`	P	proved
D	Recognize every existing direct admitted block in the richer pipeline	`PositiveInductiveBlockBridge.Direct.admitted_recognized`	PQ	proved
D	Route the existing vector block through staged admission	`PositiveInductiveBlockBridge.Controls.vector_recognized`	PQ	proved
D	Route the existing mutual block through staged admission	`PositiveInductiveBlockBridge.Controls.mutual_recognized`	PQ	proved
D	Identify the data retained by an indexed-polynomial signature equivalence	`IndexedPolynomial.SignatureEquiv.refl`	Q	proved
D	Compare direct old/new constructor polynomials fibrewise	`PositiveInductiveBlockBridge.Direct.polynomialEquiv`	PCQ	proved
D	Compare complete ordered old/new polynomial inventories	`PositiveInductiveBlockBridge.Direct.admittedPolynomialEquiv`	PCQ	proved
D	Instantiate the complete polynomial comparison for vectors	`PositiveInductiveBlockBridge.Controls.vectorPolynomialEquiv`	PCQ	proved
D	Instantiate the complete polynomial comparison for the mutual family	`PositiveInductiveBlockBridge.Controls.mutualPolynomialEquiv`	PCQ	proved
D	Refuse polynomial equivalence after changing a recursive child index	`PositiveInductiveBlockBridge.Controls.translated_odd_not_equiv_wrongChild`	Q	proved

Part	Claim	Lean declaration	Axioms	Status
D	Recover every canonical textual positive telescope after lowering	`PositiveInductiveLowering.lowerCode_quote`	P	proved
D	Recover every accepted textual positive telescope by quotation	`PositiveInductiveLowering.quote_lowerCode`	P	proved
D	Round-trip complete canonical registered-container families	`PositiveInductiveLowering.lowerFamily_quote`	PQ	proved
D	Recover every accepted registered-container family source	`PositiveInductiveLowering.quote_lowerFamily`	PCQ	proved
D	Round-trip the selected one-family module shell	`PositiveInductiveLowering.lowerModule_quote`	PQ	proved
D	Recover every accepted selected module source	`PositiveInductiveLowering.quote_lowerModule`	PCQ	proved
D	Lower the textual registered-list search-tree family	`PositiveInductiveLoweringControls.Search.family_roundtrip`	PCQ	proved
D	Lower the complete textual search-tree module	`PositiveInductiveLoweringControls.Search.module_roundtrip`	PCQ	proved
D	Identify textual lowering with the existing nested search semantics	`PositiveInductiveLoweringControls.Search.erased_family_eq_existing`	P	proved
D	Refuse an unregistered textual nesting container	`PositiveInductiveLoweringControls.Search.unknown_container_refused`	PCQ	proved
D	Refuse a negative textual recursive field	`PositiveInductiveLoweringControls.Search.negative_field_refused`	PCQ	proved
D	Refuse a changed textual family header	`PositiveInductiveLoweringControls.Search.changed_header_refused`	PCQ	proved
D	Refuse duplicate textual constructor names	`PositiveInductiveLoweringControls.Search.duplicate_constructor_refused`	PCQ	proved
D	Refuse an unselected extra module block	`PositiveInductiveLoweringControls.Search.extra_block_refused`	PCQ	proved
D	Check registered-container module positive and refusal controls	`PositiveInductiveLoweringControls.Search.all_verdicts`	PCQ	proved
D	Equate dependent List hypotheses with one witness per list position	`IndexedContainer.listAllEquiv`	PQ	proved
D	Implement the registered List induction-hypothesis bundle	`IndexedContainer.listInductionBundle`	PCQ	proved
D	Count two positions in the genuine nested-list branch	`NestedInductiveGuestBoundary.Search.nested_position_card`	PCQ	proved
D	Count one position in the direct recursive-field surrogate	`NestedInductiveGuestBoundary.Search.direct_position_card`	PCQ	proved
D	Refute equivalence between nested-list positions and one direct slot	`NestedInductiveGuestBoundary.Search.nested_positions_not_direct`	PCQ	proved
D	Refute polynomial equivalence with the direct guest surrogate	`NestedInductiveGuestBoundary.Search.nested_polynomial_not_direct_surrogate`	PCQ	proved
D	Record the direct guest surrogate's one-field count	`NestedInductiveGuestBoundary.Search.direct_surrogate_fieldCount`	none	proved
D	Supply exact list-position induction hypotheses	`NestedInductiveGuestBoundary.Search.listPositionBundle`	PCQ	proved
D	Equate a native relational List spine with its source-ordered evidence bundle	`FormationSensitiveRegisteredListGuestLift.canonicalLiftEquiv`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Equate a native relational List spine with one typed pair per position	`FormationSensitiveRegisteredListGuestLift.canonicalLiftPositionEquiv`	PQ	proved
D	Identify the native relational List spine with the registered List induction bundle	`FormationSensitiveRegisteredListGuestLift.canonicalLiftSemanticBundleEquiv`	PCQ	proved
D	Recover every registered List position from the native object representation	`FormationSensitiveRegisteredListGuestLift.canonicalLiftRegisteredPositionEquiv`	PCQ	proved
D	Form the encoded List lift and retain its exact position evidence	`FormationSensitiveRegisteredListGuestLift.QualifiedLift.judgment_and_positionEvidence`	PCQ	proved
D	Transport an encoded registered-List lift through typed substitution	`FormationSensitiveRegisteredListGuestLift.QualifiedLift.substitute`	PQ	proved
D	Recover the selected target at a singleton List position	`FormationSensitiveRegisteredListGuestLift.Controls.singleton_position_target`	PCQ	proved
D	Refuse a relational List spine with a missing target position	`FormationSensitiveRegisteredListGuestLift.Controls.empty_target_refused`	PCQ	proved
D	Instantiate the native List/relator declarations at one selected level	`IntrinsicRelatorLevels.levelInstance`	PQ	proved
D	Decode all level-instantiated List/relator roots exactly	`IntrinsicRelatorLevels.rootDecoder`	PCQ	proved
D	Generate SearchTree constructor rows from the admitted textual source	`PositiveInductiveSearchTreeGuest.sourceConstructorEntries_eq`	PCQ	proved
D	Identify the guest source with the registered-container module lowering	`PositiveInductiveSearchTreeGuest.textual_source_eq`	PQ	proved
D	Admit the generated SearchTree declaration inventory	`PositiveInductiveSearchTreeGuest.inventory_admitted`	PCQ	proved
D	Form the complete generated SearchTree signature	`PositiveInductiveSearchTreeGuest.inventory_formed`	PCQ	proved
D	Commute open nested child traversal with renaming	`PositiveInductiveSearchTreeGuest.rename_liftChildren`	PQ	proved
D	Commute open nested child traversal with substitution	`PositiveInductiveSearchTreeGuest.subst_liftChildren`	PQ	proved
D	Check solved, branch and nested-bundle endpoints with refusal control	`PositiveInductiveSearchTreeGuest.endpointVerdicts_eq_true`	PCQ	proved
D	Form the open List-position recursive-result bundle	`PositiveInductiveSearchTreeGuest.branch_bundle_formed`	PCQ	proved
D	Retain the typed solved iota receipt	`PositiveInductiveSearchTreeGuest.solvedReceipt`	PQ	proved
D	Retain the typed registered-List branch iota receipt	`PositiveInductiveSearchTreeGuest.branchReceipt`	PCQ	proved
D	Transport the registered-List branch receipt through typed substitution	`PositiveInductiveSearchTreeGuest.branchReceipt_substitute`	PQ	proved
D	Keep the receipt-only SearchTree profile free of judgmental roots	`PositiveInductiveSearchTreeGuest.receiptOnly_no_root`	PCQ	proved
D	Reject the all-undeclared substitution for the branch schema	`PositiveInductiveSearchTreeGuest.untypedAssignment_not_typed`	PQ	proved
D	Separate raw branch evidence from typed schema instantiation	`PositiveInductiveSearchTreeGuest.rawEvidence_not_typedInstance`	PCQ	proved
D	Generalize induction bundles to indexed container outputs	`IndexedContainer.InductionBundle`	none	definition

Part	Claim	Lean declaration	Axioms	Status
D	Implement the length-indexed Vector induction bundle	`IndexedContainer.vectorInductionBundle`	PCQ	proved
D	Count every Vector output's recursive positions	`IndexedContainer.vector_position_card`	PCQ	proved
D	Refute equivalence of the length-two and length-three Vector fibres	`IndexedContainer.vector_two_not_equiv_three`	PCQ	proved
D	Retain the established List container in the indexed registry	`IndexedPositiveContainer.ListInstance.entry_container_eq`	none	proved
D	Count positions in the richer indexed Vector code	`IndexedPositiveContainer.VectorControl.position_card`	PCQ	proved
D	Refute output-index erasure in the richer Vector code	`IndexedPositiveContainer.VectorControl.two_not_equiv_three`	PCQ	proved
D	Connect the Vector code to its exact induction bundle	`IndexedPositiveContainer.VectorControl.bundleAt`	PCQ	proved
D	Make scoped Vector length lookup and quotation inverse	`IndexedPositiveVectorLowering.Scope.lookup_quoteLength`	PQ	proved
D	Round-trip the textual indexed-Vector module	`IndexedPositiveVectorLowering.Controls.module_roundtrip`	PCQ	proved
D	Lower the textual branch to the exact indexed Vector code	`IndexedPositiveVectorLowering.Controls.branch_erases_to_indexed_vector`	P	proved
D	Refuse unbound and shadowed Vector length names	`IndexedPositiveVectorLowering.Controls.shadowedLength_refused`	PCQ	proved
D	Refuse a negative field in the indexed Vector source route	`IndexedPositiveVectorLowering.Controls.negativeField_refused`	PCQ	proved
D	Instantiate native Vec elimination one universe above its element family	`NativeVectorAll.signature_formed`	PCQ	proved
D	Commute the exact Vector ‘All‘ type with renaming and substitution	`NativeVectorAll.subst_allType`	PQ	proved
D	Commute constructed Vector ‘All‘ evidence with renaming and substitution	`NativeVectorAll.subst_allEvidence`	PQ	proved
D	Form the eliminator-defined Vector ‘All‘ type	`NativeVectorAll.Controls.allType_formed`	PCQ	proved
D	Type the constructed evidence at the exact source-Vector ‘All‘ family	`NativeVectorAll.Controls.allEvidence_typed`	PCQ	proved
D	Refuse a changed Vector ‘All‘ result type	`NativeVectorAll.Controls.wrong_type_refused`	PCQ	proved
D	Refuse an undeclared pointwise Vector witness	`NativeVectorAll.Controls.undeclared_witness_refused`	PCQ	proved
D	Identify the canonical Vector semantic bundle with its source-ordered witness spine	`NativeVectorAll.CanonicalAdequacy.semanticBundleEquiv`	PCQ	proved
D	Recover the expected witness at every exact Vector position	`NativeVectorAll.CanonicalAdequacy.pointwiseBundle_exact`	PQ	proved
D	Reduce native Vector ‘All‘ evidence to the quotation of the selected semantic bundle	`NativeVectorAll.CanonicalAdequacy.allEvidence_steps_semanticBundle`	PCQ	proved
D	Retain both canonical Vector witnesses in source order	`NativeVectorAll.Controls.CanonicalAdequacyControls.two_position_reduces`	PQ	proved
D	Distinguish a swap of two canonical Vector positions	`NativeVectorAll.Controls.CanonicalAdequacyControls.swapped_positions_refused`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Connect a formed object computation to one selected semantic induction bundle	`FormationSensitiveIndexedContainerBundleRealization.SelectedObjectBundle`	none	definition
D	Recover every semantic position from the connected selected object bundle	`FormationSensitiveIndexedContainerBundleRealization.SelectedObjectBundle.positionWitnesses`	PQ	proved
D	Instantiate the object-realization contract through the registered List lookup	`FormationSensitiveIndexedContainerBundleRealization.ListInstance.registeredRealization`	PCQ	proved
D	Instantiate the object-realization contract through the registered Vector lookup	`FormationSensitiveIndexedContainerBundleRealization.VectorInstance.registeredRealization`	PCQ	proved
D	Recover both witnesses from the connected two-position Vector realization	`FormationSensitiveIndexedContainerBundleRealization.VectorInstance.covers_two_positions`	PQ	proved
D	Refuse an object realization when the Reader motive has no position witnesses	`FormationSensitiveIndexedContainerBundleRealization.ReaderNegative.noSelectedObjectBundle`	PCQ	proved
D	Quote complete direct and registered-nested induction bundles in source order	`FormationSensitiveIndexedContainerBundleRealization.Code.quoteBundle`	none	definition
D	Recover every direct and nested witness from a code-wide object realization	`FormationSensitiveIndexedContainerBundleRealization.Code.SelectedObjectBundle.positionWitnesses`	PCQ	proved
D	Identify the mixed direct/Vector quotation with its nested object pair	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.quote_bundle`	PQ	proved
D	Form the mixed direct/Vector object bundle at its nested Sigma type	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.object_formed`	PCQ	proved
D	Compute the mixed native Vector evidence to the generic code quotation	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.object_steps_quote`	PCQ	proved
D	Recover all three positions of the mixed direct/Vector constructor	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.allPositionWitnesses`	PCQ	proved
D	Assemble exactly the supplied formed leaves along arbitrary constructor code	`FormationSensitiveIndexedContainerBundleRealization.Code.realizeFromLeaves`	PCQ	proved
D	Turn an assembled object value into the code-wide selected contract	`FormationSensitiveIndexedContainerBundleRealization.Code.ObjectValue.toSelectedCode`	PQ	proved
D	Match compiled and independently constructed mixed semantic bundles	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiled_value_eq`	PCQ	proved
D	Match compiled and independently constructed mixed object terms	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiled_term_eq`	PCQ	proved
D	Match compiled and independently constructed mixed Sigma types	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiled_type_eq`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Match the formation algebra's quotation with the generic code quotation	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiled_quote_eq`	PCQ	proved
D	Recover all mixed positions from the formation-algebra output	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.compiledPositionWitnesses`	PCQ	proved
D	Embed total direct and nested producers as an always-accepting provider table	`FormationSensitiveIndexedContainerBundleRealization.Code.PartialLeafProviders.ofTotal`	none	definition
D	Route formation-qualified providers recursively over arbitrary constructor code	`FormationSensitiveIndexedContainerBundleRealization.Code.realizeWithProviders?`	PCQ	definition
D	Recover every position witness from a successful provider result	`FormationSensitiveIndexedContainerBundleRealization.Code.positionWitnessesOfProviderSuccess`	PQ	proved
D	Match provider-routed and total realizations on mixed direct/Vector code	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.completeProviders_compile`	PCQ	proved
D	Recover all mixed positions from the provider-routed realization	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.providerPositionWitnesses`	PCQ	proved
D	Refuse mixed direct/Vector realization when the nested provider is absent	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.missingNestedProvider_refused`	PCQ	proved
D	Refuse mixed direct/Vector realization when direct evidence is absent	`FormationSensitiveIndexedContainerBundleRealization.MixedControl.missingDirectProvider_refused`	PCQ	proved
D	Refuse an object value when the Reader motive has no semantic bundle	`FormationSensitiveIndexedContainerBundleRealization.ReaderNegative.noObjectValue`	PCQ	proved
D	Retain occurrence-specific shape and children in a constructor realization request	`FormationSensitiveIndexedContainerBundleRealization.Code.ConstructorRequest`	none	definition
D	Route one occurrence-specific request through selected object providers	`FormationSensitiveIndexedContainerBundleRealization.Code.realizeRequest?`	PCQ	definition
D	Route every ordered constructor occurrence while retaining individual refusals	`FormationSensitiveIndexedContainerBundleRealization.Code.realizeInventory?`	PCQ	definition
D	Identify inventory routing at each occurrence with the single-request compiler	`FormationSensitiveIndexedContainerBundleRealization.Code.realizeInventory?_at`	PQ	proved
D	Recover the exact ordered VectorTree codes from the resolved textual family	`PositiveInductiveVectorTreeGuest.ProviderInventory.codes_eq`	P	proved
D	Accept both textual VectorTree occurrences with complete providers	`PositiveInductiveVectorTreeGuest.ProviderInventory.branch_provider_accepted`	PCQ	proved
D	Preserve solved and refuse branch when the Vector provider is absent	`PositiveInductiveVectorTreeGuest.ProviderInventory.missingNested_verdicts`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Generate VectorTree constructor rows from the indexed textual source	`PositiveInductiveVectorTreeGuest.sourceConstructorEntries_eq`	P	proved
D	Preserve the nested Vector traversal under renaming and substitution	`PositiveInductiveVectorTreeGuest.subst_liftChildren`	PQ	proved
D	Admit and form the complete VectorTree declaration inventory	`PositiveInductiveVectorTreeGuest.inventory_formed`	PCQ	proved
D	Form the exact source-Vector recursive-result bundle	`PositiveInductiveVectorTreeGuest.branch_bundle_formed`	PCQ	proved
D	Check all VectorTree iota endpoints and refuse an unbundled branch	`PositiveInductiveVectorTreeGuest.endpointVerdicts_eq_true`	PCQ	proved
D	Retain the typed solved VectorTree receipt	`PositiveInductiveVectorTreeGuest.solvedReceipt`	PCQ	proved
D	Retain the typed length-indexed VectorTree branch receipt	`PositiveInductiveVectorTreeGuest.branchReceipt`	PCQ	proved
D	Transport the VectorTree branch receipt through typed substitution	`PositiveInductiveVectorTreeGuest.branchReceipt_substitute`	PCQ	proved
D	Transport complete generic PTS derivations along sort-renaming morphisms	`GenericPureTypeSystem.Derives.map`	P	proved
D	Identify the maintained two-sort PTS with its generic instance	`GenericPureTypeSystem.TwoSort.derives_iff`	P	proved
D	Prove functionality of the three-stratum comparison target	`HypercubeSortRenaming.comparisonProfile_functional`	P	proved
D	Exclude a self-typed sort from the comparison target	`HypercubeSortRenaming.comparisonProfile_no_self_axiom`	none	proved
D	Expose noncommutation of the shared Type sort	`HypercubeSortRenaming.comparison_maps_disagree_on_type`	none	proved
D	Expose noncommutation of the shared Kind sort	`HypercubeSortRenaming.comparison_maps_disagree_on_kind`	none	proved
D	Transport the CoC polymorphic identity to the lower strata	`HypercubeSortRenaming.coc_polymorphicIdentity_transports`	PQ	proved
D	Transport a prototype type operator to the upper strata	`HypercubeSortRenaming.prototype_typeOperator_transports`	PQ	proved
D	Force sort collapse in every functional commuting cocone	`HypercubeSortRenaming.commuting_functional_cocone_collapses`	PQ	proved
D	Derive a self-typed sort in every functional commuting cocone	`HypercubeSortRenaming.commuting_cocone_self_axiom`	PQ	proved
D	Retain the paradox product rule after the commuting collapse	`HypercubeSortRenaming.commuting_cocone_self_product`	PQ	proved
D	Transport the concrete Hurkens inconsistency witness through the collapse	`HypercubeSortRenaming.commuting_functional_cocone_inconsistent`	PQ	proved
D	Check the finite shifted-span and noncommutation controls	`HypercubeSortRenaming.Controls.all_verdicts`	none	proved
D	Infer all 31 frozen DTTBench statement types under CoC	`LFDTTBenchDemand.allStatementsTypableInCoc_eq_true`	P	proved
D	Reconstruct declarative CoC typing for every frozen statement	`LFDTTBenchDemand.coc_typable_of_mem`	PQ	proved
D	Refute profile-independent necessity of the prototype demand rule	`LFDTTBenchDemand.prototype_demand_not_necessary_for_coc_typability`	P	proved
D	Check the intrinsic nested fold and result adapter	`PositiveInductiveSearchTreeElaboration.fold_typed`	PQ	proved
D	Apply the closed nested fold to its actual eliminator body	`PositiveInductiveSearchTreeElaboration.foldTerm_beta`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Compute the declared child/result bundle method	`PositiveInductiveSearchTreeElaboration.consBundleCase_beta`	PQ	proved
D	Retain every exact child/call pair in source order	`PositiveInductiveSearchTreeElaboration.liftChildren_fst_encode`	PQ	proved
D	Join the bundle result view with the source recursive map	`PositiveInductiveSearchTreeElaboration.bundleResults_common_output`	PQ	proved
D	Open a generated method by ordinary beta computation	`PositiveInductiveSearchTreeElaboration.applyMethod_reduces`	P	proved
D	Compile both authored nested source methods	`PositiveInductiveSearchTreeElaboration.methods_emitted`	PCQ	proved
D	Retain the existing admitted SearchTree source declarations	`PositiveInductiveSearchTreeElaboration.source_declarations_exact`	PCQ	proved
D	Emit the whole authored nested fold	`PositiveInductiveSearchTreeElaboration.function_emitted`	PCQ	proved
D	Derive selected call plans from registered constructor forms	`PositiveInductiveSearchTreeElaboration.selected_plans_derived`	PCQ	proved
D	Reconstruct both actual nested clauses in constructor order	`PositiveInductiveSearchTreeElaboration.source_methods_reconstruct`	PCQ	proved
D	Independently type-check the actual emitted nested program	`PositiveInductiveSearchTreeElaboration.source_typed`	PCQ	proved
D	Apply an accepted source-produced program to its actual body	`PositiveInductiveSearchTreeElaboration.source_application`	PCQ	proved
D	Every checked nested proposal has the displayed dependent typing	`PositiveInductiveSearchTreeElaboration.checkProposal_sound`	PCQ	proved
D	Accept the nested source and reject source and typing mutations	`PositiveInductiveSearchTreeElaboration.all_verdicts`	PCQ	proved
D	Form the fresh authored transparent declaration	`PositiveInductiveSearchTreeElaboration.Definitions.formed`	PCQ	proved
D	Retain dependent typing of the actual published body	`PositiveInductiveSearchTreeElaboration.Definitions.bodyTyped`	PCQ	proved
D	Type the actual authored constant in its declaration extension	`PositiveInductiveSearchTreeElaboration.Definitions.constant_typed`	PQ	proved
D	Preserve every displayed refined type through delta unfolding	`PositiveInductiveSearchTreeElaboration.Definitions.typedUnfolding`	PCQ	proved
D	Compute a named call to its actual compiled eliminator body	`PositiveInductiveSearchTreeElaboration.Definitions.namedApplication`	PCQ	proved
D	Expand named recursive maps to ordered object-level calls	`PositiveInductiveSearchTreeElaboration.Definitions.namedMap`	PCQ	proved
D	Join named source maps with the proof-relevant bundle result view	`PositiveInductiveSearchTreeElaboration.Definitions.namedBundleCommonOutput`	PCQ	proved
D	Remove the recursive source name from the compiled body	`PositiveInductiveSearchTreeElaboration.Definitions.body_no_recursive_name`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Refuse reuse of the existing family declaration name	`PositiveInductiveSearchTreeElaboration.Definitions.shadow_refused`	PQ	proved
D	Refuse a bare self-delta definition	`PositiveInductiveSearchTreeElaboration.Definitions.self_refused`	PQ	proved
D	Check named-declaration positive and negative controls	`PositiveInductiveSearchTreeElaboration.Definitions.all_verdicts`	PQ	proved
D	Independently check all five actual fold substitution components	`PositiveInductiveSearchTreeElaboration.Branch.slots`	PCQ	proved
D	Form the actual fold context substitution	`PositiveInductiveSearchTreeElaboration.Branch.contextMorphism`	PCQ	proved
D	Transport both fully formed branch endpoints	`PositiveInductiveSearchTreeElaboration.Branch.formed_endpoints`	PCQ	proved
D	Retain the same checked substitution in the selected guest	`PositiveInductiveSearchTreeElaboration.Branch.guestMorphism`	PCQ	proved
D	Convert the constant motive application to the result type	`PositiveInductiveSearchTreeElaboration.Branch.result_conversion`	PQ	proved
D	Type both actual branch endpoints at their displayed result	`PositiveInductiveSearchTreeElaboration.Branch.result_endpoints`	PCQ	proved
D	Retain the actual fold branch as an explicit typed iota receipt	`PositiveInductiveSearchTreeElaboration.Branch.receipt`	PCQ	proved
D	Identify the receipt source with the actual fold branch	`PositiveInductiveSearchTreeElaboration.Branch.left_is_fold`	P	proved
D	Refuse children-as-results in the actual branch method check	`PositiveInductiveSearchTreeElaboration.Branch.children_as_results_refused`	PQ	proved
D	Check the concrete typed fold-receipt controls	`PositiveInductiveSearchTreeElaboration.Branch.all_verdicts`	PCQ	proved
D	Transport guarded universal observations in every Henkin model	`HOL.GuardedQuantifierTransport.models_universal`	none	proved
D	Transport guarded existential observations in every Henkin model	`HOL.GuardedQuantifierTransport.models_existential`	none	proved
D	Retain both typed guarded transport schemas	`HOL.GuardedQuantifierTransport.all_models`	none	proved
D	Distinguish inhabited padding from an empty represented carrier	`HOL.GuardedQuantifierTransport.empty_guard_separates`	none	proved
D	Refute the actual typed statement with the target guard erased	`HOL.GuardedQuantifierTransport.guard_erasure_counterexample`	none	proved
D	Certify a nonidentity guarded Boolean comparison	`HOL.GuardedQuantifierTransport.Examples.swap_comparison`	none	proved
D	Retain the chosen comparison action on a selected observation	`HOL.GuardedQuantifierTransport.Examples.swap_changes_observation`	Q	proved
D	Witness the model hypotheses on an inhabited representation	`HOL.GuardedQuantifierTransport.standard_instance`	none	proved
D	Check nonidentity, inverse-action and padding controls	`HOL.GuardedQuantifierTransport.all_verdicts`	Q	proved

Part	Claim	Lean declaration	Axioms	Status
D	Keep the native guarded theorem names distinct	`HOLLight.GuardedQuantifierTransport.names_distinct`	none	proved
D	Characterize exact existential truth transport by the valid image	`HOL.GuardedQuantifierTransport.exists_observations_iff_image`	none	proved
D	Characterize exact universal truth transport by the valid image	`HOL.GuardedQuantifierTransport.all_observations_iff_image`	none	proved
D	Separate quantified truth transport from faithful value transport	`HOL.GuardedQuantifierTransport.truth_transport_does_not_force_injectivity`	none	proved
D	Commute registered-container constructor generation with substitution	`IndexedPositiveContainerGuestTypes.constructorType_substitute`	PQ	proved
D	Commute full dependent method generation with substitution	`IndexedPositiveContainerGuestTypes.methodBody_substitute`	PQ	proved
D	Commute named method generation with substitution	`IndexedPositiveContainerGuestTypes.methodType_substitute`	PQ	proved
D	Preserve every ordinary indexed constructor interface	`IndexedPositiveContainerGuestTypes.constructorType_ordinary`	PQ	proved
D	Preserve every ordinary indexed method body and IH order	`IndexedPositiveContainerGuestTypes.methodBody_ordinary`	PQ	proved
D	Preserve every ordinary indexed named method	`IndexedPositiveContainerGuestTypes.methodType_ordinary`	PQ	proved
D	Independently admit the generated List declaration inventory	`IndexedPositiveContainerGuestControls.Lists.declarations_admitted`	PQ	proved
D	Retain formation of the generated List inventory	`IndexedPositiveContainerGuestControls.Lists.declarations_formed`	PQ	proved
D	Independently admit the generated Vector declaration inventory	`IndexedPositiveContainerGuestControls.Vectors.declarations_admitted`	PCQ	proved
D	Retain formation of the generated Vector inventory	`IndexedPositiveContainerGuestControls.Vectors.declarations_formed`	PCQ	proved
D	Retain the mixed constructor's actual source fields	`IndexedPositiveContainerGuestControls.Vectors.Mixed.source_roundtrip`	PCQ	proved
D	Retain the mixed constructor's earlier Nat length	`IndexedPositiveContainerGuestControls.Vectors.Mixed.constructor_exact`	PCQ	proved
D	Retain the mixed method's complete ordered IHs	`IndexedPositiveContainerGuestControls.Vectors.Mixed.method_exact`	PCQ	proved
D	Independently form the mixed dependent method	`IndexedPositiveContainerGuestControls.Vectors.Mixed.method_formed`	PCQ	proved
D	Exhibit the simple branch's insufficient coverage	`IndexedPositiveContainerGuestControls.Vectors.Mixed.wrong_simple_branch_agrees`	PQ	proved
D	Refuse the mixed method with a faulty length reader	`IndexedPositiveContainerGuestControls.Vectors.Mixed.wrong_length_refused`	PCQ	proved
D	Retain the actual Nat-indexed Vec constructor	`IndexedPositiveContainerGuestControls.OrdinaryVectors.cons_constructor_exact`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Retain the actual Nat-indexed Vec method	`IndexedPositiveContainerGuestControls.OrdinaryVectors.cons_method_exact`	PCQ	proved
D	Check generated declaration positive and negative controls	`IndexedPositiveContainerGuestControls.verdicts_eq_true`	PCQ	proved
D	Count complete method-call arguments	`IndexedPositiveContainerGuestPlans.calls_length`	P	proved
D	Transport recursive-call plans with substitution	`IndexedPositiveContainerGuestPlans.calls_substitute`	P	proved
D	Match plan call count to the constructor code	`IndexedPositiveContainerGuestPlans.plan_call_count`	PQ	proved
D	Reconstruct accepted clauses from code-derived plans	`IndexedPositiveContainerGuestPlans.compile_reconstructs`	PQ	proved
D	Commute authored nested map syntax with substitution	`PositiveInductiveSearchTreeElaboration.sourceMap_substitute`	PQ	proved
D	Retain mixed source fields and call positions	`IndexedPositiveContainerGuestPlansControls.VectorCalls.generated_exact`	PCQ	proved
D	Retain the earlier Vector output index	`IndexedPositiveContainerGuestPlansControls.VectorCalls.calls_exact`	PCQ	proved
D	Compile all three mixed IH positions	`IndexedPositiveContainerGuestPlansControls.VectorCalls.compilation_exact`	PCQ	proved
D	Retain mixed clause source and recursive-name freedom	`IndexedPositiveContainerGuestPlansControls.VectorCalls.compiled_reconstructs`	PCQ	proved
D	Compile the first-child source clause	`IndexedPositiveContainerGuestPlansControls.VectorCalls.first_compiled`	PCQ	proved
D	Witness accepted mixed-source reconstruction	`IndexedPositiveContainerGuestPlansControls.VectorCalls.first_reconstructs`	PCQ	proved
D	Beta-reconstruct the actual mixed clause	`IndexedPositiveContainerGuestPlansControls.VectorCalls.first_beta`	PCQ	proved
D	Witness call-plan substitution on the mixed source	`IndexedPositiveContainerGuestPlansControls.VectorCalls.substitution_witness`	PCQ	proved
D	Retain call occurrences when terms coincide	`IndexedPositiveContainerGuestPlansControls.VectorCalls.coincident_children_keep_slots`	PQ	proved
D	Expose method-slot loss under value deduplication	`IndexedPositiveContainerGuestPlansControls.VectorCalls.value_dedup_drops_slot`	PQ	proved
D	Refuse a recursive call on the data payload	`IndexedPositiveContainerGuestPlansControls.VectorCalls.wrong_child_refused`	PCQ	proved
D	Refuse a changed nested output length	`IndexedPositiveContainerGuestPlansControls.VectorCalls.wrong_length_refused`	PCQ	proved
D	Distinguish lexical call order from reversal	`IndexedPositiveContainerGuestPlansControls.VectorCalls.correct_order_not_reversed`	PQ	proved
D	Check source-call plan and abstraction controls	`IndexedPositiveContainerGuestPlansControls.all_verdicts`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Count all authored header slots	`IndexedInductive.OperationSource.Header.fromPlan_slot_count`	PQ	proved
D	Retain ordinary constructor arity	`IndexedPositiveContainerGuestPlans.fieldCount_ordinary`	none	proved
D	Preserve ordinary index/child call templates	`IndexedPositiveContainerGuestPlans.calls_ordinary`	PCQ	proved
D	Replay source clauses at generated dependent interfaces	`IndexedPositiveContainerGuestPlans.elaborateClause_sound`	PCQ	proved
D	Retain the Nat-indexed append plan	`IndexedPositiveContainerGuestPlansControls.OrdinaryAppend.cons_plan_exact`	PCQ	proved
D	Witness ordinary calls on the actual Vec code	`IndexedPositiveContainerGuestPlansControls.OrdinaryAppend.calls_ordinary_witness`	PCQ	proved
D	Count the actual Vec source fields	`IndexedPositiveContainerGuestPlansControls.OrdinaryAppend.cons_fields_exact`	P	proved
D	Compile the named mixed clause	`IndexedPositiveContainerGuestPlansControls.VectorCalls.named_before_compiled`	PCQ	proved
D	Count every mixed source header slot	`IndexedPositiveContainerGuestPlansControls.VectorCalls.named_slots_count`	PCQ	proved
D	Refuse a named recursive payload	`IndexedPositiveContainerGuestPlansControls.VectorCalls.named_payload_refused`	PCQ	proved
D	Refuse aliased source constructor binders	`IndexedPositiveContainerGuestPlansControls.VectorCalls.source_alias_refused`	PCQ	proved
D	Resolve the authored VectorTree inventory	`PositiveInductiveVectorTreeElaboration.source_inventory_exact`	PCQ	proved
D	Compile both authored VectorTree methods	`PositiveInductiveVectorTreeElaboration.methods_emitted`	PCQ	proved
D	Retain the generated solved interface	`PositiveInductiveVectorTreeElaboration.generated_solved_type`	PQ	proved
D	Retain the generated dependent branch interface	`PositiveInductiveVectorTreeElaboration.generated_branch_type`	PQ	proved
D	Retain source compilation after method replay	`PositiveInductiveVectorTreeElaboration.checked_sound`	PCQ	proved
D	Type the authored VectorTree solved method	`PositiveInductiveVectorTreeElaboration.solved_typed`	PCQ	proved
D	Type the authored VectorTree branch method	`PositiveInductiveVectorTreeElaboration.branch_typed`	PCQ	proved
D	Beta-reconstruct the parsed VectorTree branch	`PositiveInductiveVectorTreeElaboration.branch_beta`	PCQ	proved
D	Reconstruct typing of the compiled method application	`PositiveInductiveVectorTreeElaboration.branch_application_typed`	PCQ	proved
D	Retain both typed reconstruction endpoints	`PositiveInductiveVectorTreeElaboration.branch_endpoints_typed`	PCQ	proved
D	Identify the actual parsed receipt target	`PositiveInductiveVectorTreeElaboration.branch_read_exact`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Transport the selected receipt to the parsed branch	`PositiveInductiveVectorTreeElaboration.sourceBranchReceipt`	PCQ	proved
D	Exhibit raw compilation without a valid bundle	`PositiveInductiveVectorTreeElaboration.unbundled_compiles`	PCQ	proved
D	Refuse a missing induction bundle at replay	`PositiveInductiveVectorTreeElaboration.unbundled_refused`	PCQ	proved
D	Check authored dependent-source controls	`PositiveInductiveVectorTreeElaboration.all_verdicts`	PCQ	proved
D	Commute scoped index-domain input with substitution	`IndexedPositiveContainerGuestTypes.Indexing.weakenDomain_substitute`	PQ	proved
D	Substitute the generated dependent motive type	`IndexedPositiveContainerGuestTypes.motiveType_substitute`	PQ	proved
D	Substitute the complete eliminator result interface	`IndexedPositiveContainerGuestTypes.resultType_substitute`	PQ	proved
D	Substitute all ordered dependent methods	`IndexedPositiveContainerGuestTypes.methodsType_substitute`	PQ	proved
D	Substitute the complete generated eliminator interface	`IndexedPositiveContainerGuestTypes.eliminatorType_substitute`	PQ	proved
D	Retain ordinary indexed method inventories	`IndexedPositiveContainerGuestTypes.methodsType_ordinary`	PQ	proved
D	Retain complete ordinary indexed eliminator interfaces	`IndexedPositiveContainerGuestTypes.eliminatorType_ordinary`	PQ	proved
D	Generate the actual List SearchTree eliminator	`IndexedPositiveContainerGuestControls.Lists.eliminator_exact`	PQ	proved
D	Witness complete generation at an open source parameter	`IndexedPositiveContainerGuestControls.Lists.eliminator_substitution_witness`	PQ	proved
D	Detect List constructor-order interface drift	`IndexedPositiveContainerGuestControls.Lists.reversed_order_not_exact`	PQ	proved
D	Generate the actual VectorTree eliminator	`IndexedPositiveContainerGuestControls.Vectors.eliminator_exact`	PQ	proved
D	Detect Vector constructor-order interface drift	`IndexedPositiveContainerGuestControls.Vectors.reversed_order_not_exact`	PQ	proved
D	Retain the actual Nat-indexed Vec eliminator	`IndexedPositiveContainerGuestControls.OrdinaryVectors.eliminator_exact`	PCQ	proved
D	Compose declaration-prefix typing evidence	`DeclarationAdmissionReplay.Admitted.append`	PQ	proved
D	Retain exact published source names	`IndexedInductive.Guest.Declarations.Registered.entries_names`	PQ	proved
D	Keep generated entries opaque	`IndexedInductive.Guest.Declarations.Registered.entries_opaque`	PQ	proved
D	Preserve complete ordinary indexed inventories	`IndexedInductive.Guest.Declarations.Registered.entries_ordinary`	PQ	proved
D	Replay generated entries at their source prefix	`IndexedInductive.Guest.Declarations.Registered.admitInventory_sound`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Form generated libraries with admitted dependencies	`IndexedInductive.Guest.Declarations.Registered.admitInventory_formed`	PQ	proved
D	Generate the actual parameterized SearchTree inventory	`IndexedPositiveContainerGuestControls.Lists.entries_exact`	PQ	proved
D	Refuse a family/eliminator name collision	`IndexedPositiveContainerGuestControls.Lists.collision_refused`	PQ	proved
D	Generate VectorTree entries after the Answer dependency	`IndexedPositiveContainerGuestControls.Vectors.entries_exact`	PQ	proved
D	Admit the generated VectorTree suffix	`IndexedPositiveContainerGuestControls.Vectors.generated_admitted`	PCQ	proved
D	Admit the actual Answer dependency	`IndexedPositiveContainerGuestControls.Vectors.prior_admitted`	PCQ	proved
D	Refuse a payload in a Vector length position	`IndexedPositiveContainerGuestControls.Vectors.wrong_data_refused`	PCQ	proved
D	Retain the actual Nat-indexed Vec declarations	`IndexedPositiveContainerGuestControls.OrdinaryVectors.entries_exact`	PCQ	proved
D	Preserve name uniqueness through prefix admission	`DeclarationAdmissionReplay.Admitted.names_nodup`	PQ	proved
D	Derive published-name separation from replay	`IndexedInductive.Guest.Declarations.Registered.admitInventory_names`	PQ	proved
D	Refuse endpoint aliases at every production budget	`IndexedInductive.Guest.Declarations.Registered.aliased_endpoints_refused`	PQ	proved
D	Witness List source-name separation	`IndexedPositiveContainerGuestControls.Lists.names_distinct`	PQ	proved
D	Witness Vector source-name separation	`IndexedPositiveContainerGuestControls.Vectors.names_distinct`	PCQ	proved
D	Preserve exact certificate concatenation	`IndexedInductive.Guest.Declarations.produce_append`	PQ	proved
D	Compose production and independent replay exactly	`IndexedInductive.Guest.Declarations.elaborate_append`	PQ	proved
D	Factor acceptance over consecutive prefixes	`IndexedInductive.Guest.Declarations.elaborate_isSome_append`	PQ	proved
D	Assemble whole-inventory acceptance from checked entries	`IndexedInductive.Guest.Declarations.elaborate_cons_of`	PQ	proved
D	Preserve result projection under scoped substitution	`NativeVectorAll.ResultView.subst_project`	PQ	proved
D	Project the exact ordered canonical Vector bundle	`NativeVectorAll.ResultView.canonical_steps`	PQ	proved
D	Open the closed Vector result program by beta computation	`NativeVectorAll.ResultView.program_beta`	PQ	proved
D	Replay the complete polymorphic Vector result program	`NativeVectorAll.ResultView.program_formed`	PCQ	proved
D	Form the library with the actual checked result definition	`PositiveInductiveVectorTreeGuest.ResultFold.checking_formed`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Retain formation with the existing selected Tree computation	`PositiveInductiveVectorTreeGuest.ResultFold.execution_formed`	PCQ	proved
D	Replay the whole constant-motive VectorTree fold	`PositiveInductiveVectorTreeGuest.ResultFold.fold_typed`	PCQ	proved
D	Check all five source-context substitutions	`PositiveInductiveVectorTreeGuest.ResultFold.contextMorphism`	PCQ	proved
D	Transport the original parsed branch receipt	`PositiveInductiveVectorTreeGuest.ResultFold.sourceReceipt`	PCQ	proved
D	Beta-reconstruct the specialized parsed branch target	`PositiveInductiveVectorTreeGuest.ResultFold.parsed_branch_beta`	PCQ	proved
D	Connect the selected source step to the projected node call	`PositiveInductiveVectorTreeGuest.ResultFold.source_path`	PCQ	proved
D	Retain both path endpoints at the result type	`PositiveInductiveVectorTreeGuest.ResultFold.source_target_typed`	PCQ	proved
D	Preserve displayed types under ordinary projection unfolding	`PositiveInductiveVectorTreeGuest.ResultFold.projection_typedUnfolding`	PCQ	proved
D	Preserve length, order and repeated child occurrences	`PositiveInductiveVectorTreeGuest.ResultFold.childrenResults_canonical`	PQ	proved
D	Compute the solved fold through the existing equation	`PositiveInductiveVectorTreeGuest.ResultFold.fold_solved_steps`	PQ	proved
D	Compute the finite branch with source-ordered results	`PositiveInductiveVectorTreeGuest.ResultFold.fold_branch_canonical`	PQ	proved
D	Open the actual closed VectorTree fold program	`PositiveInductiveVectorTreeGuest.ResultFold.program_application`	PQ	proved
D	Refuse original children as recursive result values	`PositiveInductiveVectorTreeGuest.ResultFold.wrongBranch_refused`	PCQ	proved
D	Check the selected result-fold positive and negative controls	`PositiveInductiveVectorTreeGuest.ResultFold.all_verdicts`	PCQ	proved
D	Form every declared constructor code from primitive syntax typing	`IndexedPositiveContainerGuestTypes.constructorType_formed`	PQ	proved
D	Close generated constructors over their formed parameter telescope	`IndexedInductive.Guest.Declarations.Registered.constructorEntry_formed`	PQ	proved
D	Admit the generated fresh opaque constructor at its actual prefix	`IndexedInductive.Guest.Declarations.Registered.constructorEntry_admitted`	PQ	proved
D	Admit the actual family-only source prefix	`IndexedPositiveContainerGuestControls.Lists.Formation.prefix_admitted`	PQ	proved
D	Form the family-only source prefix	`IndexedPositiveContainerGuestControls.Lists.Formation.prefix_formed`	PQ	proved
D	Validate the actual native List primitive typing contracts	`IndexedPositiveContainerGuestControls.Lists.Formation.primitives`	PQ	proved
D	Form every registered-List constructor before its admission	`IndexedPositiveContainerGuestControls.Lists.Formation.every_constructor_formed`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Retain the formed source branch in its actual parameter context	`IndexedPositiveContainerGuestControls.Lists.Formation.source_branch_judgment`	PQ	proved
D	Close the actual source branch over its parameter	`IndexedPositiveContainerGuestControls.Lists.Formation.source_closed_branch_formed`	PQ	proved
D	Compose family and generically derived branch admission	`IndexedPositiveContainerGuestControls.Lists.Formation.family_branch_admitted`	PQ	proved
D	Independently check the generated branch at its family prefix	`IndexedPositiveContainerGuestControls.Lists.Formation.source_branch_checked`	PQ	proved
D	Refute a primitive contract for an undeclared family	`IndexedPositiveContainerGuestControls.Lists.Formation.missing_family_refused`	PQ	proved
D	Refuse the generated branch without its family prefix	`IndexedPositiveContainerGuestControls.Lists.Formation.missing_prefix_refused`	PQ	proved
D	Replay witness-free native Vector All formation	`NativeVectorAll.TypeFormation.replay_checked`	PCQ	proved
D	Form native Vector All from typed inputs in every signature extension	`NativeVectorAll.TypeFormation.allType_typed`	PCQ	proved
D	Recover actual native Vector declaration formation	`IndexedPositiveContainerGuestControls.Vectors.Formation.nativeVecType_formed`	PCQ	proved
D	Admit the actual VectorTree family-only prefix	`IndexedPositiveContainerGuestControls.Vectors.Formation.prefix_admitted`	PQ	proved
D	Form the actual VectorTree family-only prefix	`IndexedPositiveContainerGuestControls.Vectors.Formation.prefix_formed`	PQ	proved
D	Type actual length expressions through the data environment	`IndexedPositiveContainerGuestControls.Vectors.Formation.lengthTerm_typed`	P	proved
D	Discharge actual Vector constructor primitive contracts	`IndexedPositiveContainerGuestControls.Vectors.Formation.primitives`	PCQ	proved
D	Form every constructor over the actual Vector registry	`IndexedPositiveContainerGuestControls.Vectors.Formation.every_constructor_formed`	PCQ	proved
D	Form the actual Vector bundle without assuming its evidence	`IndexedPositiveContainerGuestControls.Vectors.Formation.bundle_formed`	PCQ	proved
D	Admit the generated Vector branch at its family prefix	`IndexedPositiveContainerGuestControls.Vectors.Formation.branch_admitted`	PCQ	proved
D	Admit the generated mixed Vector constructor at its family prefix	`IndexedPositiveContainerGuestControls.Vectors.Formation.mixed_admitted`	PCQ	proved
D	Independently check the Vector branch at its family prefix	`IndexedPositiveContainerGuestControls.Vectors.Formation.branch_checked`	PCQ	proved
D	Independently check the mixed Vector constructor at its family prefix	`IndexedPositiveContainerGuestControls.Vectors.Formation.mixed_checked`	PCQ	proved
D	Refuse a payload used as the Vector length	`IndexedPositiveContainerGuestControls.Vectors.Formation.wrong_mixed_refused`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Form generated dependent method bodies from typed primitive contracts	`IndexedPositiveContainerGuestTypes.methodBody_formed`	PQ	proved
D	Form generated dependent methods from actual constructors and motives	`IndexedPositiveContainerGuestTypes.methodType_formed`	PQ	proved
D	Retain the actual authored Vector branch type	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.source_type_exact`	PQ	proved
D	Keep the tree eliminator absent from the method prefix	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.no_eliminator`	PQ	proved
D	Admit the family and branch prefix before the tree eliminator	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.prefix_admitted`	PCQ	proved
D	Discharge actual Vector dependent-method primitive contracts	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.primitives`	PCQ	proved
D	Form the actual source method through the generic theorem	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.source_method_formed`	PCQ	proved
D	Close the source method over its actual motive	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.closed_method_formed`	PCQ	proved
D	Admit the derived source method as an ordinary opaque declaration	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.method_admitted`	PCQ	proved
D	Independently check the derived source method	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.source_method_checked`	PCQ	proved
D	Refuse a source method without its constructor	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.missing_constructor_refused`	PCQ	proved
D	Refuse a source method for an undeclared constructor	`IndexedPositiveContainerGuestControls.Vectors.MethodFormation.undeclared_constructor_refused`	PCQ	proved
D	Form the generated motive at its explicit upper universe	`IndexedPositiveContainerGuestTypes.motiveType_formed`	PQ	proved
D	Form the final dependent result interface	`IndexedPositiveContainerGuestTypes.resultType_formed`	PQ	proved
D	Form the complete ordered dependent-method inventory	`IndexedPositiveContainerGuestTypes.methodsType_formed`	PQ	proved
D	Form the complete eliminator from primitive contracts	`IndexedPositiveContainerGuestTypes.eliminatorType_formed`	PQ	proved
D	Retain the full authored prefix before its eliminator	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.prefix_exact`	PQ	proved
D	Keep the source eliminator absent from its formation prefix	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.no_eliminator`	PQ	proved
D	Generically admit both source constructors at their actual prefixes	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.prefix_admitted`	PCQ	proved
D	Discharge actual VectorTree eliminator universe contracts	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.primitives`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Type the full actual constructor inventory structurally	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.constructors_typed`	PCQ	proved
D	Form the actual complete source eliminator generically	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.source_eliminator_formed`	PCQ	proved
D	Admit the structurally formed source eliminator	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.eliminator_admitted`	PCQ	proved
D	Derive full authored inventory admission from structural formation	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.full_inventory_admitted`	PCQ	proved
D	Independently check the complete generated source eliminator	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.source_checked`	PCQ	proved
D	Refuse an eliminator missing its source constructors	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.missing_constructor_refused`	PCQ	proved
D	Refuse the checked lower-universe claim for the eliminator type	`IndexedPositiveContainerGuestControls.Vectors.EliminatorFormation.lower_universe_refused`	PCQ	proved
D	Recover unrestricted typing as the universal admission policy	`IndexedPositiveContainerGuestTypes.indexTypingOn_unrestricted_iff`	P	proved
D	Form family fibres at admitted source indices	`IndexedPositiveContainerGuestTypes.ConstructorPrimitives.familyAt_formedOn`	PQ	proved
D	Form constructors from code-local index admission	`IndexedPositiveContainerGuestTypes.constructorType_formedOn`	PQ	proved
D	Type motive fibres at admitted source indices	`IndexedPositiveContainerGuestTypes.ConstructorPrimitives.predicateAt_typedOn`	PQ	proved
D	Form dependent method bodies under source-local index admission	`IndexedPositiveContainerGuestTypes.methodBody_formedOn`	PQ	proved
D	Form dependent methods under source-local index admission	`IndexedPositiveContainerGuestTypes.methodType_formedOn`	PQ	proved
D	Form ordered method inventories with admitted source indices	`IndexedPositiveContainerGuestTypes.methodsType_formedOn`	PQ	proved
D	Form complete eliminators with admitted source indices	`IndexedPositiveContainerGuestTypes.eliminatorType_formedOn`	PQ	proved
D	Close scoped-index constructor formation over its parameters	`IndexedInductive.Guest.Declarations.Registered.constructorEntry_formedOn`	PQ	proved
D	Admit a scoped-index constructor at its actual prefix	`IndexedInductive.Guest.Declarations.Registered.constructorEntry_admittedOn`	PQ	proved
D	Type supported zero-variable-successor reference indices	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.supported_index_typed`	P	proved

Part	Claim	Lean declaration	Axioms	Status
D	Type an actual open successor index in the constructor-free prefix	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.native_successor_index_typed`	PQ	proved
D	Refute unrestricted typing of the undeclared reference addition	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.native_total_contract_refused`	PQ	proved
D	Discharge the actual scoped Nat-indexed Vec primitives	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.scoped_primitives`	PQ	proved
D	Admit all indices of the original Cons constructor	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.cons_indices_admitted`	P	proved
D	Admit the original Nil result index	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.nil_indices_admitted`	P	proved
D	Form the original indexed Cons through the scoped structural theorem	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_cons_formed`	PQ	proved
D	Close the original cumulative vector constructor type	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_vcons_type_formed`	PQ	proved
D	Keep both vector constructors and eliminator absent from formation	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_prefix_has_no_vector_constructors`	PQ	proved
D	Admit the original indexed Cons through scoped generic formation	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_cons_admitted`	PQ	proved
D	Retain the original reference constructor declaration	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.cons_entry_reference`	PQ	proved
D	Independently check the scoped original indexed constructor	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_cons_checked`	PQ	proved
D	Refuse scoped indexed admission without Nat	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_missing_nat_refused`	PQ	proved
D	Refuse undeclared arithmetic at the declaration boundary	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_addition_refused`	PQ	proved
D	Refuse addition code under the supported operation policy	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.addition_code_not_admitted`	P	proved

Part	Claim	Lean declaration	Axioms	Status
D	Check the generated indexed entry through its exact reference comparison	`IndexedPositiveContainerGuestControls.OrdinaryVectors.IndexAdmission.source_generated_cons_checked`	PQ	proved
D	Admit the original constructor-only Nat/Vec prefix	`IndexedPositiveContainerUniverseControls.prefix_admitted`	PQ	proved
D	Qualify indexed Vec primitives before its eliminator	`IndexedPositiveContainerUniverseControls.constructor_primitives`	PQ	proved
D	Discharge independent data, result and method universe contracts	`IndexedPositiveContainerUniverseControls.primitives`	PQ	proved
D	Admit every original indexed Vec constructor occurrence	`IndexedPositiveContainerUniverseControls.indices_admitted`	P	proved
D	Type the original constructor inventory in its admitted prefix	`IndexedPositiveContainerUniverseControls.constructors_typed`	PQ	proved
D	Form the full generated indexed eliminator at independent universes	`IndexedPositiveContainerUniverseControls.generated_type_formed`	PQ	proved
D	Retain the original reference indexed eliminator type	`IndexedPositiveContainerUniverseControls.generated_type_exact`	PCQ	proved
D	Keep the indexed eliminator absent from its formation prefix	`IndexedPositiveContainerUniverseControls.source_eliminator_absent`	PQ	proved
D	Admit the independently polymorphic generated eliminator opaquely	`IndexedPositiveContainerUniverseControls.eliminator_admitted`	PQ	proved
D	Close the original declaration inventory from structural elimination formation	`IndexedPositiveContainerUniverseControls.source_inventory_admitted`	PCQ	proved
D	Transport generated formation through independent level instantiation	`IndexedPositiveContainerUniverseControls.instantiated_type_formed`	PQ	proved
D	Relate concrete checked types to the instantiated generated source	`IndexedPositiveContainerUniverseControls.closed_type_exact`	PCQ	proved
D	Form the original indexed source with a higher result level	`IndexedPositiveContainerUniverseControls.higher_result_formed`	PQ	proved
D	Form the original indexed source with a lower result level	`IndexedPositiveContainerUniverseControls.lower_result_formed`	PQ	proved
D	Independently check the higher-result indexed instance	`IndexedPositiveContainerUniverseControls.higher_result_checked`	PQ	proved
D	Independently check the lower-result indexed instance	`IndexedPositiveContainerUniverseControls.lower_result_checked`	PQ	proved
D	Accept the motive-binding result-universe successor	`IndexedPositiveContainerUniverseControls.motive_successor_bound_checked`	PQ	proved
D	Refuse an omitted motive-binding universe successor	`IndexedPositiveContainerUniverseControls.missing_motive_successor_refused`	PQ	proved
D	Refuse indexed elimination before its constructor dependencies	`IndexedPositiveContainerUniverseControls.missing_constructors_refused`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Directly type the original indexed Cons in its admitted constructor prefix	`IndexedPositiveContainerUniverseControls.source_cons_typed`	PQ	proved
D	Check the existing All template with independent child and predicate universes	`NativeVectorAll.Universes.accepted_eq_true`	PCQ	proved
D	Form the universe-instantiated open All program without pointwise evidence	`NativeVectorAll.Universes.formed`	PCQ	proved
D	Instantiate independent-universe All formation under typed inputs	`NativeVectorAll.Universes.allType_typed`	PCQ	proved
D	Commute universe-instantiated All with scoped substitution	`NativeVectorAll.Universes.substitute_allType`	PCQ	proved
D	Refuse an All result bound below the declared predicate universe	`NativeVectorAll.Universes.insufficient_bound_refused`	PCQ	proved
D	Refuse a natural type in the All predicate position	`NativeVectorAll.Universes.malformed_predicate_refused`	PCQ	proved
D	Admit the original VectorTree constructor prefix at the new native universe instance	`IndexedPositiveContainerUniverseTreeControls.prefix_admitted`	PCQ	proved
D	Qualify original VectorTree fields at their actual data universe	`IndexedPositiveContainerUniverseTreeControls.constructor_primitives`	PCQ	proved
D	Assemble the original VectorTree and native All at independent universe coordinates	`IndexedPositiveContainerUniverseTreeControls.primitives`	PCQ	proved
D	Type the actual VectorTree constructors before its eliminator	`IndexedPositiveContainerUniverseTreeControls.constructors_typed`	PCQ	proved
D	Directly type the original VectorTree branch from its inventory	`IndexedPositiveContainerUniverseTreeControls.source_branch_typed`	PCQ	proved
D	Form the full registered VectorTree eliminator with independent result universe	`IndexedPositiveContainerUniverseTreeControls.generated_type_formed`	PCQ	proved
D	Retain the independently written polymorphic VectorTree interface	`IndexedPositiveContainerUniverseTreeControls.source_type_exact`	PCQ	proved
D	Keep the polymorphic VectorTree eliminator absent from formation	`IndexedPositiveContainerUniverseTreeControls.source_eliminator_absent`	PQ	proved
D	Ordinarily admit the generated polymorphic VectorTree eliminator	`IndexedPositiveContainerUniverseTreeControls.eliminator_admitted`	PCQ	proved
D	Close the assembled independent-universe VectorTree declaration inventory	`IndexedPositiveContainerUniverseTreeControls.source_inventory_admitted`	PCQ	proved
D	Independently check the polymorphic VectorTree reference interface	`IndexedPositiveContainerUniverseTreeControls.source_checked`	PCQ	proved
D	Check the exact generated polymorphic VectorTree source entry	`IndexedPositiveContainerUniverseTreeControls.source_generated_checked`	PCQ	proved
D	Refuse polymorphic VectorTree elimination without its constructors	`IndexedPositiveContainerUniverseTreeControls.missing_constructor_refused`	PCQ	proved
D	Type the original source children in the actual bundle context	`IndexedPositiveContainerUniverseTreeControls.original_children_typed`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Refuse original child data in place of the required induction evidence	`IndexedPositiveContainerUniverseTreeControls.original_children_not_bundle`	PCQ	proved
D	Directly form the independent-universe induction bundle without evidence	`IndexedPositiveContainerUniverseTreeControls.source_bundle_formed`	PCQ	proved
D	Distinguish the declared evidence bundle from the child-vector interface	`IndexedPositiveContainerUniverseTreeControls.bundle_cannot_be_replaced_by_children`	PQ	proved
D	Retain the existing Vector-eliminator program under a raw universe annotation	`NativeVectorAll.Universes.Canonical.allTypeAtUniverse_eq_eliminate`	PQ	proved
D	Compute every canonical constructor Vector to its quoted dependent-pair type spine	`NativeVectorAll.Universes.Canonical.allTypeAtUniverse_steps`	PQ	proved
D	Retain one common quoted type spine across raw universe annotations	`NativeVectorAll.Universes.Canonical.universe_annotation_common_result`	PQ	proved
D	Specialize canonical type-spine computation to the checked universe profile	`NativeVectorAll.Universes.Canonical.allType_steps`	PQ	proved
D	Compute the actual two-position dependent-predicate source instance	`NativeVectorAll.Universes.Canonical.two_positions_reduce`	PQ	proved
D	Compare two raw annotations on the same concrete source vector	`NativeVectorAll.Universes.Canonical.two_raw_annotations_share_result`	PQ	proved
D	Keep two source positions distinct from a one-position type spine	`NativeVectorAll.Universes.Canonical.two_positions_not_one`	PQ	proved
D	Exhibit a predicate reduction inside a quoted type spine	`NativeVectorAll.Universes.Canonical.quoted_predicate_can_reduce`	PQ	proved
D	Commute All generation with capture-avoiding input substitution	`NativeVectorAll.Universes.Canonical.substitute_allTypeAtUniverse`	PCQ	proved
D	Commute All generation with universe instantiation	`NativeVectorAll.Universes.Canonical.substLevels_allTypeAtUniverse`	PQ	proved
D	Commute both substitutions on the actual four-input All program	`NativeVectorAll.Universes.Canonical.instantiation_substitution_square`	PCQ	proved
D	Form both open All annotations at one common upper bound	`NativeVectorAll.Universes.CumulativeElimination.both_typed`	PCQ	proved
D	Convert the two All types on every constructor vector	`NativeVectorAll.Universes.CumulativeElimination.canonical_conversion`	PQ	proved
D	Transport every inhabitant judgment on constructor vectors	`NativeVectorAll.Universes.CumulativeElimination.canonical_typing_iff`	PCQ	proved
D	Instantiate both open judgments with exact extension lookup	`NativeVectorAll.Universes.CumulativeElimination.instantiated_both_typed`	PCQ	proved
D	Admit the collapsed target despite the symbolic source refusal	`NativeVectorAll.Universes.CumulativeElimination.collapsed_upper_fits_lower`	PCQ	proved
D	Lift an actually checked two-position dependent-pair proof	`NativeVectorAll.Universes.CumulativeElimination.two_positions_lift`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Instantiate the actual two-position predicate source	`NativeVectorAll.Universes.CumulativeElimination.two_positions_instantiated`	PCQ	proved
D	Refuse the enlarged program at an insufficient symbolic bound	`NativeVectorAll.Universes.CumulativeElimination.upper_below_refused`	PCQ	proved
D	Retain the different annotations in neutral source syntax	`NativeVectorAll.Universes.CumulativeElimination.neutral_programs_distinct`	PQ	proved
D	Commute generated constructor types with head maps	`IndexedPositiveContainerGuestTypes.constructorType_mapHead`	PQ	proved
D	Commute dependent method bodies with head maps	`IndexedPositiveContainerGuestTypes.methodBody_mapHead`	PQ	proved
D	Commute complete generated eliminators with head maps	`IndexedPositiveContainerGuestTypes.eliminatorType_mapHead`	PQ	proved
D	Retain the actual source index interpretation under head maps	`IndexedInductive.Guest.TypedControls.indexTerm_mapHead`	P	proved
D	Instantiate the registered native Vector bundle template	`IndexedPositiveContainerUniverseTreeControls.containers_substLevels`	PCQ	proved
D	Instantiate the whole nested VectorTree eliminator	`IndexedPositiveContainerUniverseTreeControls.generated_type_substLevels`	PCQ	proved
D	Check a nonidentity instance of the whole nested source type	`IndexedPositiveContainerUniverseTreeControls.generated_collapse_exact`	PCQ	proved
D	Instantiate the complete indexed Vec eliminator	`IndexedPositiveContainerUniverseControls.generated_type_substLevels`	PQ	proved
D	Distinguish a collapsed generated annotation from the original	`IndexedPositiveContainerUniverseTreeControls.generated_annotation_changes`	PCQ	proved
D	Map All evidence to the upper annotation at neutral vectors	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.forward_typed`	PCQ	proved
D	Map All evidence back to the lower annotation at neutral vectors	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.backward_typed`	PCQ	proved
D	Compare inhabitation of both All annotations at arbitrary typed inputs	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.inhabited_iff`	PCQ	proved
D	Refuse a projection shortcut under the indexed map certificate	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.projection_certificate_refused`	PCQ	proved
D	Refuse reversed evidence for the forward universe map	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.swapped_certificate_refused`	PCQ	proved
D	Preserve a finite head observation under pair expansion	`ParameterizedPiSigmaId.Presentation.PairSpine.head_expand`	P	proved
D	Preserve the final tail under pair expansion	`ParameterizedPiSigmaId.Presentation.PairSpine.tail_expand`	P	proved
D	Compute a universe map on opaque evidence over constructor vectors	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.canonical_steps`	PQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Preserve each source head through the actual Vector map	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.canonical_head`	PQ	proved
D	Preserve the final source tail through the actual Vector map	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.canonical_tail`	PQ	proved
D	Preserve head observations through a universe round trip	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.roundtrip_head`	PQ	proved
D	Preserve the tail observation through a universe round trip	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.roundtrip_tail`	PQ	proved
D	Contract repeated pair expansion without terminal eta	`ParameterizedPiSigmaId.Presentation.PairSpine.expand_idempotent`	P	proved
D	Compute the complete universe round trip on a constructor vector	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.roundtrip_steps`	PQ	proved
D	Reconstruct a formed dependent spine by typed pair eta	`ParameterizedPiSigmaId.Presentation.PairSpine.TypedSpine.expand_typed_and_equal`	P	proved
D	Produce an object identity witness for pair-eta reconstruction	`ParameterizedPiSigmaId.Presentation.PairSpine.TypedSpine.expand_identity`	P	proved
D	Transport a typed spine through context substitution	`ParameterizedPiSigmaId.Presentation.PairSpine.TypedSpine.substitute`	PQ	proved
D	Transport a typed spine through a rule-package morphism	`ParameterizedPiSigmaId.Presentation.PairSpine.TypedSpine.mapHead`	PQ	proved
D	Reconstruct typed equality from retained projection comparisons	`ParameterizedPiSigmaId.Presentation.PairSpine.equal_of_readout`	P	proved
D	Transport the unknown-bundle identity witness through universe substitution	`ParameterizedPiSigmaId.Presentation.TypedEquality.DependentPairSpine.neutral_identity_levels`	PQ	proved
D	Separate typed pair eta from raw beta/projection conversion	`ParameterizedPiSigmaId.Presentation.TypedEquality.DependentPairSpine.neutral_expansion_not_raw_convertible`	PQ	proved
D	Read arbitrary supplied evidence at constructor-vector positions	`NativeVectorAll.ResultView.canonical_steps_observe`	PQ	proved
D	Read independent assembled witnesses with an arbitrary final tail	`NativeVectorAll.ResultView.canonical_steps_assembled`	PQ	proved
D	Read the actual registered Vector induction bundle	`NativeVectorAll.ResultView.semanticBundle_steps`	PCQ	proved
D	Separate raw expansion syntax from its opaque input	`NativeVectorAll.Universes.CumulativeElimination.EvidenceMaps.expansion_syntax_distinct`	PQ	proved
D	Exhibit distinct raw inputs sharing one result observation	`NativeVectorAll.ResultView.shared_result_distinct_inputs`	PQ	proved
D	Check independent caller bounds in the enlarged Vector result profile	`NativeVectorAll.ResultView.Universes.program_formed`	PCQ	proved
D	Refuse the same result certificate in the original native profile	`NativeVectorAll.ResultView.Universes.original_certificate_refused`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Transport formed judgments through the actual result-profile level instance	`NativeVectorAll.ResultView.Universes.judgment_map`	PQ	proved
D	Compute ordered mapped witnesses with the actual result program	`NativeVectorAll.ResultView.Universes.program_computes_mapped_witnesses`	PQ	proved
D	Accept the mapped two-position conversion certificate	`NativeVectorAll.ResultView.Universes.mapped_conversion_accepted`	PCQ	proved
D	Refuse the same code for swapped result witnesses	`NativeVectorAll.ResultView.Universes.mapped_swapped_refused`	PCQ	proved
D	Distinguish the result program's raw universe annotation	`NativeVectorAll.ResultView.Universes.program_annotation_changes`	PQ	proved
D	Form compatible declaration unions through shared signature transport	`ParameterizedPiSigmaId.Presentation.Declaration.Signature.Formed.merge`	PQ	proved
D	Retain the original lower input in a formed higher-result signature	`NativeVectorAll.ResultView.OriginalInputUniverses.signature_formed`	PCQ	proved
D	Check the actual original-input higher-result program	`NativeVectorAll.ResultView.OriginalInputUniverses.program_formed`	PCQ	proved
D	Refuse its certificate with a changed result family	`NativeVectorAll.ResultView.OriginalInputUniverses.changed_result_certificate_refused`	PCQ	proved
D	Expose the exact unrenamed declaration-entry conflict	`NativeVectorAll.ResultView.OriginalInputUniverses.unrenamed_signatures_incompatible`	PQ	proved
D	Accept ordered independent position evidence	`NativeVectorAll.ResultView.OriginalInputUniverses.independent_conversion_accepted`	PCQ	proved
D	Refuse the same code for swapped position witnesses	`NativeVectorAll.ResultView.OriginalInputUniverses.swapped_conversion_refused`	PCQ	proved
D	Transport the joined signature through universe substitution	`NativeVectorAll.ResultView.OriginalInputUniverses.result_signature_level_formed`	PCQ	proved
D	Transport the complete original-input program judgment	`NativeVectorAll.ResultView.OriginalInputUniverses.program_level_transport`	PCQ	proved
D	Derive ordered Vector evidence readout from actual roots and case computations	`NativeVectorAll.ResultView.EvidenceElimination.fold_reads`	P	proved
D	Read all constructor positions in the original-input higher-result profile	`NativeVectorAll.ResultView.OriginalInputUniverses.Computation.project_reads`	PCQ	proved
D	Compute the actual original-input program on independent assembled witnesses	`NativeVectorAll.ResultView.OriginalInputUniverses.Computation.program_witnesses`	PCQ	proved
D	Specialize the original-input result computation to the registered Vector bundle	`NativeVectorAll.ResultView.OriginalInputUniverses.Computation.registered_bundle_steps`	PCQ	proved
D	Transport the complete result computation through universe substitution	`NativeVectorAll.ResultView.OriginalInputUniverses.Computation.program_level_witnesses`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Separate constructor-prefix injection from global argument renaming in raw syntax	`NativeVectorAll.ResultView.OriginalInputUniverses.Computation.constructor_prefix_renaming_is_distinct`	PCQ	proved
D	Form the checked closed result definition	`NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.formed`	PCQ	proved
D	Type its declared source name	`NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.constant_typed`	PCQ	proved
D	Preserve the displayed type when the checked result definition unfolds	`NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.typed_unfolding`	PCQ	proved
D	Type the closed original-input application returning a Vector of types	`NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.application_typed`	PCQ	proved
D	Independently type its returned Vector of types	`NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.output_typed`	PCQ	proved
D	Check the finite conversion from the named application to its output	`NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.application_conversion`	PCQ	proved
D	Refuse the same Vector-of-types certificate for swapped result witnesses	`NativeVectorAll.ResultView.OriginalInputUniverses.ClosedResultDefinition.swapped_refused`	PCQ	proved
D	Reconstruct guarded raw trees with canonical unused slots	`IndexedPolynomial.GuardedRaw.algebraRepresentation`	PQ	proved
D	Compare indexed polynomial families with their guarded recursive-space image	`IndexedPolynomial.RecursiveSpaceRepresentation.recursiveAlgebraRepresentation`	PCQ	proved
D	Construct the relational recursive-space algebra	`HOLLight.RecursiveSpaceSemantics.model`	PCQ	proved
D	Interpret least membership in the existing HOL syntax	`HOLLight.RecursiveSpaceSemantics.predicateMember_iff_rawMember`	PCQ	proved
D	Retain ordered registered blocks through the representation	`IndexedPositiveContainer.RecursiveSpaceRepresentation.blockRepresentation`	PCQ	proved
D	Recover indexed shapes from fixed constructor data	`IndexedPolynomial.Description.constructorDataEquiv`	none	proved
D	Refuse a changed parameter index for the same payload	`IndexedInductive.ConstructorDataControls.wrong_parameter_index_refused`	P	proved
D	Recover the chronological history of received constant definitions	`HOLLight.DefinitionReplayModels.replay_constant_history`	PCQ	proved
D	Construct models of source-profile reference definitions at all type instances	`HOLLight.DefinitionReplayModels.loaded_reference_models`	PCQ	proved
D	Complete the actual registered type-instantiation certificate finder	`HOLLight.TypeInstantiationCertificates.instanceCertificates_complete`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Decide inference-parameter formation with the actual certificates	`HOLLight.TypeInstantiationCertificates.inferenceFormationCheck_exact`	PCQ	proved
D	Specialize the generic pointed witness to its exact native conclusion	`HOLLight.PointedPredicateInstantiation.specialized_conclusion`	PCQ	proved
D	Preserve registered theorem formation under type-correct substitution	`HOLLight.DefinitionAdapters.TheoremFormed.substitute`	PCQ	proved
D	Execute checked type and term instantiation with two retained receipts	`HOLLight.RegisteredSubstitution.apply_available`	PCQ	proved
D	Preserve earlier proof addresses through registered instantiation	`HOLLight.RegisteredSubstitution.preserves_lookup`	PCQ	proved
D	Retain the actual instantiation execution as a GSLT path	`HOLLight.RegisteredSubstitution.execution_path`	PCQ	proved
D	Observe the specialized theorem by OSLF reachability	`HOLLight.RegisteredSubstitution.Controls.final_observed`	PCQ	proved
D	Check the cached generic pointed-witness certificate	`HOLLight.PointedPredicateTemplate.accepted`	PCQ	proved
D	Check every cached source-formation obligation	`HOLLight.PointedPredicateTemplate.graph_formed`	PCQ	proved
D	Characterize registered proof reception by primitive replay and formation	`HOLLight.ProofExecution.Registered.replayProofs_iff`	PCQ	proved
D	Receive specialized witnesses from ordinary formed inputs	`HOLLight.RegisteredPointedPredicate.replay_available`	PCQ	proved
D	Preserve source derivations in specialized witness reception	`HOLLight.RegisteredPointedPredicate.replay_preserves_derivations`	PCQ	proved
D	Introduce a registered pointed type from ordinary formed inputs	`HOLLight.RegisteredPointedTypeDefinition.introduce_available`	PCQ	proved
D	Retain the exact addresses of both type-definition laws	`HOLLight.RegisteredPointedTypeDefinition.introduce_roots`	PCQ	proved
D	Derive both returned laws in the registered source calculus	`HOLLight.RegisteredPointedTypeDefinition.introduce_derived_laws`	PCQ	proved
D	Count two theorem outputs and one type-definition receipt	`HOLLight.RegisteredPointedTypeDefinition.finish_counts`	PCQ	proved
D	Preserve relative provability through all native inference operations	`HOLLight.InferenceInterpretation.inference`	PCQ	proved
D	Construct a model from actual axiom-free registered replay	`HOLLight.RegisteredReplayModels.replay_models`	PCQ	proved
D	Exclude translated falsity from the interpreted cache	`HOLLight.RegisteredReplayModels.Frame.cached_falsity_refused`	PCQ	proved
D	Validate both actual pointed-type laws in one constructed model	`HOLLight.RegisteredPointedModels.introduce_laws_models`	PCQ	proved
D	Join formed-input introduction and constructed model availability	`HOLLight.RegisteredPointedModels.introduce_available_models`	PCQ	proved
D	Identify the Boolean interpretation with its admitted reference history	`HOLLight.StandardTheoryInterpretation.referenceHistory_model`	PCQ	proved
D	Validate the actual source eta commitment in the reference model	`HOLLight.SourceProfileModels.reference_models_eta`	PCQ	proved
D	Validate the actual source choice commitment in the reference model	`HOLLight.SourceProfileModels.reference_models_selection`	PCQ	proved
D	Validate the actual source infinity commitment with a constructed witness	`HOLLight.SourceProfileModels.reference_models_infinity`	PCQ	proved

Part	Claim	Lean declaration	Axioms	Status
D	Replay all nine reference declarations with exact outputs and receipts	`HOLLight.SourceProfileModels .reference_replayed`	PCQ	proved
D	Construct models for accepted replay after the three-axiom reference state	`HOLLight.SourceProfileModels .replay_models`	PCQ	proved
A3	Finite-stack realization of the HOL Light to MM0 translation	—	—	open
A2	Existing shared-lemma HOL translation equals the node-by-node translation	—	—	open
A4	Free symmetric monoidal category of consuming processes	—	—	open
C	Portable modules for rho, MeTTa equations, recipes, rule-code kernels	—	—	open

15 Open problems

1. **Finite-stack realization of the HOL Light to MM0 translation.** The certificate bound is proved and the declared time account is bounded below by output weight, but no finite-stack machine computing the translation is given; a pushdown transducer for planar rule images is the natural first case.
2. **The existing lemma translation.** The generic node-by-node translation commutes with unfolding; prove that the existing shared-lemma HOL Light translation, which works over cumulative contexts and raw proofs, computes the same per-node rule images.
3. **Monoidal processes for consuming steps.** Independent consuming steps commute and the split/join example exceeds tree-shaped single-wire cuts; the free symmetric monoidal category of processes, with traces identified exactly up to independence, is not formalized.
4. **Memory of the executed MM0 compilation.** On a sixteen-event export with ten definitional roots, the existing compiler exhausted its memory allowance while compiling the last of sixteen shared proofs; witness sharing in the compiled MM0 proofs is the evident target.
5. **Remaining portable modules.** Rho, MeTTa equation and authored recipe kernels, and the propositional rule-code kernels, have faithful checkers but no portable module presentation.
6. **Admitted dependent carve-outs and native specialization.** Complete the context, substitution and theory comparisons for declared dependent fragments; give their smaller packages executable decoders and computational-admission proofs; connect the actual native checking machines through adequate operational routes. Proof-relevant route agreement, model assumptions and executed cost bounds remain separate obligations. Compare derived checking bases with the existing native kernels under the same fragment and cost model.
7. **Coding closure beyond detour-free source evidence.** The checked STTfa roundtrip retains detour-freedom and core witnesses. Extend the coding-preservation argument to arbitrary typed reducts before using a broader normal-form reflection theorem. This obligation remains separate from termination of normalization on every typed hub term.
8. **Lean-family native qualification.** Instantiate the four-outcome service and acceptance-refinement interfaces with a pinned external checker and export format. Bind parser, declaration stream, checker mode, semantic assumptions and runtime artifact in one release receipt. Prove the selected elaboration/reconstruction route into that checker, and keep official-kernel parity, model soundness and computational adequacy as distinct edges. Generalize the current finite-catalog exact node to declarations, inductive installation, quotients, primitives and the complete typed checker.
9. **Generic registered-container and mutual guest generation.** Generalize the checked List and Vector specializations to every admitted registered container and to the mutual-family source fragment. Retain each container's output expression, position bundle and executable-traversal capability. Then connect finite-position evidence to executable MeTTa traversal and retain an

explicit refusal or semantic-only status for positive infinitary containers.

References

[1] Dharmakirti. *Nyayabindu*, chapters I–III, with Dharmottara's commentary. English translation in Th. Stcherbatsky, *Buddhist Logic*, vol. II, Bibliotheca Buddhica XXVI, 1930.

[2] D. J. Chalmers and F. Jackson. Conceptual analysis and reductive explanation. *The Philosophical Review* 110 (2001), 315–361, Section 2. https://consc.net/papers/analysis.html.

[3] D. J. Chalmers. *Constructing the World*. Oxford University Press, 2012, especially chapters 1–3 and the sixth excursus on totality and indexical truths. https://consc.net/constructing-the-world/.

[4] S. A. Kripke. Outline of a theory of truth. *The Journal of Philosophy* 72 (1975), 690–716.

[5] J. Lambek. Deductive systems and categories II: standard constructions and closed categories. In *Category Theory, Homology Theory and their Applications I*, Lecture Notes in Mathematics 86, Springer, 1969, 76–122.

[6] C. Williams and M. Stay. Native Type Theory. *Electronic Proceedings in Theoretical Computer Science* 372 (2022), 116–132. https://doi.org/10.4204/EPTCS.372.9.

[7] P. Saville. Clones, closed categories, and combinatory logic. *FoSSaCS 2024*, Lecture Notes in Computer Science 14575, pp. 160–181. https://arxiv.org/abs/2405.01675.

[8] M. Carneiro. Metamath Zero: the Cartesian theorem prover. Preprint, 2019. https://arxiv.org/abs/1910.10703.

[9] J. Jang, A. Gaulin, J. Z. S. Hu and B. Pientka. McTT: A verified kernel for a proof assistant. *Proceedings of the ACM on Programming Languages*, ICFP, 2025. https://doi.org/10.1145/3747511.

[10] H. P. Barendregt. Lambda calculi with types. In S. Abramsky, D. M. Gabbay and T. S. E. Maibaum (eds.), *Handbook of Logic in Computer Science*, volume 2. Oxford University Press, 1992. https://www.cs.ru.nl/~henk/papers.html.

[11] R. Harper, F. Honsell and G. Plotkin. A framework for defining logics. *Journal of the ACM* 40(1), 1993, 143–184. Earlier report ECS-LFCS-91-162, 1991. https://publish.lfcs.inf.ed.ac.uk/reports/91/ECS-LFCS-91-162/.

[12] L. Pujet and N. Tabareau. Observational equality: now for good. *Proceedings of the ACM on Programming Languages* 6, POPL, 2022. https://doi.org/10.1145/3498693.

[13] F. Blanqui, G. Dowek, E. Grienenberger, G. Hondet and F. Thiré. A modular construction of type theories. *Logical Methods in Computer Science* 19(1), 2023, 12:1–12:28. https://doi.org/10.46298/lmcs-19(1:12)2023.

[14] M. Hofmann. *Extensional Concepts in Intensional Type Theory*. PhD thesis, University of Edinburgh, 1995. https://hdl.handle.net/1842/399.

[15] K. Kapulkin and Y. Li. Extensional concepts in intensional type theory, revisited. *Theoretical Computer Science* 1029, 2025, 115051. https://arxiv.org/abs/2310.05706.

[16] M. Carneiro. Lean4Lean: towards a verified typechecker for Lean, in Lean. Preprint, 2024. https://arxiv.org/abs/2403.14064.

[17] J. Breitner. *con-leche: a consistent Lean checker*. Lean FRO, 2026. https://github.com/leanprover/con-leche.

[18] N. Behr and P. Sobociński. Rule Algebras for Adhesive Categories. Version 4. https://arxiv.org/abs/1807.00785v4.

[19] N. Behr and J. Krivine. Compositionality of Rewriting Rules with Conditions. Version 2. https://arxiv.org/abs/1904.09322v2.